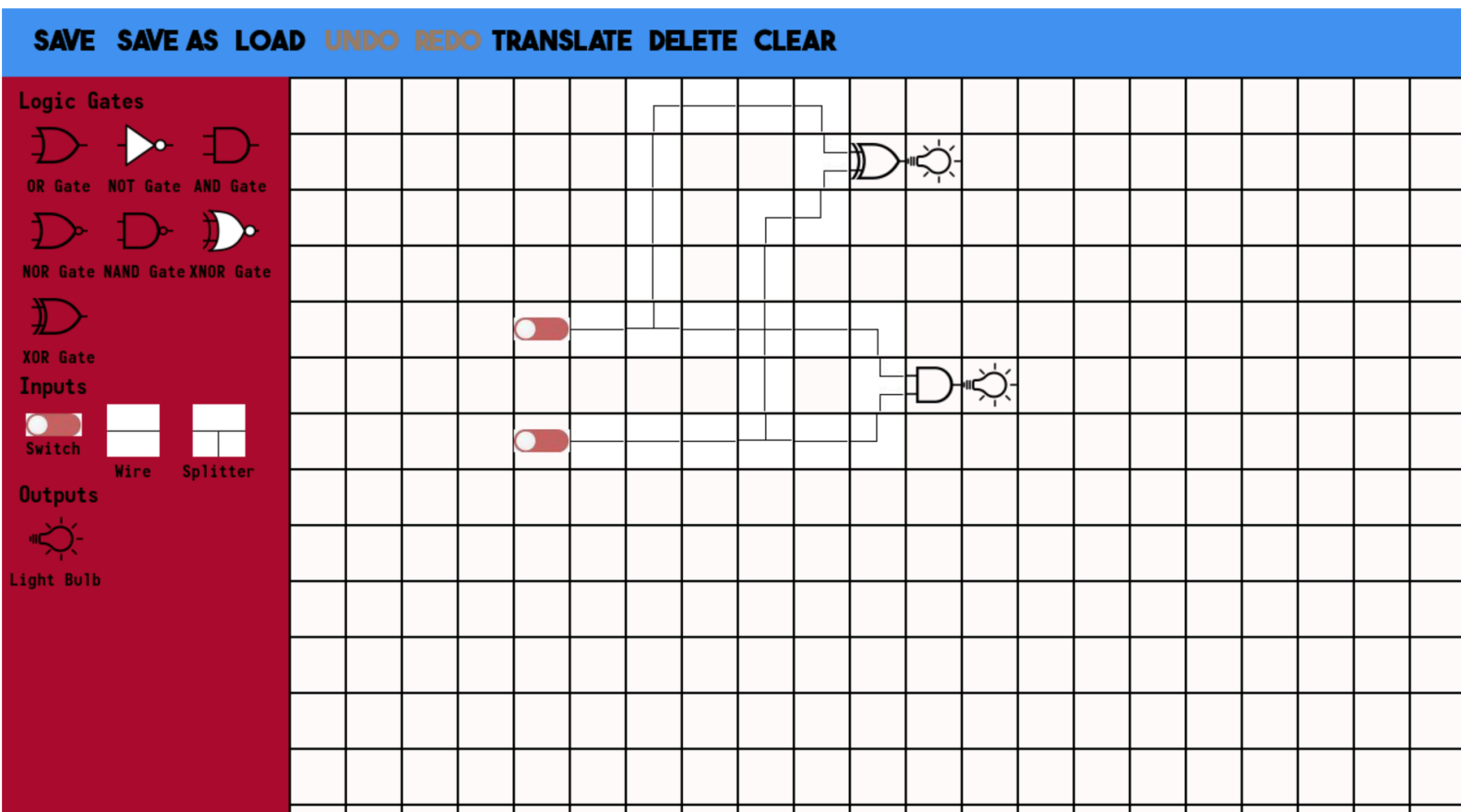


Logic Gate Simulator

Harry Taylor - 7173

The Angmering School -

65301



https://www.youtube.com/watch?v=t9S-TN_5w08

Contents

Contents	2
Analysis	5
Introduction to the project	5
Planned components	5
Simulator function	5
Logic gate definition	5
Logic gate types	6
OR Gate	6
AND Gate	6
NOT gate	7
NOR and NAND gates	7
XOR gate	8
XNOR gate	9
Adder definition	9
Adder types	10
Half adder	10
Full adder	11
Flip-flop definition	12
Flip-flop types	12
D type flip-flop	12
Tutorial	13
Initial research	13
Logic.ly	13
Logic gate Circuit Builder - Scratch	16
Potential Users	18
User information form	18
Test users	25
MOSCOW Chart	26
Objectives List	28
Time management	35
Documented Design	37
Data structures	37
Logic grid	37
Logic elements	39
Undo and redo elements	39
Truth tables	40
Structure diagram	41
Key algorithms	42
Menu subroutines	42

Save and Save As	42
Load file	48
Truth table creation	51
Tutorial	61
Adders	81
Turning off/on	83
Main loop	89
Logic element reinsertion and removal	95
Logic elements	99
Menu implementation	99
Grid placement	107
Boolean algebra translation	112
Button functionality	113
User interface	115
Main menu	115
Logic interface	118
Tutorial	120
Save menu	120
Load menu	122
Adders	123
Truth tables	126
Persistent storage	127
Saving	127
Testing	128
Test plan	128
Test evidence	153
Objective 1: User opens simulation	153
Objective 2: Options menu	168
Objective 3: Logic interface	181
Objective 4: Logic gate interactions and shortcuts	199
User testing	211
Technical solution	219
Subroutine and class definition and skills utilised	219
Full code	223
Imported algorithms	223
Button class	223
Binary translator	224
Main code	224
Evaluation	295
MOSCOW chart	295
Requirements	296

Improvements and further features	297
Boolean algebra translation	297
Tutorial	297
Truth table functionality	298
Bibliography	298
Appendices	301

Analysis

Introduction to the project

Logic gates are continually used within the GCSE and A level syllabus, although sometimes they can be hard to visualise, especially if multiple gates and inputs are involved. Given my experience and enjoyment of logic gates, I have a clear idea of what I would like to achieve.

My simulator will involve both a GUI based environment built in python likely using appjar or pygame as I have prior experience in both. The simulator will allow users to add and link different gates and inputs together, simplifying the experience of drawing out logic symbols and predicting their output. My unique selling point however will be the ability to convert a logic circuit into a truth table, or a boolean expression and vice versa.

By researching pre-existing systems, and expanding my expertise in logic gates further I hope to understand the project requirements and complete the project successfully.

Planned components

Here I will research the functions of logic gates themselves as well as pre-existing instances of logic gate simulators to provide inspiration for features to include in my NEA as well as how logic circuits will display on-screen.

Simulator function

The goal of the simulator is to allow users to drag and drop logic gates, flip-flops, half/full adders, input switches or buttons and outputs onto the screen and link them together. This interface will be accomplished using a GUI with multiple submenus allowing users access to other features such as saving their work, converting their work into boolean logic expressions and creating new work files.

Logic gate definition

A logic gate is an idealised or physical device capable of performing a Boolean algebra function on one or more binary inputs producing a single binary output. My NEA functions to create an idealistic workspace for logic gates although in reality physical circuits are built out of transistors and diodes.

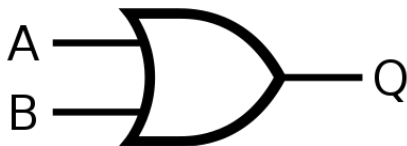
Logic gate types

The logic gates and boolean expressions I plan to use in my project are the ones found in both the AQA GCSE and A-Level computer science syllabuses. This is so the simulator can work effectively for students studying those syllabuses as well as effectively utilising my preexisting knowledge.

Here are each of the logic gates I plan to include:

OR Gate

An OR gate returns True if any of its inputs are true; otherwise it returns false. This gate is represented by the boolean expression (+). its symbol and truth table are as follows:



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1

AND Gate

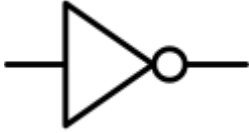
An AND gate returns True if all of its inputs are true; otherwise it returns false. This gate is represented by the boolean expression (.). Its symbol and truth table are as follows:



A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

NOT gate

A NOT gate inverts the input passed into it; if the input is True the output will be false and vice versa. This gate is represented in boolean by a line over the input ($\bar{A}$). Its symbol and truth table are as follows:

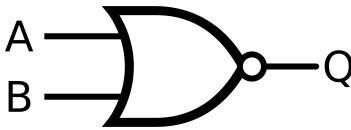


A	Q
0	1
1	0

NOR and NAND gates

NOR and NAND gates function by combining the function of a NOT gate with an OR and AND gate; Thus if the output of an OR gate or an AND gate is true in a NAND or NOR gate it will be false. These gates are represented in boolean by a line over an AND or OR expression. Their symbols and truth tables are as follows:

NOR gate



NAND gate



NOR gate

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

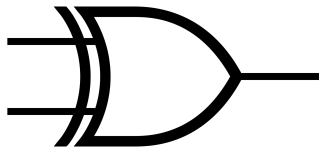
NAND gate

A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0

XOR gate

A XOR gate produces a true output every time the amount of inputs that are true are odd. This gates boolean expression, symbol and truth table are as follows:

$$A \cdot \bar{B} + \bar{A} \cdot B$$

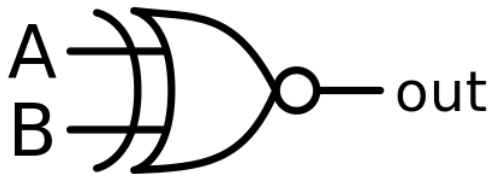


A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

XNOR gate

A XNOR gate combines the function of an XOR gate with the function of a NOT gate; thus if the output of an XOR gate is true the output of an XNOR gate is false and vice versa. This gates boolean representation, symbol and truth table are as follows:

$$(A + \bar{B}) \cdot (\bar{A} + B)$$



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

Adder definition

An adder is a digital circuit that performs addition of numbers. This type of circuit is often used within the arithmetic logic units on a processor, although they can be found in other areas of the processor. An adder usually performs addition on binary numbers whether that be two's complement or one's complement.

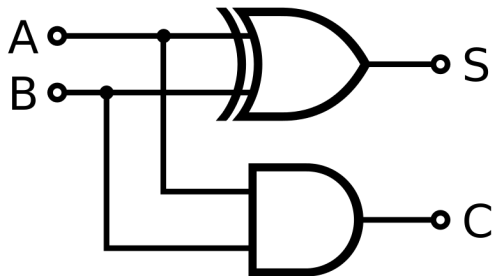
Adder types

There are two main adders that I plan to include within my project, both of which are common places. Adders would be pre-implemented circuits that users could access and work atop of. Implementing these would work to expand a user's knowledge of logic gates, as well as provide them with an effective demonstration of how logic gates can be used.

Here are the adders I plan to include:

Half adder

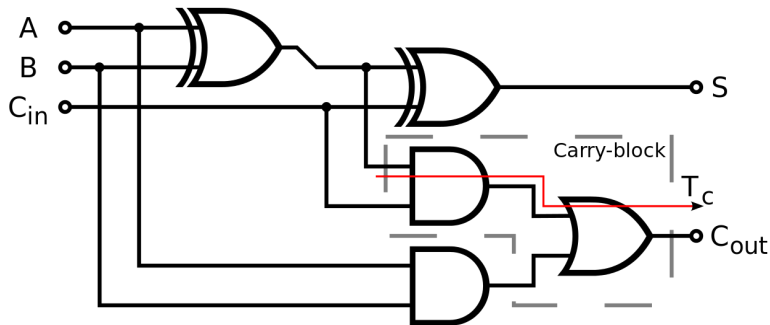
A half adder adds two single digit binary digits together, outputting a sum as well as a carry (overflow). This type of adder connects two inputs to two separate XOR and AND gates then those logic gates to the sum and carry outputs respectively. The half adders circuit and truth table are shown below:



Inputs		Outputs	
A	B	S	C
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Full adder

A full adder adds two binary numbers as well as a carry in digit, outputting a sum as well as an output carry (overflow). The same XOR and AND gates are implemented in the same way as in a half adder with other AND and OR gates used for the carry digit. The full adders circuit diagram and truth table are shown below:



Inputs			Outputs	
A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Flip-flop definition

Flip-flops have two stable states that store state information; The states are changed by passing through input signals. Flip-flops are examples of sequential circuits where outputs are dependent on both current inputs and past inputs. A flip-flop allows a previous input to be stored and acts as a simple volatile memory device. The operations in a flip-flop act on the system's clock which is a continuous alternating stream of binary ones and zeros which is essential in a computer system.

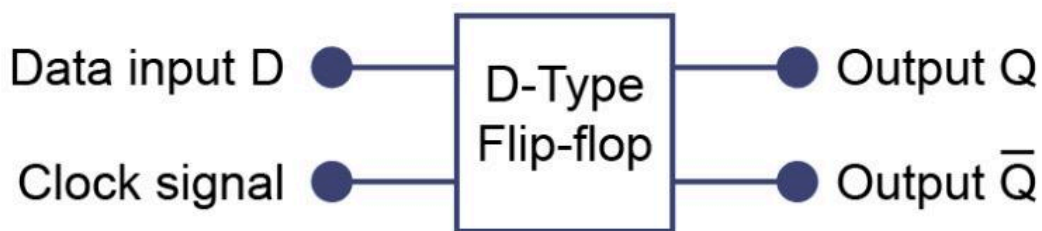
Flip-flop types

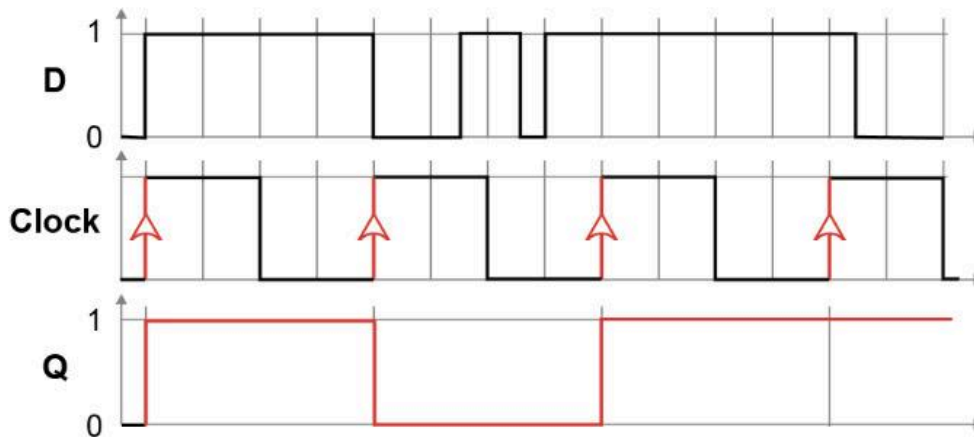
I plan to include the one type of flip-flop in my project, which is included in the second paper syllabus of the AQA A-level. Flip-flops would be added to the document in the same way as any other input or logic gate, with inputs and outputs corresponding to that of the flip-flop.

Here is the flip-flop I plan to include:

D type flip-flop

A D or Delay type flip-flop contains two inputs with one of them being a clock. This flip-flop prevents a change in state until the point of a positive edge clock pulse (0 to 1). If the input changes but the clock input is not at a positive edge the output will not change until the clock reaches its next positive edge. A D type flip-flop produces two outputs, which can be represented at Q and not Q. A diagram of a D-type flip-flop and its truth table is as follows:





Tutorial

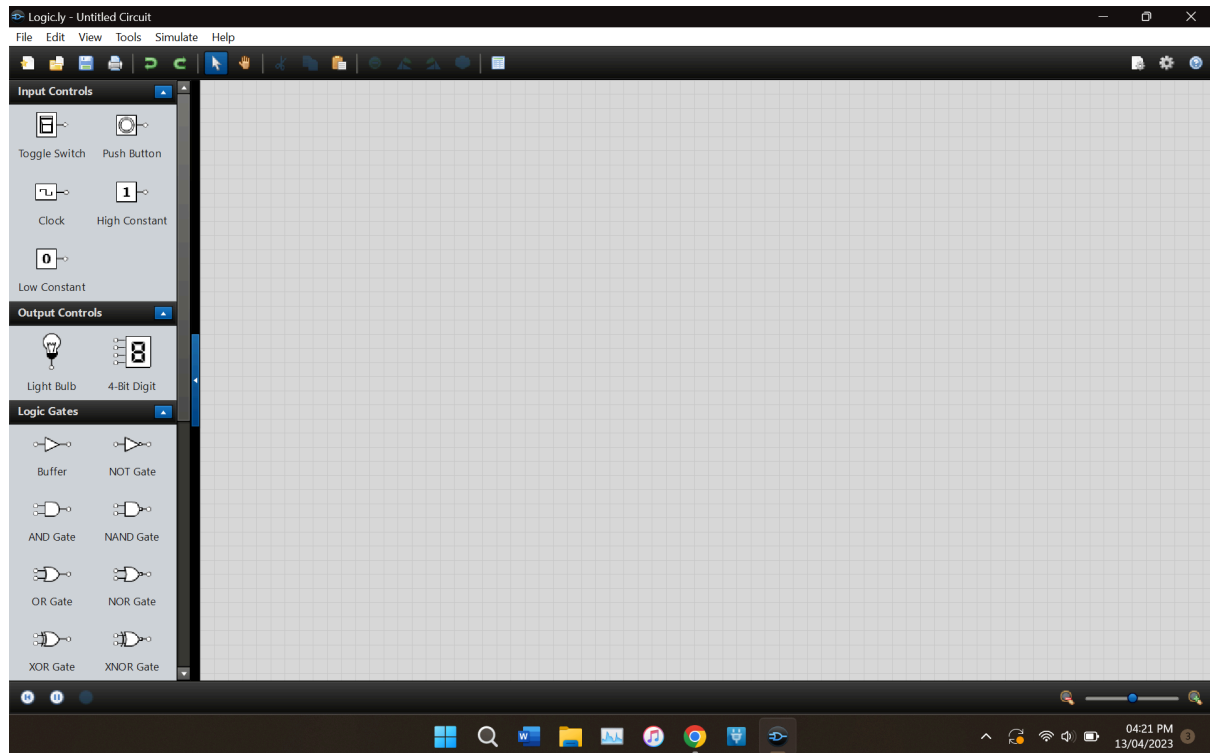
A function I intend on adding to the project as a whole is a tutorial system, which will guide users interactively showing them how to connect all of the logic gates to one another and to inputs/outputs. This feature will be targeted at people with a limited working knowledge of logic gates and will be accessible from the main menu.

Initial research

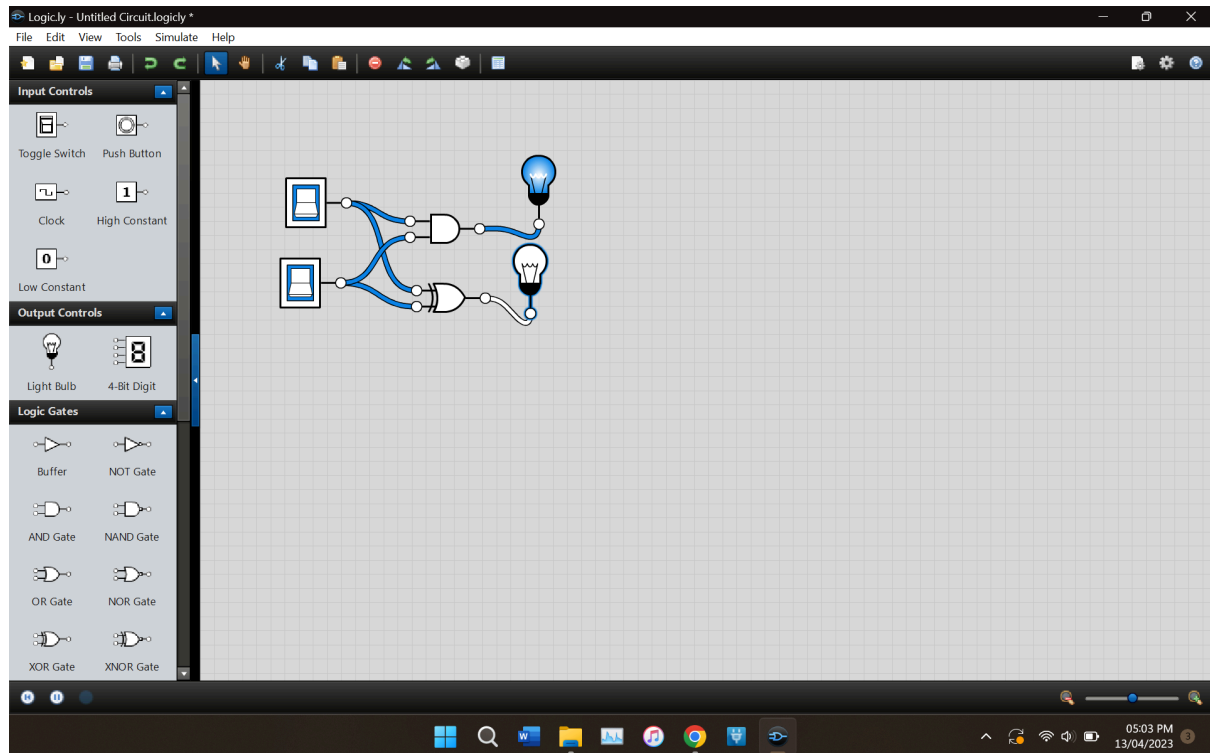
For insight into how my project could be completed, I downloaded and utilised the free trial of logic.ly for windows, as well as using the more readily available online client. A more basic version of a logic gate simulator is in a scratch coded version by user Proto-Eric, which also includes a tutorial section on how to connect inputs, gates and outputs.

[Logic.ly](https://logic.ly)

As mentioned before, logic.ly can be installed on Windows and Mac computers as well as accessed online.

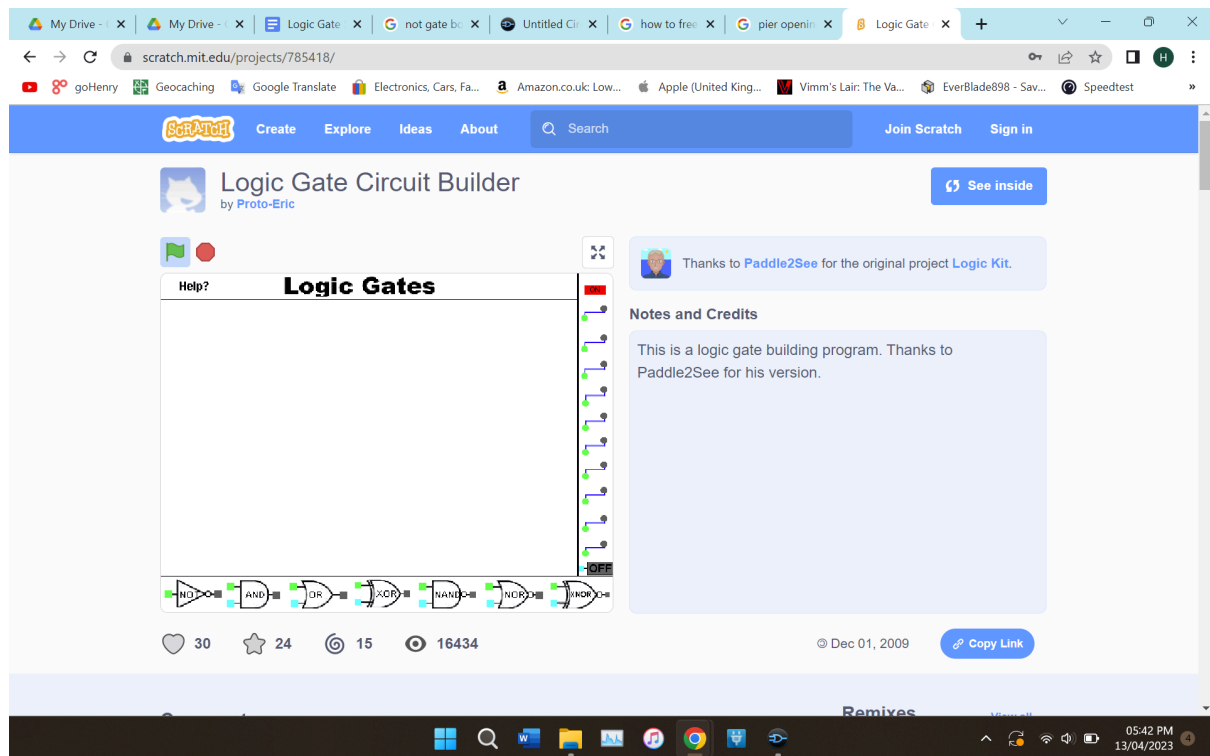


Shown above this is the interface a user is greeted with upon opening a new document. To the left are the options of logic gates, inputs and outputs along with flip flop gates (not shown in image). This software contains all types of logic gates I plan on including in my project, as well as examples of similar inputs and outputs. Along the top are options for saving, undoing and redoing; as well as a print option which is unlikely that I am going to include in my project. Upon opening the software the user is not greeted with a tutorial, which I find to be the biggest shortcoming of the software.



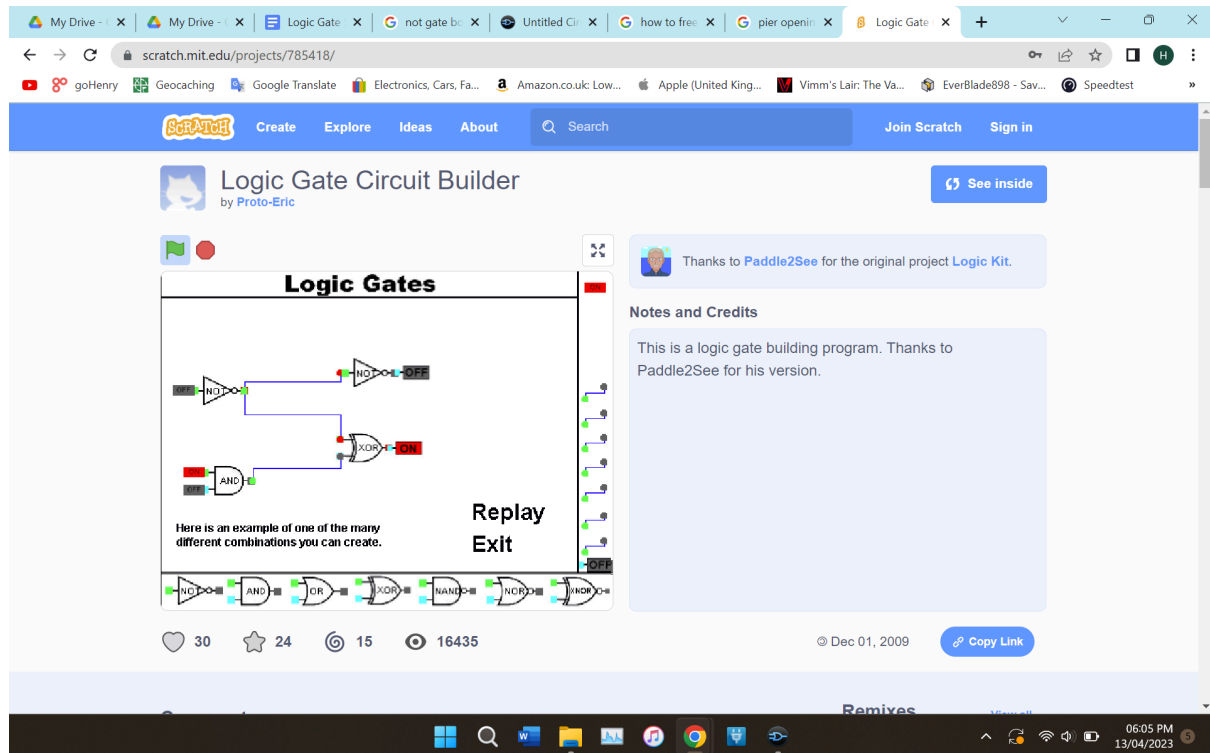
This is a simple half adder created with the software. As you can see, any area that passes a true signal is highlighted in blue, which is a feature I would like to include in my project to increase ease of use. The wires that link the components of the adder connect seamlessly to one another, and slot into place when they are passed over components, depending on complexity I would like to implement this feature into my project as well.

Logic gate Circuit Builder - Scratch



This example is built utilising the programming tool scratch and involves only logic gates. In the top left the help button leads the user into an audio guided tutorial demonstrating how the logic gates link together. Despite this the interface for adding wires, inputs and outputs wasn't very intuitive as the wires do not click into place. You also cannot delete parts of your work or clear your work without refreshing the entire page. The simulator also only has a limited number of wires.

Logic gate simulator - Harry Taylor - 7173 - 65301

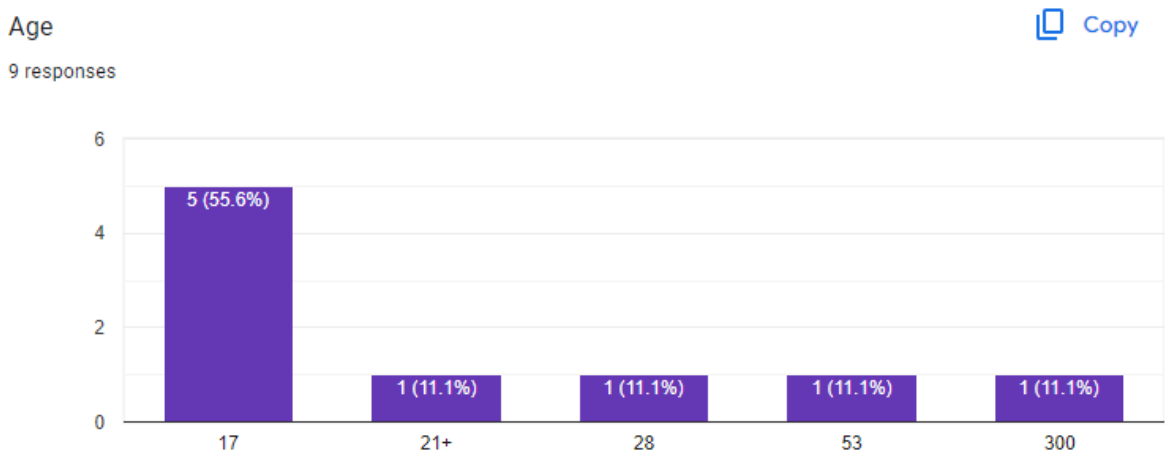


The example given above is the example given in the tutorial. As shown the green end of the wires have to be connected to the output of a gate or switch, whilst the red end has to be connected to the input of another gate or an output switch. The input and output switches are also overlapped over the logic gates. Although the input switches do not have nodes making the experience less intuitive.

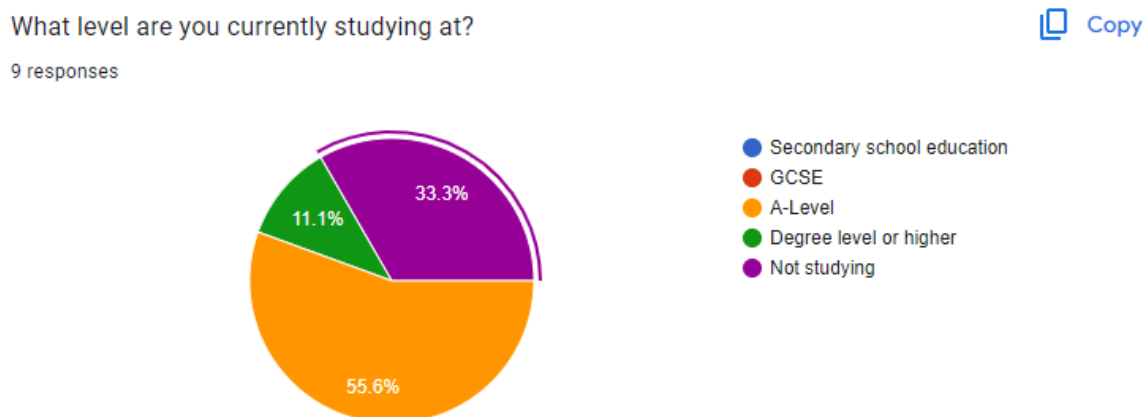
Potential Users

User information form

To gather information about my potential users and their wants from my project idea, I created a google form allowing potential users to contribute their ideas. This inspiration contributed to my finalised objective list. Using a form also allowed me to find users who were willing to test my project and select users to test my project based on their responses. Below are questions I requested of users and their responses:



As shown the majority of users who responded were of college age. Due to the similar demographic I had to choose between a variety of the most appropriate college age students as well as the limited number of adults that submitted a response. Although some adults were unwilling to submit their proper age.



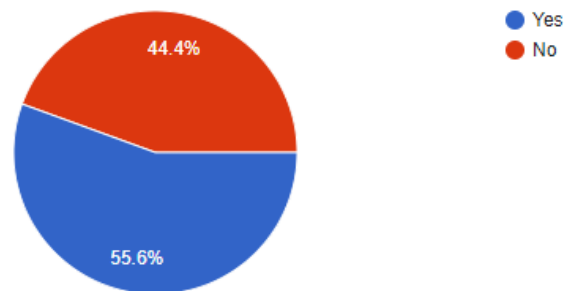
Partially indicated by the age submission, the majority of my potential users were studying at A-level. This meant that the large majority of these users would be at

least somewhat familiar with the use of logic gates, and would be able to determine the effectiveness of my tutorial system with their own knowledge.

Do you have a background in the computer science industry?

 Copy

9 responses



If yes, what is your background?

5 responses

Professional software engineer

Programmer / Systems analyst / Computing teacher

I am studying Computer Science

Computer Science student

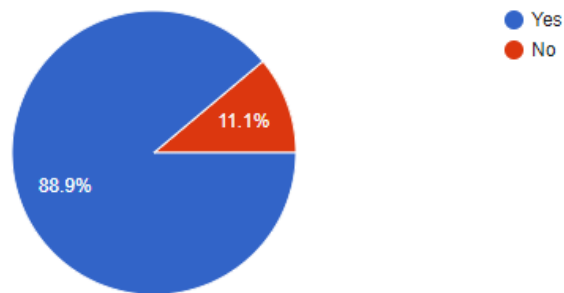
Worked as an Engineer and Teacher of Computer Science

To gain a better understanding of the types of people who would be potentially testing my project, I asked users if they had a background in the computer science industry, and if so what their background was. This allowed me to understand the careers of people who weren't studying and therefore gain an insight into how their expertise could best help my project.

Have you previously used any logic gate simulators e.g. logic.ly?

 Copy

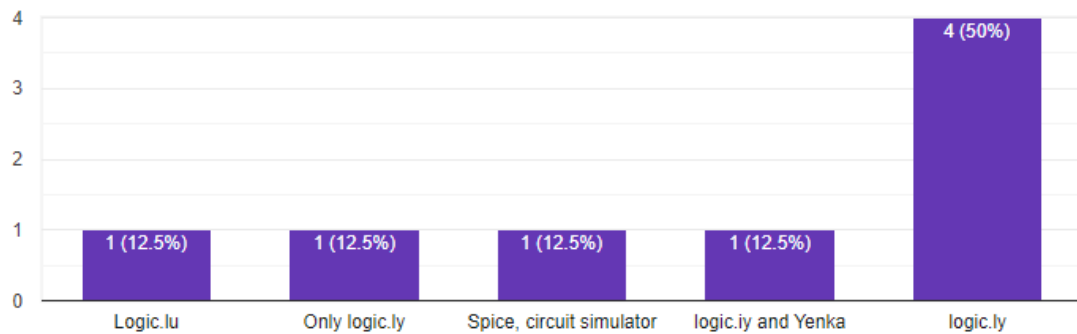
9 responses



If yes, which logic gate simulators have you used?

 Copy

8 responses



As shown nearly all my users have familiarity with logic gate simulators. In particular most users have used logic.ly, a system I have used as a pre existing system in my initial research. This means that lots of users will be familiar with how to operate my simulator, making them more able to give me constructive feedback.

What features do you like about the simulator(s) you have used?

8 responses

Smooth running, easy to use

Has all the common gates. Can generate truth tables

It was very visual, helping me memorise the symbols for my exams

Clear UI, Nice images to represent the gates and the fact it works smoothly

simple to use, with light bulb showing on/off

Ability to drag icons out , and when you drag a wire, it will highlight possible connections

Ease of connecting gates

Click and drag simplicity

Most users found that simplicity in dragging and dropping connections and gates helped them in the pre-existing systems they have used. Some users also specified that the functionality in pre existing systems functioned smoothly with one user noting that their system included truth table functionality, which was a feature I intended on including.

Is there any features lacking from these simulators that you would like to see implemented?

7 responses

Automated simplification of circuits

To be able to generate the boolean logic for a built circuit / be able to generate a circuit from inputted boolean logic

I would like an option to save my circuits

bit clunky to drag and drop at times and the colours clash at times making it hard to visualise

graphics of each gate is clearly different and allows multiple connections between them

Highlighting of possible connections

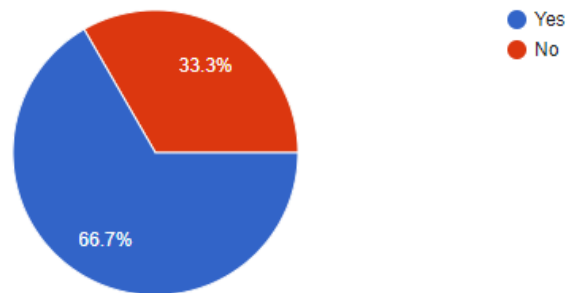
Ease and visual clarity of inputs and outputs

There were a variety of different improvements that users stated about pre existing systems that they had used, users stated that they would prefer the ability to save, and to not drag and drop items onto their circuit; noting that this produced a lack of visualisation for circuit designs. I planned on implementing saving into my project, as well as improving upon the stated logic element placement functionality.

Have you had any previous experience working with/using logic gates?

 Copy

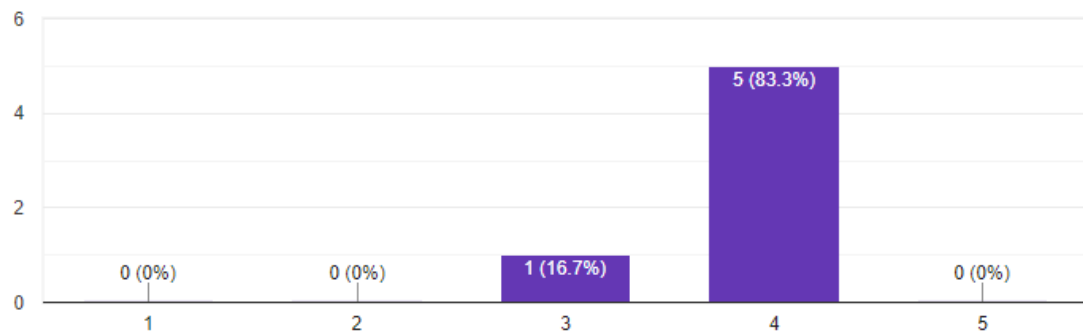
9 responses



If yes, how would you rate your experience?

 Copy

6 responses

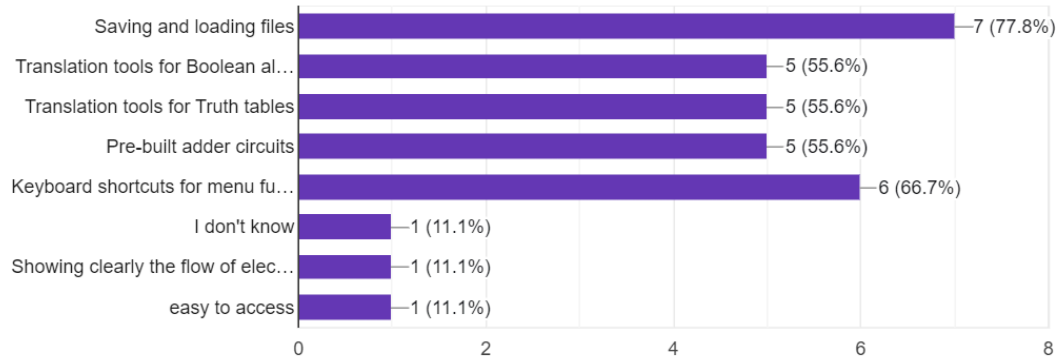


To better understand the experience of potential users, I asked them to state if they had experience in logic gates, and how they would rate their experience. This would give me a greater understanding of how well users would be able to use my project without assistance, whilst also understanding who does not have any experience and therefore require the assistance of a tutorial function.

Which of these features do you find necessary in a logic gate simulator?

 Copy

9 responses

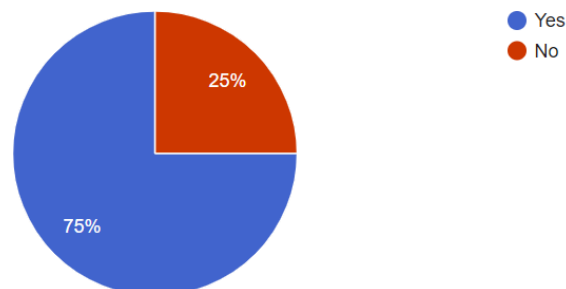


To best understand what users would definitely from a basic logic system, I asked them to state what they would require out of a variety of pre-stated options, whilst also allowing users to contribute their own ideas. Most users stated that saving a loading file and keyboard shortcuts were a necessity, with most users stating that translation tools and pre built adders would also be necessary.

Would you like to be a test user for the logic simulator software?

 Copy

8 responses



As a final question I asked all potential users if they would be willing to test my project, this way I could work out which users would be most appropriate out of users that were willing to test, whilst still gaining insight from others who didn't wish to test my project.

Test users

Below is a finalised list of users I intend to test my project based on responses to my previous user information form:

User 1: Leon Dobson

This person has a similar level of experience as myself regarding computer science and logic gates having completed the GCSE syllabus and currently attending the same A-level computer science course as myself, leaving them in the same position regarding their NEA.

User 2: Kai Oliver Page

Another A-level student with a knowledge of logic gates, whose experience extends through doing a level 2 IT BTEC as well as participating in the same A-level computer science course as myself,.I have allowed them to often view my project during development stages, giving them an advantage in knowing how my interface works.

User 3: Alan Taylor

This person has a limited knowledge of logic gate based systems. They may not fully understand logic gates and their functions, but I believe their feedback will be useful in gauging how well my tutorial system works in teaching new users how to utilise both my simulator interface and logic gates themselves.

User 4: Adrian Sefton

A secondary school teacher teaching GCSE computer science. Their knowledge of logic circuits is quite similar to that of user 2. This person taught me logic gates and therefore understands my thought process towards logic gates and by extension the NEA itself.

User 5: Joel Nicholl

Another A-level student with a complex working knowledge of logic gates, this student has also completed the GCSE syllabus and is currently completing their own NEA project in the same course as myself.

User 6: Alex Jones

A professional degree level programmer coding simulators for a career, giving me an opinion from a professional standpoint about my project from the same field as my project idea.

MOSCOW Chart

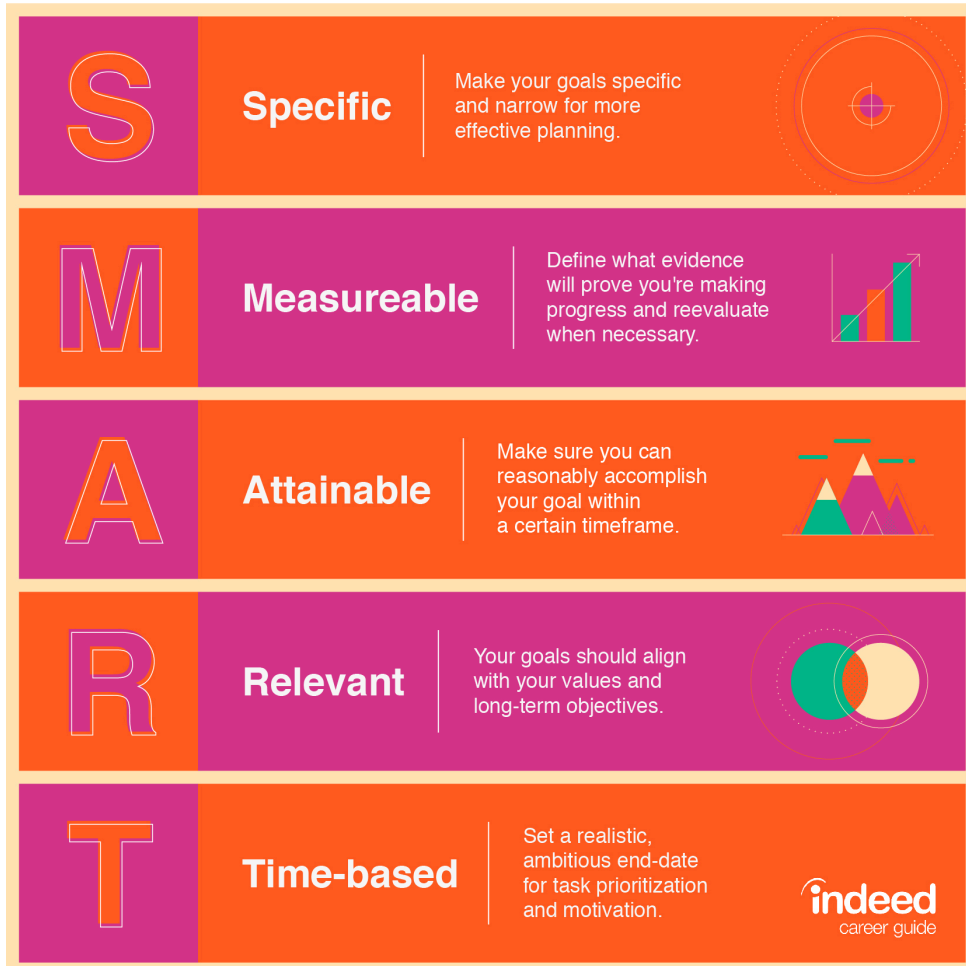
A MOSCOW chart is a list of features my project should include. Including features that must be included in order to make the simulator function; Features that should be implemented to add depth to the functionality of the simulator; Features that may be feasible to add although won't necessarily impact the overall function of the simulator; As well as features that should not be implemented due to complexity or lack of necessity.

Must	Should	Could	Won't
Contain all logic gate types from GCSE and A-Level	Allow users to swap out logic gates that have already been placed in the circuit	Contain a feature that allows auto saving of logic circuits	Allow sharing of files between multiple users
Drag and drop logic gates, input buttons and output lights onto a display	Contain a description for each logic gate providing a logic table for that specific gate	Allow users to select an area of their work they would like to replicate for use in another part of the circuit	
Contain at least a tutorial screen instructing users how to use the software	Contain an undo and redo function for ease of use	Contain a login system for users to access their individual files	
Connect wires to different inputs, gates and outputs	Allow a user to select the exam board and syllabus they are learning so that the logic gates that are part of their syllabus can be highlighted or those that are not can be hidden	Allow a user to save their circuit and reuse it at a later date	
Contain switches for linking to logic gates for input and	Allow the user to view a truth table of their work	Allow a user to change the zoom and menu size	

lights for outputs			
	Contain prebuilt adder functions	Translate boolean expressions to logic circuits and vice versa	
	Contain flip-flop gates		

Objectives List

After ranking the importance of each section of my project, I will implement the SMART requirement system standing for specific, measurable, attainable, relevant and time-based. SMART requirements are:



1. User opens simulation

1.1. User is given the option to open a new document, edit/view an existing one, open a pre-built adder, open the tutorial or exit the application

1.2. New document is opened

1.2.1. User can select to open a new blank document

1.3. Existing document is opened

1.3.1. User can select load document option

1.3.1.1. User can type in the name of an existing file and press enter to open that document

1.3.1.1.5. User is presented with the document they selected to edit/view

1.3.1.1.6. User is presented with an error message if file they specified is not found

1.3.1.2. If a user cannot find a file, they can back out of the menu back into the same menu as would be presented by opening the software

1.4. Adder menu is opened

1.4.1. User can select to open adder menu

1.4.1.1. User can select half-adder, doing so will open up a pre-built half adder circuit

1.4.1.2. User can select full-adder, doing so will open up a pre-built full adder circuit

1.4.1.3. User can select the back option, returning them to the initial menu

1.5. Tutorial menu is opened

1.5.1. User can select a tutorial option

1.5.1.1. User is shown how to build a NOT gate circuit

1.5.1.1.1. Users can test the NOT circuit once it is complete

1.5.1.2. User is shown how to build an AND gate circuit

1.5.1.2.1. Users can test the AND circuit once it is complete

1.5.1.3. User is shown how to replace the AND gate with another gate of their choosing

1.5.1.3.1. Users can change the and gate in the circuit with any of the logic gates in the menu

1.5.1.3.2. Users can test their circuit at any point

1.5.1.3.3. Users can press next at any time to return to the main menu

1.5.2. User can press back at any time to return to main menu

1.6. Exit button pressed

1.6.1. Window closes

2. Options menu

2.1. Undo

2.1.1. Most recent logic item placed is removed from the document

2.1.2. If there is no items in the file the option is greyed out

2.2. Redo

2.1.1. Item most recently undone is added to the document

2.1.2. If there is no item to be restored the option is greyed out

2.3. Save

2.3.1. Document being saved overwrites the current data stored for the file with the current file state

2.3.2. if a save file does not exist one can be named and created

2.4. Save as

2.4.1. Save as menu opens and a new save file can be named and created

2.4.1.1. Selecting enter with user text input creates a new save document with the users assigned name

2.4.1.2. Enter button is greyed out when there is no text input present

2.4.1.3. Back button returns user to the logic interface with the document they are currently working on

2.5. Load

2.5.1. Load menu opens and an existing document can be loaded

2.5.1.1. Selecting enter with valid user text input loads the specified document

2.5.1.2. Selecting enter with invalid user text input shows an error message

2.5.1.3. Enter button is greyed out when there is no text input present

2.5.1.4. Back button returns user to the logic interface with the document they are currently working on

2.6. Clear

2.6.1. Current logic document is cleared

2.7. Translate between logic circuits and boolean algebra

2.7.1. Existing logic circuit can be translated to boolean logic

2.7.2. If there are no items in the logic circuit option is greyed out

2.8. Delete logic element

2.8.1. When the delete button is pressed the button is highlighted

2.8.1.1. Clicking on a logic element causes it to be deleted

2.8.2. When the delete button is pressed whilst highlighted, the button returns to its original state

2.9. Truth table

2.9.1. A truth table is generated for the logic circuit within the open document

2.9.2. If a completed logic circuit is not present option is greyed out

2.10. New

2.10.1. When the new button is pressed logic document is cleared and file name is reset

2.11. Main menu

2.11.1. User is returned to main menu

2.12. Auto Save

2.12.1. If the file has been previously saved user can turn auto save on

2.12.1.1. Auto save saves the file every 30 seconds

3. Logic interface

3.1. Inputs

3.1.1. User can place switches onto their document

3.1.2. User can place clocks onto their document

3.2. Wires

3.2.1. Wires can be placed onto the logic circuit

3.2.2. If there are wires above/below each other, both turn into vertical wires

3.2.3. If there are wires above and to the left of the wire, wire turns into a bottom right corner wire

3.2.3.1. If the wire to the left is deleted, wire corner turns to a vertical wire

3.2.3.2. If the wire above is deleted, wire corner turns to a horizontal wire

3.2.4. If there are wires above and to the right of the wire, wire turns into a bottom left corner wire

3.2.4.1. If the wire to the right is deleted, wire corner turns to a vertical wire

3.2.4.2. If the wire to above is deleted, wire corner turns to a horizontal wire

3.2.5. If there are wires below and to the left of the wire, wire turns into a top right corner wire

3.2.5.1. If the wire to the left is deleted, wire corner turns to a vertical wire

3.2.5.2. If the wire below is deleted, wire corner turns to a horizontal wire

3.2.6. If there are wires below and to the right of the wire, wire turns into a top left corner wire

3.2.6.1. If the wire to the right is deleted, wire corner turns into a vertical wire

3.2.6.2. If the wire below is deleted, wire corner turns into a horizontal wire

3.2.7. If there are wires to the right, left and below the wire, wire turns into a wire splitter facing down

3.2.7.1. If the wire below is deleted, wire splitter turns into a horizontal wire

3.2.7.2. If the wire to the right is deleted, wire splitter turns into a top right corner wire

3.2.7.3. If the wire to the left is deleted, wire splitter turns into a top left corner wire

3.2.8. If there are wires to the right, left and above the wire, wire turns into a wire splitter facing up

3.2.8.1. If the wire above is deleted, wire splitter turns into a horizontal wire

3.2.8.2. If the wire to the right is deleted, wire splitter turns into a bottom right corner wire

3.2.8.3. If the wire to the left is deleted, wire splitter turns into a bottom left corner wire

3.2.9. If there are wires above, below and to the right of the wire, wire turns into a wire splitter facing right

3.2.9.1. If the wire to the right is deleted, wire splitter turns into a vertical wire

3.2.9.2. If the wire above is deleted, wire splitter turns into a top left wire

3.2.9.3. If the wire below is deleted, wire splitter turns into a bottom left wire

3.2.10. If there are wires above below and to the left of the wire, wire turns into a wire splitter facing left

3.2.10.1. If the wire to the left is deleted, wire splitter turns into a vertical wire

3.2.10.2. If the wire above is deleted, wire splitter turns into a bottom right wire

3.2.10.3. If the wire below is deleted, wire splitter turns into a top right wire

3.2.11. If there are wires above, below, to the left and to the right of the wire, wire turns into a wire cross

3.2.11.1. If the wire above is deleted, wire cross turns into a wire splitter down

3.2.11.2. If the wire below is deleted, wire cross turns into a wire splitter facing up

3.2.11.3. If the wire to the left is deleted, wire cross turns into a wire splitter facing right

3.2.11.4. If the wire to the right is deleted, wire cross turns into a wire splitter facing left

3.3. Outputs

3.3.1. User can place light bulbs onto their document

3.4. Logic gates

3.4.1. User can place OR gates onto their document

3.4.2. User can place AND gates onto their document

3.4.3. User can place NOT gates onto their document

3.4.4. User can place NOR gates onto their document

3.4.5. User can place NAND gates onto their document

3.4.6. User can place XOR gates onto their document

3.4.7. User can place XNOR gates onto their document

3.4.8. Clicking on any logic gate twice will reset the active gate

3.5. Flip-flop gates

3.5.1. User can place an edge-triggered D type flip flop gate onto their document

4. Logic gate interactions and shortcuts

4.1. Inputs

4.1.1. Switch changes state from on to off or vice versa

4.1.1.1. Wires connected to the switch change colour when button is pressed

4.1.2. Clock

4.1.2.1. Clocks change state based on a pulse

4.2. Logic gate interactions

4.2.1. AND gate

4.2.1.1. An AND operation is performed based on the states of the wire intersection to the left, this affects the state of the wires connected on the right

4.2.2. OR gate

4.2.2.1. An OR operation is performed based on the states of the wire intersection to the left, this affects the state of the wires connected on the right

4.2.3. NOT gate

4.2.3.1. A NOT operation is performed based on the state of the wire to the left, this affects the state of the wires connected on the right

4.2.4. NAND Gate

4.2.4.1. A NAND operation is performed based on the states of the wire intersection to the left, this affects the state of the wires connected on the right

4.2.5. NOR Gate

4.2.5.1. A NOR operation is performed based on the states of the wire intersection to the left, this affects the state of the wires connected on the right

4.2.6. XNOR Gate

4.2.6.1. A XNOR operation is performed based on the states of the wire intersection to the left, this affects the state of the wires connected on the right

4.2.7. XOR Gate

4.2.7.1. A XOR operation is performed based on the states of the wire intersection to the left, this affects the state of the wires connected on the right

4.3. Flip-flops

4.3.1. Edge triggered D-type flip flop

4.3.1.1. A Edge triggered D-type flip flop operation is performed based on the states of the wire intersection to the left, this affects the state of the wires connected on the right

4.4. Light bulb

4.4.1. The state of the wire to the left of a light switch causes it to turn on and off

Time management

Starting from April 2023 I will have a year to complete this project. Due to this being an extended project, I have created this section to better manage my time to eliminate procrastination and ensure better quality work.

Project	Status	Deadline	Notes
Work on research and analysis section	Completed ▾	18 Apr 2023	Set as work for easter break
Complete research and analysis section	Completed ▾	4 May 2023	
Complete formal project proposal	Completed ▾	14 May 2023	
Analysis: Submit core design deliverables	Completed ▾	4 Jun 2023	
Complete first draft of initialisation UI code	Completed ▾	20 Jun 2023	
Complete test plan	Completed ▾	20 Jun 2023	
Complete user survey	Completed ▾	20 Jun 2023	
Complete first draft of circuit UI code	Completed ▾	30 Jul 2023	
Design: Submission of core design deliverables	Completed ▾	July-August	
Complete main logic for circuit UI	Completed ▾	9 Nov 2023	
Complete undo/redo functionality	Completed ▾	13 Nov 2023	
Complete save/load functionality	Completed ▾	11 Dec 2023	
Complete main menu code	Completed ▾	28 Dec 2023	

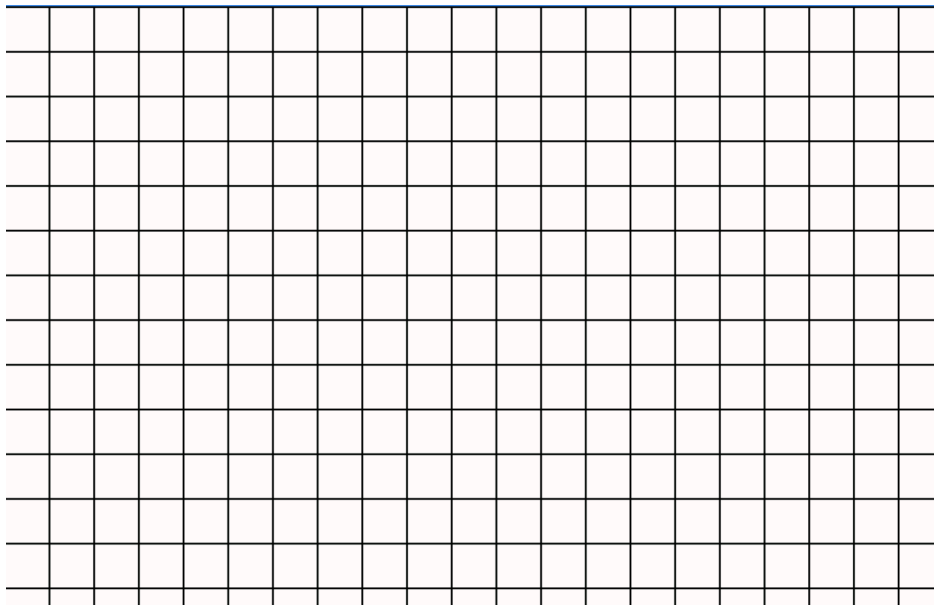
Project	Status	Deadline	Notes
Complete wire code	Completed ▾	15 Jan 2024	
Complete truth table functionality	Completed ▾	31 Jan 2024	
Complete adder section	Completed ▾	14 Feb 2024	
Complete tutorial section	Completed ▾	14 Feb 2024	
Complete test plan	Completed ▾	18 Feb 2024	
Complete boolean algebra translation	Not Compl... ▾	21 Feb 2024	
Complete project report	In progress ▾	29 Feb 2024	
Final draft report submission	Completed ▾	12 Feb 2024	
Final testing and user acceptance survey	Completed ▾	6 Mar 2024	
Demonstration of final build	Completed ▾	15 Dec 2023	
Project completion	Completed ▾	8 Apr 2024	

Documented Design

Data structures

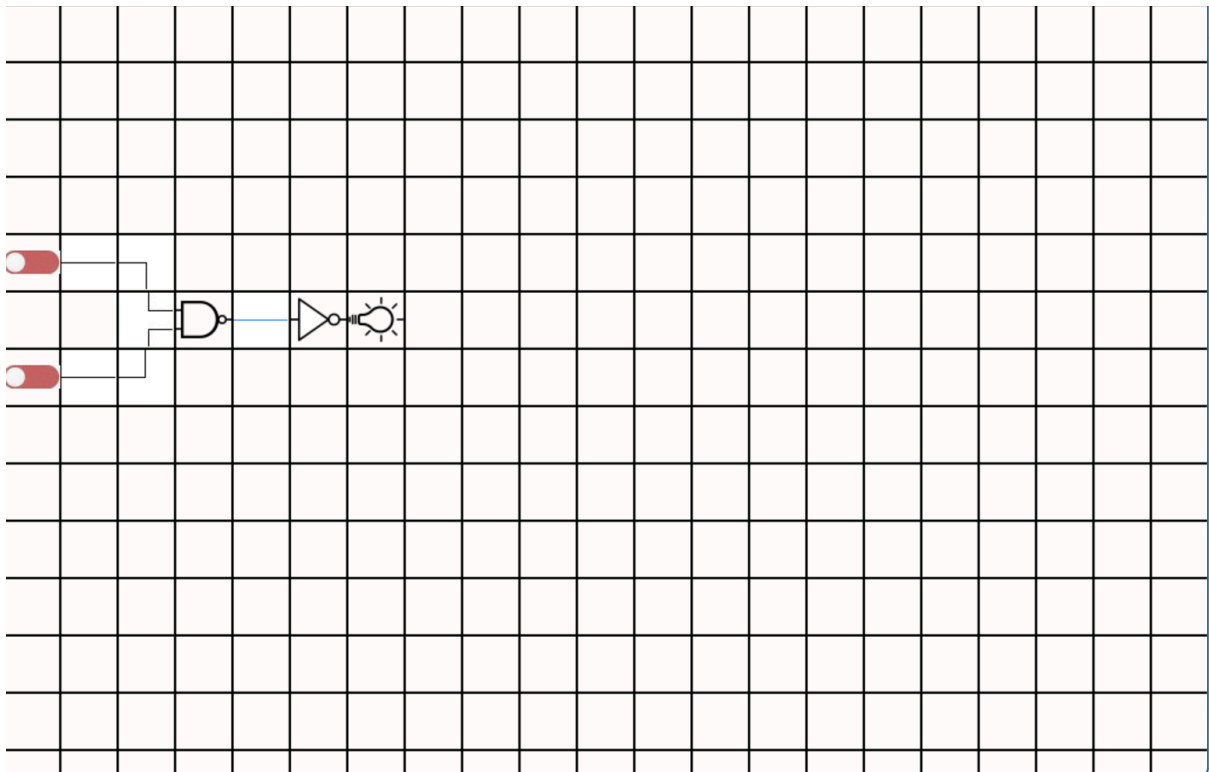
Logic grid

In order to hold all data necessary for logic systems created in my simulator I chose to utilise a 3D array holding a position for every visual square in a grid corresponding to a logic document. When the array is created, the amount of squares required by the visual grid are made available as null values in a 3D array. When a user places a logic element into the document, a one dimensional array takes the place of the null value corresponding to the square clicked.

[illegible]

Above is an example of the contents of the 3D array for the empty grid shown with 21 spaces across and 13 spaces down.

[illegible]



Above is an example of the contents of the 3D array with a logic circuit for the same grid size of 21 across and 13 down.

Logic elements

As mentioned above, each logic element is stored in a one dimensional array, this is done so that each square can hold data relating to the logic element being stored. The elements in the array hold data for the specific logic element type, state of the logic element's output, as well as in some cases, the direction of power, when the element in question is a wire. The exact format of the array elements are contextual to the logic element being represented.

```
['ANDGate', False]
```

Above is an example of an array linking to an AND gate, showing the logic element type and the state output.

```
['wireHorizontal', False, None]
```

Above is an example of an array linking to an horizontal wire, showing the logic element type, state output as well as the direction of power, which maintains a null value when the wires state is false.

Undo and redo elements

To functionally undo and redo elements in the document, I chose to utilise a stack data structure for each function in the form of a 2D array. The undo and redo array once populated hold data relating to the removal or addition of logic elements to the main array, including the logic element in question, as well as the coordinate position of the element being added or removed.

During the course of editing a logic document, every placement and removal of logic elements is pushed onto the undo stack. When the undo option is selected by the user, the top of the undo stack is popped, then the command of the undo statement array is executed, this array is then pushed onto the redo stack. This process happens vice versa for the redo function.

```
[['add', 'wireHorizontal', 6, 1], ['add', 'wireHorizontal', 4, 1], ['add', 'wireHorizontal', 5, 4]]
```

Above is an example of a populated undo stack, each array within the stack represents an element that can be undone. Including the change the undo function will cause to the main logic array, the logic item that is to be added or removed as well as the Y and X coordinates of the element to be added or removed.

Truth tables

For a truth table to be created, using a combination of subroutines a 3D array is generated which is used by the simulator to create a visible logic grid. The 3D array holds information relating to the inputs and outputs, with the first array in a 2D array being the inputs and the latter being the outputs. The states of the inputs and outputs are then appended into either list as arrays themselves, where each input or output in a particular state represents an item in the array, adding an extra dimension to the array. Below are some examples of the truth table array:

```
[[[]], []]
```

As shown, the first array within the truth table would be where combinations of inputs determined by the truth table function are appended. With the second array being for outputs generated.

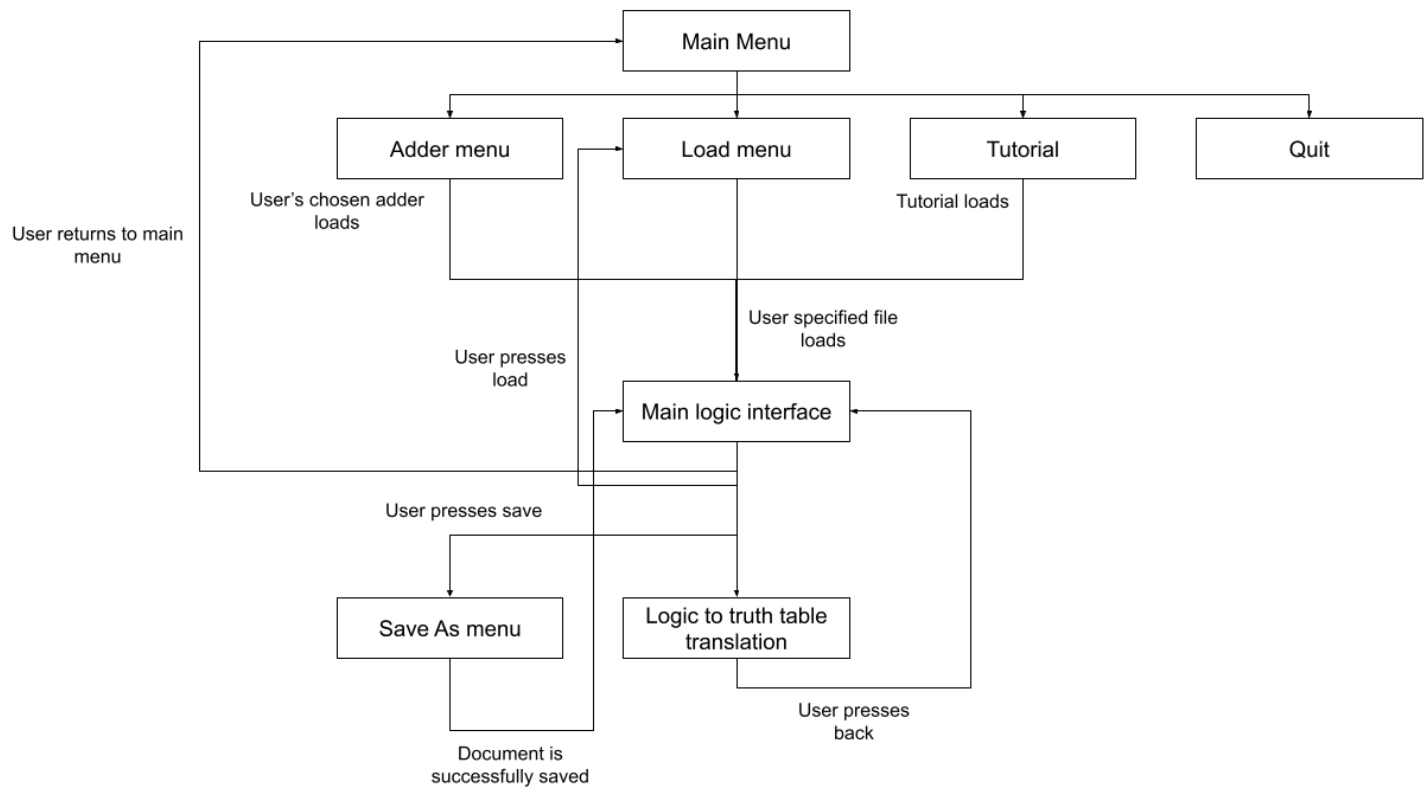
```
[[[0, 0], [0, 1], [1, 0], [1, 1]], [[0, 0], [1, 0], [1, 0], [0, 1]]]
```

Inputs		Outputs	
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Above is an example of a full truth table array for a half adder, alongside the produced truth table grid.

Structure diagram

To better understand the links between different menus and the ways in which they interact with one another, I created a structure diagram. This details all menus within the simulator and how they can be opened and from what menus:



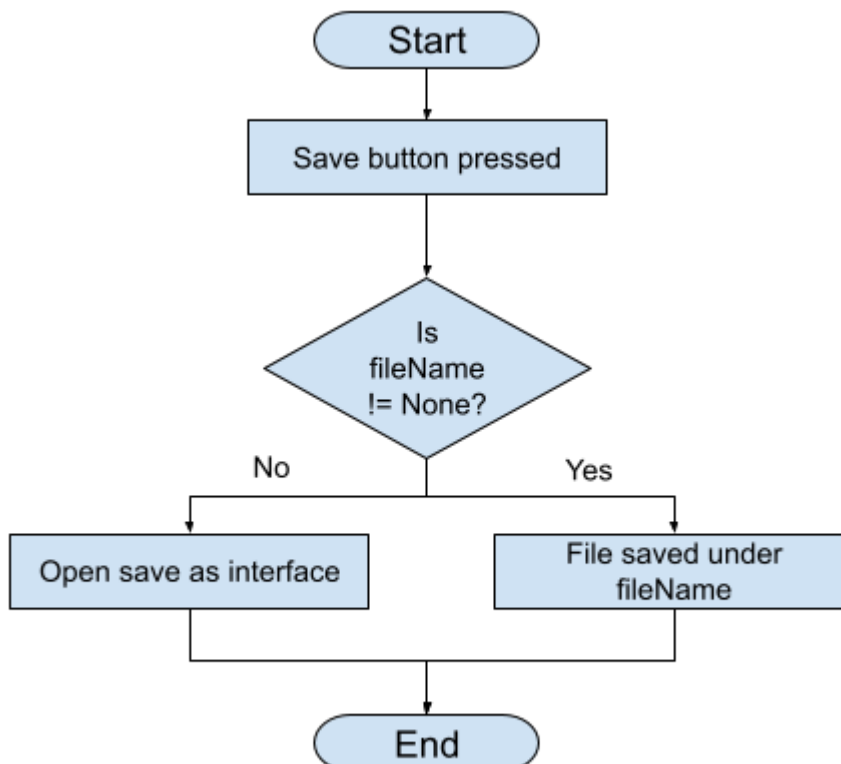
Key algorithms

Menu subroutines

In order to effectively change the GUI output as well as the exact algorithms running based on user choice, I chose to create individual subroutines for each menu within my simulator. This way my code can be better structured and easier for the user to navigate. Below are the menus I plan on integrating:

Save and Save As

In order to create new save files using a save as function, I used an algorithm to receive user keyboard input and display it. This is so the user can visually confirm it as their file name. The user can then confirm this using a button causing the file to be made and a confirmation message to appear. If the user presses the save button, the program checks if the file has been previously saved, if it has the file is resaved over that name, overwriting the files original contents. Otherwise it opens the save as menu. Below is a flowchart of how the save algorithm works as well as code for the respective save algorithm:



#user presses save button in options menu

if logicInterfaceSave.checkForInput(logicInterface_MOUSE_POS):

#file is a new document, pressing save directs the user to the save as menu

if fileName == None:

saveAsInterface(logicArray, fileName)

#file is a previously saved document, pressing save overwrites file contents

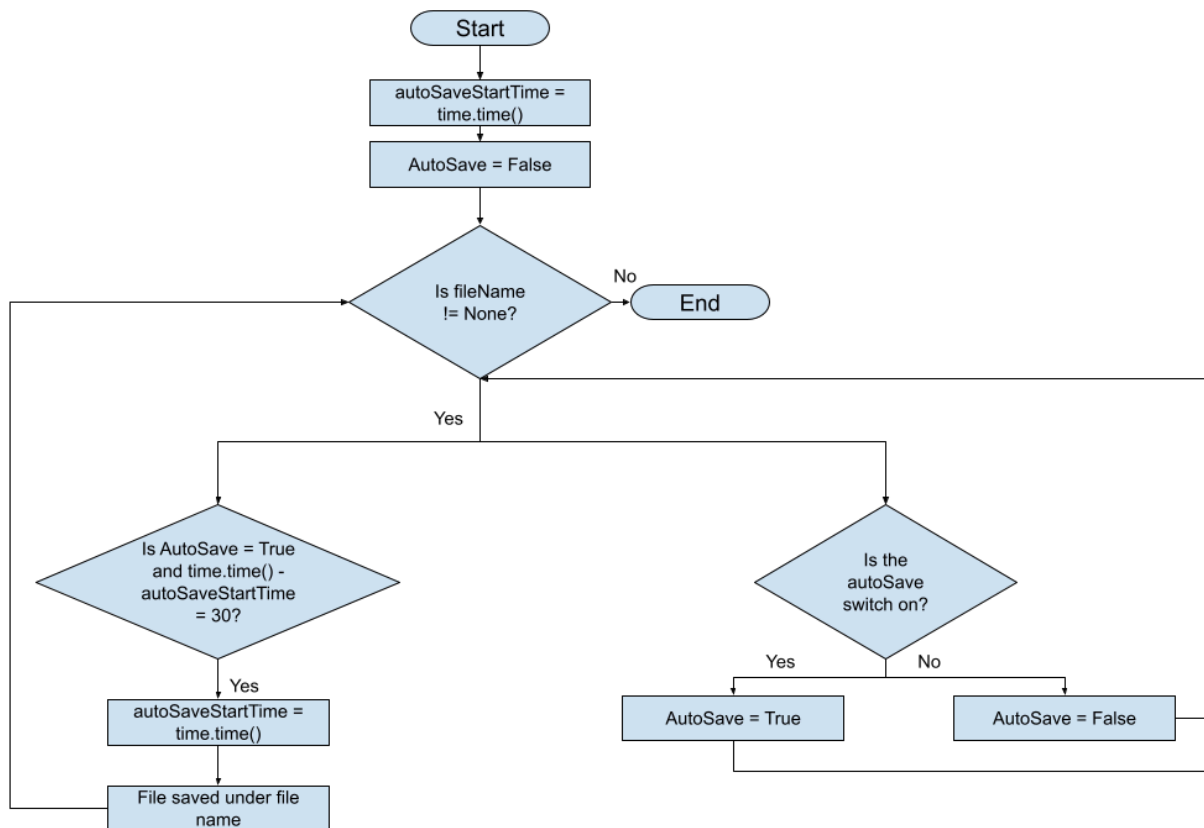
else:

logicFile = open(fileName, "wb")

pickle.dump(logicArray, logicFile)

logicFile.close()

For the user to save their file automatically I utilised python's time function to save files that have already been saved previously constantly after a timeframe of 30 seconds has passed. The user can enable and disable this feature via the options menu, the feature only works if the file has been previously saved and exists in the users file directory. Below is a flowchart for how this system works as well as the code for the related algorithm:



Auto save display

```
#if autosave is off, switch is off
    if autoSave == False:
        autoSaveSwitch = pygame.image.load("off_switch.jpg")
    #if autosave is on, switch is on
    else:
        autoSaveSwitch = pygame.image.load("on_switch.jpg")
    autoSaveSwitchRect = autoSaveSwitch.get_rect()
    autoSaveSwitchRect = (1050,30)
    #auto save unavailable
    if fileName == None or tutorialMode == True:
        autoSaveText = get_font3(15).render("Auto Save", True, GREY)
    #auto save off
    elif autoSave == False:
        autoSaveText = get_font3(15).render("Auto Save", True, BLACK)
    #auto save on
    elif autoSave == True:
        autoSaveText = get_font3(15).render("Auto Save", True, GREEN)
    autoSaveTextRect = autoSaveText.get_rect()
    autoSaveTextRect.center = (1150,42)
```

Auto save switch logic

```
elif mouseX >= 1050 and mouseX <= 1100 and mouseY >= 30 and mouseY <= 50:
    #toggle on/off for auto save, only will turn on if filename is not
    none
    if autoSave == False and fileName != None:
        autoSave = True
    else:
        autoSave = False
```

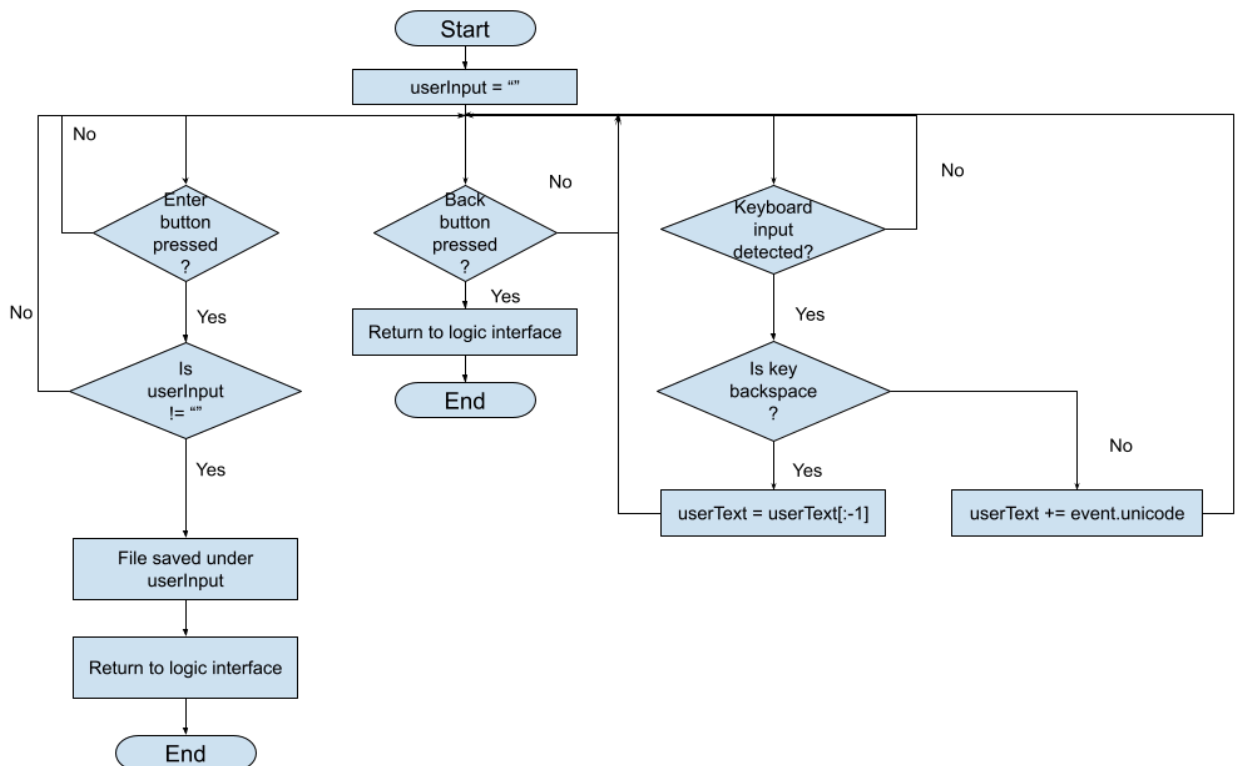
Auto save logic

```
if autoSave == True:
    # if time passed since last save is 30 seconds
    if time.time() - autoSaveStartTime >= 30:
        print("Auto Saving")
        #time reset
        autoSaveStartTime = time.time()
        logicFile = open(fileName, "wb")
        #file saved
        pickle.dump(logicArray, logicFile)
        logicFile.close()
```


For the user to have a save as function for saving new documents I needed to design an interface that would allow users to input a name stored as a string, and have the pickle python module dump the logic array into the programs directory folder under the assigned name. The menu takes both button input as well as input from the user's keyboard as unicode characters. When the user selects the enter button in the menu, the file is dumped under the user's name if they have inputted one.

When a saved document is created using the aforementioned save as menu, a file name variable is set as the name of the file; this variable is also set as the file name of a loaded document. When the user presses the save button instead of the save as button in the options menu, the program dumps the contents of the logic array under the file name specified by the user, overwriting its original contents, unless the user has not previously saved the file in which case they are directed to the save as menu.

Below is the flowchart for the save as function as well as the algorithms for the input statements within the main logic interface, as well as the save as graphical menu:



Input statement

#user presses save as button in options menu

```
if logicInterfaceSaveAs.checkForInput(logicInterface_MOUSE_POS):
    saveAsInterface(logicArray, fileName)
```

Save as menu

#logicArray and fileName parameters are used to return the user to their document once they have saved or go back

```
def saveAsInterface(logicArray, fileName):
    #font for user text
    baseFont = get_font3(32)
    userText = ""
    #text input is shown to user in a rectangle
    inputRect = pygame.Rect(390,315,200,40)
    colour = pygame.Color(WHITE)
    while True:
        saveAs_MOUSE_POS = pygame.mouse.get_pos()
        SCREEN.fill(BLACK)
        saveFile_TEXT = get_font2(45).render("Enter New File Name", True,
        WHITE)
        saveFile_RECT = saveFile_TEXT.get_rect(center=(640, 260))
        SCREEN.blit(saveFile_TEXT, saveFile_RECT)
        saveFileBack = Button(pos=(100, 675), text_input="BACK",
        font=get_font2(35), base_color=WHITE, hovering_color=GREY)
        #enter button only works if the user has inputted a name
        if userText != "":
            saveFileEnter = Button(pos=(640, 420), text_input="ENTER",
            font=get_font2(35), base_color=WHITE, hovering_color=GREY)
        else:
            saveFileEnter = Button(pos=(640, 420), text_input="ENTER",
            font=get_font2(35), base_color=GREY, hovering_color=GREY)
        #user text is placed onto pygame window
        textSurface = baseFont.render(userText,True,(255,255,255))
        SCREEN.blit(textSurface, (inputRect.x + 5, inputRect.y - 5))
        pygame.draw.rect(SCREEN,colour,inputRect,2)
        #rectangle changes with the length of the text
        inputRect.w = textSurface.get_width()+10

        for button in [saveFileBack, saveFileEnter]:
            button.changeColor(saveAs_MOUSE_POS)
            button.update(SCREEN)

    for event in pygame.event.get():
```

```
if event.type == pygame.QUIT:
    pygame.quit()
    sys.exit()
if event.type == pygame.MOUSEBUTTONDOWN:
    #pressing back returns the user to the logic interface in
    the state it was before
    if saveFileBack.checkForInput(saveAs_MOUSE_POS):
        logicInterface(logicArray, fileName, False)
    if saveFileEnter.checkForInput(saveAs_MOUSE_POS):
        #enter button only works if the user has inputted a
        name
        if userText != "":
            logicFile = open(userText, 'wb')
            pickle.dump(logicArray, logicFile)
            logicFile.close()
            print("Save Successful")
            #save successful prompt displayed
            saveText = get_font2(35).render("Save
            Successful", True, GREEN)
            saveTextRect = saveText.get_rect()
            saveTextRect.center = (640,380)
            SCREEN.blit(saveText, saveTextRect)
            pygame.display.update()
            #prompt is displayed for one second
            time.sleep(1)
            #returns user to logic document
            logicInterface(logicArray, userText, False)
if event.type == pygame.KEYDOWN:
    #pressing backspace removes a letter from the text
    if event.key == pygame.K_BACKSPACE:
        userText = userText[:-1]
    #pressing enter attempts a save
    elif event.key == pygame.K_RETURN:
        if userText != "":
            logicFile = open(userText, 'wb')
            pickle.dump(logicArray, logicFile)
            logicFile.close()
            print("Save Successful")
            saveText = get_font2(35).render("Save
Successful", True, GREEN)
            saveTextRect = saveText.get_rect()
            saveTextRect.center = (640,380)
            SCREEN.blit(saveText, saveTextRect)
            pygame.display.update()
```

```
time.sleep(1)
logicInterface(logicArray, userText, False)
```

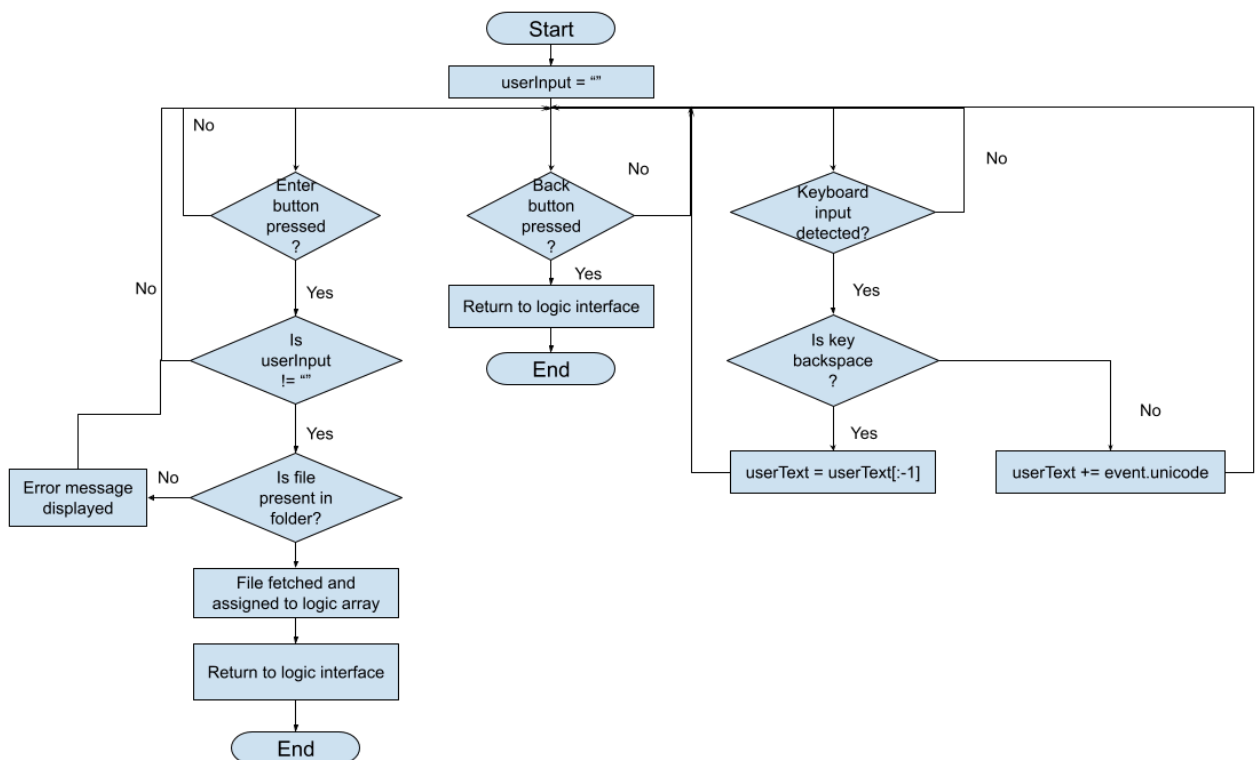
```
#pressing any other key adds it to the text input
else:
```

```
userText += event.unicode
```

```
pygame.display.update()
```

Load file

So that the user can load existing files, an algorithm similar but opposite in function to the save as function is used. User input is taken and displayed in the same way, however when the confirmation button is pressed the user text input is compared to files in persistent storage. If a file is found of the same name as the user inputted, the file is fetched and opened by the simulator. Below is a flowchart of how this algorithm works:



In a similar but opposite fashion to the save and save as functions, I used pickle python to open files and assign the contents of the file to the logic array, allowing users to access files they had previously saved. Similar to the functionality of the save as menu, text input is accepted and displayed. This text input however is used to load a file from the same directory folder as the program itself; if the file does not exist, exception handling is used to produce an error message that is displayed to the user, prompting them to reenter their file name. Since the user can also access the load menu from the main menu as well as the logic interface, I added in an extra boolean parameter specifying whether or not to return to the main menu when the user presses the back button. Below is the code for my load menu:

```
#parameters used to determine the menu to return to, as well the contents of the
logic interface, if the user is accessing the menu from there
def loadInterface(logicArray, fileName, menuBack):
    #font for user text
    baseFont = get_font3(32)
    userText = ""
    #text input is shown in a rectangle
    inputRect = pygame.Rect(390,315,200,40)
    colour = pygame.Color(WHITE)
    while True:
        load_MOUSE_POS = pygame.mouse.get_pos()
        SCREEN.fill(BLACK)
        loadFile_TEXT = get_font2(45).render("Enter file name to load", True,
        WHITE)
        loadFile_RECT = loadFile_TEXT.get_rect(center=(640, 260))
        SCREEN.blit(loadFile_TEXT, loadFile_RECT)
        loadFileBack = Button(pos=(100, 675), text_input="BACK",
        font=get_font2(35), base_color=WHITE, hovering_color=GREY)
        #enter button only works if the user has inputted a name
        if userText != "":
            loadFileEnter = Button(pos=(640, 420), text_input="ENTER",
            font=get_font2(35), base_color=WHITE, hovering_color=GREY)
        else:
            loadFileEnter = Button(pos=(640, 420), text_input="ENTER",
            font=get_font2(35), base_color=GREY, hovering_color=GREY)
        #user text is placed into pygame window
        textSurface = baseFont.render(userText,True,(255,255,255))
        SCREEN.blit(textSurface, (inputRect.x + 5, inputRect.y - 5))
        pygame.draw.rect(SCREEN,colour,inputRect,2)
        #rectangle changes with the length of the text
        inputRect.w = textSurface.get_width()+10

        for button in [loadFileBack, loadFileEnter]:
```

```
button.changeColor(load_MOUSE_POS)
button.update(SCREEN)

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.MOUSEBUTTONDOWN:

        if loadFileBack.checkForInput(load_MOUSE_POS):
            #if the user accessed the load interface from the
main menu they are returned to the main menu
            if menuBack == True:
                main_menu()
            #otherwise the user is returned to their logic circuit
in the state it was previously
        else:
            logicInterface(logicArray,fileName, False)
    if loadFileEnter.checkForInput(load_MOUSE_POS):
        print(logicArray)
        #exception handling used to check if file exists
        try:
            logicFile = open(userText, 'rb')
        Except:
            loadFailText = get_font2(35).render("File Not
Found", True, RED)
            loadFailTextRect = loadFailText.get_rect()
            loadFailTextRect.center = (640,380)
            SCREEN.blit(loadFailText, loadFailTextRect)
            pygame.display.update()
            time.sleep(1)
        else:
            logicArray = pickle.load(logicFile)
            logicFile.close()
            logicInterface(logicArray,userText, False)
    if event.type == pygame.KEYDOWN:
        #pressing backspace removes a letter from the text
        if event.key == pygame.K_BACKSPACE:
            userText = userText[:-1]
        #pressing enter attempts to load a file
        elif event.key == pygame.K_RETURN:
            try:
                logicFile = open(userText, 'rb')
            except:
```

```
        loadFailText = get_font2(35).render("File Not
Found", True, RED)
        loadFailTextRect = loadFailText.get_rect()
        loadFailTextRect.center = (640,380)
        SCREEN.blit(loadFailText, loadFailTextRect)
        pygame.display.update()
        time.sleep(1)
    else:
        logicArray = pickle.load(logicFile)
        logicFile.close()
        logicInterface(logicArray,userText, False)
        #pressing any other key adds it to text input
    else:
        userText += event.unicode

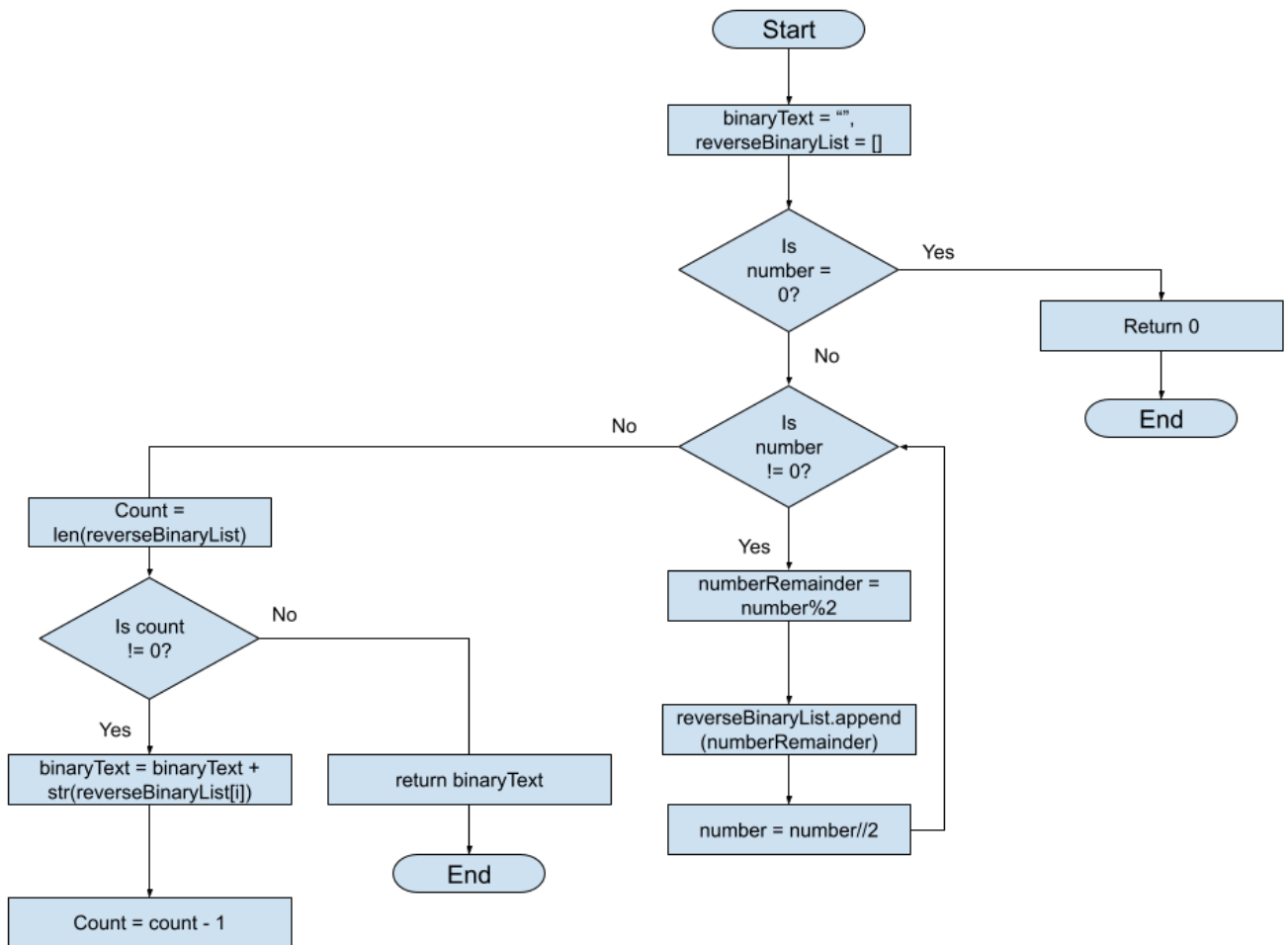
pygame.display.update()
```

Truth table creation

For truth tables to be generated based on the user's created logic circuit I had to locate the number of switches (inputs) and light switches (outputs) and their position within the logic array, as well as utilise a 3D array for the truth table, to differentiate the inputs and outputs. A number of switch combinations are created and each number below that number, including zero, is converted to binary using a separate program, converted to a list, then appended to the truth table array. These binary combinations are then iterated through and put into a check output subroutine. The check output subroutine utilises the locations of the inputs and outputs to determine how changing the state of the switches changes the output of the light bulbs in the circuit. The value of the outputs are then converted to a list and appended onto the truth table array.

The full truth table is then passed into the truth table menu, which draws a grid based on the amount of inputs, outputs and combinations of switch configurations. The outputs are differentiated from the inputs by different coloured text as well as headers above the grid itself. The window size is also increased vertically as I found during the development process that larger logic circuits would not be fully displayed by the menu. Below is the code utilised for the binary conversion subroutine, the check output subroutine, the translation display subroutine, the main logic as well as flow charts explaining each of these processes:

Binary converter



#number is passed in from main logic

```
def binaryConvert(number):
```

```
    binaryText = ""
```

```
    reverseBinaryList = []
```

#denary number zero produces zero in binary, so zero is returned

```
    if number == 0:
```

```
        return 0
```

#number is continually divided by two to generate binary bits

```
    while number != 0:
```

```
        numberRemainder = number%2
```

```
        reverseBinaryList.append(numberRemainder)
```

```
        number = number//2
```

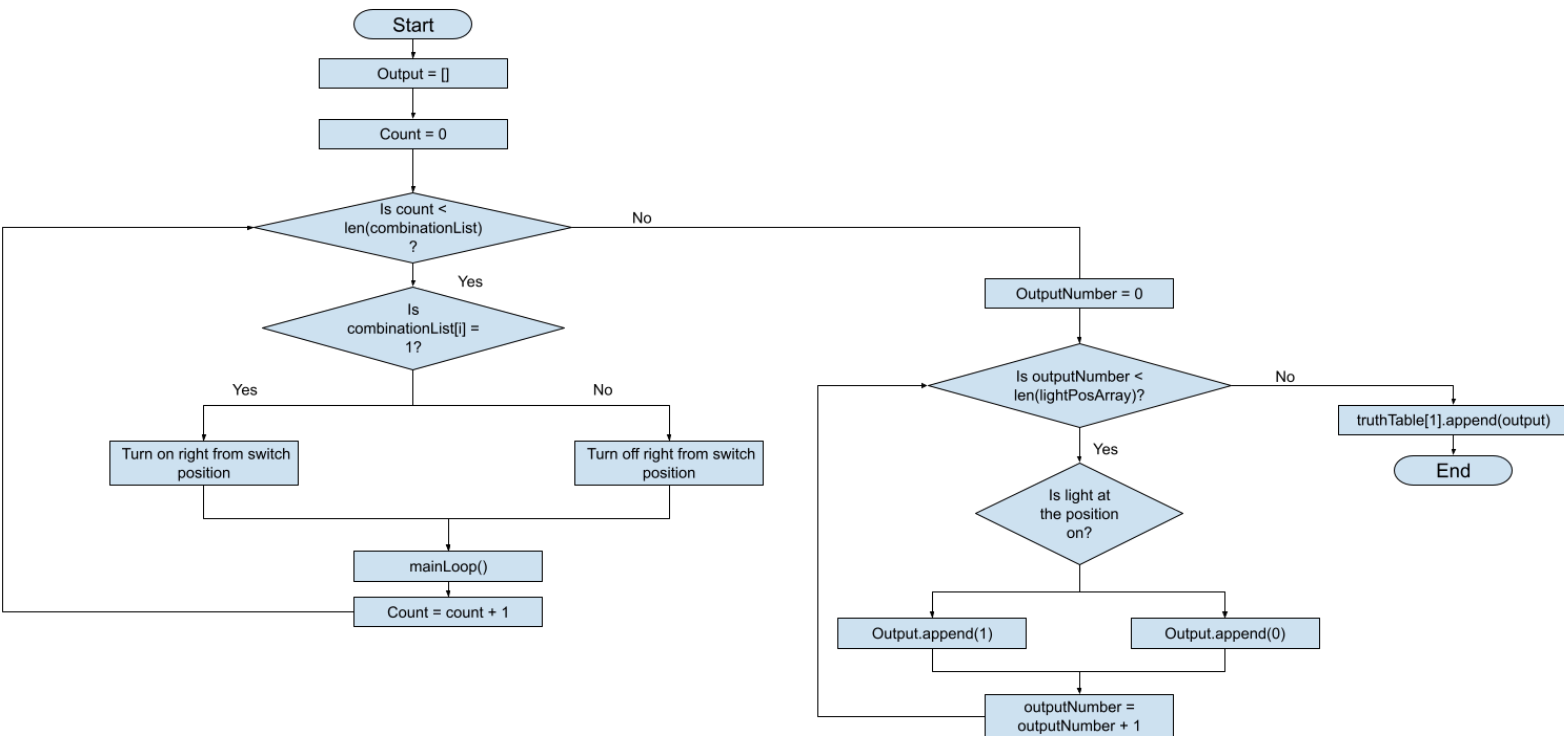
#list is converted to string

```
    for i in range(len(reverseBinaryList)-1,-1, -1):
```

```
        binaryText = binaryText + str(reverseBinaryList[i])
```

```
    return binaryText
```


Check output



#finds the outputted information from a combination of inputs in the truth table

```
def checkOutput(combinationList,switchPosArray,lightPosArray):
```

```
    output = []
```

```
    print("combination list: " + str(combinationList))
```

```
    #iterates through the combination of inputs
```

```
    for i in range(0,len(combinationList)):
```

```
        #switch is turned on, if combination specifies
```

```
        if combinationList[i] == 1:
```

```
            #i value correspond to position in switchPosArray as well
```

```
            turnOnHorizontalRight(switchPosArray[i][0],switchPosArray[i][1])
```

```
                logicArray[switchPosArray[i][0]][switchPosArray[i][1]] =
```

```
                ["Switch_ON",True]
```

```
            #main loop run to produce changes in logic array
```

```
            mainLoop()
```

```
            #switch is turned off, if combination specifies
```

```
            elif combinationList[i] == 0:
```

```
            turnOffHorizontalRight(switchPosArray[i][0],switchPosArray[i][1])
```

```
                logicArray[switchPosArray[i][0]][switchPosArray[i][1]] =
```

```
                ["Switch_OFF",False]
```

```
        #main loop run to produce changes in logic array
        mainLoop()
    print("the length: " + str(len(lightPosArray)))
    #checking if light bulbs turn on in correspondence, iterates through
list of light positions
    outputNumber = 0
    while outputNumber < len(lightPosArray):
        if
logicArray[lightPosArray[outputNumber][0]][lightPosArray[outputNumber][1]][0] ==
"Light_ON":
            #output of light is appended to overall output
            output.append(1)
        elif
logicArray[lightPosArray[outputNumber][0]][lightPosArray[outputNumber][1]][0] ==
"Light_OFF":
            #output of light is appended to overall output
            output.append(0)
            outputNumber += 1
        print(output)
        #overall output appended to truth table row
        truthTable[1].append(output)
```

Translation display

```
def translate(truthTable, switchCount, lightCount, switchCombinations, logicArray,
fileName):
```

```
    #screen height increased to 850 from 720
    SCREEN = pygame.display.set_mode((1280, 850))
    #combined count determined
    combinedCount = switchCount + lightCount
    print(combinedCount)
    print((combinedCount*49))
    print(switchCombinations)
    while True:
        #grid pixel values change depending on number of switches and lights
        if switchCount == 1:
            SCREEN.fill(BLACK)
            #column lines drawn
            for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+40),2)
            #row lines drawn
            for i in range(0,(switchCombinations*49)+1,49):
```

```
pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+542,i+40),2)
    elif switchCount == 2:
        SCREEN.fill(BLACK)
        for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+40),2)
    for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+542,i+40),2)
    elif switchCount == 3:
        SCREEN.fill(BLACK)
        if lightCount == 2:
            for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+40+147),2
)
    for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+542-147,i+4
0),2)
    elif lightCount == 1:
        for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+40+147+4
9),2)
    for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+542-196,i+4
0),2)
    #max number of inputs accepted is 4
    elif switchCount == 4:
        SCREEN.fill(BLACK)
        if lightCount == 1:
            for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+578),2)
    for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+4,i+40),2)
    elif lightCount == 2:
        for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+529),2)
```

```
    for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+53,i+40),2)
    else:
        SCREEN.fill(BLACK)

#used to define position in logic array
countHorizontal = 0
countVertical = 0
#used to determine where to place inputs and outputs text
rectangleWidth = 0
#input states are generated in grid
for i in range(40,(switchCombinations*49)+40,49):
    rectangleWidth += 49
    for j in range(542,(switchCount*49)+542,49):
        #state being placed depends on horizontal and vertical position in truth
table array
        truthTable_TEXT =
get_font1(25).render(str(truthTable[0][countVertical][countHorizontal]), True, WHITE)
        #i and j values equal the pixel count at the top left corner of the grid
square, 25 is added to each value to find the centre
        truthTable_RECT = truthTable_TEXT.get_rect(center=(j+25, i+25))
        SCREEN.blit(truthTable_TEXT, truthTable_RECT)
        #if the number of the position of the input value <= number of inputs
        if countHorizontal < switchCount-1:
            countHorizontal += 1
        #otherwise move onto next row
        else:
            countHorizontal = 0
            countVertical += 1

#input header text is generated at a fixed position
inputsText = get_font1(15).render("Inputs", True, WHITE)
inputTextRect = pygame.Rect(542,20,rectangleWidth, 20)
SCREEN.blit(inputsText, inputTextRect)

countHorizontal = 0
countVertical = 0
rectangleWidth = 0
#output states are generated in grid
for i in range(40,(switchCombinations*49)+40,49):
    for j in
range((switchCount*49)+542,(lightCount*49)+(switchCount*49)+542,49):
```

```
    truthTable_TEXT =
get_font1(25).render(str(truthTable[1][countVertical][countHorizontal]), True, GREY)
    truthTable_RECT = truthTable_TEXT.get_rect(center=(j+25, i+25))
    SCREEN.blit(truthTable_TEXT, truthTable_RECT)
    if countHorizontal < lightCount-1:
        countHorizontal += 1
    else:
        countHorizontal = 0
        countVertical += 1

    #output header text is generated based on the amount of inputs compared to
inputs and outputs
    outputText = get_font1(15).render("Outputs", True, WHITE)
    outputTextRect = pygame.Rect((switchCount*49)+542,20,rectangleWidth, 20)
    SCREEN.blit(outputText, outputTextRect)

    translate_MOUSE_POS = pygame.mouse.get_pos()

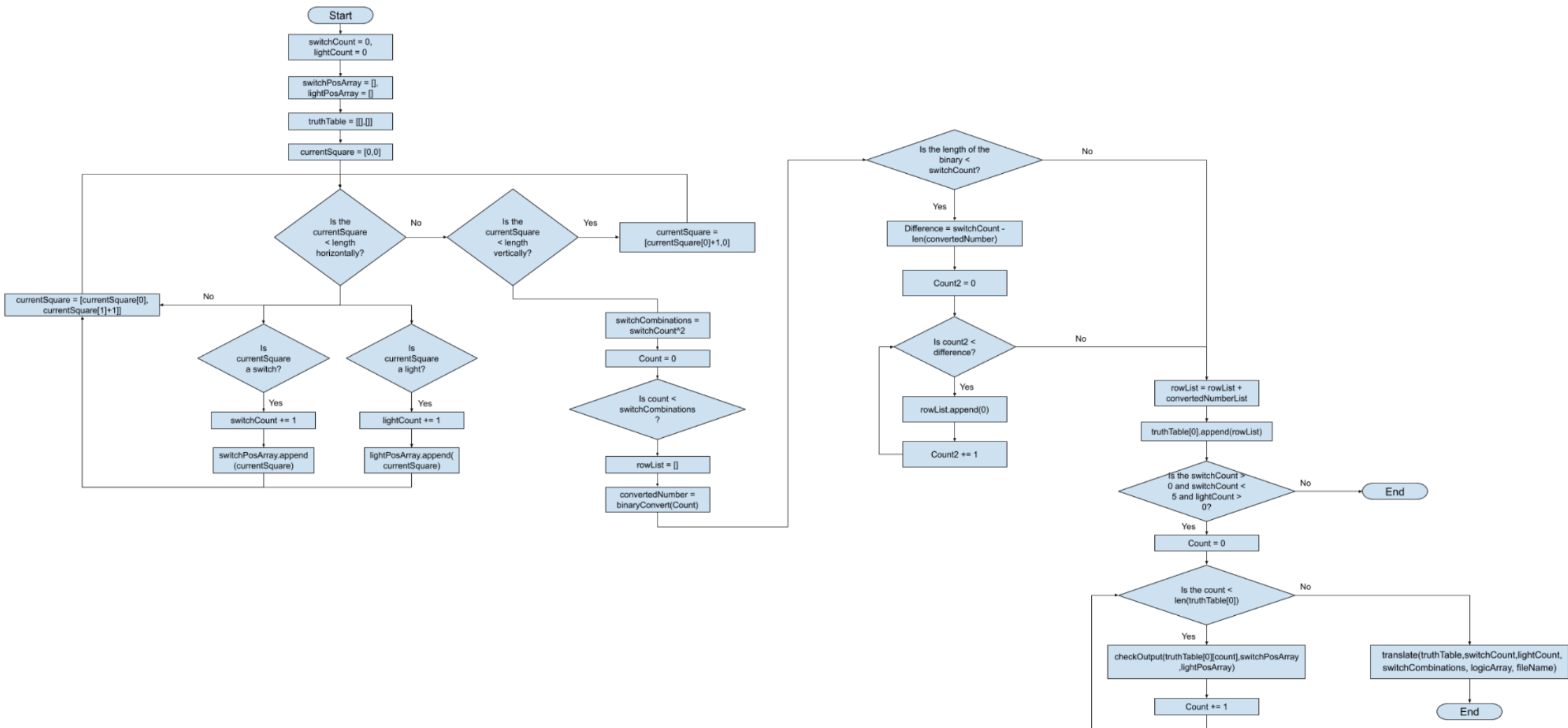
    translateBack = Button(pos=(100, 675), text_input="BACK", font=get_font2(35),
base_color=WHITE, hovering_color=GREY)

    translateBack.changeColor(translate_MOUSE_POS)
    translateBack.update(SCREEN)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            #back button returns user to original document
            if translateBack.checkForInput(translate_MOUSE_POS):
                logicInterface(logicArray, fileName, False)

    pygame.display.update()
```

Main logic



```
#input registered on truth table button in the logic interface
if logicInterfaceTruthTable.checkForInput(logicInterface_MOUSE_POS):
#initial values and data structures set
    switchCount = 0
    lightCount = 0
    switchPosArray = []
    lightPosArray = []
    truthTable = [[],[]]
    print(switchCount)
    print(lightCount)
#logic array is searched for switches and lights
for i in range(0,len(logicArray)):
    for j in range(0,len(logicArray[i])):
        #switch found
        if logicArray[i][j][0] == "Switch_OFF" or logicArray[i][j][0] ==
"Switch_ON":
            switchCount += 1
            switchPosArray.append([i,j])
        #light found
        elif logicArray[i][j][0] == "Light_OFF" or logicArray[i][j][0] ==
"Light_ON":
            lightCount += 1
            lightPosArray.append([i,j])

#amount of possible switch combinations is calculated as 2^n
switchCombinations = 2**switchCount
#print values in shell
print("switchCount: " + str(switchCount))
print("switchPosArray: " + str(switchPosArray))
print("lightCount: " + str(lightCount))
print("lightPosArray: " + str(lightPosArray))
for i in range(0,switchCombinations):
    rowList = []
    #each switch combination is converted to binary
    convertedNumber = binaryConvert(i)
    if len(str(convertedNumber)) < switchCount:
        difference = switchCount - len(str(convertedNumber))
        #if binary number is less than the number of bits, zeros are
appended to the combination
        for i in range(0,difference):
            rowList.append(0)
```

```
#binary number is converted to list format
convertedNumberList = list(str(convertedNumber))
#items in list are converted to integers
for j in range(0, len(convertedNumberList)):
    convertedNumberList[j] = int(convertedNumberList[j])
#binary number is added to contents of rowList
rowList = rowList + convertedNumberList
#binary combination is appended to truth table
truthTable[0].append(rowList)

#translation only runs if switches and lights are detected
if switchCount > 0 and switchCount < 5 and lightCount > 0:
    for i in range(0,len(truthTable[0])):
        checkOutput(truthTable[0][i],switchPosArray,lightPosArray)

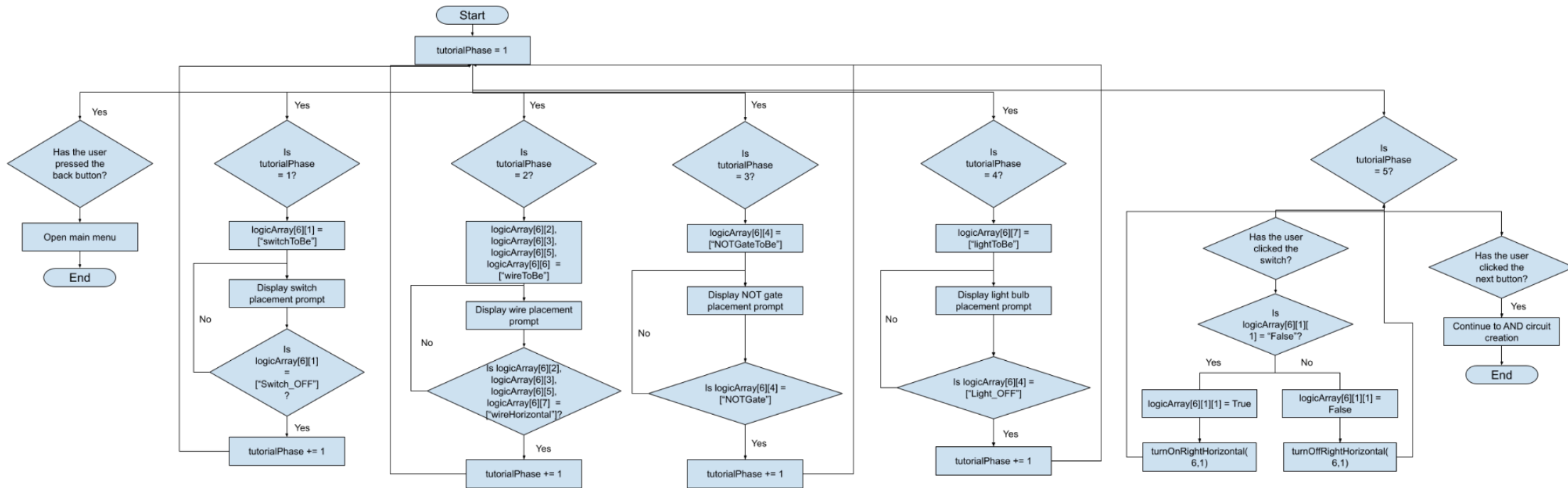
    print(truthTable)

    translate(truthTable,switchCount,lightCount, switchCombinations,
logicArray, fileName)
```

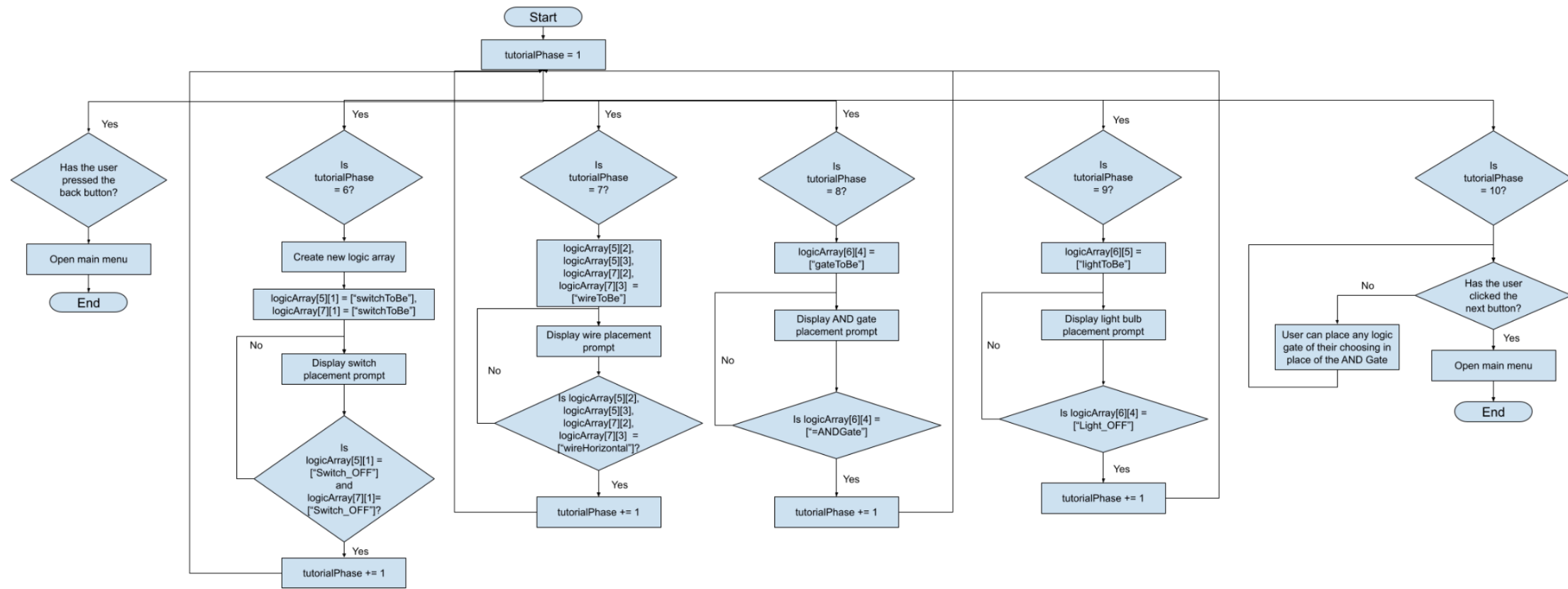

Tutorial

To allow the user to understand how to build basic logic circuits in the simulator, the tutorial function will tutor users into how to create a NOT circuit and an AND circuit, with the option of changing the AND gate in the circuit at the end and testing how that affects the output. The user is guided using text prompts as well as visual clues such as changes in text colour of logic gates in the options menu and changes in the colour of the grid squares in which gates are meant to be placed in. Below are flowcharts for each stage of the tutorial:

NOT circuit creation



Logic gate simulator - Harry Taylor - 7173 - 65301



For the tutorial function to work, I created a system whereby the tutorial would proceed in stages defined by a tutorialPhase variable. I needed to alter my user input code to allow for the changes in the allowed input from the user between the aforementioned stages within my tutorial system. The text displayed to the user, and the locations in which the user is allowed to place their logic elements vary by which stage they are in in the tutorial. Once the user has successfully placed their logic element, the program proceeds with the next stage in the tutorial, changing the allowed inputs and text displayed. Below is the code utilised for creating the text of the tutorial, as well as the first stage of the tutorial:

Tutorial text

#text changes in accordance with the tutorial stage

if tutorialPhase == 1:

 #text and constants

 paragraphText = [

 "Welcome to the Tutorial",

 "First we are going to build a NOT Gate circuit",

 "Select the switch and place it in the glowing square"]

 yPos = 280

 lineSpacing = 20

 #width and height definitions

 textWidth = 400

 textHeight = len(paragraphText) * lineSpacing

 rectangleWidth = textWidth + 30

 rectangleHeight = textHeight + 20

 #drawing background rectangle

 rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))

 rectangleSurface.fill(BLUE)

 rectangleRect = rectangleSurface.get_rect()

 rectangleRect.topleft = (830, 280)

 SCREEN.blit(rectangleSurface, rectangleRect)

 #drawing lines of text

 for line in paragraphText:

 textSurface = get_font3(15).render(line, True, BLACK)

 SCREEN.blit(textSurface, (835, yPos))

 yPos += lineSpacing

elif tutorialPhase == 2:

 paragraphText = [

 "Well done!",

 "Now to place the wires that will be used to connect the circuit",

 "Select the wire and place it in the glowing squares"]

```
yPos = 280  
lineSpacing = 20
```

```
textWidth = 480  
textHeight = len(paragraphText) * lineSpacing  
rectangleWidth = textWidth + 30  
rectangleHeight = textHeight + 20
```

```
rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))  
rectangleSurface.fill(BLUE)  
rectangleRect = rectangleSurface.get_rect()  
rectangleRect.topleft = (760, 280)  
SCREEN.blit(rectangleSurface, rectangleRect)  
for line in paragraphText:  
    textSurface = get_font3(15).render(line, True, BLACK)  
    SCREEN.blit(textSurface, (765, yPos))  
    yPos += lineSpacing
```

```
elif tutorialPhase == 3:  
    paragraphText = [  
        "Now we need to add the logic gate to the circuit",  
        "The gate we will be using is a NOT gate, which inverts the input",  
        "Select the NOT gate and place it in the glowing square"]  
    yPos = 280  
    lineSpacing = 20
```

```
textWidth = 490  
textHeight = len(paragraphText) * lineSpacing  
rectangleWidth = textWidth + 30  
rectangleHeight = textHeight + 20
```

```
rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))  
rectangleSurface.fill(BLUE)  
rectangleRect = rectangleSurface.get_rect()  
rectangleRect.topleft = (755, 280)  
SCREEN.blit(rectangleSurface, rectangleRect)  
for line in paragraphText:  
    textSurface = get_font3(15).render(line, True, BLACK)  
    SCREEN.blit(textSurface, (760, yPos))  
    yPos += lineSpacing
```

```
elif tutorialPhase == 4:  
    paragraphText = [  
        "We now need to put in the light bulb, completing the circuit",  
        "Select the light bulb and place it in the glowing square"]  
    yPos = 280
```

```
lineSpacing = 20
```

```
textWidth = 465
```

```
textHeight = len(paragraphText) * lineSpacing
```

```
rectangleWidth = textWidth + 30
```

```
rectangleHeight = textHeight + 20
```

```
rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
```

```
rectangleSurface.fill(BLUE)
```

```
rectangleRect = rectangleSurface.get_rect()
```

```
rectangleRect.topleft = (775, 280)
```

```
SCREEN.blit(rectangleSurface, rectangleRect)
```

```
for line in paragraphText:
```

```
    textSurface = get_font3(15).render(line, True, BLACK)
```

```
    SCREEN.blit(textSurface, (780, yPos))
```

```
    yPos += lineSpacing
```

```
elif tutorialPhase == 5:
```

```
    paragraphText = [
```

```
        "Now the circuit has been completed",
```

```
        "Feel free to test the circuit by turning it on and off",
```

```
        "Press the next button to continue"]
```

```
    yPos = 280
```

```
    lineSpacing = 20
```

```
textWidth = 410
```

```
textHeight = len(paragraphText) * lineSpacing
```

```
rectangleWidth = textWidth + 30
```

```
rectangleHeight = textHeight + 20
```

```
rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
```

```
rectangleSurface.fill(BLUE)
```

```
rectangleRect = rectangleSurface.get_rect()
```

```
rectangleRect.topleft = (800, 280)
```

```
SCREEN.blit(rectangleSurface, rectangleRect)
```

```
for line in paragraphText:
```

```
    textSurface = get_font3(15).render(line, True, BLACK)
```

```
    SCREEN.blit(textSurface, (805, yPos))
```

```
    yPos += lineSpacing
```

```
elif tutorialPhase == 6:
```

```
    paragraphText = [
```

```
        "Now we are going to make an AND circuit",
```

```
        "We are going to need two switches for this circuit",
```

```
        "Select the switch and place it in the glowing squares"]
```

```
    yPos = 280
```

```
lineSpacing = 20

textWidth = 410
textHeight = len(paragraphText) * lineSpacing
rectangleWidth = textWidth + 30
rectangleHeight = textHeight + 20

rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
rectangleSurface.fill(BLUE)
rectangleRect = rectangleSurface.get_rect()
rectangleRect.topleft = (830, 280)
SCREEN.blit(rectangleSurface, rectangleRect)
for line in paragraphText:
    textSurface = get_font3(15).render(line, True, BLACK)
    SCREEN.blit(textSurface, (835, yPos))
    yPos += lineSpacing
elif tutorialPhase == 7:
    paragraphText = [
        "Now we need to place the wires needed for the circuit",
        "This will allow both of the switches to connect to the AND gate",
        "Select the wire and place it in the glowing squares"]
    yPos = 280
    lineSpacing = 20

    textWidth = 480
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20

    rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
    rectangleSurface.fill(BLUE)
    rectangleRect = rectangleSurface.get_rect()
    rectangleRect.topleft = (750, 280)
    SCREEN.blit(rectangleSurface, rectangleRect)
    for line in paragraphText:
        textSurface = get_font3(15).render(line, True, BLACK)
        SCREEN.blit(textSurface, (755, yPos))
        yPos += lineSpacing
elif tutorialPhase == 8:
    paragraphText = [
        "Now we need to place the logic gate needed for the circuit",
        "The gate we will be using an AND gate, which turns on when both inputs
are on",
        "Select the AND gate and place it in the glowing square"]
```

```
yPos = 280
lineSpacing = 20
```

```
textWidth = 595
textHeight = len(paragraphText) * lineSpacing
rectangleWidth = textWidth + 30
rectangleHeight = textHeight + 20
```

```
rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
rectangleSurface.fill(BLUE)
rectangleRect = rectangleSurface.get_rect()
rectangleRect.topleft = (650, 280)
SCREEN.blit(rectangleSurface, rectangleRect)
for line in paragraphText:
    textSurface = get_font3(15).render(line, True, BLACK)
    SCREEN.blit(textSurface, (655, yPos))
    yPos += lineSpacing
```

```
elif tutorialPhase == 9:
    paragraphText = [
        "We now need to put in the light bulb, completing the circuit",
        "Select the light bulb and place it in the glowing square"]
    yPos = 280
    lineSpacing = 20
```

```
textWidth = 465
textHeight = len(paragraphText) * lineSpacing
rectangleWidth = textWidth + 30
rectangleHeight = textHeight + 20
```

```
rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
rectangleSurface.fill(BLUE)
rectangleRect = rectangleSurface.get_rect()
rectangleRect.topleft = (775, 280)
SCREEN.blit(rectangleSurface, rectangleRect)
for line in paragraphText:
    textSurface = get_font3(15).render(line, True, BLACK)
    SCREEN.blit(textSurface, (780, yPos))
    yPos += lineSpacing
```

```
elif tutorialPhase == 10:
    paragraphText = [
        "The AND gate circuit is now complete",
        "Feel free to test it again, you can also change the gate to one of your
choosing",
```


"Select the gate you want and place it in the same place as the AND Gate",

"Press next to finish the tutorial"]

yPos = 280

lineSpacing = 20

textWidth = 620

textHeight = len(paragraphText) * lineSpacing

rectangleWidth = textWidth + 30

rectangleHeight = textHeight + 20

rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))

rectangleSurface.fill(BLUE)

rectangleRect = rectangleSurface.get_rect()

rectangleRect.topleft = (620, 280)

SCREEN.blit(rectangleSurface, rectangleRect)

for line in paragraphText:

textSurface = get_font3(15).render(line, True, BLACK)

SCREEN.blit(textSurface, (625, yPos))

yPos += lineSpacing

Tutorial function

#functionality of stage 1 of the tutorial

if tutorialPhase == 1:

#location for the user to place a switch is set

logicArray[6][1] = ["switchToBe"]

for event in pygame.event.get():

if event.type == pygame.QUIT:

pygame.quit()

sys.exit()

if event.type == pygame.MOUSEBUTTONDOWN:

#pixel position of click determined

mouseX = event.pos[0]

mouseY = event.pos[1]

print("X pos " + str(mouseX))

print("Y pos " + str(mouseY))

if event.button == 1:

#selection only allows for a switch to be selected

if mouseX >= 20 and mouseX <= 69 and mouseY >= 355 and

mouseY <= 375:

if activeGate == "Switch_OFF":

activeGate = None

else:

```
        activeGate = "Switch_OFF"
        print(activeGate)
    elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
        #x and y squares in grid are determined
        Xsquare = (mouseX-251)//49
        Ysquare = (mouseY-60)//49
        print(Xsquare)
        print(Ysquare)
        if activeGate == "Switch_OFF":
            #user can only place in a specific square
            if logicArray[Ysquare][Xsquare][0] == "switchToBe":
                logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
                activeGate = None
                #proceed to next phase
                tutorialPhase += 1
        if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
        #user returns to main menu if back button is pressed
        main_menu()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_UP:
            print(logicArray)

elif tutorialPhase == 2:
    for i in range(2,7):
        if i != 4:
            #location for each wire piece is set
            if logicArray[6][i][0] == None:
                logicArray[6][i] = ["wireToBe"]
            #variable checking if all wires have been placed
            complete = True
            for i in range(2,7):
                if i != 4:
                    if logicArray[6][i][0] == "wireToBe":
                        complete = False
            if complete == True:
                tutorialPhase += 1
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    sys.exit()
                if event.type == pygame.MOUSEBUTTONDOWN:
                    mouseX = event.pos[0]
```

```
        mouseY = event.pos[1]
        print("X pos " + str(mouseX))
        print("Y pos " + str(mouseY))
        if event.button == 1:
            if mouseX >= 90 and mouseX <= 140 and mouseY >= 345
and mouseY <= 395:
                if activeGate == "wireHorizontal":
                    activeGate = None
                else:
                    activeGate = "wireHorizontal"
            elif mouseX >= 251 and mouseX <= 1280 and mouseY >=
60 and mouseY <= 697:
                Xsquare = (mouseX-251)//49
                Ysquare = (mouseY-60)//49
                print(Xsquare)
                print(Ysquare)
                if activeGate == "wireHorizontal":
                    if logicArray[Ysquare][Xsquare][0] == "wireToBe":
                        logicArray[Ysquare][Xsquare] = ["wireHorizontal",
False, None]
                    if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
                        main_menu()

elif tutorialPhase == 3:
    #activeGate is reset
    if activeGate == "wireHorizontal":
        activeGate = None
    #location for NOT gate is set
    if logicArray[6][4][0] == None:
        logicArray[6][4][0] = "NOTGateToBe"
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            if event.button == 1:
                #selection only allows for NOT gates to selected
                if mouseX >= 99 and mouseX <= 125 and mouseY >= 100 and
mouseY <= 140:
```

```
        if activeGate == "NOTGate":
            activeGate = None
        else:
            activeGate = "NOTGate"
    elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
        Xsquare = (mouseX-251)//49
        Ysquare = (mouseY-60)//49
        print(Xsquare)
        print(Ysquare)
        if activeGate == "NOTGate":
            if logicArray[Ysquare][Xsquare][0] == "NOTGateToBe":
                logicArray[Ysquare][Xsquare] = ["NOTGate", False]
                activeGate = None
                tutorialPhase += 1
        if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    main_menu()
```

```
elif tutorialPhase == 4:
    #location for user to place a light is set
    if logicArray[6][7][0] == None:
        logicArray[6][7][0] = "lightToBe"
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            if event.button == 1:
                #selection only allows lights to be selected
                if mouseX >= 27 and mouseX <= 67 and mouseY >= 440 and
mouseY <= 491:
                    if activeGate == "Light_OFF":
                        activeGate = None
                    else:
                        activeGate = "Light_OFF"
                elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                    Xsquare = (mouseX-251)//49
```

```
Ysquare = (mouseY-60)//49
print(Xsquare)
print(Ysquare)
if activeGate == "Light_OFF":
    if logicArray[Ysquare][Xsquare][0] == "lightToBe":
        logicArray[Ysquare][Xsquare] = ["Light_OFF", False]
        activeGate = None
        tutorialPhase += 1
    if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    main_menu()

elif tutorialPhase == 5:
    #next button becomes visible to user
    logicInterfaceTutorialNext.changeColor(logicInterface_MOUSE_POS)
    logicInterfaceTutorialNext.update(SCREEN)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            if event.button == 1:
                if mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                    Xsquare = (mouseX-251)//49
                    Ysquare = (mouseY-60)//49
                    print(Xsquare)
                    print(Ysquare)
                    #user can turn the switch on and off
                    if logicArray[Ysquare][Xsquare][0] == "Switch_OFF":
                        logicArray[Ysquare][Xsquare] = ["Switch_ON", True]
                    elif logicArray[Ysquare][Xsquare][0] == "Switch_ON":
                        logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
                if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    main_menu()
    if
logicInterfaceTutorialNext.checkForInput(logicInterface_MOUSE_POS):
    #pressing next proceeds to AND circuit building
    tutorialPhase += 1
```

```
elif tutorialPhase == 6:
    #logic array reset
    if logicArray[6][1][0] == "Switch_OFF" or logicArray[6][1][0] ==
"Switch_ON":
        logicArray = []
        for i in range(0,14):
            logicArray.append([None], [None], [None],[None], [None],
[None],[None], [None], [None],[None], [None], [None],[None],[None],
[None], [None],[None], [None], [None]])
            #locations for switches to be placed are set
            logicArray[5][1][0] = "switchToBe"
            logicArray[7][1][0] = "switchToBe"
            if logicArray[5][1][0] == "Switch_OFF" and logicArray[7][1][0] ==
"Switch_OFF":
                tutorialPhase += 1
                for event in pygame.event.get():
                    if event.type == pygame.QUIT:
                        pygame.quit()
                        sys.exit()
                    if event.type == pygame.MOUSEBUTTONDOWN:
                        mouseX = event.pos[0]
                        mouseY = event.pos[1]
                        print("X pos " + str(mouseX))
                        print("Y pos " + str(mouseY))
                        if event.button == 1:
                            if mouseX >= 20 and mouseX <= 69 and mouseY >= 355 and
mouseY <= 375:
                                if activeGate == "Switch_OFF":
                                    activeGate = None
                                else:
                                    activeGate = "Switch_OFF"
                                    print(activeGate)
                            elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                                Xsquare = (mouseX-251)//49
                                Ysquare = (mouseY-60)//49
                                print(Xsquare)
                                print(Ysquare)
                                if activeGate == "Switch_OFF":
                                    if logicArray[Ysquare][Xsquare][0] == "switchToBe":
                                        logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
                                        activeGate = None
```

```
        if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    main_menu()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_UP:
            print(logicArray)

elif tutorialPhase == 7:
    if logicArray[5][2][0] == None:
        #locations for wires to be placed are set
        for i in range(2,4):
            logicArray[5][i][0] = "wireToBe"
            logicArray[7][i][0] = "wireToBe"
        if logicArray[5][2][0] == "wireHorizontal" and logicArray[7][2][0] ==
"wireHorizontal" and logicArray[5][3][0] == "wireHorizontal" and logicArray[7][3][0] ==
"wireHorizontal":
            activeGate = None
            tutorialPhase += 1
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.MOUSEBUTTONDOWN:
                mouseX = event.pos[0]
                mouseY = event.pos[1]
                print("X pos " + str(mouseX))
                print("Y pos " + str(mouseY))
                if event.button == 1:
                    if mouseX >= 90 and mouseX <= 140 and mouseY >= 345 and
mouseY <= 395:
                        if activeGate == "wireHorizontal":
                            activeGate = None
                        else:
                            activeGate = "wireHorizontal"
                    elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                        Xsquare = (mouseX-251)//49
                        Ysquare = (mouseY-60)//49
                        print(Xsquare)
                        print(Ysquare)
                        if activeGate == "wireHorizontal":
                            if logicArray[Ysquare][Xsquare][0] == "wireToBe":
                                logicArray[Ysquare][Xsquare] = ["wireHorizontal", False,
None]
```

```
        if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    main_menu()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_UP:
            print(logicArray)
    elif tutorialPhase == 8:
        #location for AND gate is set
        if logicArray[6][4][0] == None:
            logicArray[6][4][0] = "gateToBe"
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.MOUSEBUTTONDOWN:
                mouseX = event.pos[0]
                mouseY = event.pos[1]
                print("X pos " + str(mouseX))
                print("Y pos " + str(mouseY))
                if event.button == 1:
                    if mouseX >= 175 and mouseX <= 225 and mouseY >= 100 and
mouseY <= 140:
                        if activeGate == "ANDGate":
                            activeGate = None
                        else:
                            activeGate = "ANDGate"
                    elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                        Xsquare = (mouseX-251)//49
                        Ysquare = (mouseY-60)//49
                        print(Xsquare)
                        print(Ysquare)
                        if activeGate == "ANDGate":
                            if logicArray[Ysquare][Xsquare][0] == "gateToBe":
                                logicArray[Ysquare][Xsquare] = ["ANDGate", False]
                                logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]
                                activeGate = None
                                tutorialPhase += 1
                                print(activeGate)
        if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    main_menu()
    if event.type == pygame.KEYDOWN:
```



```
        if event.key == pygame.K_UP:
            print(logicArray)
elif tutorialPhase == 9:
    #location for light is set
    if logicArray[6][5][0] == None:
        logicArray[6][5][0] = "lightToBe"
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            if event.button == 1:
                if mouseX >= 27 and mouseX <= 67 and mouseY >= 440 and
mouseY <= 491:
                    if activeGate == "Light_OFF":
                        activeGate = None
                    else:
                        activeGate = "Light_OFF"
                elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                    Xsquare = (mouseX-251)//49
                    Ysquare = (mouseY-60)//49
                    print(Xsquare)
                    print(Ysquare)
                    if activeGate == "Light_OFF":
                        if logicArray[Ysquare][Xsquare][0] == "lightToBe":
                            logicArray[Ysquare][Xsquare] = ["Light_OFF", False]
                            activeGate = None
                            tutorialPhase += 1
            if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    main_menu()
elif tutorialPhase == 10:
    #next button appears visible to user
    logicInterfaceTutorialNext.changeColor(logicInterface_MOUSE_POS)
    logicInterfaceTutorialNext.update(SCREEN)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
```

```
if event.type == pygame.MOUSEBUTTONDOWN:
    mouseX = event.pos[0]
    mouseY = event.pos[1]
    print("X pos " + str(mouseX))
    print("Y pos " + str(mouseY))
    if event.button == 1:
        #any logic gate can be placed in place of the AND gate
        if mouseX >= 25 and mouseX <= 75 and mouseY >= 100 and
mouseY <= 140:
            if activeGate == "ORGate":
                activeGate = None
            else:
                activeGate = "ORGate"
        elif mouseX >= 175 and mouseX <= 225 and mouseY >= 100 and
mouseY <= 140:
            if activeGate == "ANDGate":
                activeGate = None
            else:
                activeGate = "ANDGate"
        elif mouseX >= 25 and mouseX <= 75 and mouseY >= 175 and
mouseY <= 215:
            if activeGate == "NORGate":
                activeGate = None
            else:
                activeGate = "NORGate"
        elif mouseX >= 99 and mouseX <= 125 and mouseY >= 175 and
mouseY <= 215:
            if activeGate == "NANDGate":
                activeGate = None
            else:
                activeGate = "NANDGate"
        elif mouseX >= 175 and mouseX <= 225 and mouseY >= 175 and
mouseY <= 215:
            if activeGate == "XNORGate":
                activeGate = None
            else:
                activeGate = "XNORGate"
        elif mouseX >= 25 and mouseX <= 75 and mouseY >= 250 and
mouseY <= 290:
            if activeGate == "XORGate":
                activeGate = None
            else:
                activeGate = "XORGate"
```

```
elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
    Xsquare = (mouseX-251)//49
    Ysquare = (mouseY-60)//49
    print(Xsquare)
    print(Ysquare)
    if logicArray[Ysquare][Xsquare][0] == "ORGate" or
logicArray[Ysquare][Xsquare][0] == "ANDGate" or logicArray[Ysquare][Xsquare][0]
== "NORGate" or logicArray[Ysquare][Xsquare][0] == "NANDGate" or
logicArray[Ysquare][Xsquare][0] == "XNORGate" or logicArray[Ysquare][Xsquare][0]
== "XORGate":
        if activeGate == "ORGate":
            logicArray[Ysquare][Xsquare] = ["ORGate", False]
            logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

            activeGate = None
            print(activeGate)
        elif activeGate == "ANDGate":
            logicArray[Ysquare][Xsquare] = ["ANDGate", False]
            logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

            activeGate = None
            undoStack.append(["remove", "ANDGate", Ysquare,
Xsquare])

            print(activeGate)
        elif activeGate == "NORGate":
            logicArray[Ysquare][Xsquare] = ["NORGate", False]
            logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

            activeGate = None
            print(activeGate)
        elif activeGate == "NANDGate":
            logicArray[Ysquare][Xsquare] = ["NANDGate", False]
            logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

            activeGate = None
            print(activeGate)
        elif activeGate == "XNORGate":
            logicArray[Ysquare][Xsquare] = ["XNORGate", False]
            logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

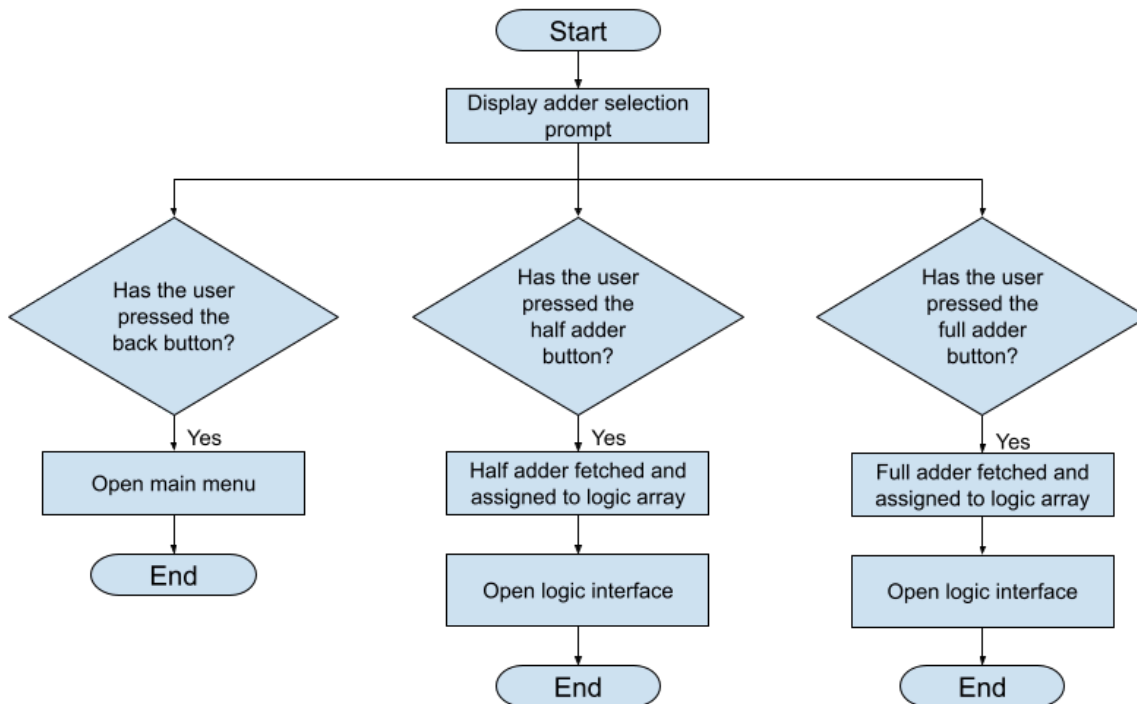
            activeGate = None
            print(activeGate)
        elif activeGate == "XORGate":
```

```
logicArray[Ysquare][Xsquare] = ["XORGate", False]
logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    print(activeGate)
    if logicArray[Ysquare][Xsquare][0] == "Switch_OFF":
        logicArray[Ysquare][Xsquare] = ["Switch_ON", True]
    elif logicArray[Ysquare][Xsquare][0] == "Switch_ON":
        logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
    if
logicInterfaceTutorialNext.checkForInput(logicInterface_MOUSE_POS):
    #tutorial ends
    main_menu()
```

Adders

To give users a demonstration of a practical use case for logic gates, with the added benefit of demonstrating what can be built within my simulator, I chose to implement pre-built adders circuits. Users can select from the main menu an adder option that will allow them to select from either a half or full adder that I myself have made within the simulator. This will then be loaded using pickle python from an adder folder, allowing the user to interact and modify the adder as they please. Below is a flowchart for how this menu works as well as the code for the menu interface itself:



```

def adders():
    while True:
        adders_MOUSE_POS = pygame.mouse.get_pos()

        SCREEN.fill(BLACK)

        #adder text created
        adders_TEXT = get_font1(45).render("Please select an adder to load", True,
        WHITE)
        adders_RECT = adders_TEXT.get_rect(center=(640, 260))
        SCREEN.blit(adders_TEXT, adders_RECT)

        #buttons created
        addersBack = Button(pos=(640,440),text_input="Back", font=get_font2(30),
        base_color=WHITE, hovering_color=GREEN)
    
```

```
addersHalfAdder = Button(pos=(550, 360), text_input="Half Adder",
font=get_font2(30), base_color=WHITE, hovering_color=GREEN)
addersFullAdder = Button(pos=(730,360), text_input="Full Adder", font =
get_font2(30), base_color=WHITE, hovering_color=GREEN)
```

```
#buttons draw in in interface
```

```
for button in [addersBack, addersHalfAdder, addersFullAdder]:
    button.changeColor(adders_MOUSE_POS)
    button.update(SCREEN)
```

```
for event in pygame.event.get():
```

```
    if event.type == pygame.QUIT:
```

```
        pygame.quit()
```

```
        sys.exit()
```

```
    if event.type == pygame.MOUSEBUTTONDOWN:
```

```
        #back button pressed
```

```
        if addersBack.checkForInput(adders_MOUSE_POS):
```

```
            main_menu()
```

```
        #half adder button pressed
```

```
        if addersHalfAdder.checkForInput(adders_MOUSE_POS):
```

```
            #file is loaded from adders folder
```

```
            currentDirectory = os.path.dirname(os.path.abspath(__file__))
```

```
            folderPath = os.path.join(currentDirectory, "adders")
```

```
            filePath = os.path.join(folderPath, "halfAdder")
```

```
            logicFile = open(filePath,"rb")
```

```
            logicArray = pickle.load(logicFile)
```

```
            #logic interface opens
```

```
            logicInterface(logicArray,"halfAdder", False)
```

```
        #full adder button pressed
```

```
        if addersFullAdder.checkForInput(adders_MOUSE_POS):
```

```
            #file is loaded from adders folder
```

```
            currentDirectory = os.path.dirname(os.path.abspath(__file__))
```

```
            folderPath = os.path.join(currentDirectory, "adders")
```

```
            filePath = os.path.join(folderPath, "fullAdder")
```

```
            logicFile = open(filePath,"rb")
```

```
            logicArray = pickle.load(logicFile)
```

```
            #logic interface opens
```

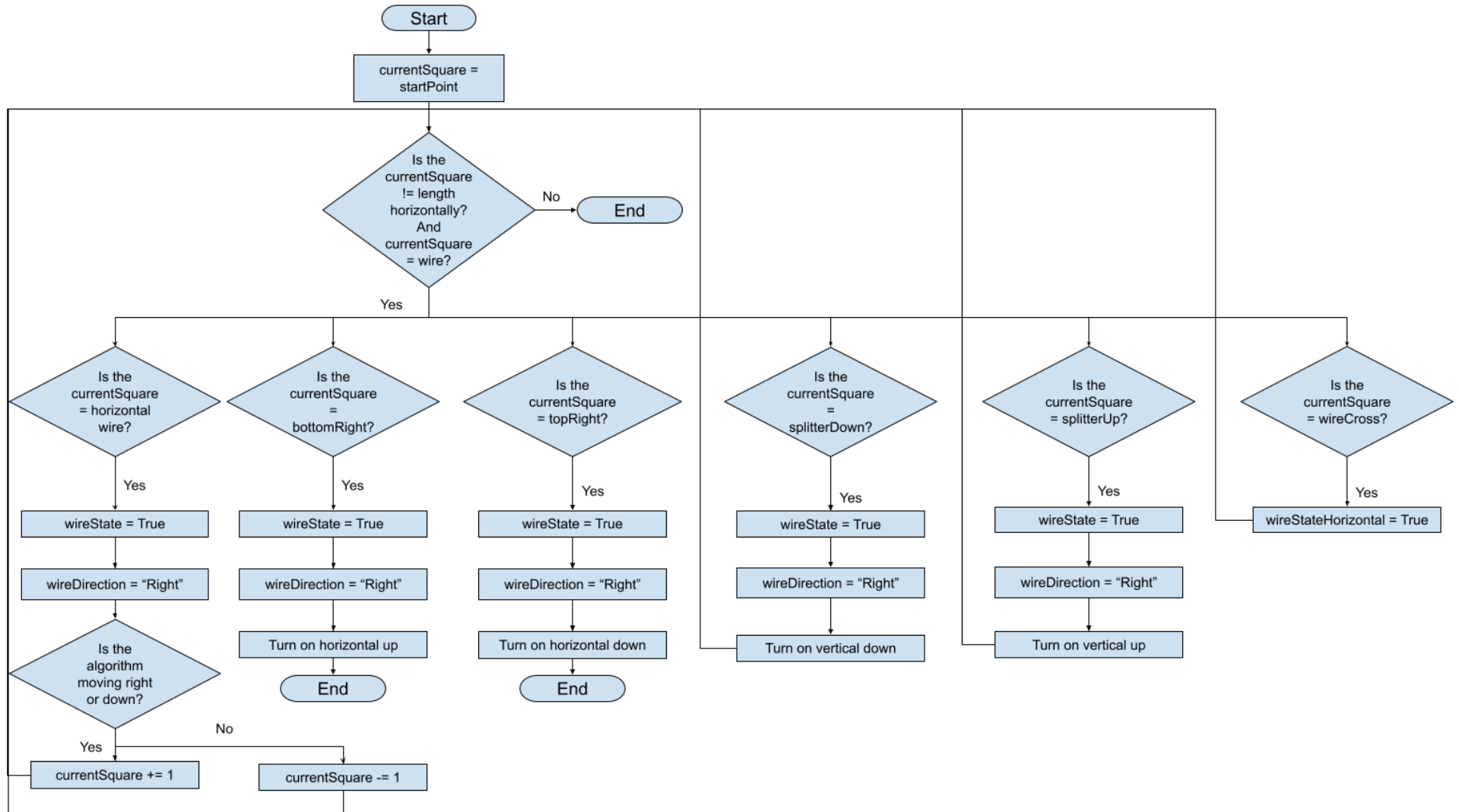
```
            logicInterface(logicArray,"fullAdder", False)
```

```
pygame.display.update()
```

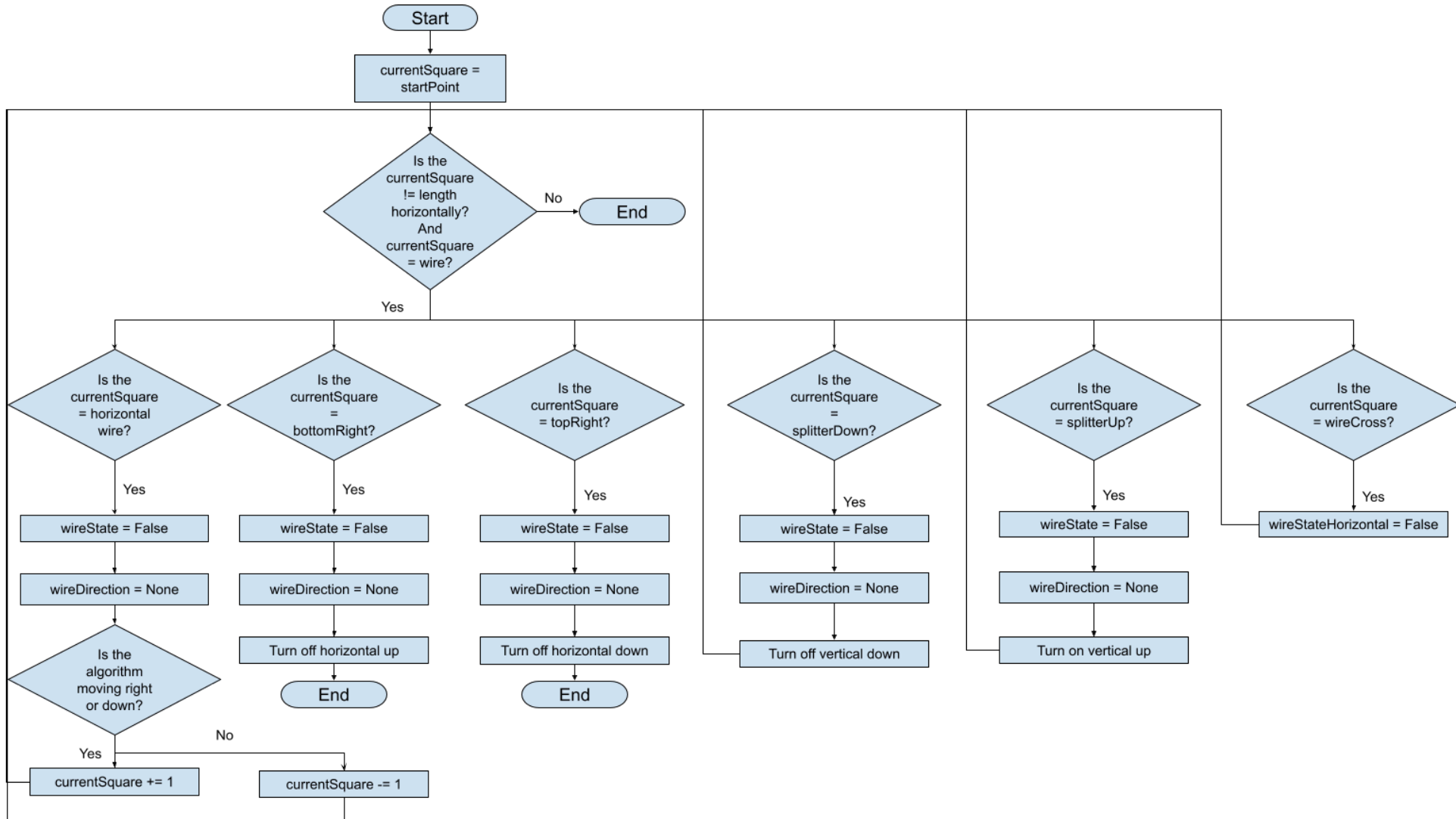
Turning off/on

To simulate an interactive logic circuit, the circuit will need to be turned on and off by the user using switches. This change in user input will need to allow corresponding wires to turn on/off utilising subroutines that turn wires on or off either horizontally or vertically. When another element is detected or no value is detected in the grid square, the subroutine returns and the main loop continues. When a wire corner or splitter is detected, the corresponding subroutine for turning on/off vertically or horizontally is called. Below are flowcharts of these subroutines:

Turning on



Turning off



If a switch is detected in the main loop, its state is checked. Depending on the state of the switch, the on or off subroutines are called in all directions causing the corresponding wires to turn on or off. When the subroutine comes into contact with a direction changing wire, the subroutine for that respective change in direction is called nested within the initial subroutine. Eventually when the subroutine meets an empty square the subroutines end.

Below are the subroutines for the on and off algorithms for power flowing right:

Turn on

#i refers to vertical position, j refers to horizontal position

```
def turnOnHorizontalRight(i,j):
```

```
    startPoint = j+1
```

```
    #entire row is searched
```

```
    for k in range(startPoint,21):
```

```
        #when blank square is found, subroutine ends
```

```
        if logicArray[i][k] == [None]:
```

```
            break
```

```
        #state of wire is changed, direction of power flow is defined as "Right"
```

```
        elif logicArray[i][k][0] == "wireHorizontal":
```

```
            logicArray[i][k][1] = True
```

```
            logicArray[i][k][2] = "Right"
```

```
        elif logicArray[i][k][0] == "wireBottomRight":
```

```
            logicArray[i][k][1] = True
```

```
            logicArray[i][k][2] = "Up"
```

```
            #wire direction changes, so vertical up subroutine is called
```

```
            turnOnVerticalUp(i,k)
```

```
        elif logicArray[i][k][0] == "wireTopRight":
```

```
            logicArray[i][k][1] = True
```

```
            logicArray[i][k][2] = "Down"
```

```
            #wire direction changes, so vertical down subroutine is called
```

```
            turnOnVerticalDown(i,k)
```

```
        elif logicArray[i][k][0] == "wireSplitterDown":
```

```
            logicArray[i][k][1] = True
```

```
            logicArray[i][k][2] = "Right"
```

```
            #power is split, so both vertical down and horizontal right  
            subroutines are called
```

```
            turnOnVerticalDown(i,k)
```

```
            turnOnHorizontalRight(i,k)
```

```
        elif logicArray[i][k][0] == "wireSplitterUp":
```

```
            logicArray[i][k][1] = True
```

```
            logicArray[i][k][2] = "Right"
```

```
            #power is split, so both vertical up and horizontal right  
            subroutines are called
```

```
        turnOnVerticalUp(i,k)
        turnOnHorizontalRight(i,k)
    elif logicArray[i][k][0] == "wireSplitterLeft":
        logicArray[i][k][1] = True
        logicArray[i][k][2] = "Right"
        #power is split, so both vertical up and vertical down subroutines
        #are called
        turnOnVerticalDown(i,k)
        turnOnVerticalUp(i,k)
    elif logicArray[i][k][0] == "wireCross":
        #wire cross functions in the same way as a horizontal wire, but
        #with a different index
        logicArray[i][k][2] = True
```

Turn off

#i refers to vertical position, j refers to horizontal position

def turnOffHorizontalRight(i,j):

```
    startPoint = j+1
```

```
    #entire row is searched
```

```
    for k in range(startPoint,21):
```

```
        #when blank square is found, subroutine ends
```

```
        if logicArray[i][k] == [None]:
```

```
            break
```

```
    elif logicArray[i][k][0] == "wireHorizontal":
```

```
        logicArray[i][k][1] = False
```

```
        #state of wire is changed to off, direction of power flow is reset
```

```
        logicArray[i][k][2] = None
```

```
    elif logicArray[i][k][0] == "wireBottomRight":
```

```
        logicArray[i][k][1] = False
```

```
        logicArray[i][k][2] = None
```

```
        #wire direction changes, so vertical up subroutine is called
```

```
        turnOffVerticalUp(i, k)
```

```
    elif logicArray[i][k][0] == "wireTopRight":
```

```
        logicArray[i][k][1] = False
```

```
        logicArray[i][k][2] = None
```

```
        #wire direction changes, so vertical down subroutine is called
```

```
        turnOffVerticalDown(i,k)
```

```
    elif logicArray[i][k][0] == "wireSplitterDown":
```

```
        logicArray[i][k][1] = False
```

```
        logicArray[i][k][2] = None
```

```
        #power is split, both horizontal right and vertical down
        #subroutines are called
```

```
        turnOffVerticalDown(i,k)
```

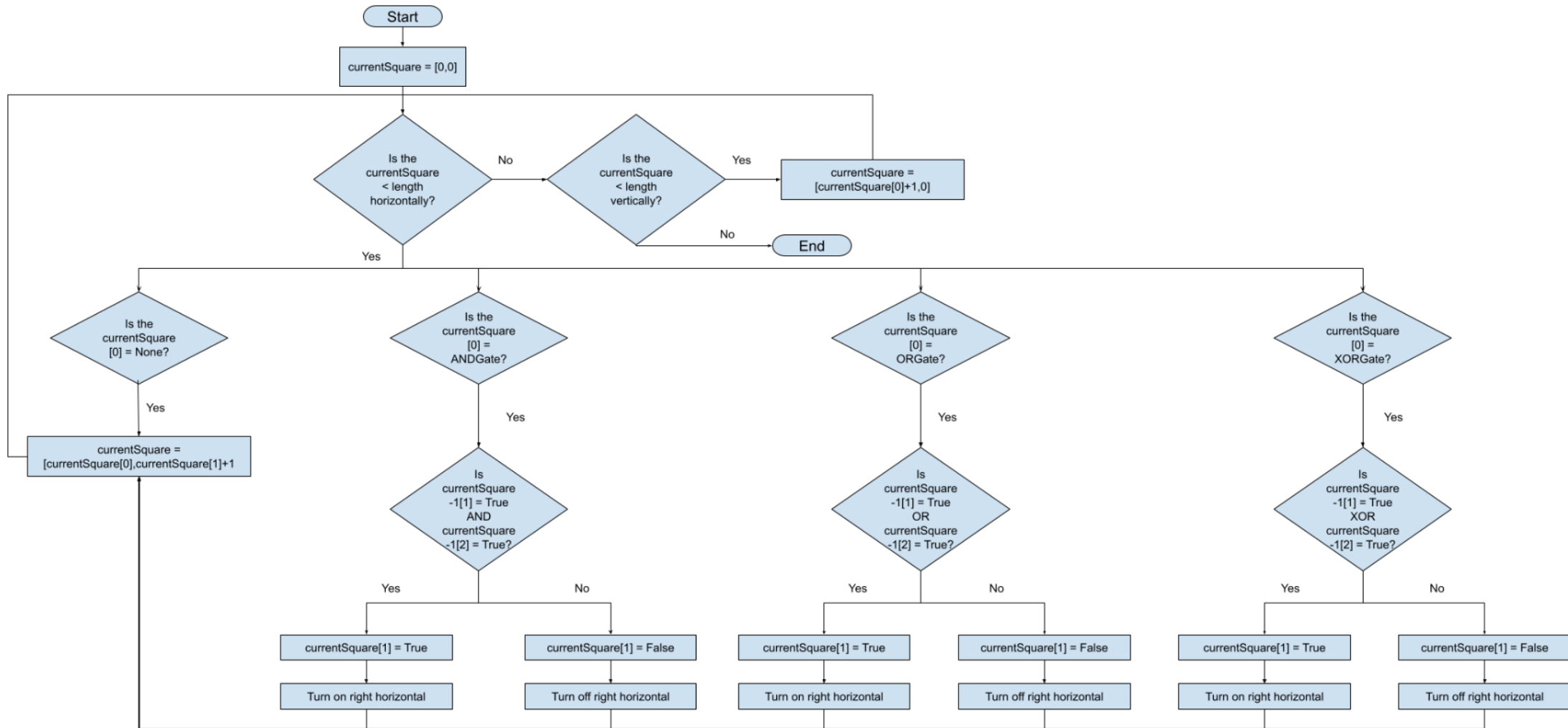
```
        turnOffHorizontalRight(i,k)
```

```
elif logicArray[i][k][0] == "wireSplitterUp":
    logicArray[i][k][1] = False
    logicArray[i][k][2] = None
    #power is split, both horizontal right and vertical up subroutines
    are called
    turnOffVerticalUp(i,k)
    turnOffHorizontalRight(i,k)
elif logicArray[i][k][0] == "wireSplitterLeft":
    logicArray[i][k][1] = False
    logicArray[i][k][2] = None
    #power is split, both vertical up and down subroutines are called
    turnOffVerticalDown(i,k)
    turnOffVerticalUp(i,k)
elif logicArray[i][k][0] == "wireCross":
    #wire cross functions in the same way as a horizontal wire, but
    with a different index
    logicArray[i][k][2] = False
```

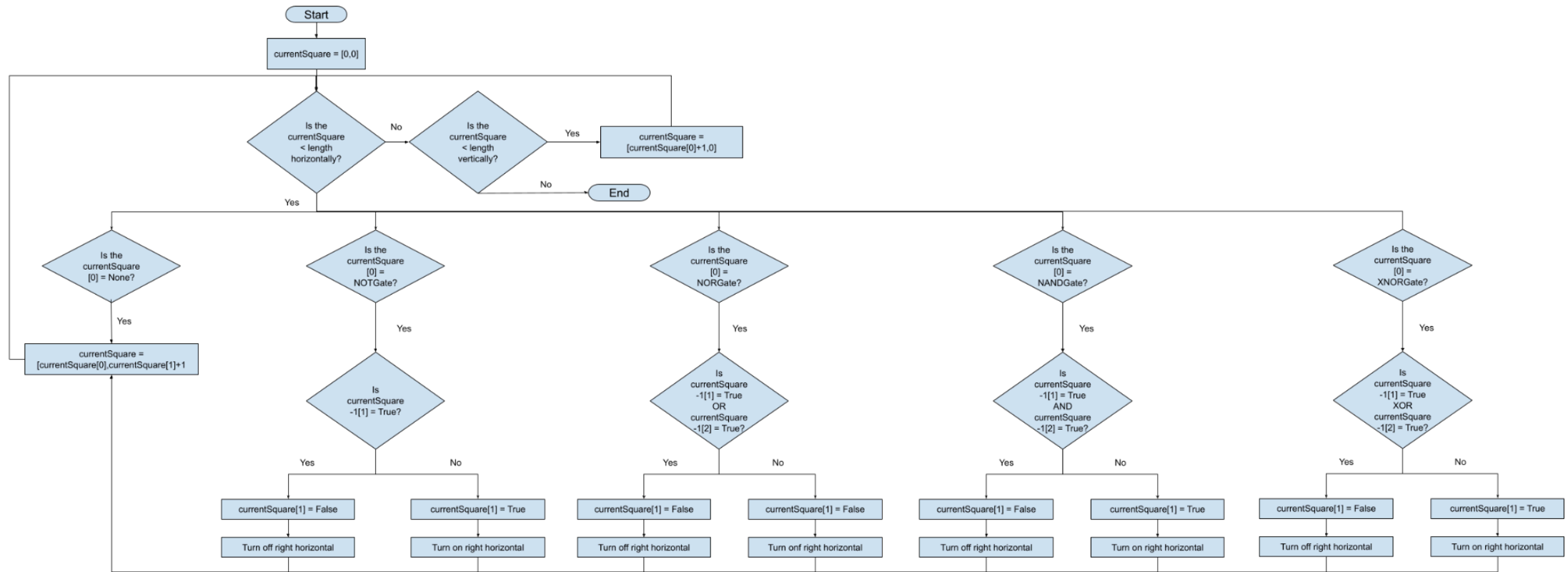
Main loop

The main loop defines the logic operations needed for the simulator. The entire array is searched until a logic element is encountered. When one is encountered the algorithm makes a decision based on the element's type; for example, if the element were a wire the main loop would search the corresponding grid squares around it to make sure the wire takes the correct shape. The main loop is also responsible for managing the gates themselves based on the states of the wire intersection square on the left of the gate. Below are flowcharts for the different sections of the main loop, including the functions of each logic gate:

Regular gates



NOT gates



To allow continuous changes to the logic grid, the entire grid is searched. If a switch is detected, the state of it is checked and any wires connected directly to the right of the switch will change state in accordance. If a logic gate is detected, the state of the two boolean state variables in the wire intersection one square to the left of the gate is checked, then the state of the logic gate changes in correspondence with the gate type; this in turn changes the state of any wires connected to the gate on the right hand side. If a light is detected, the square to the left is checked. If the state of the wire on the left is True, the wire turns on, if no wire is detected or the wire state is false, the light turns off.

The main loop is also responsible for changing the orientation of wires in correspondence with the surrounding wires. This includes each wire corner, as well as vertical wires. For example if two wires are placed atop each other, they both become vertical or if wires are placed perpendicular to one another a corner piece is created.

Below is the algorithm for the main loops functionality for a switch, OR gate, as well as a horizontal wire piece:

```
def mainLoop():
    #global variables
    global lastCountChange
    global count
    global finalTime
    global clockPulse
    global lastChange
    #clock pulse changes
    if time.time() > finalTime:
        if clockPulse == False:
            clockPulse = True
        else:
            clockPulse = False
        lastChange = finalTime
        finalTime = time.time() + 1

    for i in range(0,len(logicArray)):
        for j in range(0,len(logicArray[i])):
            #algorithm passes over empty squares
            if logicArray[i][j][0] == None:
                pass
            #turning a switch on causes corresponding elements on the right
            #to turn on
            elif logicArray[i][j][0] == "Switch_ON":
```



```
turnOnHorizontalRight(i,j)
turnOnHorizontalLeft(i,j)
turnOnVerticalUp(i,j)
turnOnVerticalDown(i,j)
```

```
#turning a switch off causes corresponding elements in all
directions to turn off
```

```
elif logicArray[i][j][0] == "Switch_OFF":
```

```
turnOffHorizontalRight(i,j)
turnOffHorizontalLeft(i,j)
turnOffVerticalUp(i,j)
turnOffVerticalDown(i,j)
```

```
elif logicArray[i][j][0] == "Clock":
```

```
#clock changes state based on pulse
```

```
if clockPulse == True:
```

```
logicArray[i][j][1] = True
turnOnHorizontalRight(i,j)
turnOnHorizontalLeft(i,j)
turnOnVerticalDown(i,j)
turnOnVerticalUp(i,j)
```

```
else:
```

```
logicArray[i][j][1] = False
turnOffHorizontalRight(i,j)
turnOffHorizontalLeft(i,j)
turnOffVerticalDown(i,j)
turnOffVerticalUp(i,j)
```

```
elif logicArray[i][j][0] == "ORGate":
```

```
#OR operation
```

```
if logicArray[i][j-1][1] == True or logicArray[i][j-1][2] ==
True:
```

```
logicArray[i][j][1] = True
```

```
#corresponding elements on the right turn on
turnOnHorizontalRight(i,j)
```

```
else:
```

```
#corresponding elements on the left turn off
```

```
logicArray[i][j][1] = False
```

```
elif logicArray[i][j][0] == "DFlipFlop":
```

```
#state of D flip flop changes based on last switch state
change and time since last clock pulse
```

```
if time.time() < lastChange + 0.5 and count ==
lastCountChange + 1:
```

```
if logicArray[i][j-1][1] == True:
```

```
if logicArray[i][j-1][2] == True:
```

```
logicArray[i][j][1] = False
```

```
        turnOffHorizontalRight(i,j)
    elif logicArray[i][j-1][2] == False:
        logicArray[i][j][1] = True
        turnOnHorizontalRight(i,j)

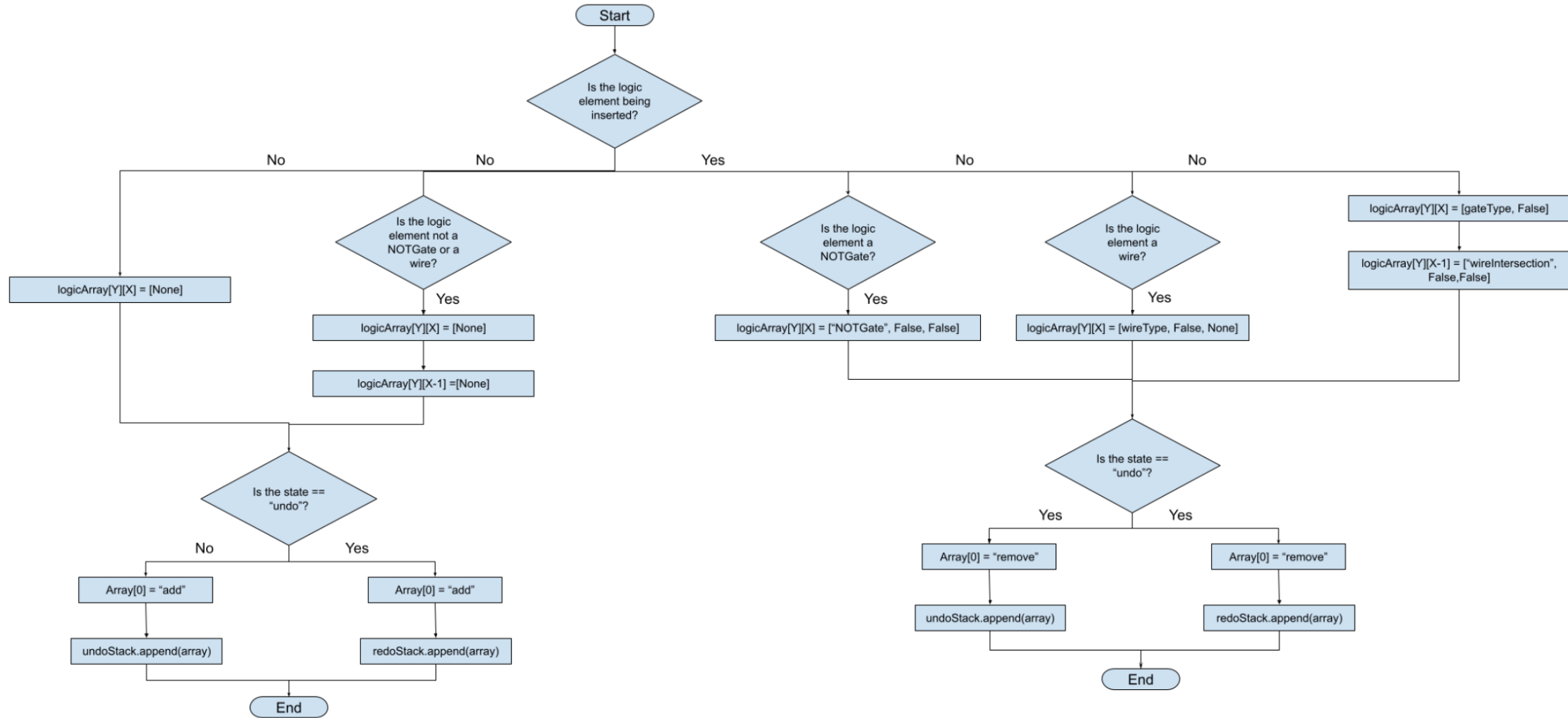
elif logicArray[i][j][0] == "wireHorizontal":
    #change to wire splitter up
    if (logicArray[i-1][j][0] == "wireHorizontal" or
        logicArray[i-1][j][0] == "wireVertical") and
        logicArray[i][j-1][0] == "wireHorizontal" and
        logicArray[i][j+1][0] == "wireHorizontal":
        logicArray[i][j][0] = "wireSplitterUp"
    #change to bottom right
    elif (logicArray[i-1][j][0] == "wireHorizontal" or
        logicArray[i-1][j][0] == "wireIntersection") and
        (logicArray[i][j-1][0] == "wireHorizontal" or
        logicArray[i][j-1][0] == "Switch_ON" or logicArray[i][j-1][0]
        == "Switch_OFF"):
        logicArray[i][j][0] = "wireBottomRight"
        if logicArray[i-1][j][0] == "wireHorizontal":
            logicArray[i][j][0] = "wireVertical"
    #change to top right
    elif (logicArray[i+1][j][0] == "wireHorizontal" or
        logicArray[i+1][j][0] == "wireIntersection") and
        logicArray[i][j-1][0] == "wireHorizontal":
        logicArray[i][j][0] = "wireTopRight"
        if logicArray[i+1][j][0] == "wireHorizontal":
            logicArray[i+1][j][0] = "wireVertical"
    #change to vertical
    elif logicArray[i+1][j][0] == "wireVertical" or
        logicArray[i+1][j][0] == "wireIntersection" or
        logicArray[i-1][j][0] == "wireSplitterUp":
        logicArray[i][j][0] = "wireVertical"
    elif logicArray[i-1][j][0] == "wireVertical" or
        logicArray[i-1][j][0] == "wireIntersection" or
        logicArray[i-1][j][0] == "wireSplitterDown" or
        logicArray[i-1][j][0] == "wireSplitterLeft" or
        logicArray[i-1][j][0] == "wireSplitterRight":
        logicArray[i][j][0] = "wireVertical"
    elif logicArray[i+1][j][0] == "wireHorizontal":
        logicArray[i+1][j][0] = "wireVertical"
        logicArray[i][j][0] = "wireVertical"
    elif logicArray[i-1][j][0] == "wireHorizontal":
        logicArray[i-1][j][0] = "wireVertical"
```

```
logicArray[i][j][0] = "wireVertical"
```

Logic element reinsertion and removal

In order to utilise the functionality of both undoing and redoing elements in the logic document, an algorithm is used to insert or remove the logic element in question. This is done by taking the popped item from the top of either the undo or redo stack and depending on its contents add or remove it from its respective coordinate in the logic grid. The reinsertion algorithm also compensates for the power flow around the circuit from that specific element, as well as the wire intersection piece placed to the left of a logic gate. The algorithm is also responsible for popping the data from the undo/redo stack and pushing it onto the other stack. Below is the flowchart for this algorithm:

Logic gate simulator - Harry Taylor - 7173 - 65301



To include the undo and redo functionality needed for the final project, I combined the functionality of deleting and adding elements to the document into a single subroutine. The undo and redo function then call this subroutine with a 2D array specifying the item to be deleted or added and its position within the logic array. The subroutine is also called with a parameter specifying whether the item is to be undone or redone, allowing the action to be popped from the appropriate stack then pushed onto the other with the opposite function. Below is the subroutine for reinsertion and removal of items in undo and redo stack:

#reArrayLast consists of: ["add/remove", "logicElement", Yposition, Xposition]

def reInsert(reArrayLast, state):

 #logic element removal

 if reArrayLast[0] == "remove":

 #if logic element is a wire or NOT gate

 if reArrayLast[1] == "NOTGate" or reArrayLast[1] == "Switch_OFF" or
 reArrayLast[1] == "Light_OFF" or reArrayLast[1] == "wireHorizontal" or
 reArrayLast[1] == "wireVertical" or reArrayLast[1] == "wireTopLeft" or
 reArrayLast[1] == "wireTopRight" or reArrayLast[1] ==
 "wireBottomRight" or reArrayLast[1] == "wireBottomLeft":

 #turning off connected wires

 if len(logicArray[reArrayLast[2]][reArrayLast[3]]) == 3:

 print(logicArray[reArrayLast[2]][reArrayLast[3]][2])

 if logicArray[reArrayLast[2]][reArrayLast[3]][2] == "Up":

 turnOffVerticalUp(reArrayLast[2],reArrayLast[3])

 elif logicArray[reArrayLast[2]][reArrayLast[3]][2] ==
 "Down":

 turnOffVerticalDown(reArrayLast[2],reArrayLast[3])

 elif logicArray[reArrayLast[2]][reArrayLast[3]][2] ==

 "Right":

 turnOffHorizontalRight(reArrayLast[2],reArrayLast[3])

 elif logicArray[reArrayLast[2]][reArrayLast[3]][2] == "Left":

 turnOffHorizontalLeft(reArrayLast[2],reArrayLast[3])

 #if element is a clock

 elif reArrayLast[1] == "Clock":

 pass

 else:

 turnOffHorizontalRight(reArrayLast[2],reArrayLast[3])

 logicArray[reArrayLast[2]][reArrayLast[3]] = [None]

 #if logic element is a logic gate

 else:

 turnOffHorizontalRight(reArrayLast[2],reArrayLast[3])

```
        logicArray[reArrayLast[2]][reArrayLast[3]] = [None]
        logicArray[reArrayLast[2]][reArrayLast[3]-1] = [None]
    if state == "undo":
        #reArrayLast is popped from undo stack and pushed to redo
        stack with function reversed from add to remove
        reArrayLast[0] = "add"
        redoStack.append(reArrayLast)
        del undoStack[len(undoStack)-1]
    elif state == "redo":
        #reArrayLast is popped from redo stack and pushed to undo
        stack with function reversed from add to remove
        reArrayLast[0] = "add"
        undoStack.append(reArrayLast)
        del redoStack[len(redoStack)-1]
#logic element insertion
elif reArrayLast[0] == "add":
    #NOT gate is reinserted
    if reArrayLast[1] == "NOTGate":
        logicArray[reArrayLast[2]][reArrayLast[3]] = ["NOTGate", False,
        False]
    #switch or light is reinserted
    elif reArrayLast[1] == "Switch_OFF" or reArrayLast[1] == "Light_OFF":
        logicArray[reArrayLast[2]][reArrayLast[3]] = [reArrayLast[1],
        False]
    #wire is reinserted
    elif reArrayLast[1] == "wireHorizontal" or reArrayLast[1] ==
    "wireVertical" or reArrayLast[1] == "wireTopLeft" or reArrayLast[1] ==
    "wireTopRight" or reArrayLast[1] == "wireBottomRight" or
    reArrayLast[1] == "wireBottomLeft":
        logicArray[reArrayLast[2]][reArrayLast[3]] = ["wireHorizontal",
        False, None]
    #logic gate is reinserted
    else:
        logicArray[reArrayLast[2]][reArrayLast[3]] = [reArrayLast[1],
        False]
        logicArray[reArrayLast[2]][reArrayLast[3]-1] = ["wireIntersection",
        False, False]
    if state == "redo":
        #reArrayLast is popped from redo stack and pushed to undo
        stack with function reversed from add to remove
        reArrayLast[0] = "remove"
        undoStack.append(reArrayLast)
        del redoStack[len(redoStack)-1]
    elif state == "undo":
```

```
#reArrayLast is popped from undo stack and pushed to redo  
stack with function reversed from add to remove  
reArrayLast[0] = "remove"  
redoStack.append(reArrayLast)  
del undoStack[len(undoStack)-1]
```

Logic elements

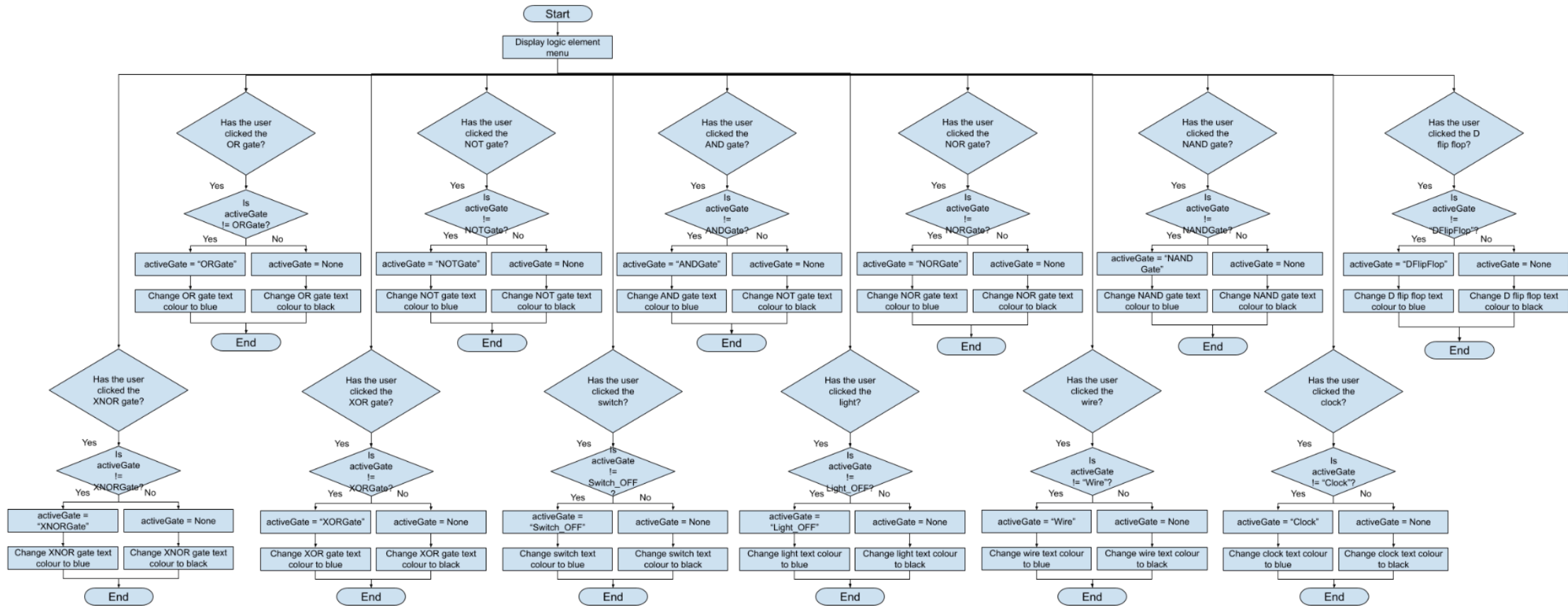
In order to program all logic elements and integrate them with one another I chose to implement them using a procedural main logic loop that repeats constantly until the logic interface subroutine is interrupted. This way the function of each individual logic gate blends seamlessly with each other, alongside all inputs, wires and outputs. This approach allows constant updates to the grid array, meaning changes to the wire type can be constantly made based on user input. This is done alongside turning sections of the logic system on and off, via subroutines, based on the direction of power.

Menu implementation

For the user to access all logic elements, the left section of the logic interface contains a menu allowing the user access to all elements on offer by the simulator. Clicking on each element makes it active for selection indicated by a change in colour of the text for the element selected, or if the element is unavailable in the case of the tutorial.

The implementation comes in two sections, one in which the buttons of the menu are defined and placed, and one in which the inputs for each element are checked, allowing a gate to become active for placement. Below is a flowchart for this feature as well as example code:

Logic gate simulator - Harry Taylor - 7173 - 65301



While True:

```
#load gate image
    ORGate = pygame.image.load("ORGate.png")
    #creation of background rectangle needed for the OR gate to be
displayed
    ORrect = ORGate.get_rect()
    ORrect.center = (50,120)
    #creation of the text description of the gate, colours change depending
on events in the logic interface
    if activeGate == "ORGate":
        ORText = get_font3(15).render("OR Gate", True, BLUE)
        #black text only appears if the gate is not active, and either in document
or on the tenth stage of the tutorial
        elif tutorialPhase == None or tutorialPhase == 10:
            ORText = get_font3(15).render("OR Gate", True, BLACK)
            #grey text appears when option is unavailable in the context of the
tutorial
        elif tutorialPhase != None:
            ORText = get_font3(15).render("OR Gate", True, GREY)
            #creation of the background rectangle needed for text description
            ORrectText = ORText.get_rect()
            ORrectText.center = (50,155)

    NOTGate = pygame.image.load("NOTGate.png")
    NOTrect = NOTGate.get_rect()
    NOTrect.center = (125,120)
    if activeGate == "NOTGate":
        NOTText = get_font3(15).render("NOT Gate", True, BLUE)
        elif tutorialPhase == None or tutorialPhase == 3:
            NOTText = get_font3(15).render("NOT Gate", True, BLACK)
        elif tutorialPhase != None:
            NOTText = get_font3(15).render("NOT Gate", True, GREY)
        NOTrectText = NOTText.get_rect()
        NOTrectText.center = (125,155)

    ANDGate = pygame.image.load("ANDGate.png")
    ANDrect = ANDGate.get_rect()
    ANDrect.center = (200,120)
    if activeGate == "ANDGate":
        ANDText = get_font3(15).render("AND Gate", True, BLUE)
        elif tutorialPhase == None or tutorialPhase == 8 or tutorialPhase == 10:
```

```
    ANDText = get_font3(15).render("AND Gate", True, BLACK)
elif tutorialPhase != None:
    ANDText = get_font3(15).render("AND Gate", True, GREY)
ANDRectText = ANDText.get_rect()
ANDRectText.center = (200,155)

NORGate = pygame.image.load("NORGate.png")
NORrect = NORGate.get_rect()
NORrect.center = (50,195)
if activeGate == "NORGate":
    NORText = get_font3(15).render("NOR Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 10:
    NORText = get_font3(15).render("NOR Gate", True, BLACK)
elif tutorialPhase != None:
    NORText = get_font3(15).render("NOR Gate", True, GREY)
NORrectText = NORText.get_rect()
NORrectText.center = (50,230)

NANDGate = pygame.image.load("NANDGate.png")
NANDrect = NANDGate.get_rect()
NANDrect.center = (125,195)
if activeGate == "NANDGate":
    NANDText = get_font3(15).render("NAND Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 10:
    NANDText = get_font3(15).render("NAND Gate", True, BLACK)
elif tutorialPhase != None:
    NANDText = get_font3(15).render("NAND Gate", True, GREY)
NANDrectText = NANDText.get_rect()
NANDrectText.center = (125,230)

XNORGate = pygame.image.load("XNORGate.png")
XNORrect = ANDGate.get_rect()
XNORrect.center = (200,195)
if activeGate == "XNORGate":
    XNORText = get_font3(15).render("XNOR Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 10:
    XNORText = get_font3(15).render("XNOR Gate", True, BLACK)
elif tutorialPhase != None:
    XNORText = get_font3(15).render("XNOR Gate", True, GREY)
XNORrectText = XNORText.get_rect()
XNORrectText.center = (200,230)

XORGate = pygame.image.load("XORGate.png")
XORrect = XORGate.get_rect()
```

```
XORrect.center = (50,270)
if activeGate == "XORGate":
    XORText = get_font3(15).render("XOR Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 10:
    XORText = get_font3(15).render("XOR Gate", True, BLACK)
elif tutorialPhase != None:
    XORText = get_font3(15).render("XOR Gate", True, GREY)
XORrectText = XORText.get_rect()
XORrectText.center = (50,305)

offSwitch = pygame.image.load("off_switch.jpg")
switchRect = offSwitch.get_rect()
switchRect.center = (45,365)
if activeGate == "Switch_OFF":
    switchText = get_font3(15).render("Switch", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 1 or tutorialPhase == 6:
    switchText = get_font3(15).render("Switch", True, BLACK)
elif tutorialPhase != None:
    switchText = get_font3(15).render("Switch", True, GREY)
switchRectText = switchText.get_rect()
switchRectText.center = (45,385)

wireHorizontal = pygame.image.load("wire_horizontal.png")
wireRect = wireHorizontal.get_rect()
wireRect.center = (115,370)
if activeGate == "wireHorizontal":
    wireText = get_font3(15).render("Wire", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 2 or tutorialPhase == 7:
    wireText = get_font3(15).render("Wire", True, BLACK)
elif tutorialPhase != None:
    wireText = get_font3(15).render("Wire", True, GREY)
wireRectText = wireText.get_rect()
wireRectText.center = (115,405)

clock = pygame.image.load("Clock_OFF.png")
clockRect = clock.get_rect()
clockRect.center = (190,370)
if activeGate == "Clock":
    clockText = get_font3(15).render("Clock", True, BLUE)
elif tutorialPhase != None:
    clockText = get_font3(15).render("Clock", True, GREY)
else:
    clockText = get_font3(15).render("Clock", True, BLACK)
clockTextRect = clockText.get_rect()
```

```
clockTextRect.center = (190,405)
```

```
offLight = pygame.image.load("off_bulb.png")
lightRect = offLight.get_rect()
lightRect.center = (47,465)
if activeGate == "Light_OFF":
    lightText = get_font3(15).render("Light Bulb", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 4 or tutorialPhase == 9:
    lightText = get_font3(15).render("Light Bulb", True, BLACK)
elif tutorialPhase != None:
    lightText = get_font3(15).render("Light Bulb", True, GREY)
lightRectText = lightText.get_rect()
lightRectText.center = (47,500)
```

```
DFlipFlop = pygame.image.load("DflipFlop.png")
DFlipFlopRect = DFlipFlop.get_rect()
DFlipFlopRect.center = (47,575)
if activeGate == "DFlipFlop":
    DFlipFlopText = get_font3(15).render("Edge D Type", True, BLUE)
elif tutorialPhase == None:
    DFlipFlopText = get_font3(15).render("Edge D Type", True, BLACK)
else:
    DFlipFlopText = get_font3(15).render("Edge D Type", True, GREY)
DFlipFlopTextRect = DFlipFlopText.get_rect()
DFlipFlopTextRect.center = (47,610)
```

```
#block transfer of image and text
```

```
SCREEN.blit(ORGate, ORrect)
SCREEN.blit(ORText, ORrectText)
```

```
SCREEN.blit(NOTGate, NOTrect)
SCREEN.blit(NOTText, NOTrectText)
```

```
SCREEN.blit(ANDGate, ANDrect)
SCREEN.blit(ANDText, ANDrectText)
```

```
SCREEN.blit(NORGate, NORrect)
SCREEN.blit(NORText, NORrectText)
```

```
SCREEN.blit(NANDGate, NANDrect)
SCREEN.blit(NANDText, NANDrectText)
```

```
SCREEN.blit(XNORGate, XNORrect)
SCREEN.blit(XNORText, XNORrectText)
```

```
SCREEN.blit(XORGate, XORrect)
SCREEN.blit(XORText, XORrectText)
```

```
SCREEN.blit(offSwitch, switchRect)
SCREEN.blit(switchText, switchRectText)
```

```
SCREEN.blit(offLight, lightRect)
SCREEN.blit(lightText, lightRectText)
```

```
SCREEN.blit(wireHorizontal, wireRect)
SCREEN.blit(wireText, wireRectText)
```

```
SCREEN.blit(clock, clockRect)
SCREEN.blit(clockText, clockTextRect)
```

```
SCREEN.blit(DFlipFlop, DFlipFlopRect)
SCREEN.blit(DFlipFlopText, DFlipFlopTextRect)
```

```
#input checker
for event in pygame.event.get():
    if event.type == pygame.MOUSEBUTTONDOWN:
        #pixel position determined
        mouseX = event.pos[0]
        mouseY = event.pos[1]
        if event.button == 1:
            #logic menu input checker
            if mouseX >= 25 and mouseX <= 75 and mouseY >= 100 and
mouseY <= 140:
                #if gate is already selected, gate resets
                if activeGate == "ORGate":
                    activeGate = None
                #if no gate or a gate other than an ORgate is selected,
ORGate becomes active for placement
            else:
                activeGate = "ORGate"
            elif mouseX >= 99 and mouseX <= 125 and mouseY >= 100
and mouseY <= 140:
                if activeGate == "NOTGate":
                    activeGate = None
```

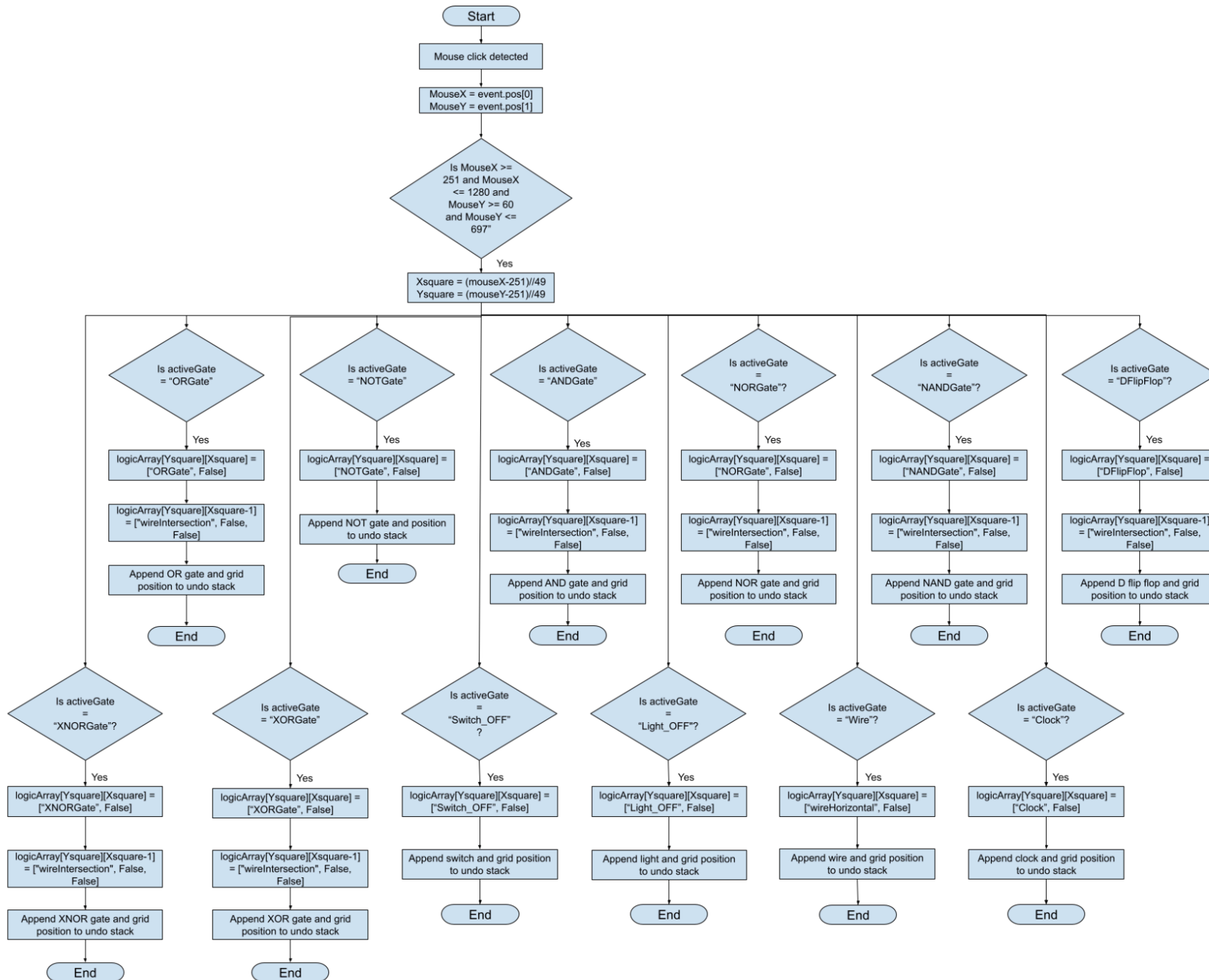
```
        else:
            activeGate = "NOTGate"
        elif mouseX >= 175 and mouseX <= 225 and mouseY >= 100
and mouseY <= 140:
            if activeGate == "ANDGate":
                activeGate = None
            else:
                activeGate = "ANDGate"
        elif mouseX >= 25 and mouseX <= 75 and mouseY >= 175
and mouseY <= 215:
            if activeGate == "NORGate":
                activeGate = None
            else:
                activeGate = "NORGate"
        elif mouseX >= 99 and mouseX <= 125 and mouseY >= 175
and mouseY <= 215:
            if activeGate == "NANDGate":
                activeGate = None
            else:
                activeGate = "NANDGate"
        elif mouseX >= 175 and mouseX <= 225 and mouseY >= 175
and mouseY <= 215:
            if activeGate == "XNORGate":
                activeGate = None
            else:
                activeGate = "XNORGate"
        elif mouseX >= 25 and mouseX <= 75 and mouseY >= 250
and mouseY <= 290:
            if activeGate == "XORGate":
                activeGate = None
            else:
                activeGate = "XORGate"
        elif mouseX >= 20 and mouseX <= 69 and mouseY >= 355
and mouseY <= 375:
            if activeGate == "Switch_OFF":
                activeGate = None
            else:
                activeGate = "Switch_OFF"
        elif mouseX >= 27 and mouseX <= 67 and mouseY >= 440
and mouseY <= 491:
            if activeGate == "Light_OFF":
                activeGate = None
            else:
                activeGate = "Light_OFF"
```

```
        elif mouseX >= 90 and mouseX <= 140 and mouseY >= 345
and mouseY <= 395:
            if activeGate == "wireHorizontal":
                activeGate = None
            else:
                activeGate = "wireHorizontal"
        elif mouseX >= 165 and mouseX <= 215 and mouseY >= 348
and mouseY <= 398:
            if activeGate == "Clock":
                activeGate = None
            else:
                activeGate = "Clock"
        elif mouseX >= 25 and mouseX <= 75 and mouseY >= 550 and
mouseY <= 605:
            if activeGate == "DFlipFlop":
                activeGate = None
            else:
                activeGate = "DFlipFlop"
```

Grid placement

Once an element has been selected, it can be placed onto the document. In order to do so, when an input is registered on the logic grid, the program calculates the exact grid square that has been selected relative to the logic array variables positions. The program then checks what gate is active at the moment, places it onto the grid alongside a wire intersection element if the gate accepts two inputs. This action is also pushed onto the undo stack. Below is a flowchart and example code for this algorithm:

Logic gate simulator - Harry Taylor - 7173 - 65301



While True:

for event in pygame.event.get():

if event.type == pygame.MOUSEBUTTONDOWN:

#mouse click pixel position determined

mouseX = event.pos[0]

mouseY = event.pos[1]

if event.button == 1:

if mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and mouseY <= 697:

#grid position is calculated

Xsquare = (mouseX-251)//49

Ysquare = (mouseY-60)//49

print(Xsquare)

print(Ysquare)

if activeGate != None:

redoStack = []

if activeGate == "ORGate":

#OR gate is added to grid

logicArray[Ysquare][Xsquare] =

["ORGate", False]

#wire intersection is added to grid,

this is the same for all 2 output gates

logicArray[Ysquare][Xsquare-1] =

["wireIntersection", False, False]

#active gate is reset

activeGate = None

#action is added to undo stack

undoStack.append(["remove",

"ORGate", Ysquare, Xsquare])

print(activeGate)

elif activeGate == "NOTGate":

#NOT gate is added to grid

logicArray[Ysquare][Xsquare] =

["NOTGate", False, False]

activeGate = None

undoStack.append(["remove",

"NOTGate", Ysquare, Xsquare])

print(activeGate)

elif activeGate == "ANDGate":

#AND gate is added to grid

logicArray[Ysquare][Xsquare] =

["ANDGate", False]

logicArray[Ysquare][Xsquare-1] =

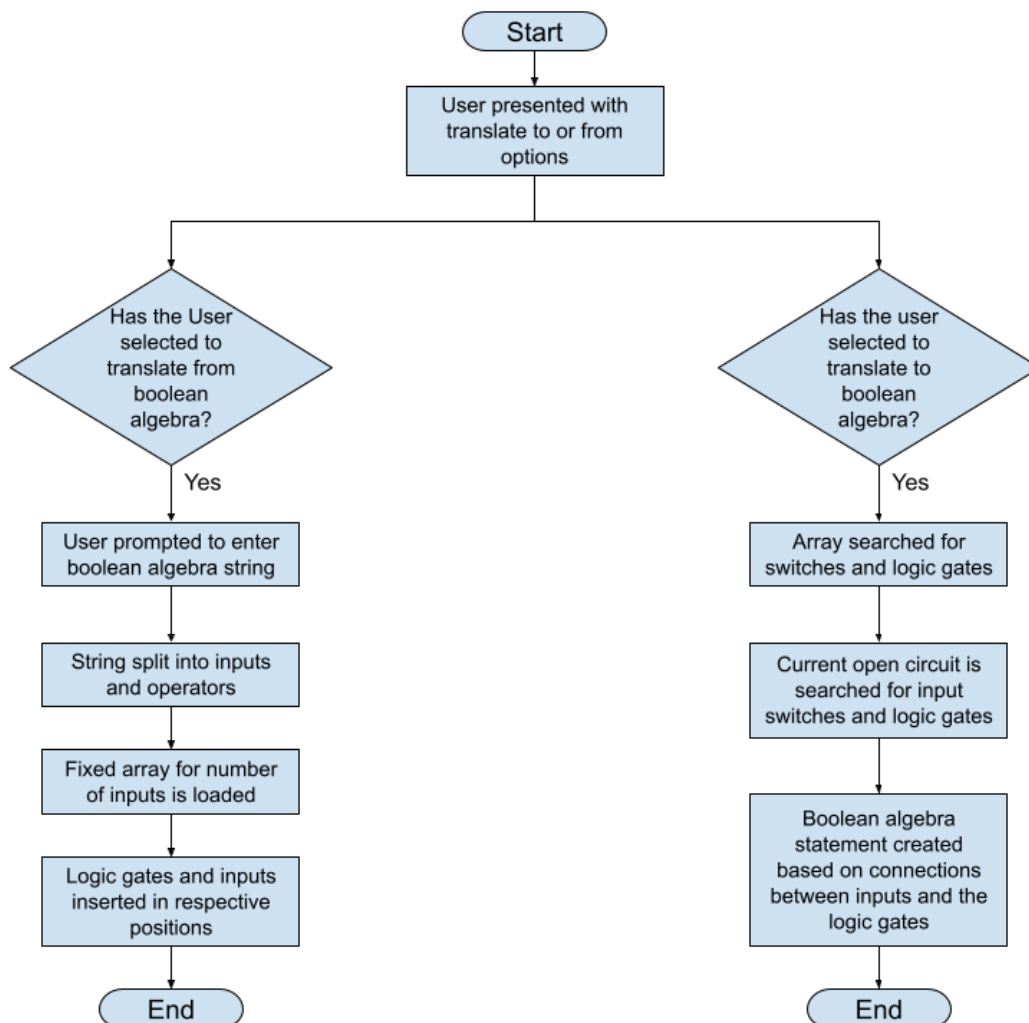
["wireIntersection", False, False]

```
        activeGate = None
        undoStack.append(["remove",
"ANDGate", Ysquare, Xsquare])
        print(activeGate)
        elif activeGate == "NORGate":
            #NOR gate is added to grid
            logicArray[Ysquare][Xsquare] =
["NORGate", False]
            logicArray[Ysquare][Xsquare-1] =
["wireIntersection", False, False]
            activeGate = None
            undoStack.append(["remove",
"NORGate", Ysquare, Xsquare])
            print(activeGate)
            elif activeGate == "NANDGate":
                #NAND gate is added to grid
                logicArray[Ysquare][Xsquare] =
["NANDGate", False]
                logicArray[Ysquare][Xsquare-1] =
["wireIntersection", False, False]
                activeGate = None
                undoStack.append(["remove",
"NANDGate", Ysquare, Xsquare])
                print(activeGate)
                elif activeGate == "XNORGate":
                    #XNOR gate is added to grid
                    logicArray[Ysquare][Xsquare] =
["XNORGate", False]
                    logicArray[Ysquare][Xsquare-1] =
["wireIntersection", False, False]
                    activeGate = None
                    undoStack.append(["remove",
"XNORGate", Ysquare, Xsquare])
                    print(activeGate)
                    elif activeGate == "XORGate":
                        #XOR gate is added to grid
                        logicArray[Ysquare][Xsquare] =
["XORGate", False]
                        logicArray[Ysquare][Xsquare-1] =
["wireIntersection", False, False]
                        activeGate = None
                        undoStack.append(["remove",
"XORGate", Ysquare, Xsquare])
                        print(activeGate)
```

```
elif activeGate == "Light_OFF":
    #light is added to grid
    logicArray[Ysquare][Xsquare] =
["Light_OFF", False]
    activeGate = None
    undoStack.append(["remove",
"Light_OFF", Ysquare, Xsquare])
    print(activeGate)
elif activeGate == "Switch_OFF":
    #switch is added to grid
    logicArray[Ysquare][Xsquare] =
["Switch_OFF", False]
    activeGate = None
    undoStack.append(["remove",
"Switch_OFF", Ysquare, Xsquare])
    print(activeGate)
elif activeGate == "wireHorizontal":
    #horizontal wire is added to grid
    logicArray[Ysquare][Xsquare] =
["wireHorizontal", False, None]
    undoStack.append(["remove",
"wireHorizontal", Ysquare, Xsquare])
elif activeGate == "Clock":
    #clock is added to grid
    logicArray[Ysquare][Xsquare] =
["Clock", False]
    activeGate = None
    undoStack.append(["remove",
"Clock", Ysquare, Xsquare])
elif activeGate == "DFlipFlop":
    #D type flip flop is added to grid
    logicArray[Ysquare][Xsquare] =
["DFlipFlop", False]
    logicArray[Ysquare][Xsquare-1] =
["wireIntersection", False, False]
    activeGate = None
    undoStack.append(["remove",
"DFlipFlop", Ysquare, Xsquare])
```

Boolean algebra translation

Whilst not included in my final project due to its complexity, the boolean algebra translation tool was going to allow users to input a string of boolean algebra, which the simulator would then interpret and display in the form of a logic simulator circuit, this feature would likely have been accessible from both the main menu and options bar within an open document. The functionality would also allow for user created circuits within the simulator to be translated to boolean expressions from the options menu. Below is a flowchart for how this feature may have worked:



Button functionality

Within all menus in the program, buttons are utilised by the user for accessing other menus and functionality within the main interface. To code the button behaviour I created a button class following a YouTube tutorial and imported it into my main program. This class defines: the position, text displayed, font used, and colour both when hovering the cursor over the button and not hovering as attributes. When a new button is created, a method is called placing it in its intended position and allowing user input. The program also continuously checks the mouse position and if it coincides with the position of a button, the colour of the button changes. If the user clicks on the button, the function of that button is executed. Below is the class diagram for the button class:

Button
- Image: surface - x_Pos: integer - y_Pos: integer - Font: object - Base_color: string - Text_input: string - Text: surface - Rect: rect - Text_rect: rect
+ update(screen) + checkForInput(position) + changeColor(position)

Below is the class code itself:

```
class Button():
    def __init__(self, pos, text_input, font, base_color, hovering_color):
        self.image = None
        self.x_pos = pos[0]
        self.y_pos = pos[1]
        self.font = font
        self.base_color, self.hovering_color = base_color, hovering_color
        self.text_input = text_input
        self.text = self.font.render(self.text_input, True, self.base_color)
        if self.image is None:
            self.image = self.text
        self.rect = self.image.get_rect(center=(self.x_pos, self.y_pos))
        self.text_rect = self.text.get_rect(center=(self.x_pos, self.y_pos))

    def update(self, screen):
        if self.image is not None:
            screen.blit(self.image, self.rect)
        screen.blit(self.text, self.text_rect)

    def checkForInput(self, position):
        if position[0] in range(self.rect.left, self.rect.right) and position[1] in
            range(self.rect.top, self.rect.bottom):
            return True
        return False

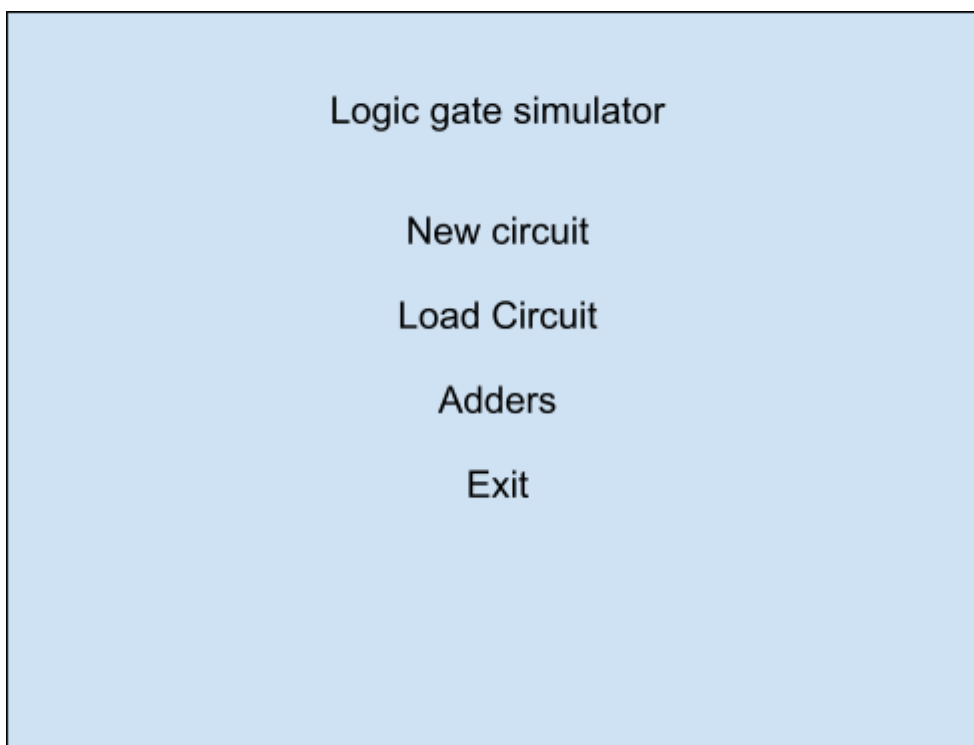
    def changeColor(self, position):
        if position[0] in range(self.rect.left, self.rect.right) and position[1] in
            range(self.rect.top, self.rect.bottom):
            self.text = self.font.render(self.text_input, True,
                self.hovering_color)
        else:
            self.text = self.font.render(self.text_input, True, self.base_color)
```

User interface

Main menu

Given the wide amount of features I plan to include in my simulator users will require a method of intuitively accessing them. This is why I have elected to design an efficient user interface where users can access logic gates, inputs and outputs to drag them into the simulator instance. The interface is also to include the ability to undo and redo work. A file management system for saving and opening files will also be included in this menu.

A visual first draft of my opening menu is as follows:



As shown, my main menu contains multiple options upon startup: new circuit, which opens a new circuit document for the user to create a circuit on, load circuit, which opens the file explorer menu for the user to enter a pre-existing saved document, adders, which opens a premade file of either a half or full adder circuit and exit, which closes the application.

The first section of code I started working on was for the user interface for the main menu within my simulator, so that I could design the general look and feel of the simulator. My main menu has a simple design with a large title alongside buttons for each of the main menu's functions. Hovering over each button on the interface changes its colour. Clicking on each menu button presents the user with that options menu, or a placeholder one in the case of early development. This menu has been made using a separate code document containing a button class, which is imported to the main user interface file and used in defining each button by its text, font size, colour, and colour the button changes to when hovering over it.

I begun by following a tutorial for a game menu with a play menu, options menu, and exit button, producing this result:



Given that this menu doesn't really apply to my project, nor does the font match what I would like of the simulator, I altered the code and font to produce a user interface similar to the one drafted above, producing this result:

Logic Gate Simulator

New File

Load File

Tutorial

Adders

Quit

As shown, the names of the buttons have been changed, and 2 extra buttons have been added for the adder menu and the tutorial menu.

Logic interface

A visual first draft of my logic interface is as follows:



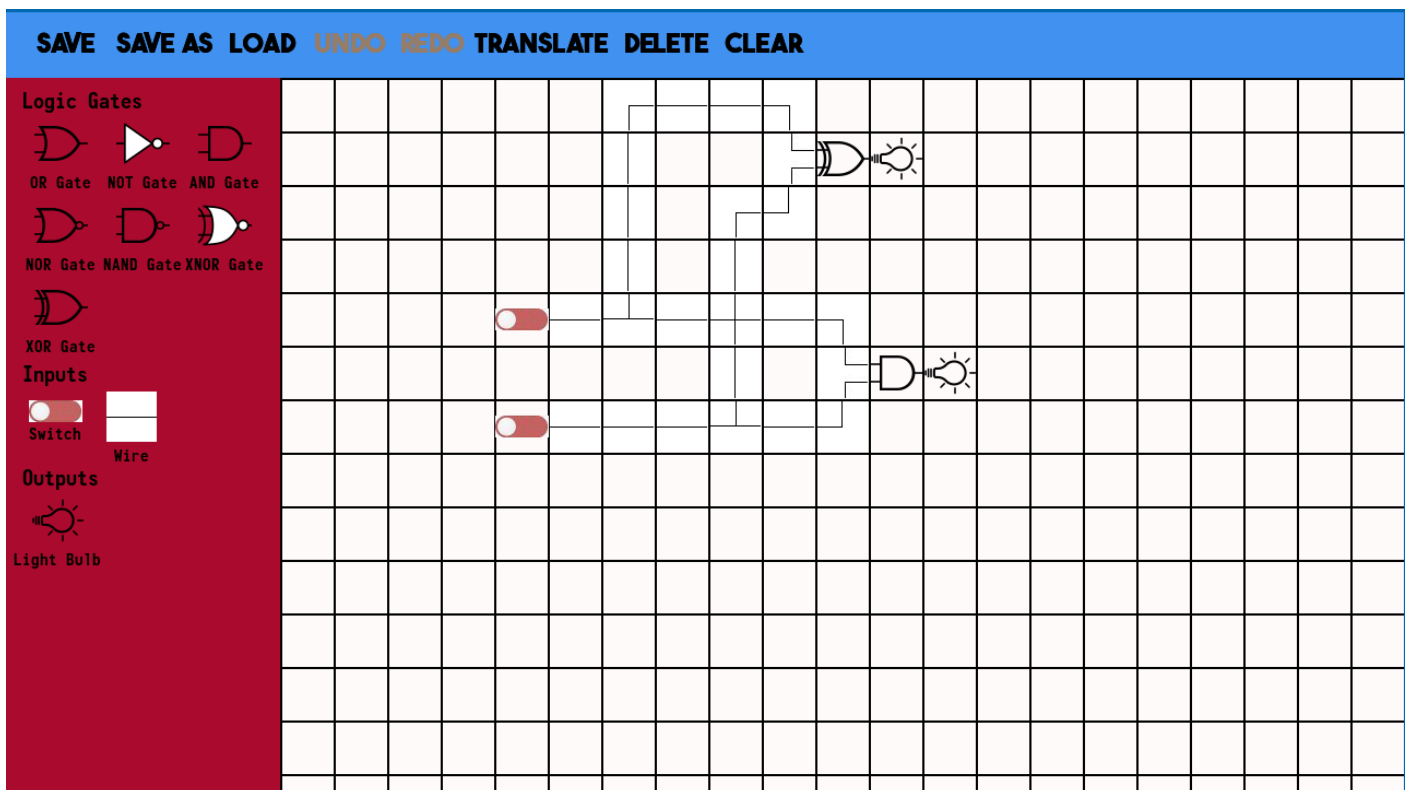
As shown, on the left I intended on including inputs, outputs and logic gates which were initially planned to be dragged into place within the grid. However I decided to allow the user to select a logic element on the left menu, causing the text below it to change colour to indicate its selection, then click on a grid square to place it within the document.

The grid idea was chosen as the method for users to create their logic circuits so that each element can connect seamlessly to one another and so that a save function can be easily implemented utilising a 2D array which can be saved via pickle. The top menu is to include functions such as file management, as well as the translation tool for logic systems to boolean algebra or truth tables, options that can be chosen by the user to execute their specific function, or open a submenu.

For the user to create their custom logic circuits, I needed to develop the interface needed for the user to interact with. This allows users to edit their circuits, test them and access features of the options menu. My logic interface contains a large grid for users to place and customise their logic circuits, with a bar along the top for the interface's options, such as saving, loading, deleting logic elements and translating the current circuit into a truth table or boolean algebra selecting these options executes their specific function. Along the left hand side I placed the logic elements that can be selected to be placed within a user's document. These can be selected and placed onto the document by clicking an element on the menu then a corresponding grid square. Below is an example of this menu at a late development stage:

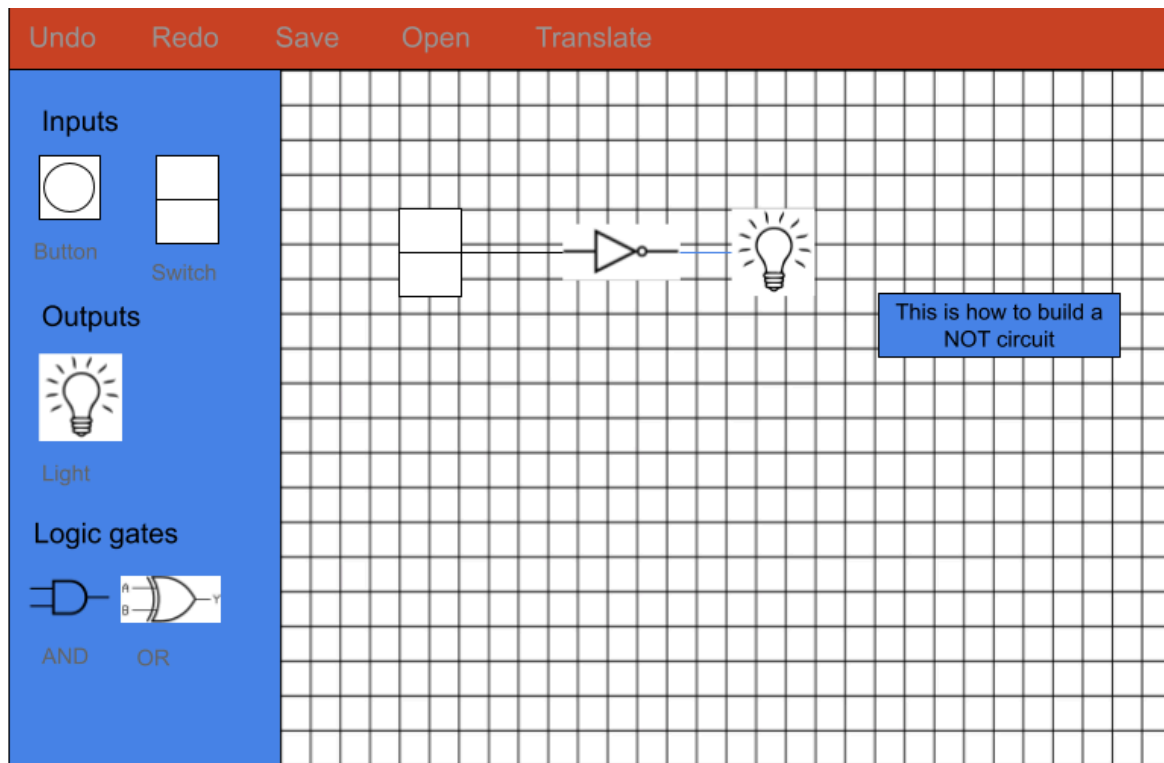


This is the same interface with the grid populated with a logic circuit:



Tutorial

A visual first draft of my tutorial is as follows:



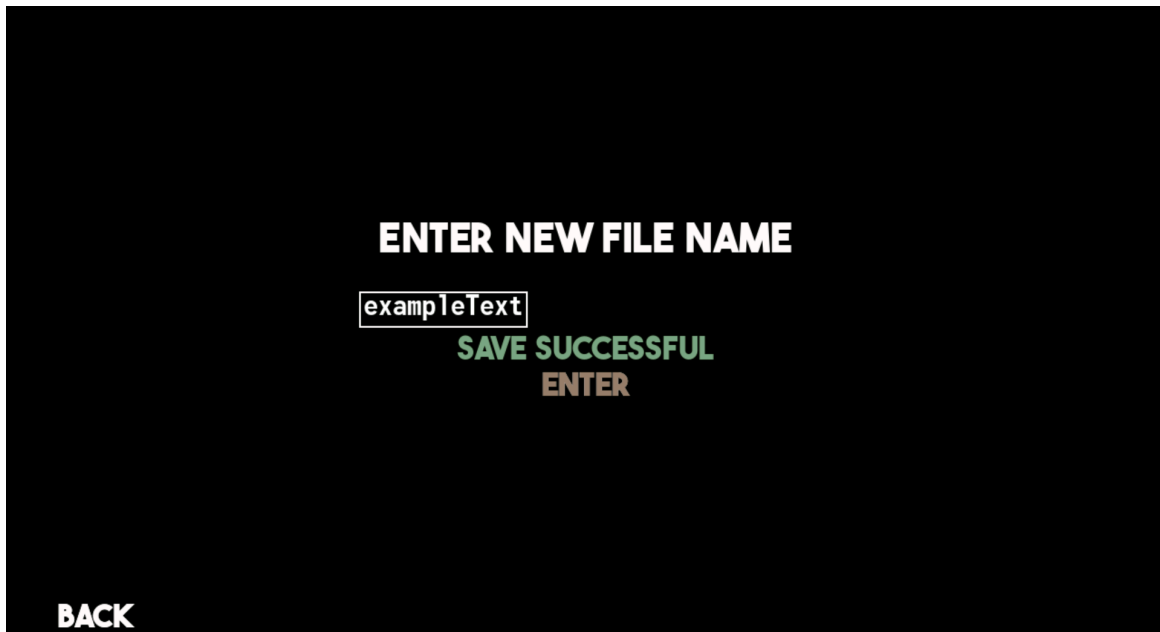
As shown, the tutorial system is displaying a NOT gate circuit. This is to be built by the user based on instruction from the blue text box on the right. The options menu and gates have grey text and are blocked from the user depending on the stage in the tutorial. This makes the experience less complicated for the user as they not only have text to prompt them what to place, but also a specific logic element to click on to place in the circuit. When the user is prompted to place an element a square or squares in the grid light up indicating where to place the logic element in the circuit.

Save menu

To allow the user to create a custom save name to save a logic file to, I developed a simple yet intuitive save menu, allowing the user to accomplish such a task. The menu opens by selecting the save or save as options whilst in the logic interface menu. The user is then prompted by the menu to enter a name for the file, when they do so, their input appears in a rectangle below the prompt. Once the user has finished inputting their file name, they can select the enter option. This prompts the user of a successful save before returning them to the document they had been working on. At any point the user can also select the back option, returning them to the document they were working on below is an example of this menu:



Below is the success prompt a user receives for saving a file:



Load menu

For the user to load their saved files, another menu is needed for the user to input the text name of the file they wish to view/edit. Whilst fundamentally similar to the save menu, this menu includes exception handling, since the user could input an invalid file name to load. When this happens an error message appears showing the file as not found. If the file name is valid, the file opens for the user to manipulate. Below is an example of this menu:



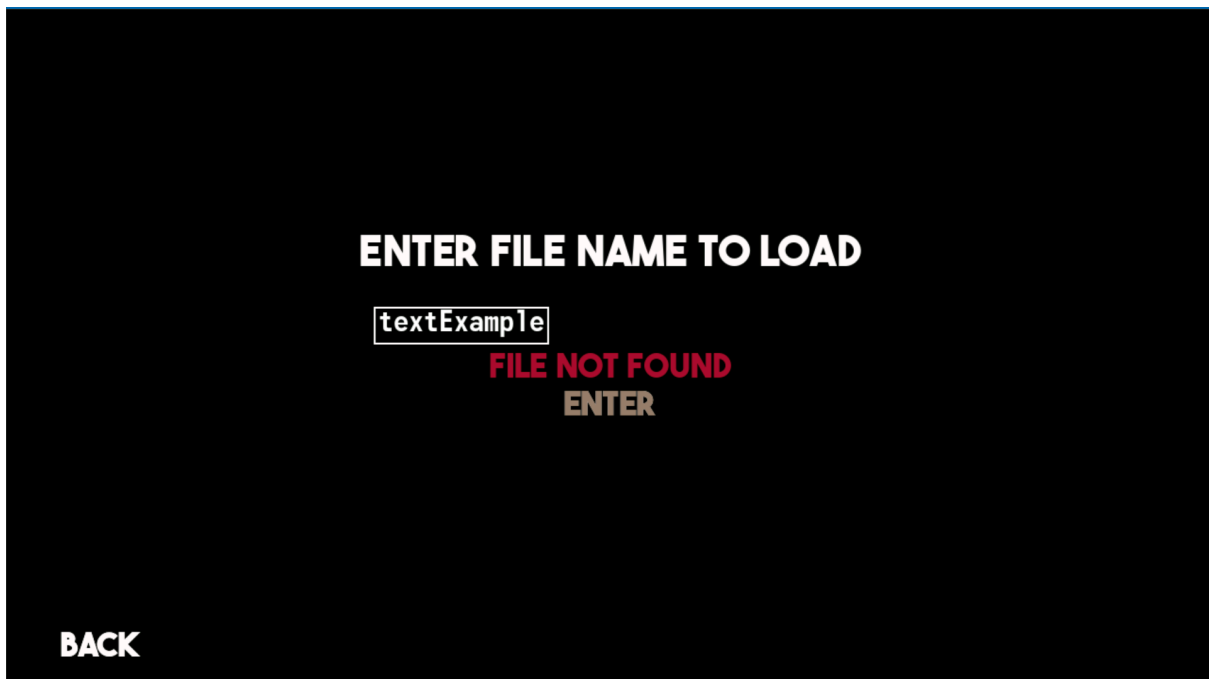
ENTER FILE NAME TO LOAD

textExample

ENTER

BACK

Below is the file not found prompt a user receives for entering an invalid file name:



ENTER FILE NAME TO LOAD

textExample

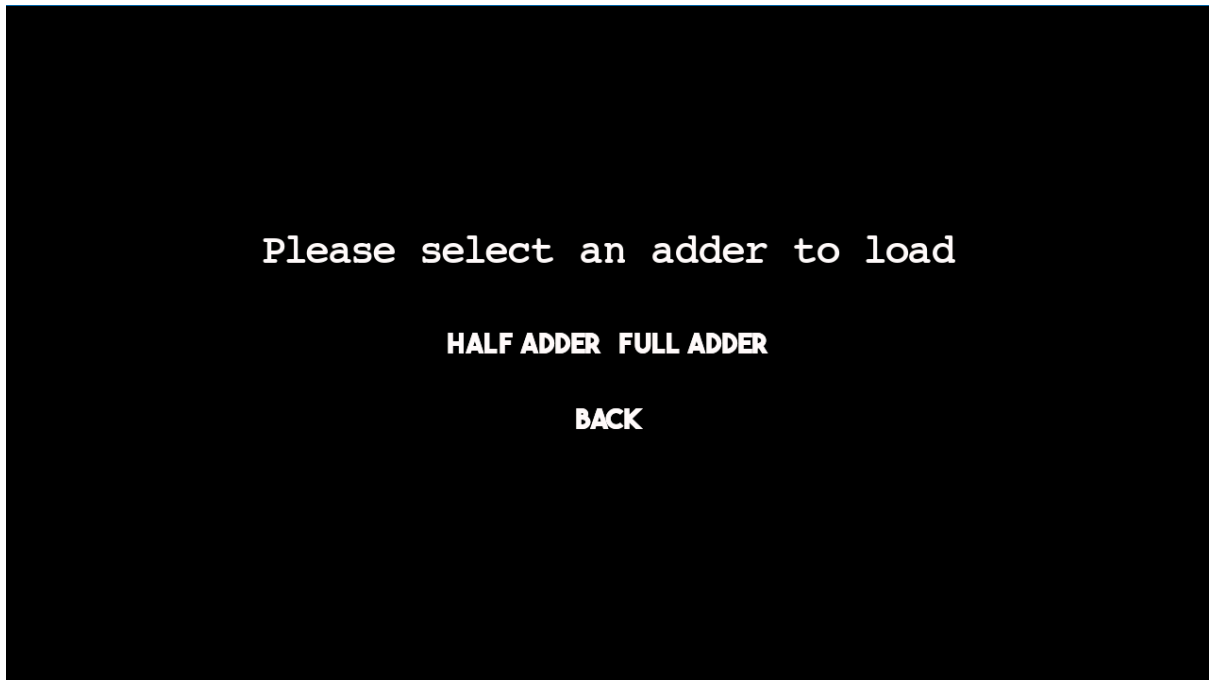
FILE NOT FOUND

ENTER

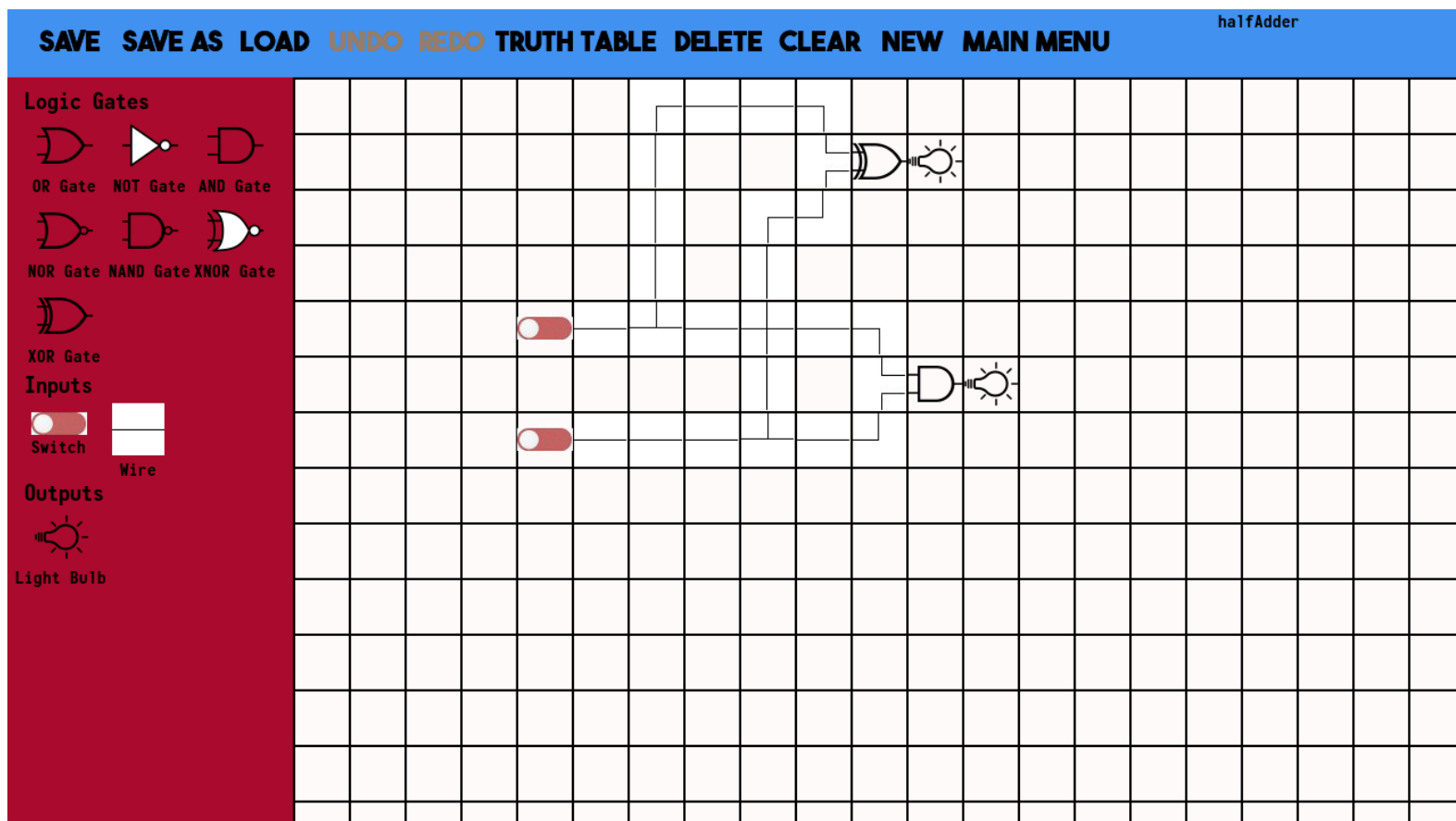
BACK

Adders

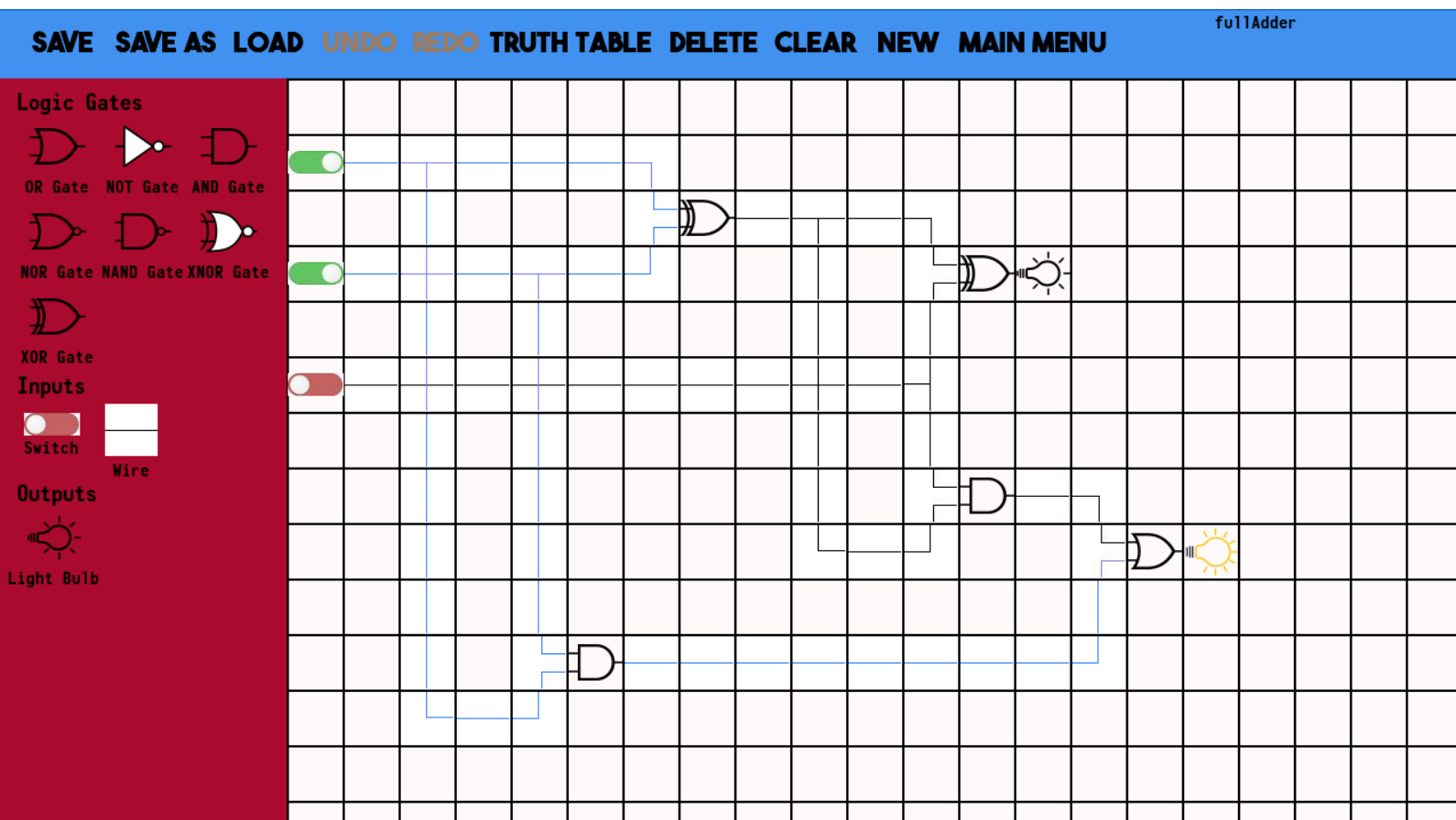
For both an example of what can be achieved using my simulator, and as a learning tool, I chose to include prebuilt adders for the user to access from the main menu. The user is presented with the option of either a half or full adder, and upon selection the adder of their choice is loaded as a document within the logic interface. These adders are loaded in from binary files in the same way as if the user were to save and load a document themselves. This means that the user can make changes to the document and save it themselves. Below are examples of the adder menu, as well as the adder documents that are produced:



This is the half adder produced by selecting it as an option:



This is the full adder produced by selecting it as an option:



Truth tables

When truth tables are generated using the main logic stemming from the truth table button. The translate menu creates a grid filled with the values from the truth tables 3D array data structure. The size of the grid is based on the amount of switch combinations possible for the circuit configuration as well as the combined amount of switches and light bulbs. The states of inputs and outputs are placed in the centre of each grid square. The colour of the font of the states are different between inputs and outputs with inputs having white text and outputs having grey text. The grid also includes headers displaying the input and output columns. As expected a back button can be used by the user to return to their original document. Below is a screenshot of an example truth table generated for a full adder:

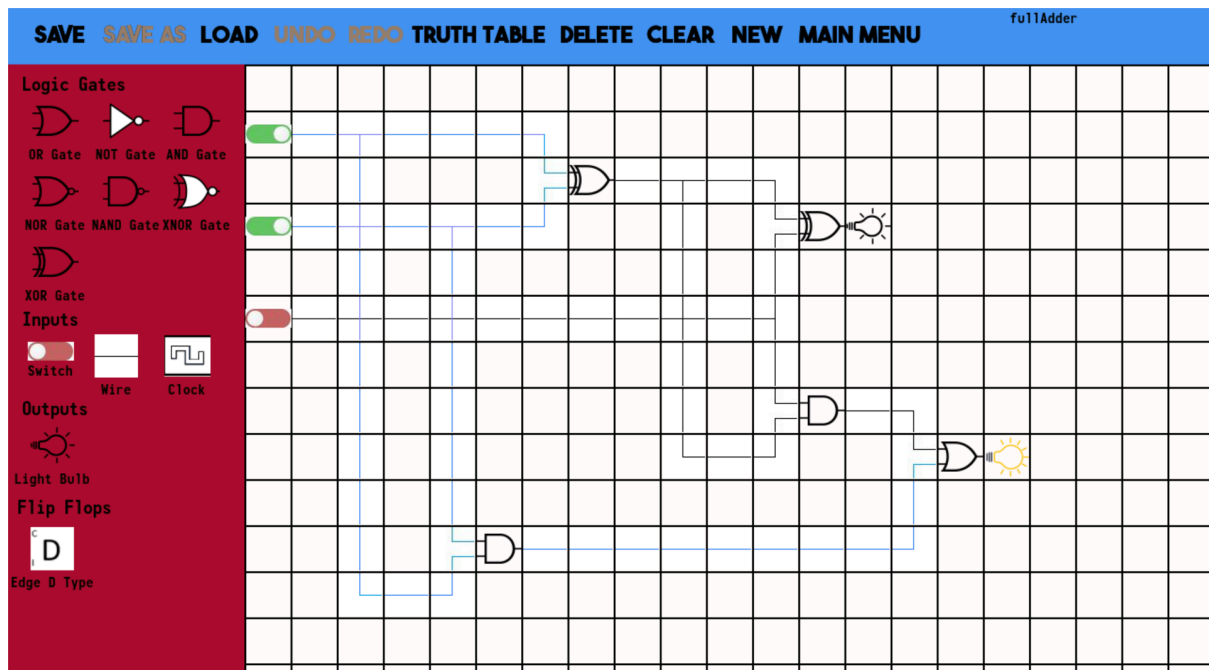
Inputs		Outputs		
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

BACK

Persistent storage

Saving

Use of saving within my simulator is relatively simple, due to there only needing to be one variable to save (main logic array). This is because the rest of my variables can be recreated through the main logic of the program. Due to this I am using the pickle package allowing me to create a file based on a user inputted name, dump the main logic array and then load the file at the users request. Below is a screenshot of an example circuit built in my simulator, and what the pickle python save file looks like as a result of saving the circuit:

[illegible]

Testing

Test plan

In order to effectively and efficiently troubleshoot my project code, I elected to devise a test plan which will outlay the components and whether they function in usability which determines how easy it is to use the system and what error messages pop up in case of an unexpected input, robustness which determines whether the function works under extreme or erroneous conditions and functionality tests which determine how the program functions under normal conditions. If the component does not function properly, the errors sustained can be logged so that they can be resolved quickly.

Objective 1: User opens simulation

Test objective	Test number	Test type	Test input	Test description	Expected output	Actual Result and timestamp
1.2.1	1	Functionality	Selecting new document option	On startup the player should be presented with the option of loading a new document	New simulator document opens	Pass
1.3.1	1	Functionality	Selecting load document option	On startup the user should be presented with the option of loading an existing document	Load file menu opens	Pass
1.3.1.1	1	Functionality	User can type in the name of an existing document and press	The load menu allows a user to open a previously saved	Saved document opens	Pass

			enter	document		
1.3.1.1.6.	1	Robu... ▾	User can type in the name of an existing document and press enter	The load menu allows a user to open a previously saved document	Error message is displayed	Pass ▾
1.3.1.2	1	Func... ▾	Selecting back option in file explorer menu	In the existing document menu the user should be presented with an option of backing out of the document	Main menu reopens	Pass ▾
1.4.1.	1	Func... ▾	Selecting adder option	On startup the user should be presented with the option of opening an adder menu	Adder menu opens	Pass ▾
1.4.1.1.	1	Func... ▾	Selecting half adder option	Within the adders menu the user is presented with half-adder and full-adder options	Half adder loads	Pass ▾
1.4.1.2.	1	Func... ▾	Selecting full-adder option	Within the adders menu the user is presented with half-adder	Full adder loads	Pass ▾

				and full-adder options		
1.4.1.3.	1	Functionality ▾	Selecting back option	Within the adders menu the user is presented with half-adder and full-adder options	Main menu is displayed	Pass ▾
1.5.1.	1	Functionality ▾	Selecting tutorial option	On startup the user should be presented with the option of opening a tutorial menu	Tutorial menu opens	Pass ▾
1.5.1.1.	1	Functionality ▾	User follows a tutorial demonstrating how to build a NOT gate circuit	Within the tutorial menu, the user should first be shown how to build a NOT circuit	User moves onto building an AND gate circuit	Pass ▾
1.5.1.1.	2	Robustness ▾	Users attempts to place logic elements in non specified areas	Within the tutorial menu, the user should first be shown how to build a NOT circuit	Nothing is placed	Pass ▾
1.5.1.1.	3	Robustness ▾	User attempts to click on logic gates in the menu that are not	Within the tutorial menu, the user should first be shown how to build a	Gate does not become active for placement	Pass ▾

			supposed to be placed	NOT circuit		
1.5.1.1.1.	1	Functionality	User changes the state of the switch	After creating a NOT circuit in the tutorial, users can test the circuit	Output changes state depending on the NOT gate	Pass
1.5.1.2.	1	Functionality	User follows a tutorial demonstrating how to build a NOT gate circuit	Within the tutorial menu, the user should first be shown how to build an AND circuit	User moves onto building any other type of circuit	Pass
1.5.1.2.	2	Robustness	User attempts to place logic elements in non specified areas	Within the tutorial menu, the user should first be shown how to build an AND circuit	Nothing is placed	Pass
1.5.1.2.1.	1	Functionality	User changes the state of the switches	After creating an AND circuit in the tutorial, users can test the circuit	Output changes state depending on the AND gate	Pass
1.5.1.3.1.	1	Functionality	User changes the AND logic gate with any other logic gate	Users can create their own custom circuits by changing the AND gate for a different logic gate	Logic gate changes to user selected gate	Pass

1.5.1.3.1.	2	Robu... ▾	User attempts to place a logic element that is not allowed in place of the AND gate (e.g. wire)	Users can create their own custom circuits by changing the AND gate for a different logic gate	AND gate doesn't change	Pass ▾
1.5.1.3.2.	1	Funci... ▾	User changes the state of the switches	After changing the gate in the circuit in the tutorial, users can test the circuit	Output changes state depending on the gate of the users choosing	Pass ▾
1.5.1.3.3.	1	Funci... ▾	User presses the next button	After changing the gate in the circuit in the tutorial, users can test the circuit	User returns to the main menu	Pass ▾
1.5.2.	1	Funci... ▾	User presses the back button	During the course of the tutorial the user is presented with a back button to return to the main menu	User returns to the main menu	Pass ▾
1.6.1.	1	Funci... ▾	Selecting exit option	On startup the player should be presented with the option of exiting the program	Program ends	Pass ▾

Objective 2: Options menu

Test objective	Test number	Test type	Test input	Test description	Expected output	Actual Result and timestamp
2.1.1	1	Funct... ▾	Selecting undo option with items in document	Options menu presents user with undo option	Most recent logic element is removed	Pass ▾
2.1.2.	1	Funct... ▾	Selecting undo option with no items in document	Options menu presents user with undo option	Selecting undo does nothing	Pass ▾
2.1.2.	2	Robu... ▾	Repeatedly selecting redo option with no items in document	Options menu presents user with undo option	Selecting undo does nothing	Pass ▾
2.2.1.	1	Funct... ▾	Selecting redo with items on redo stack	Options menu presents user with redo option	Most recently undone item is added back to the document	Pass ▾
2.2.2.	1	Funct... ▾	Selecting redo with no items on undo stack	Options menu presents user with redo option	Selecting redo does nothing	Pass ▾
2.2.2.	2	Robu... ▾	Selecting redo repeatedly with no items on undo stack	Options menu presents user with redo option	Selecting redo does nothing	Pass ▾
2.3.1.	1	Funct... ▾	Selecting	Options	Selecting	Pass ▾

			save on a document that has been previously saved	menu presents user with save option	save overwrites the current data stored for the file with the current document state	
2.3.1	2	Robu... ▾	Repeatedly selecting save on a document that has been previously saved	Options menu presents user with save option	Selecting save overwrites the current data stored for the file with the current document state	Pass ▾ Document freezes for a few seconds, but resumes functionality
2.3.1	3	Usabi... ▾	Document that has been previously saved is deleted	Options menu presents user with save option	User is presented with an option to resave the document	Pass ▾ Passed after fix; initial problem was that the document was resaved in the directory folder under the original file name
2.3.2.	1	Funct... ▾	Selecting save on a document that has not been previously saved	Options menu presents user with save option	Selecting save opens the save as menu for a user to save a new document	Pass ▾
2.4.1	1	Funct... ▾	Selecting save as on a document that has	Options menu presents user with save as	Selecting save as opens the save as menu for a	Pass ▾

			not been previously saved	option	user to save a new document	
2.4.1	2	Funct... ▾	Selecting save as on a document that has been previously saved	Options menu presents user with save as option	Selecting save as opens the save as menu for a user to save a new document	Pass ▾
2.4.1.1.	1	Funct... ▾	Selecting enter with text input present	Save as menu allows user to save a new file	Selecting enter saves document with name specified by user	Pass ▾
2.4.1.1.	2	Usabi... ▾	Selecting enter key with text input present	Save as menu allows user to save a new file	Selecting enter saves document with name specified by user	Pass ▾
2.4.1.2.	1	Funct... ▾	Selecting enter with no text input present	Save as menu allows user to save a new file	Selecting enter does nothing	Pass ▾
2.4.1.3.	1	Funct... ▾	Selecting back button	Save as menu allows user to save a new file	Re-opens the logic document the user was currently working on	Pass ▾
2.5.1.	1	Funct... ▾	Load option is selected	Options menu presents user with	Selecting load presents user with	Pass ▾

				load option	load menu to open a pre-existing document	
2.5.1.1.	1	Funct... ▾	Enter button is pressed with valid text input	Load menu allows user to input a file to load	Selecting enter loads document with name specified by user	Pass ▾
2.5.1.1.	2	Usabi... ▾	Enter key is pressed with valid text input	Load menu allows user to input a file to load	Selecting enter loads document with name specified by user	Pass ▾
2.5.1.2.	1	Robu... ▾	Enter button is pressed with invalid text input	Load menu allows user to input a file to load	Selecting enter presents a "not found" error message	Pass ▾
2.5.1.3.	1	Funct... ▾	Enter button is pressed without text input	Load menu allows user to input a file to load	Selecting enter does nothing	Pass ▾
2.5.1.4.	1	Funct... ▾	Selecting back button	Load menu allows user to input a file to load	Re-opens the logic document the user was currently working on	Pass ▾
2.6.1.	1	Funct... ▾	Current document is cleared	Options menu presents user with clear option	Document is cleared	Pass ▾

2.7.1.	1	Funct... ▾	Translate option is selected whilst a completed logic circuit has been created	Options menu presents user with an option to translate their circuit to boolean algebra	Logic circuit is translated to boolean algebra	Fail ▾ Functionality not included in final project
2.7.2.	1	Funct... ▾	Translate option is selected whilst a completed logic circuit has not been created	Options menu presents user with an option to translate their circuit to boolean algebra	Selecting translate option does nothing	Fail ▾ Functionality not included in final project
2.7.2.	2	Robu... ▾	Translate option is repeatedly selected whilst a completed logic circuit has not been created	Options menu presents user with an option to translate their circuit to boolean algebra	Selecting translate option does nothing	Fail ▾ Functionality not included in final project
2.8.1.	1	Funct... ▾	Delete option is selected	Options menu presents user with option to delete logic gates	Delete button becomes highlighted	Pass ▾
2.8.1.1.	1	Funct... ▾	Logic element is selected	Options menu presents user with option to delete logic gates	Chosen element is deleted	Pass ▾
2.8.1.1.	2	Usabi... ▾	Empty square is selected	Options menu presents	Selecting element does	Pass ▾

				user with option to delete logic gates	nothing	
2.8.2.	1	Funct... ▾	Delete option is selected	Options menu presents user with option to delete logic gates	Delete button returns to its original state	Pass ▾
2.9.1.	1	Funct... ▾	Truth table option is selected with completed logic circuit created	Options menu presents user with option to create a truth table	Selecting option generates a truth table for the logic circuit	Pass ▾
2.9.1.	2	Usabi... ▾	Truth table option is selected in a variety of scenarios	Options menu presents user with option to create a truth table	Selecting option generates a truth table for every logic circuit	Pass ▾
2.9.2.	1	Funct... ▾	Truth table option is selected with completed logic circuit not present	Options menu presents user with option to create a truth table	Selecting truth table option does nothing	Pass ▾
2.9.2.	2	Robu... ▾	Truth table option is repeatedly selected with completed logic circuit not present	Options menu presents user with option to create a truth table	Selecting truth table option does nothing	Pass ▾
2.10.1.	1	Funct... ▾	New file button is pressed	Options menu presents user with	Logic document is cleared and file	Pass ▾

				option to create a new document	name is reset	
2.11.1.	1	Funct... ▾	Main menu button is pressed	Options menu presents user with the option to return to the main menu	Main menu is opened	Pass ▾
2.12.1	1	Funct... ▾	Auto save button is pressed when file has not been previously saved	Options menu presents user with the option to auto save their document	Nothing happens	Pass ▾
1.12.1.	2	Funct... ▾	Auto save button is pressed when file has been previously saved	Options menu presents user with the option to auto save their document	Switch turns on and auto save is enabled	Pass ▾
1.12.1.1.	1	Funct... ▾	Auto save is enabled and 30 seconds pass	Options menu presents user with the option to auto save their document	Document is saved again under original name	Pass ▾

Objective 3: Logic interface

Test objective	Test number	Test type	Test input	Test description	Expected output	Actual Result and timestamp
3.1.1.	1	Functi... ▾	User selects switch	User is presented with an	Selected switch becomes	Pass ▾

			input from menu	inputs menu	active for placement	
3.1.1.	2	Funci... ▾	User clicks a square on logic grid with switch selected	User is presented with an inputs menu	Selected switch appears on document in region selected	Pass ▾
3.1.2.	1	Funci... ▾	User selects clock input	User is presented with an inputs menu	Selected clock becomes active for placement	Pass ▾
3.1.2.	2	Funci... ▾	User clicks a square on logic grid with clock selected	User is presented with an inputs menu	Selected clock appears on document in region selected	Pass ▾
3.2.1.	1	Funci... ▾	User clicks on a wire in the logic menu	User is presented with a logic gate menu	Wire becomes active for placement	Pass ▾
3.2.1.	2	Funci... ▾	User clicks on a square on the logic grid with a wire selected	User is presented with a logic gate menu	Wire appears in document	Pass ▾
3.2.2.	1	Funci... ▾	Wire is located below another wire	Wires are able to change orientation based on surrounding wires	Both wires become vertical wires	Pass ▾
3.2.2.	2	Funci... ▾	Wire is located above another	Wires are able to change orientation	Both wires become vertical wires	Pass ▾

			wire	based on surrounding wires		
3.2.3.	1	Funci... ▾	Wires are located above and to the left of another wire	Wires are able to change orientation based on surrounding wires	Wire becomes a bottom right wire	Pass ▾
3.2.3.1.	1	Funci... ▾	Wire to the left of a bottom right wire corner is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a vertical wire	Pass ▾
3.2.3.2.	1	Funci... ▾	Wire above a bottom right wire corner is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a horizontal wire	Pass ▾
3.2.4.	1	Funci... ▾	Wires are located above and to the right of another wire	Wires are able to change orientation based on surrounding wires	Wire becomes a bottom left wire	Pass ▾
3.2.4.1.	1	Funci... ▾	Wire to the right of a bottom left corner is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a vertical wire	Pass ▾
3.2.4.2.	1	Funci... ▾	Wire above a bottom left corner is deleted	Wires are able to change orientation based on surrounding	Wire becomes a horizontal wire	Pass ▾

				wires		
3.2.5.	1	Funci... ▾	Wires are located below and to the left of another wire	Wires are able to change orientation based on surrounding wires	Wire becomes a top right wire	Pass ▾
3.2.5.1.	1	Funci... ▾	Wire to the left of the top right wire is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a vertical wire	Pass ▾
5.2.5.2.	1	Funci... ▾	Wire below a top right wire is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a horizontal wire	Pass ▾
3.2.6.	1	Funci... ▾	Wires are located below and to the right of another wire	Wires are able to change orientation based on surrounding wires	Wire becomes a top left wire	Pass ▾
3.2.6.1.	1	Funci... ▾	Wire to the right of a top left corner wire is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a vertical wire	Pass ▾
3.2.6.2.	1	Funci... ▾	Wire below a top left wire is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a horizontal wire	Pass ▾

3.2.7.	1	Function...	Wires are located to the right, left and below another wire	Wires are able to change orientation based on surrounding wires	Wire becomes a wire splitter down	Pass
3.2.7.1.	1	Function...	Wire below a wire splitter down is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a wire horizontal	Pass
3.2.7.2.	1	Function...	Wire to the right of a wire splitter down is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a top right wire	Pass
3.2.7.3.	1	Function...	Wire to left of a wire splitter of a wire splitter down is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a top left wire	Pass
3.2.8.	1	Function...	Wires are located to the left, below and above another wire	Wires are able to change orientation based on surrounding wires	Wire becomes a wire splitter facing up	Pass
3.2.8.1.	1	Function...	Wire located above a wire splitter up is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a horizontal wire	Pass
3.2.8.2.	1	Function...	Wire located to	Wires are able to	Wire becomes	Pass

			the right of a wire splitter up is deleted	change orientation based on surrounding wires	a bottom right corner wire	
3.2.8.3.	1	Function...	Wire located to the left of a wire splitter up is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a bottom left corner wire	Pass
3.2.9.	1	Function...	Wires are located to the right, below and above another wire	Wires are able to change orientation based on surrounding wires	Wire becomes a wire splitter facing right	Pass
3.2.9.1.	1	Function...	Wire above a wire splitter right is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a vertical wire	Pass
3.2.9.2.	1	Function...	Wire above a wire splitter right is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a top left corner wire	Pass
3.2.9.3.	1	Function...	Wire below a wire splitter right is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a bottom left corner wire	Pass
3.2.10.	1	Function...	Wires are located to the left, below and	Wires are able to change orientation	Wire becomes a wire splitter	Pass

			above another wire	based on surrounding wires	facing left	
3.2.10.1.	1	Funci... ▾	Wire to the left of a wire splitter left is deleted	Wires are able to change orientation based on surrounding wires	wire becomes a vertical wire	Pass ▾
3.2.10.2.	1	Funci... ▾	Wire above a wire splitter left is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a top right wire	Pass ▾
3.2.10.3.	1	Funci... ▾	Wire below a wire splitter left is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a bottom right wire	Pass ▾
3.2.11.	1	Funci... ▾	Wires are located to the left, right, below and above another wire,	Wires are able to change orientation based on surrounding wires	Wire becomes a wire cross	Pass ▾
3.2.11.1.	1	Funci... ▾	Wire above a wire cross is deleted	Wires are able to change orientation based on surrounding wires	Wire cross becomes a wire splitter down	Pass ▾
3.2.11.2.	1	Funci... ▾	Wire below a wire cross is deleted	Wires are able to change orientation based on surrounding	Wire becomes a wire splitter facing up	Pass ▾

				wires		
3.2.11.3	1	Functionality	Wire to the left of a wire cross is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a wire splitter facing right	Pass
3.2.11.4	1	Functionality	Wire to the right of a wire cross is deleted	Wires are able to change orientation based on surrounding wires	Wire becomes a wire splitter facing left	Pass
3.3.1.	1	Functionality	User clicks on light bulb	User is presented with an outputs menu	Light bulb becomes active for placement	Pass
3.3.1.	2	Functionality	User clicks a square on logic grid with light bulb selected	User is presented with an outputs menu	Light bulb appears on document in region selected	Pass
3.4.1.	1	Functionality	User clicks on OR gate in the logic menu	User is presented with a logic gate menu	OR gate becomes active for placement	Pass
3.4.1.	2	Functionality	User clicks on square on logic grid with OR gate selected	User is presented with a logic gate menu	OR gate appears on document in region selected	Pass
3.4.2.	1	Functionality	User clicks on AND gate in the logic menu	User is presented with a logic gate menu	AND gate becomes active for placement	Pass
3.4.2.	2	Functionality	User clicks on square on logic	User is presented with a logic	AND gate appears on	Pass

			grid with AND gate selected	gate menu	document in region selected	
3.4.3.	1	Funci... ▾	User clicks on NOT gate in the logic menu	User is presented with a logic gate menu	NOT gate becomes active for placement	Pass ▾
3.4.3.	2	Funci... ▾	User clicks on square on logic grid with NOT gate selected	User is presented with a logic gate menu	NOT gate appears on document in region selected	Pass ▾
3.4.4.	1	Funci... ▾	User clicks on NOR gate in the logic menu	User is presented with a logic gate menu	NOR gate becomes active for placement	Pass ▾
3.4.4.	2	Funci... ▾	User clicks on square on logic grid with NOR gate selected	User is presented with a logic gate menu	NOR gate appears on document in region selected	Pass ▾
3.4.5.	1	Funci... ▾	User clicks on NAND gate in the logic menu	User is presented with a logic gate menu	NAND gate becomes active for placement	Pass ▾
3.4.5.	2	Funci... ▾	User clicks on square on logic grid with NAND gate selected	User is presented with a logic gate menu	NAND gate appears on document in region selected	Pass ▾
3.4.6.	1	Funci... ▾	User clicks on XOR gate in the logic menu	User is presented with a logic gate menu	XOR gate becomes active for placement	Pass ▾
3.4.6.	2	Funci... ▾	User clicks on square on logic grid with	User is presented with a logic gate menu	XOR gate appears on document	Pass ▾

			XOR gate selected		in region selected	
3.4.7.	1	Funci... ▾	User clicks on XNOR gate in the logic menu	User is presented with a logic gate menu	XNOR gate becomes active for placement	Pass ▾
3.4.7.	2	Funci... ▾	User clicks on square on logic grid with XNOR gate selected	User is presented with a logic gate menu	XNOR gate appears on document in region selected	Pass ▾
3.5.1.	1	Funci... ▾	User clicks on edge triggered D-type flip flop in the logic menu	User is presented with a flip-flop menu	Edge triggered D-type flip flop becomes active for placement	Pass ▾
3.5.1.	2	Funci... ▾	User clicks on square on logic grid with edge triggered D-type flip flop selected	User is presented with a flip-flop menu	Edge triggered D-type flip flop appears on document in region selected	Pass ▾

Objective 4: Logic gate interaction and shortcuts

Test objective	Test number	Test type	Test input	Test description	Expected output	Actual Result and timestamp
4.1.1.	1	Funci... ▾	User left clicks on a switch in the document	User can click on logic elements to manipulate them	Switch changes state	Pass ▾

4.1.1.1.	1	Functionality	User left clicks on a switch in the document	User can click on logic elements to manipulate them	Any logic elements and wires connected to the switch change colour	Pass
4.1.2.1.	1	Functionality	Clock is detected in logic grid	User can utilise clocks in their circuits	Clock changes state based on pulse	Pass
4.2.1.1.	1	Functionality	AND gate is detected in logic array grid	User can utilise logic gates in their circuits	An AND operation is performed using the states of the wire intersection to the left, the wire to the right changes in correspondence	Pass
4.2.2.1.	1	Functionality	OR gate is detected in logic array grid	User can utilise logic gates in their circuits	An OR operation is performed using the states of the wire intersection to the left, the wire to the right changes in correspondence	Pass
4.2.3.1.	1	Functionality	NOT gate	User can	A NOT	Pass

			is detected in logic array grid	utilise logic gates in their circuits	operation is performed based on the state of the wire to the left, the wire to the right changes in correspondence	
4.2.4.1.	1	Funci... ▾	NAND gate is detected in logic array grid	User can utilise logic gates in their circuits	A NAND operation is performed using the states of the wire intersection to the left, the wire to the right changes in correspondence	Pass ▾
4.2.5.1.	1	Funci... ▾	NOR gate is detected in logic array grid	User can utilise logic gates in their circuits	A NOR operation is performed using the states of the wire intersection to the left, the wire to the right changes in correspondence	Pass ▾
4.2.6.1.	1	Funci... ▾	XNOR gate is detected	User can utilise logic	An XNOR operation	Pass ▾

			in logic array grid	gates in their circuits	is performed using the states of the wire intersection to the left, the wire to the right changes in correspondence	
4.2.7.1.	1	Function... ▾	XOR gate is detected in logic array grid	User can utilise logic gates in their circuits	An XOR operation is performed using the states of the wire intersection to the left, the wire to the right changes in correspondence	Pass ▾
4.3.1.1.	1	Function... ▾	D-type flip flop is detected in logic grid	User can utilise flip-flops in their circuits	A edge triggered D-type flip flop operation is performed based on the states of the wire intersection to the left, this affects the state of the wires connected on the	Pass ▾

					right	
4.4.1.	1	Function... ▾	Light bulb is detected in logic grid	Users can utilise light bulbs as outputs in their circuit	The state of the wire to the left of a light bulb causes it to turn on and off	Pass ▾

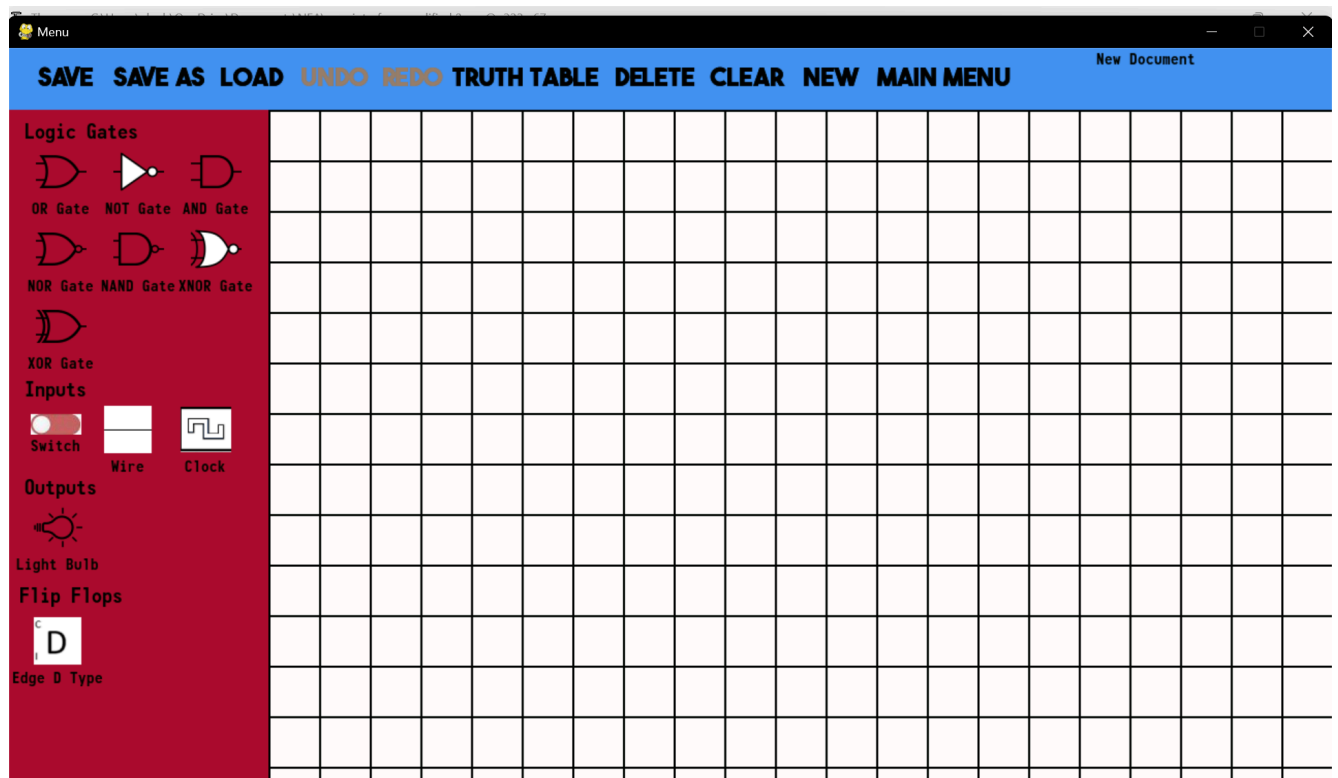
Test evidence

Objective 1: User opens simulation

1.2. New document is opened - Pass

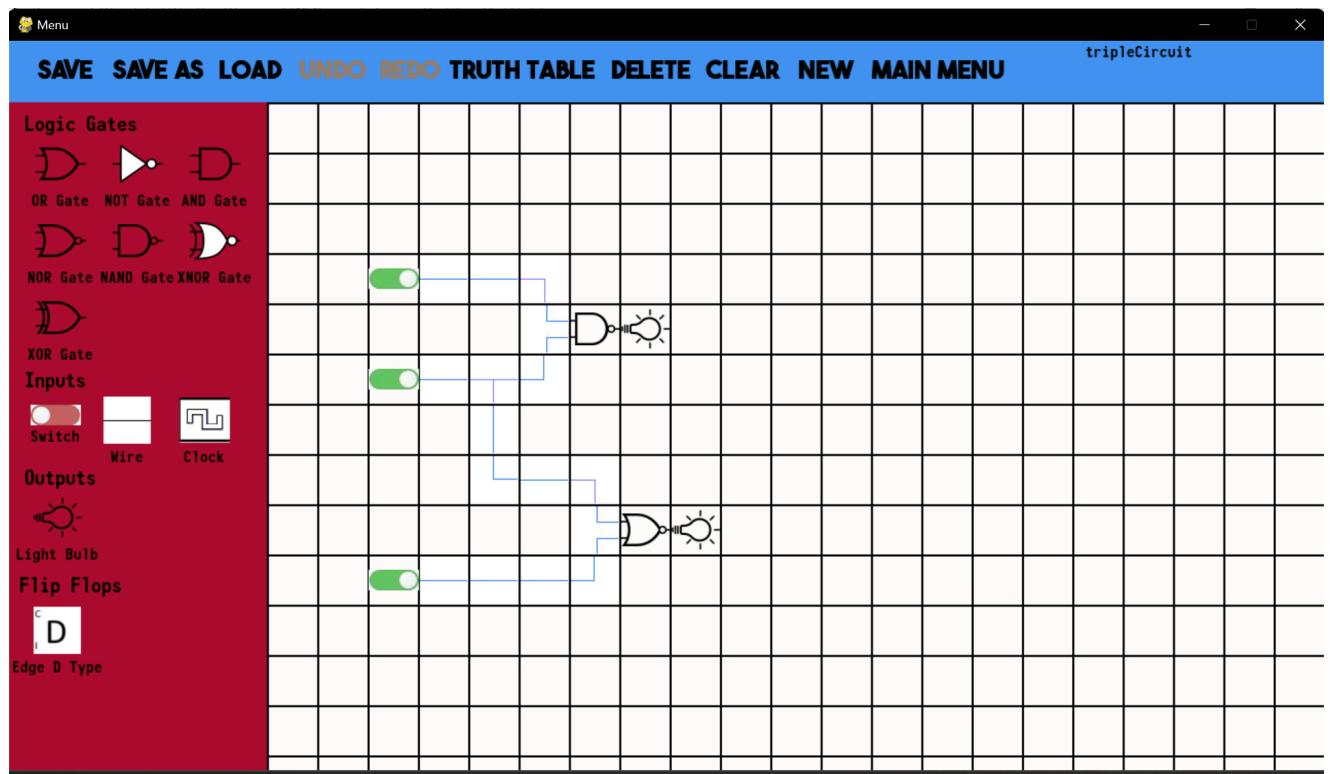
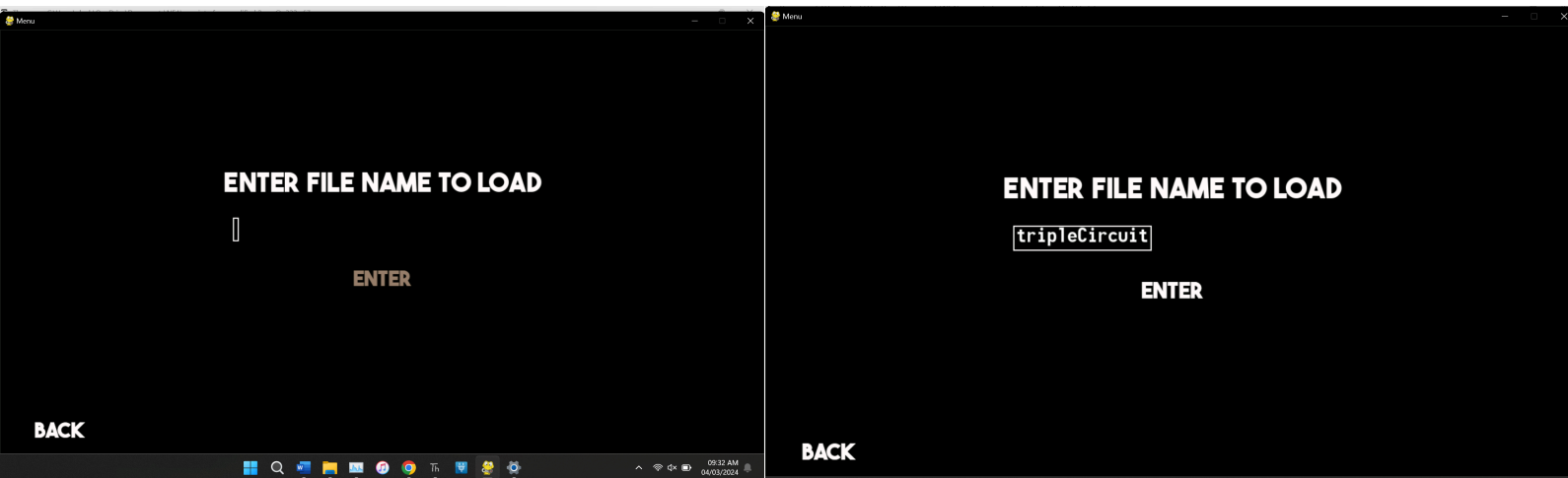
New document is opened when a new file button is pressed.

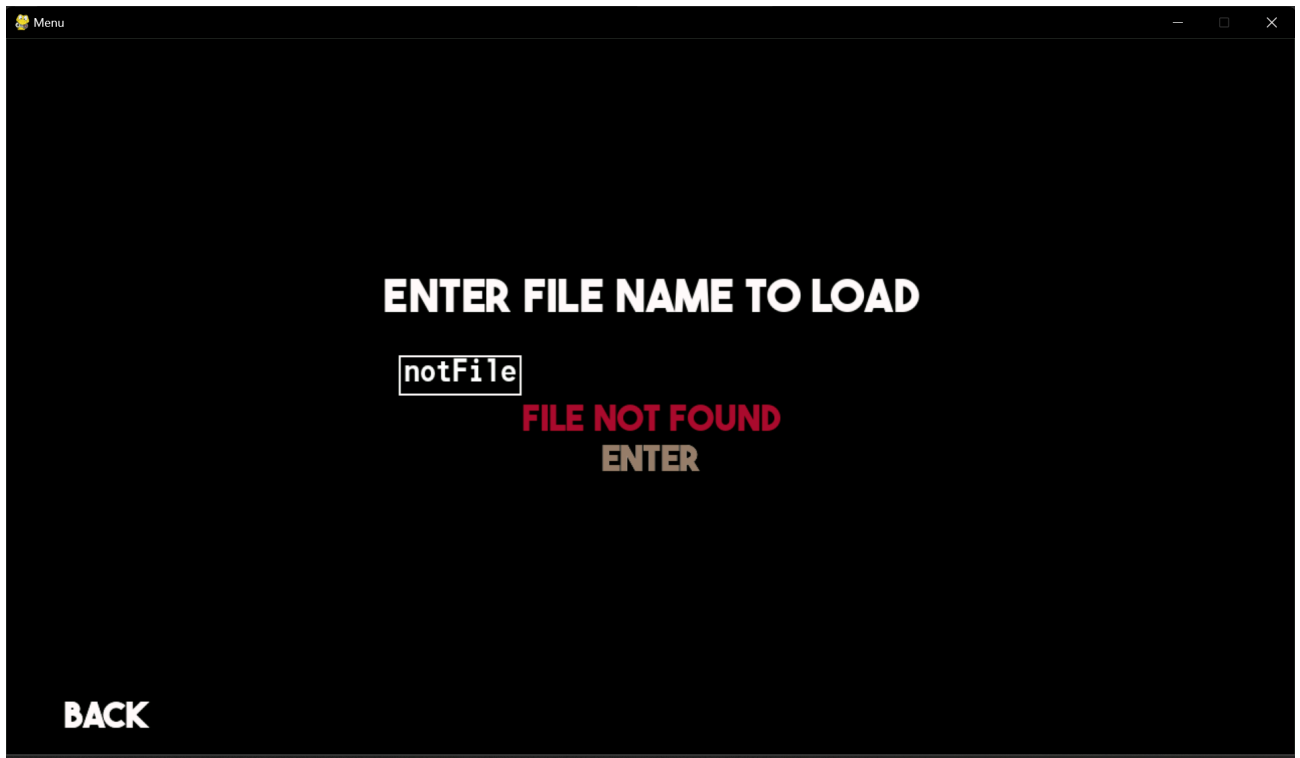




1.3. Existing document is opened - **Pass**

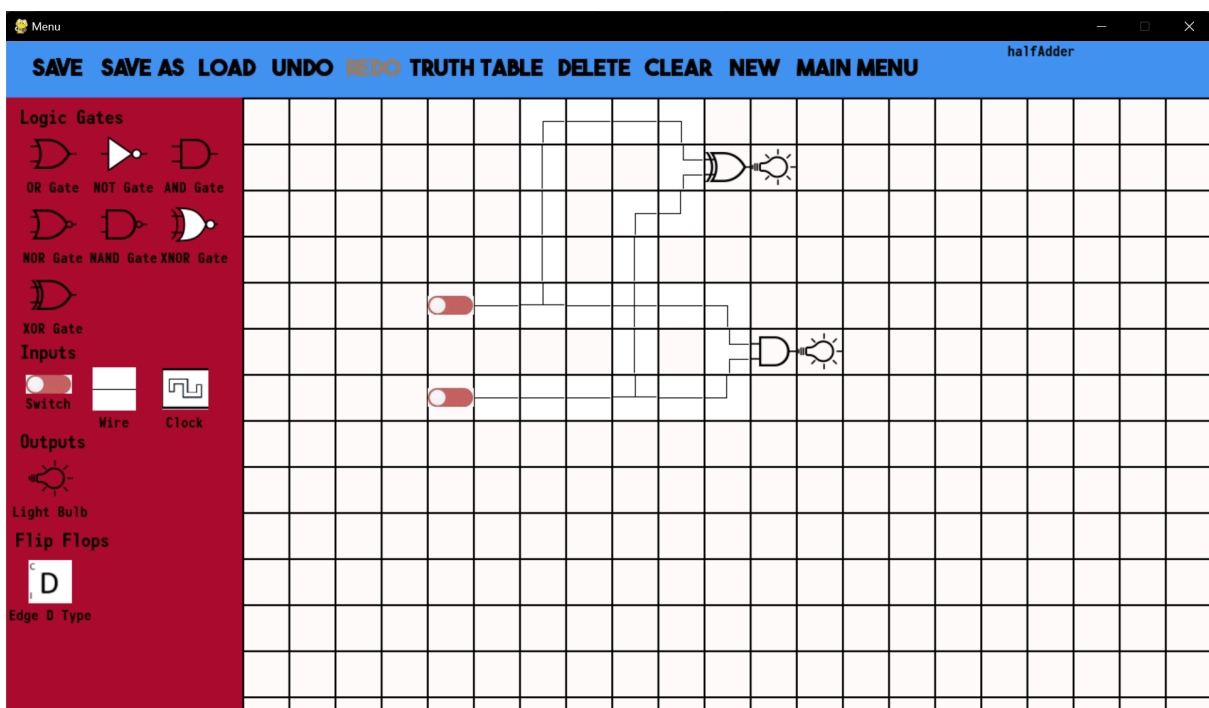
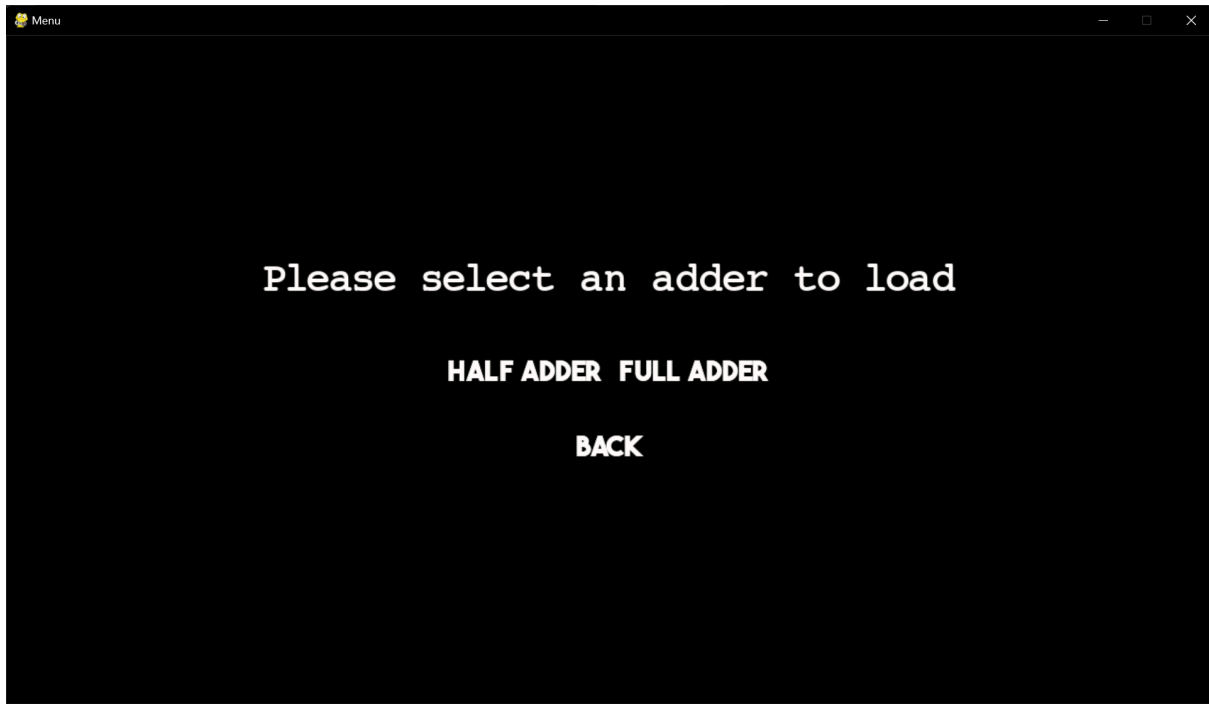
Users can load files, menu opens, exception handling works as expected.

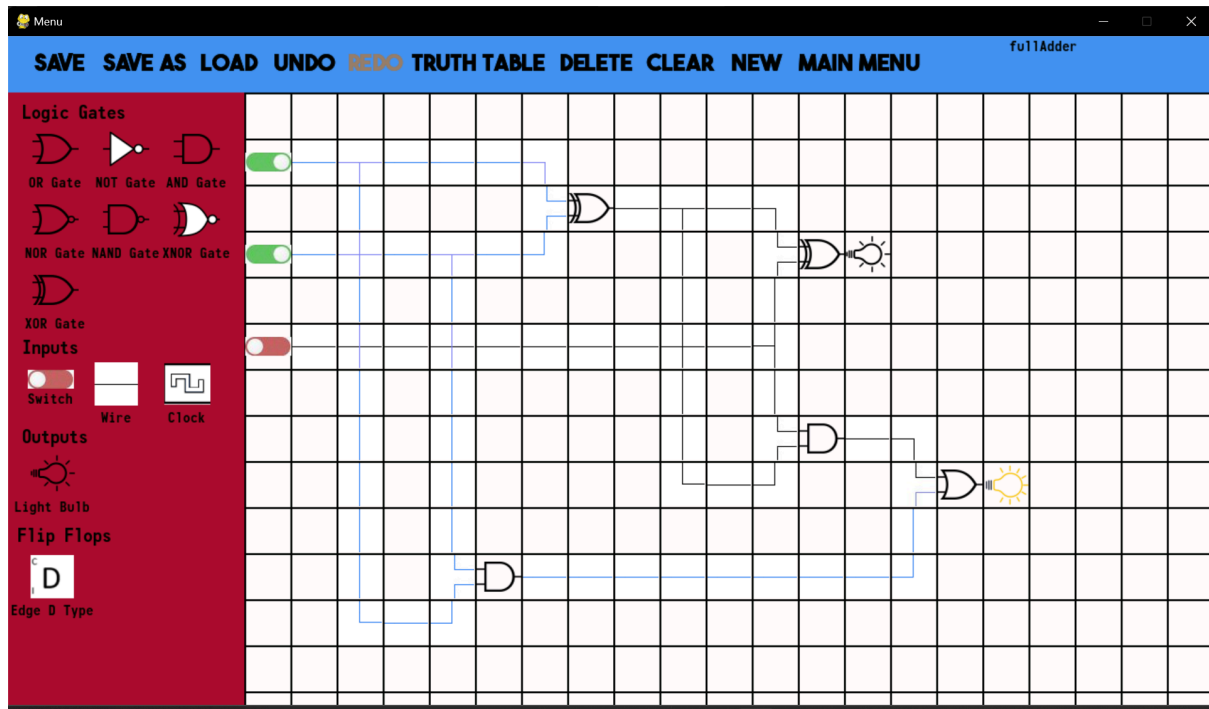




1.4. Adder menu is opened - **Pass**

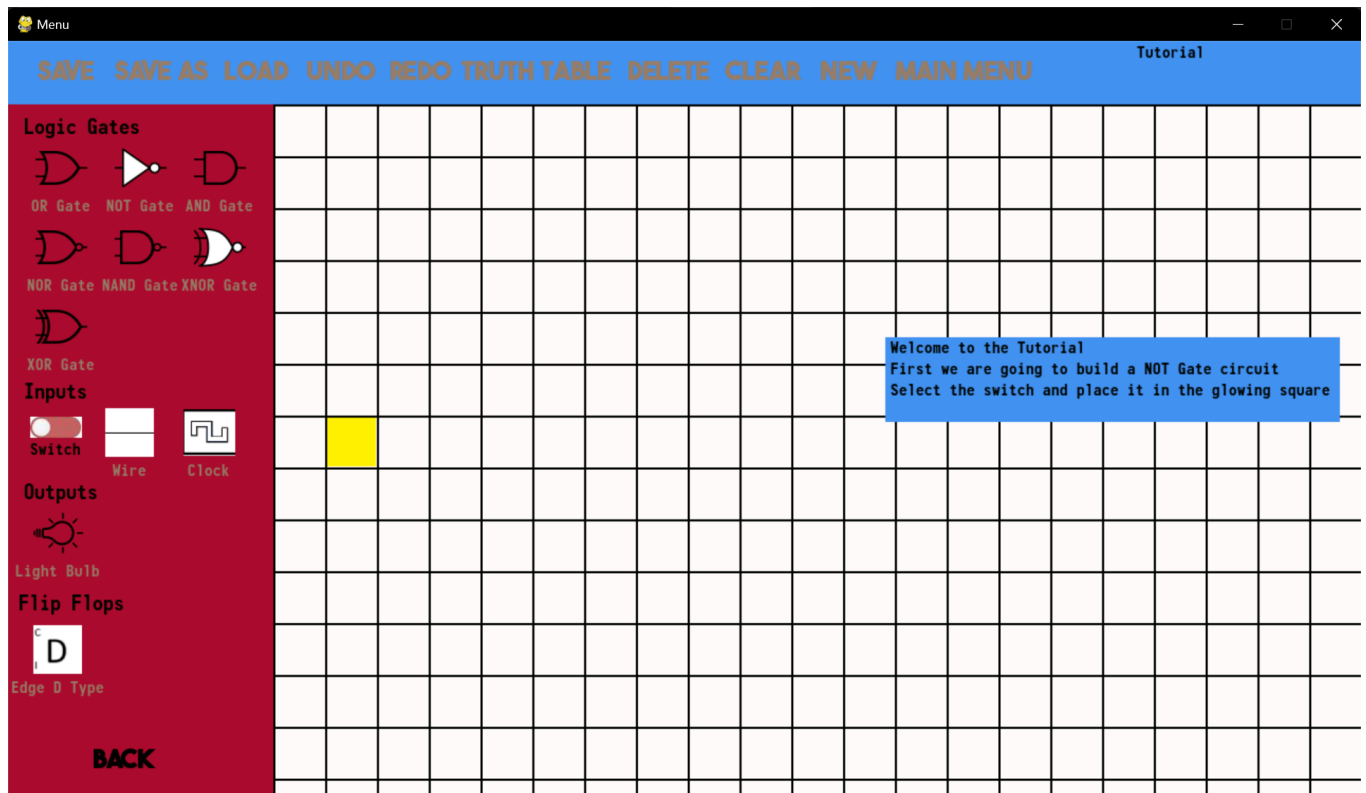
Adder menu opens and adders load as expected.





1.5. Tutorial menu is opened - Pass

Tutorial menu opens, tutorial functions behave as expected.



Tutorial
Tutorial

SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

Logic Gates





OR Gate NOT Gate AND Gate





NOR Gate NAND Gate XNOR Gate



XOR Gate

Inputs





Switch Wire Clock

Outputs



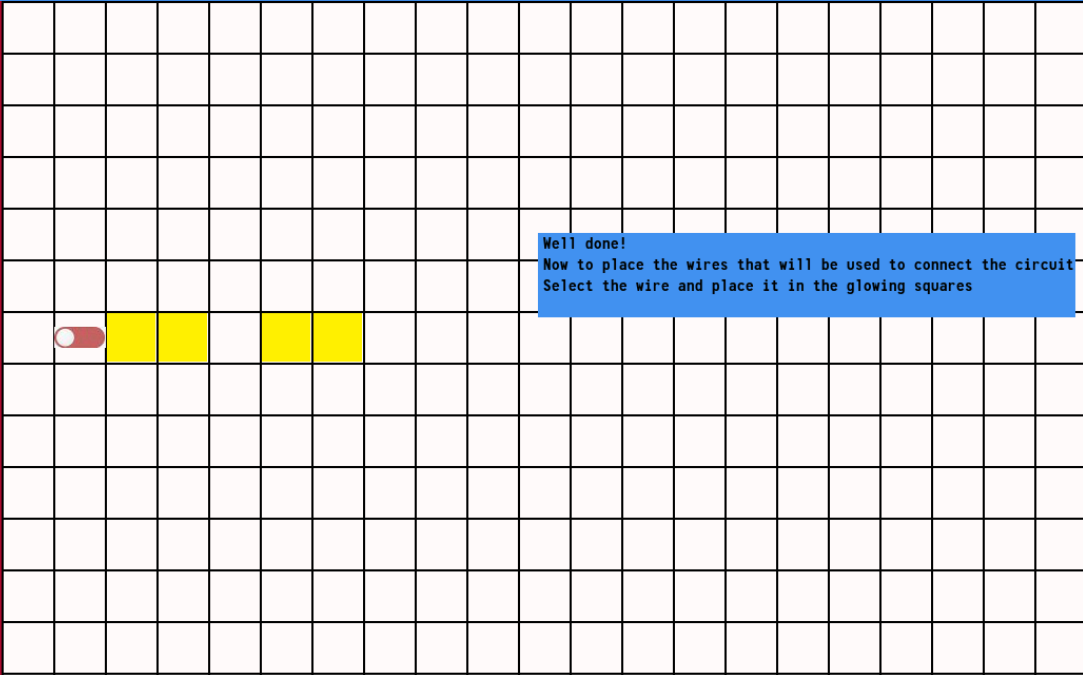
Light Bulb

Flip Flops



Edge D Type

BACK



We'll done!
 Now to place the wires that will be used to connect the circuit
 Select the wire and place it in the glowing squares

Menu
Tutorial

SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

Logic Gates





OR Gate NOT Gate AND Gate





NOR Gate NAND Gate XNOR Gate



XOR Gate

Inputs





Switch Wire Clock

Outputs



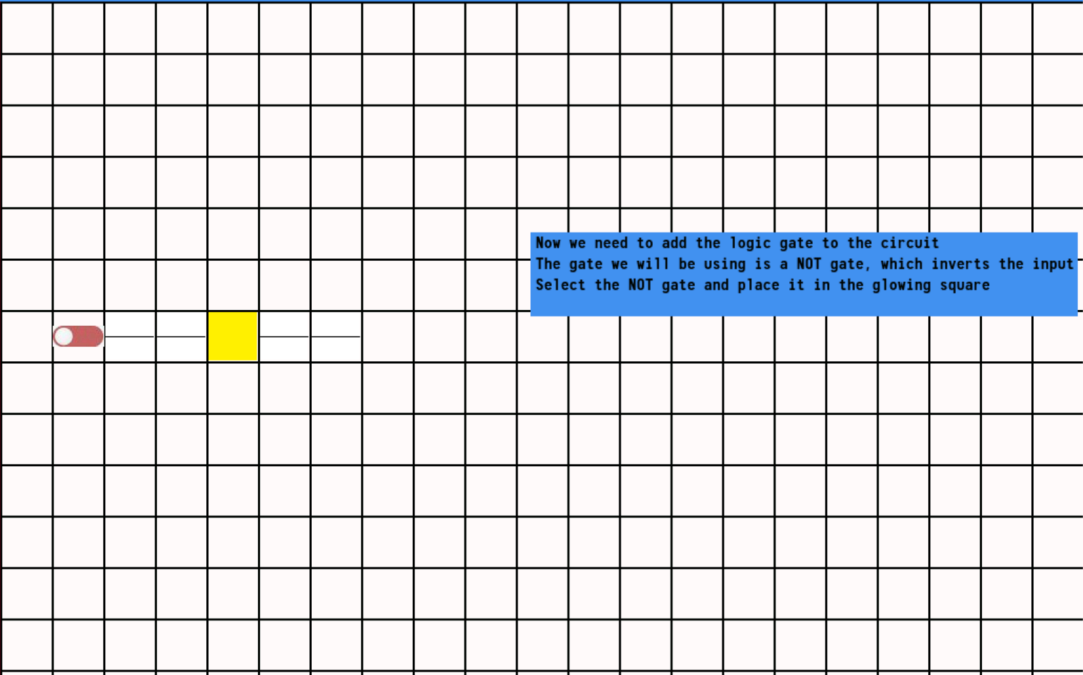
Light Bulb

Flip Flops

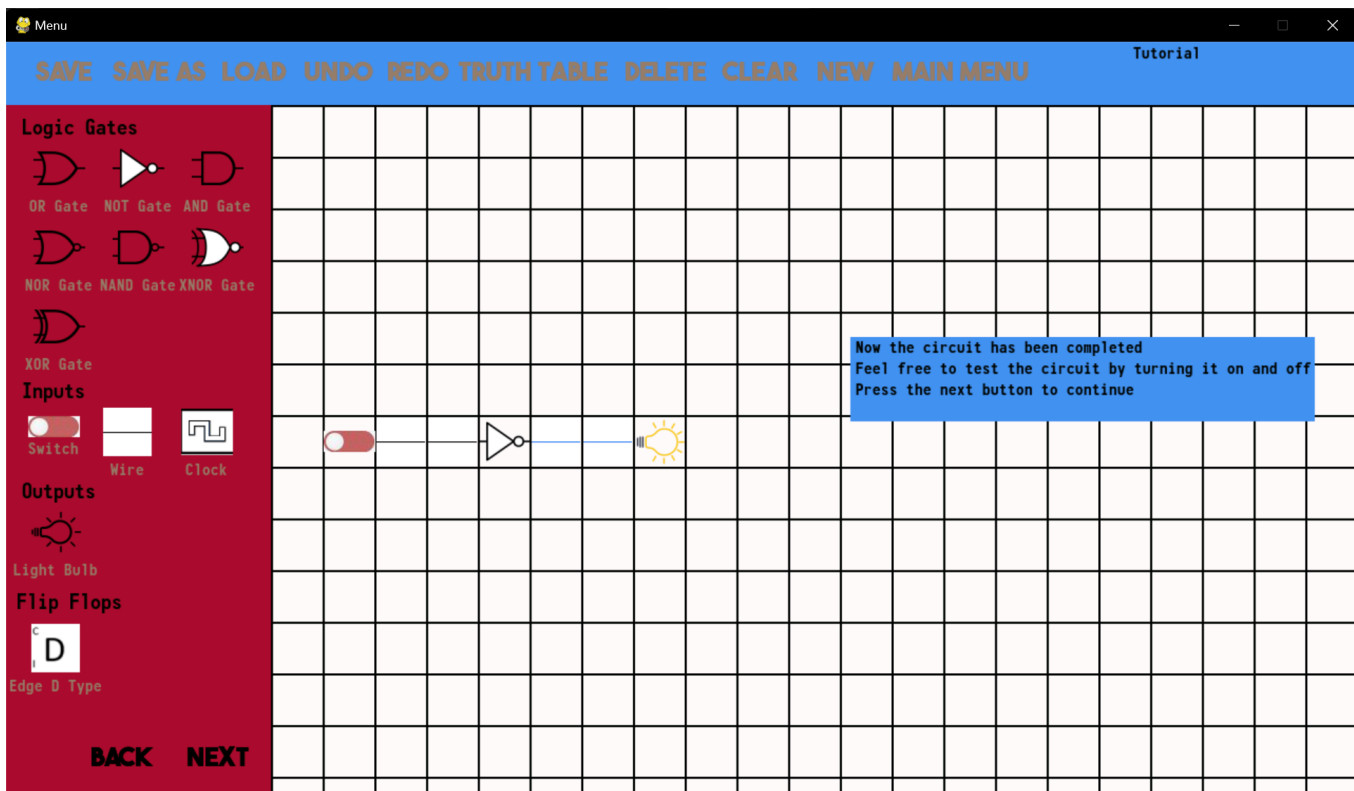
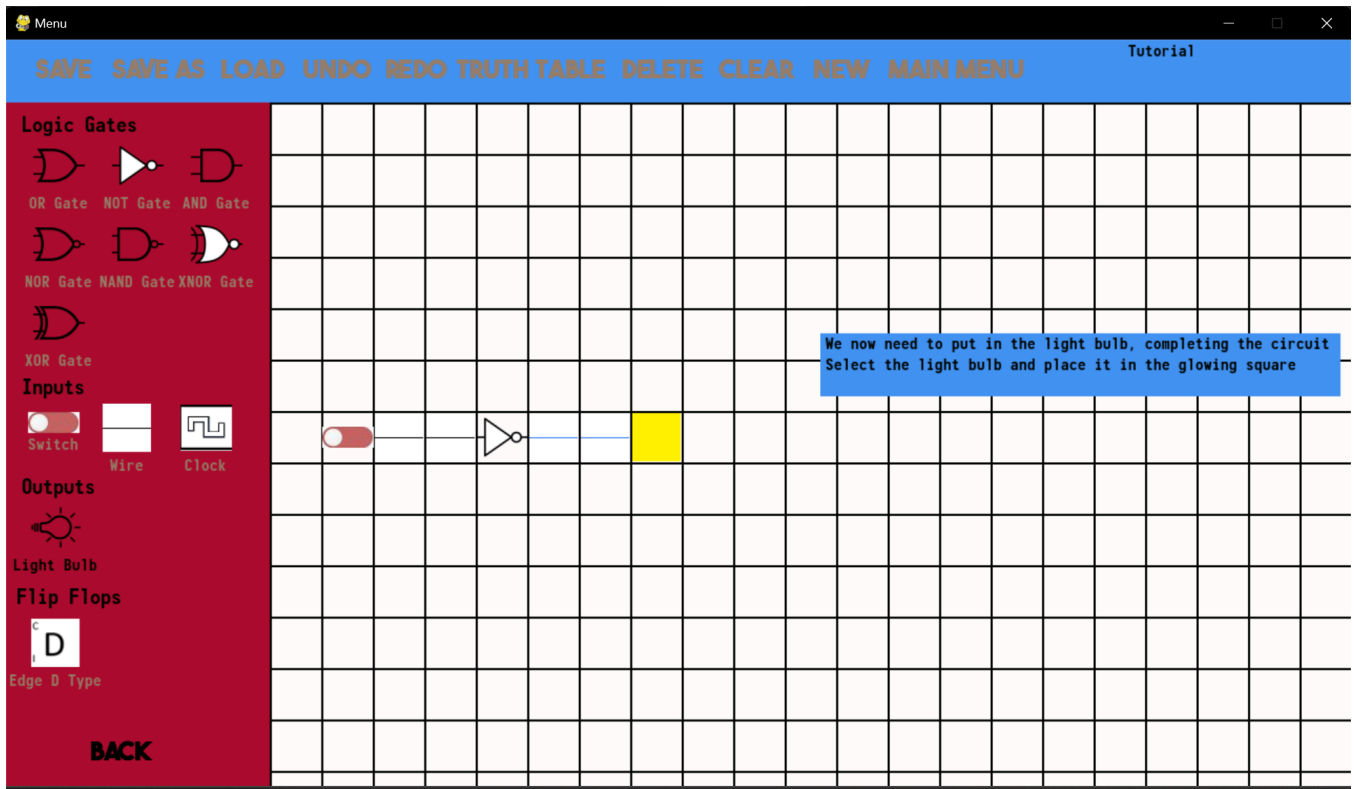


Edge D Type

BACK



Now we need to add the logic gate to the circuit
 The gate we will be using is a NOT gate, which inverts the input
 Select the NOT gate and place it in the glowing square



Tutorial

SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

Tutorial

Logic Gates



OR Gate NOT Gate AND Gate



NOR Gate NAND Gate XNOR Gate



XOR Gate

Inputs



Switch Wire Clock

Outputs



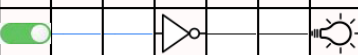
Light Bulb

Flip Flops

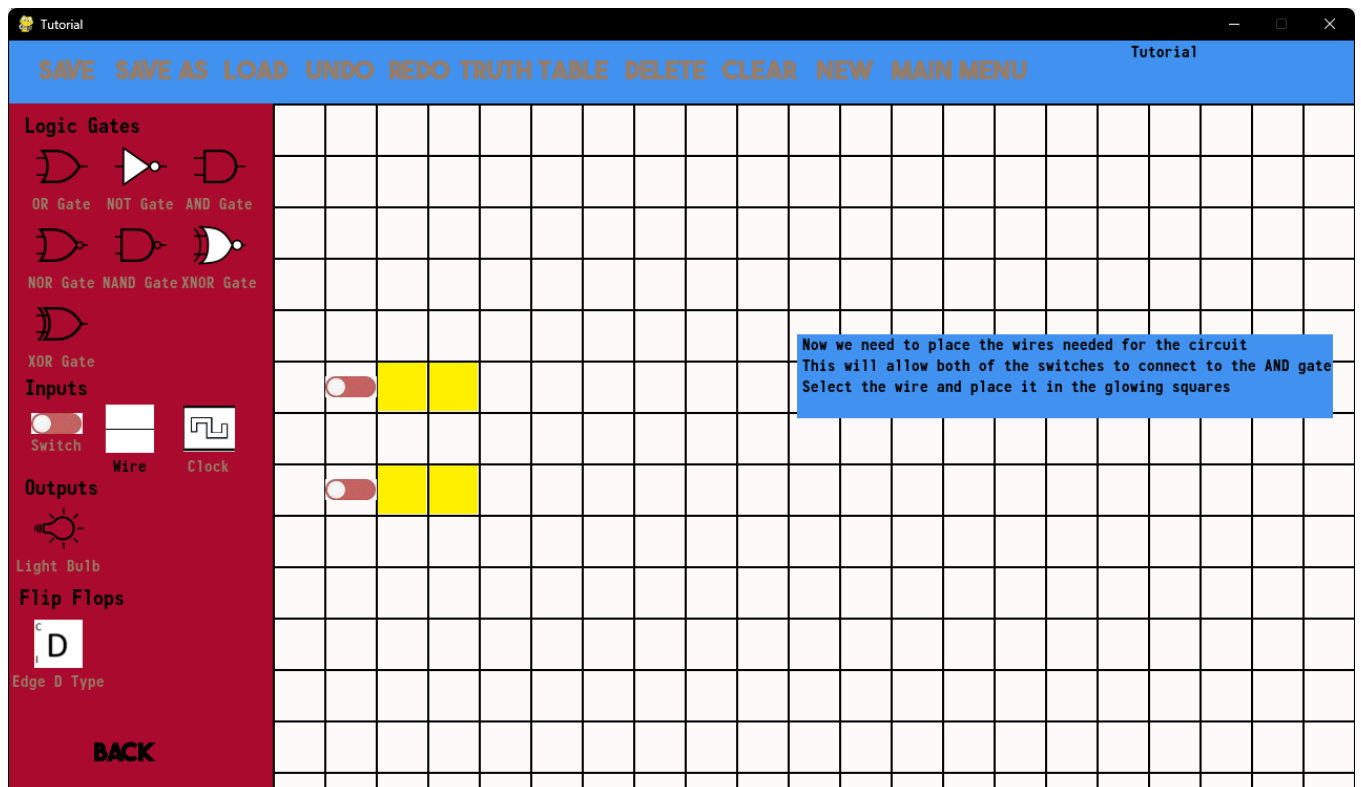
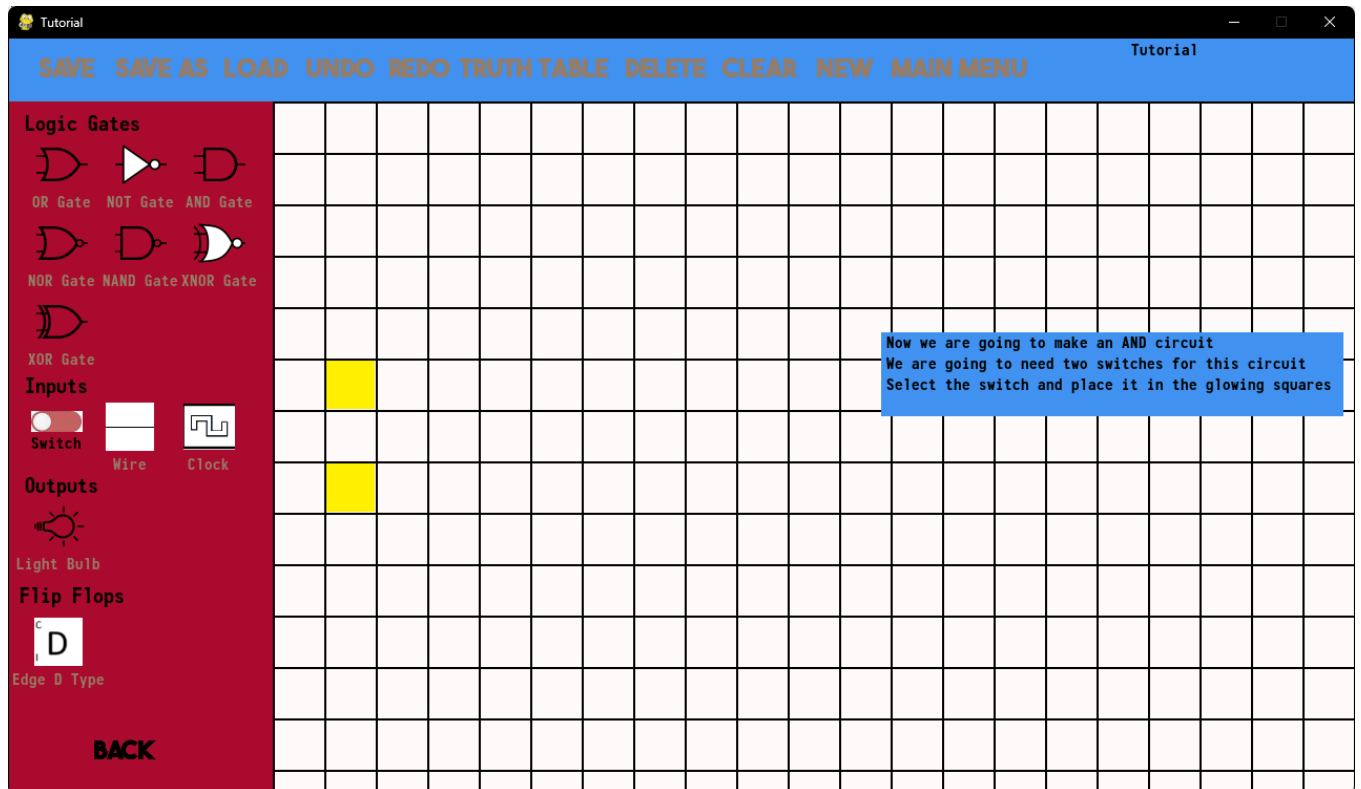


Edge D Type

BACK NEXT



Now the circuit has been completed
Feel free to test the circuit by turning it on and off
Press the next button to continue



Tutorial
SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

Logic Gates





OR Gate NOT Gate AND Gate





NOR Gate NAND Gate XNOR Gate



XOR Gate

Inputs





Switch Wire Clock

Outputs



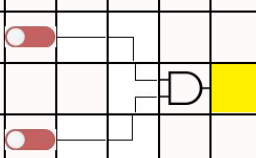
Light Bulb

Flip Flops



Edge D Type

BACK



We now need to put in the light bulb, completing the circuit
Select the light bulb and place it in the glowing square

Tutorial
SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

Logic Gates





OR Gate NOT Gate AND Gate





NOR Gate NAND Gate XNOR Gate



XOR Gate

Inputs





Switch Wire Clock

Outputs



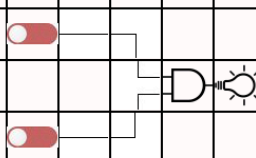
Light Bulb

Flip Flops



Edge D Type

NEXT



The AND gate circuit is now complete
Feel free to test it again, you can also change the gate to one of your choosing
Select the gate you want and place it in the same place as the AND Gate
Press next to finish the tutorial

Tutorial

SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

Tutorial

Logic Gates





OR Gate NOT Gate AND Gate





NOR Gate NAND Gate XNOR Gate



XOR Gate

Inputs





Switch Wire Clock

Outputs



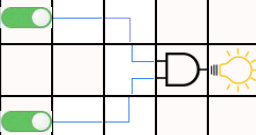
Light Bulb

Flip Flops



Edge D Type

NEXT



The AND gate circuit is now complete

Feel free to test it again, you can also change the gate to one of your choosing

Select the gate you want and place it in the same place as the AND Gate

Press next to finish the tutorial

Tutorial
Tutorial

SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

Logic Gates





OR Gate NOT Gate AND Gate





NOR Gate NAND Gate XNOR Gate



XOR Gate

Inputs





Switch Wire Clock

Outputs



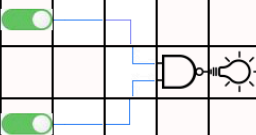
Light Bulb

Flip Flops



Edge D Type

NEXT



The AND gate circuit is now complete
 Feel free to test it again, you can also change the gate to one of your choosing
 Select the gate you want and place it in the same place as the AND Gate
 Press next to finish the tutorial

Tutorial
Tutorial

SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

Logic Gates





OR Gate NOT Gate AND Gate





NOR Gate NAND Gate XNOR Gate



XOR Gate

Inputs





Switch Wire Clock

Outputs



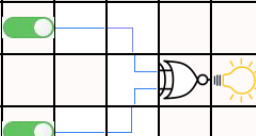
Light Bulb

Flip Flops

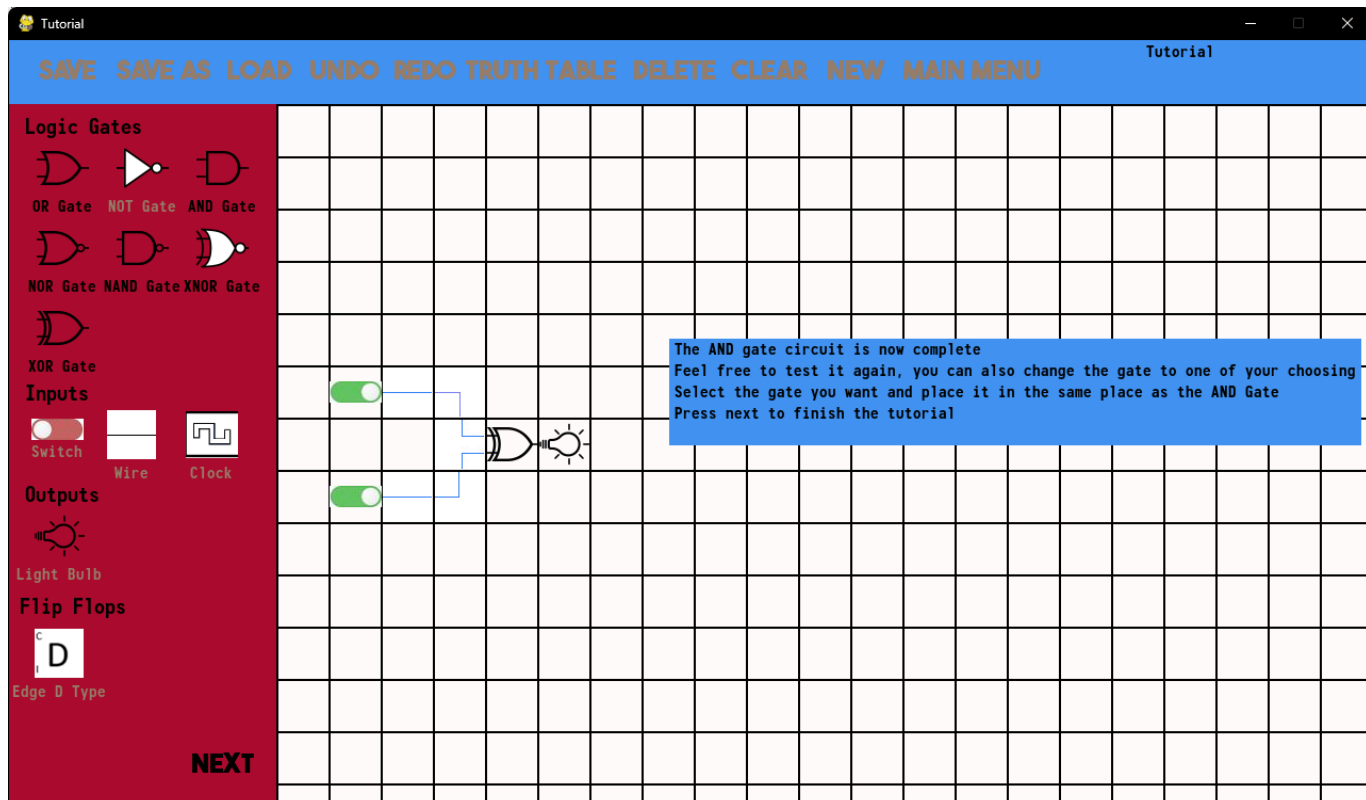
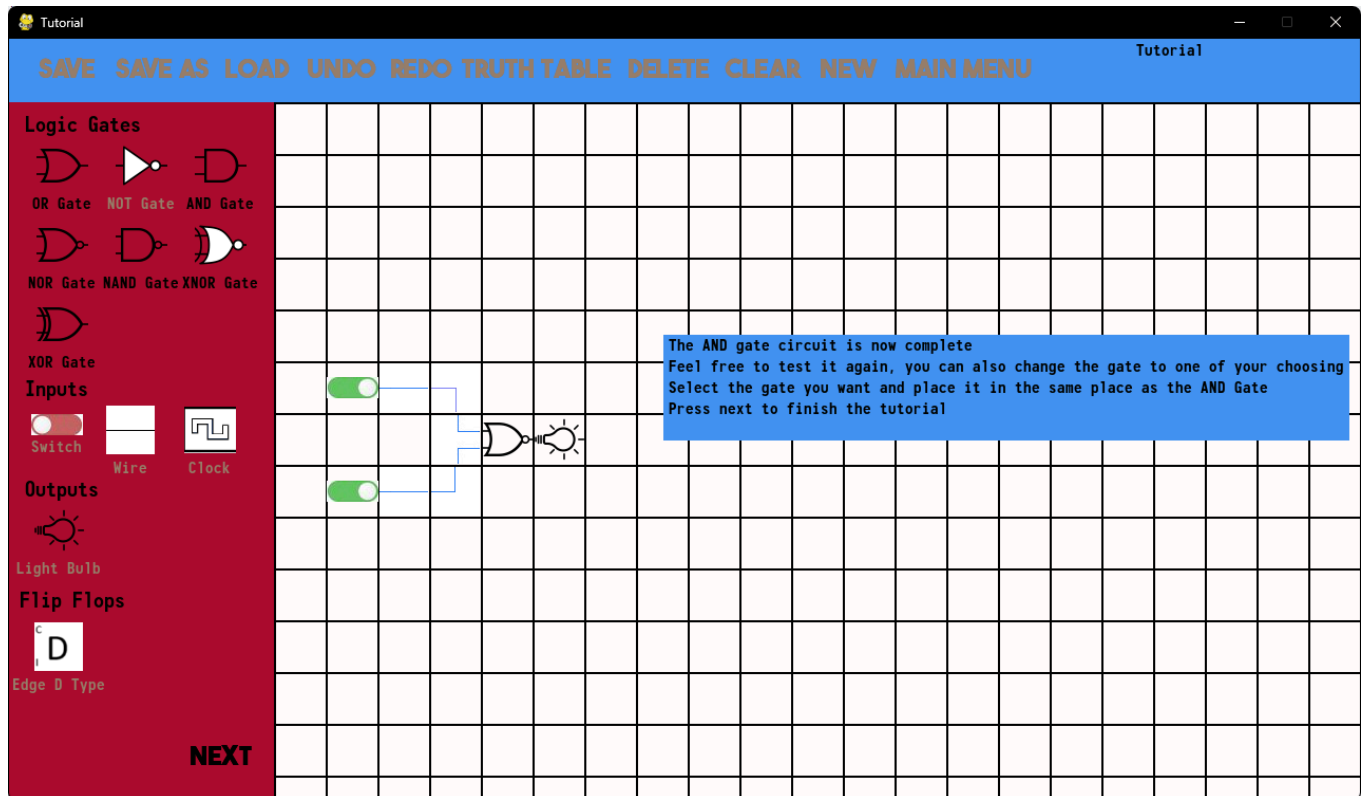


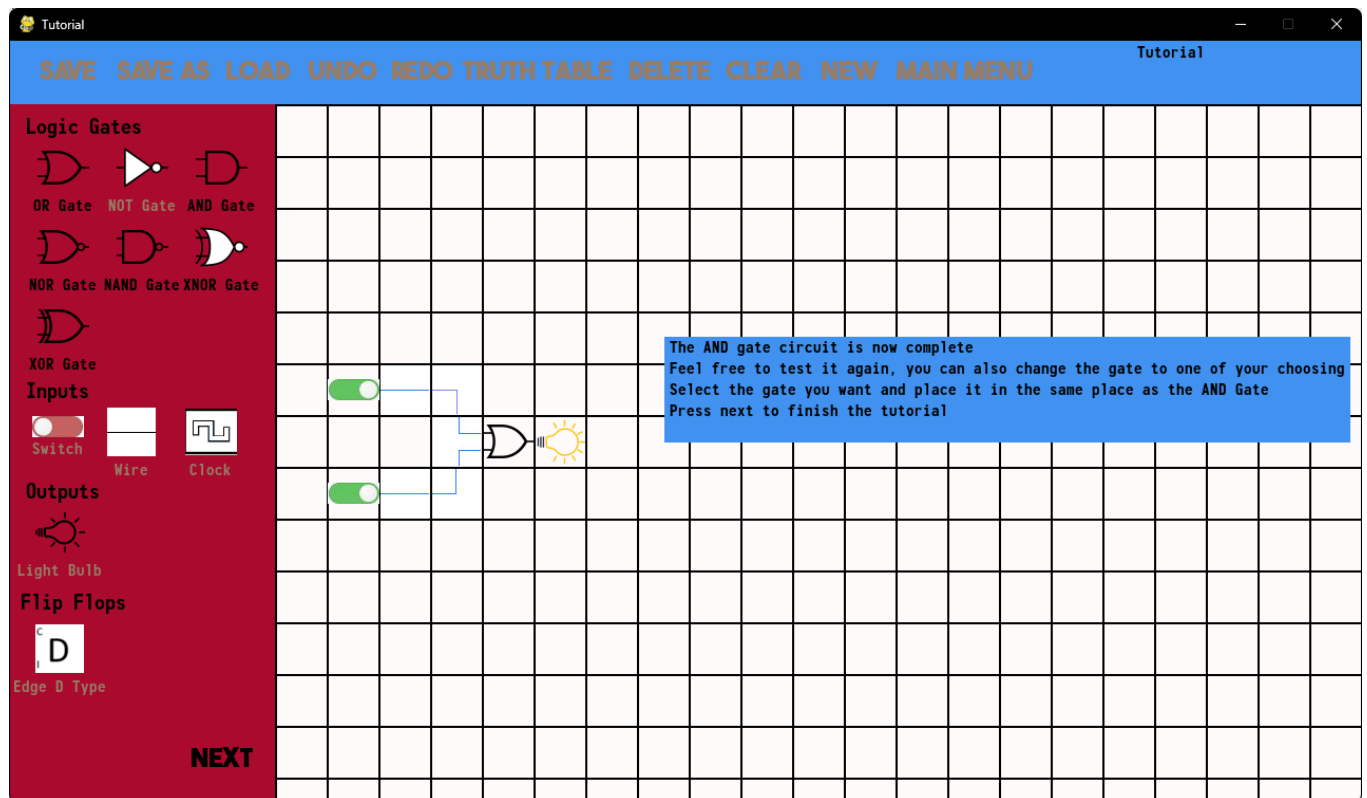
Edge D Type

NEXT



The AND gate circuit is now complete
 Feel free to test it again, you can also change the gate to one of your choosing
 Select the gate you want and place it in the same place as the AND Gate
 Press next to finish the tutorial





1.6. Exit button pressed - **Pass**

Program ended as expected.

Objective 2: Options menu

2.1. Undo - **Pass**

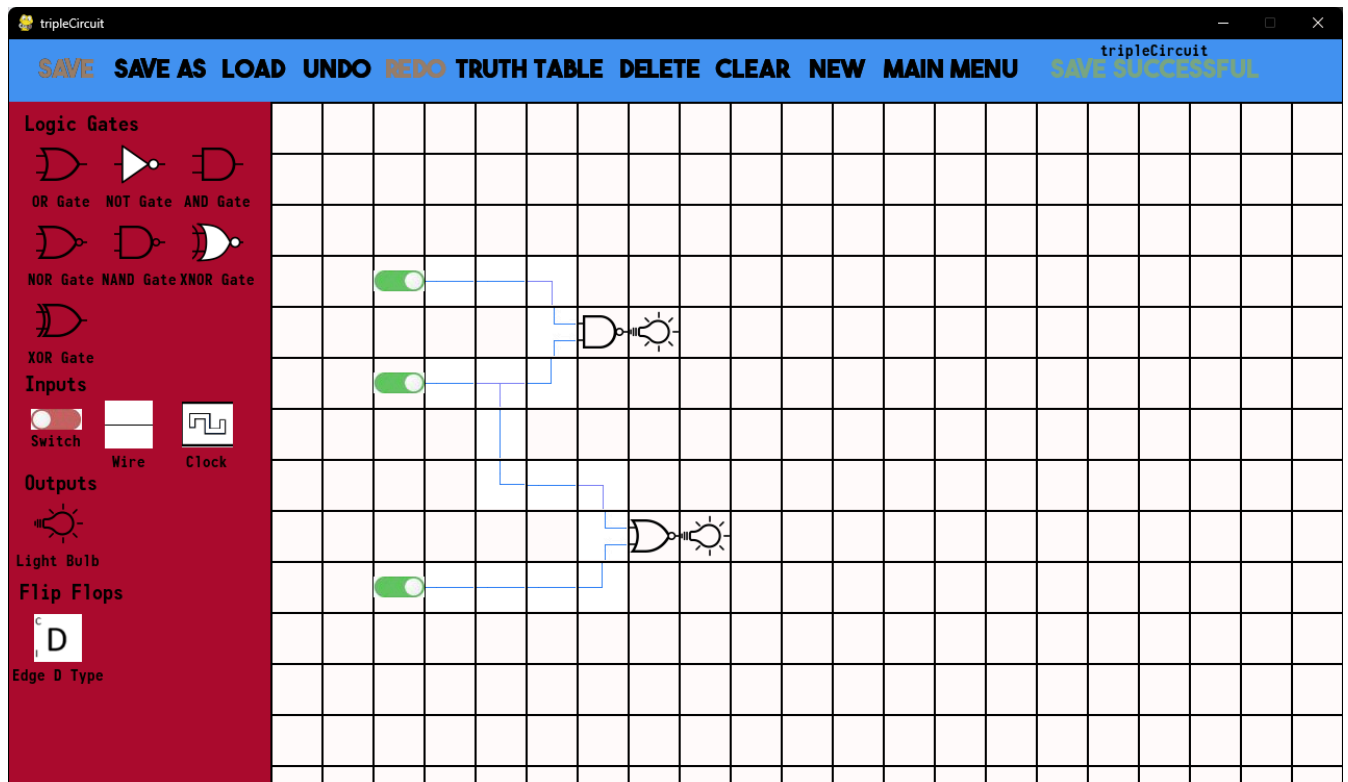
Undo functionality works as expected.

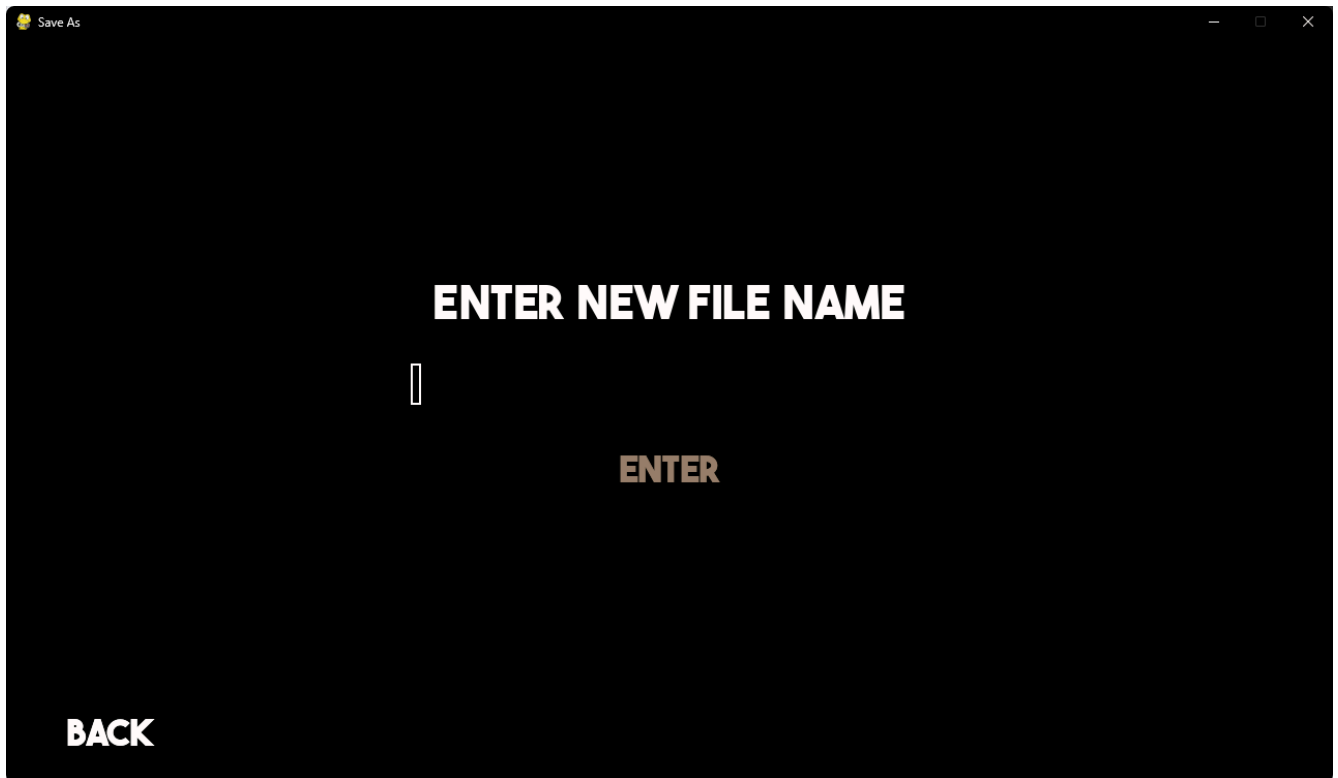
2.2. Redo - **Pass**

Redo functionality works as expected.

2.3. Save - **Pass**

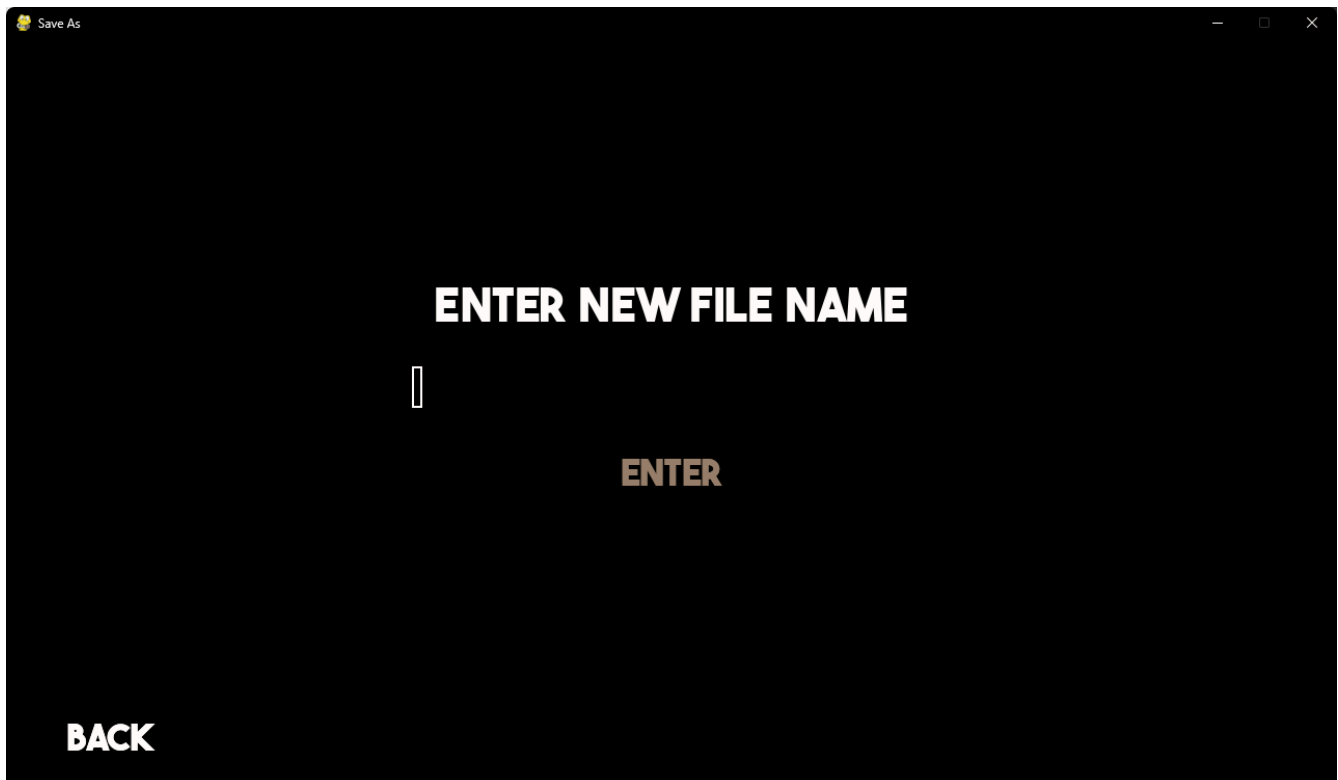
Save functionality works as expected, users are directed to the save as menu if the file has not been previously saved.

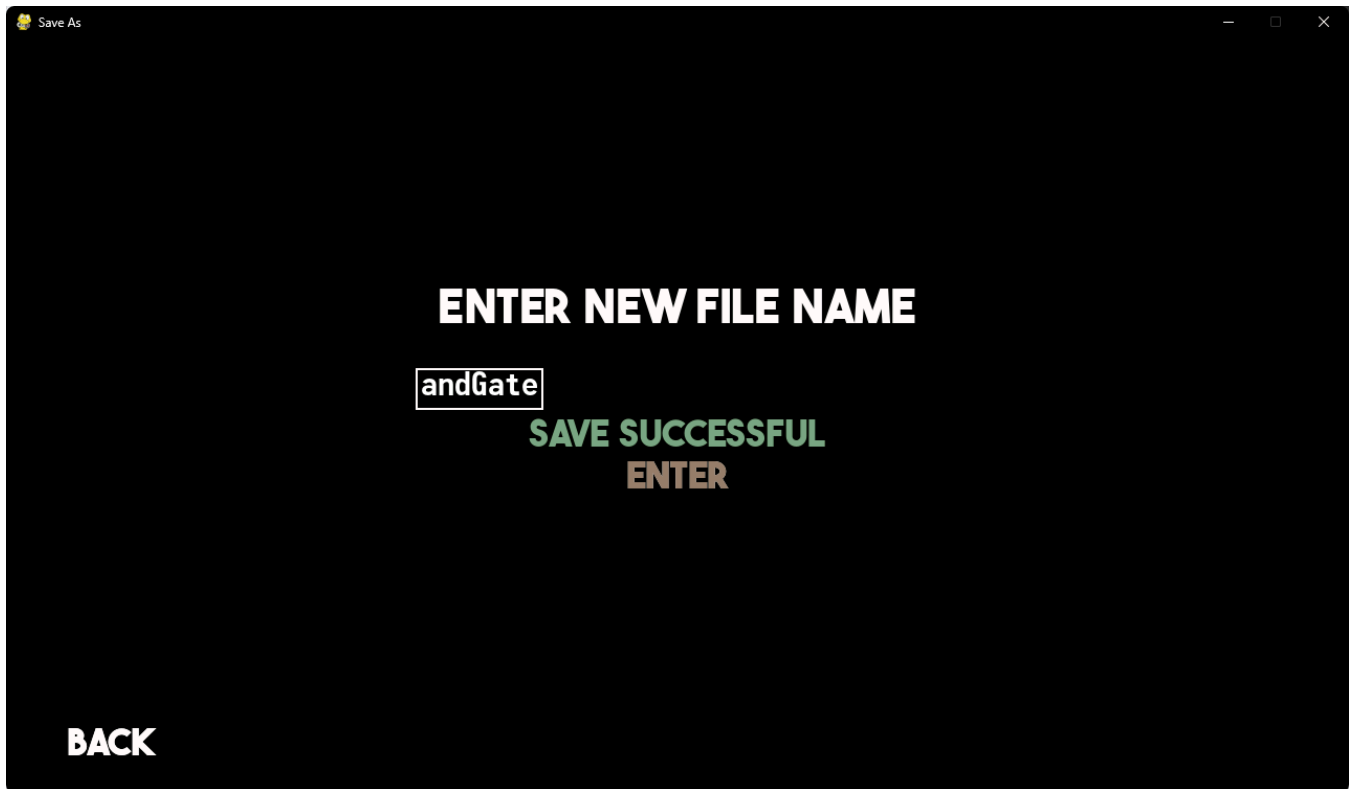




2.4. Save as - **Pass**

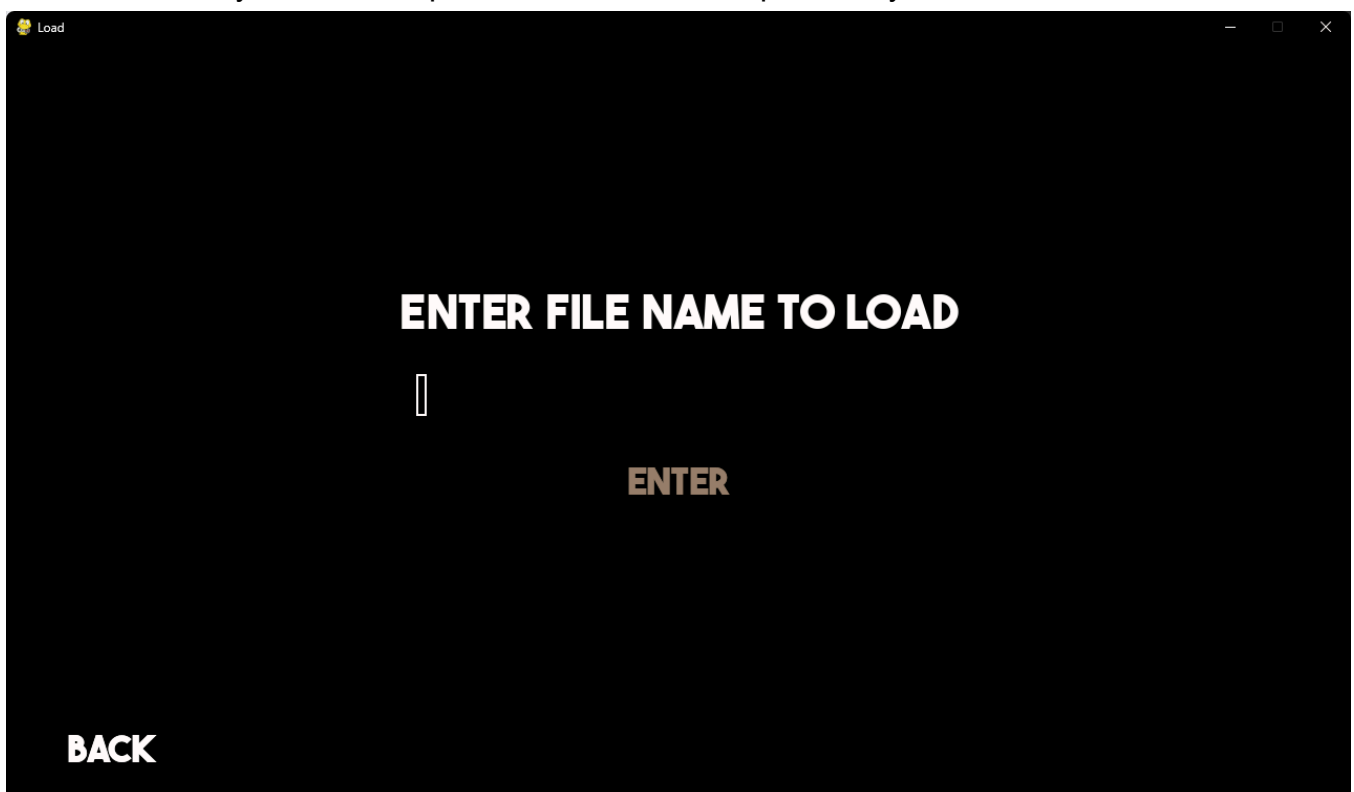
Save as functionality works as expected, new files are created.

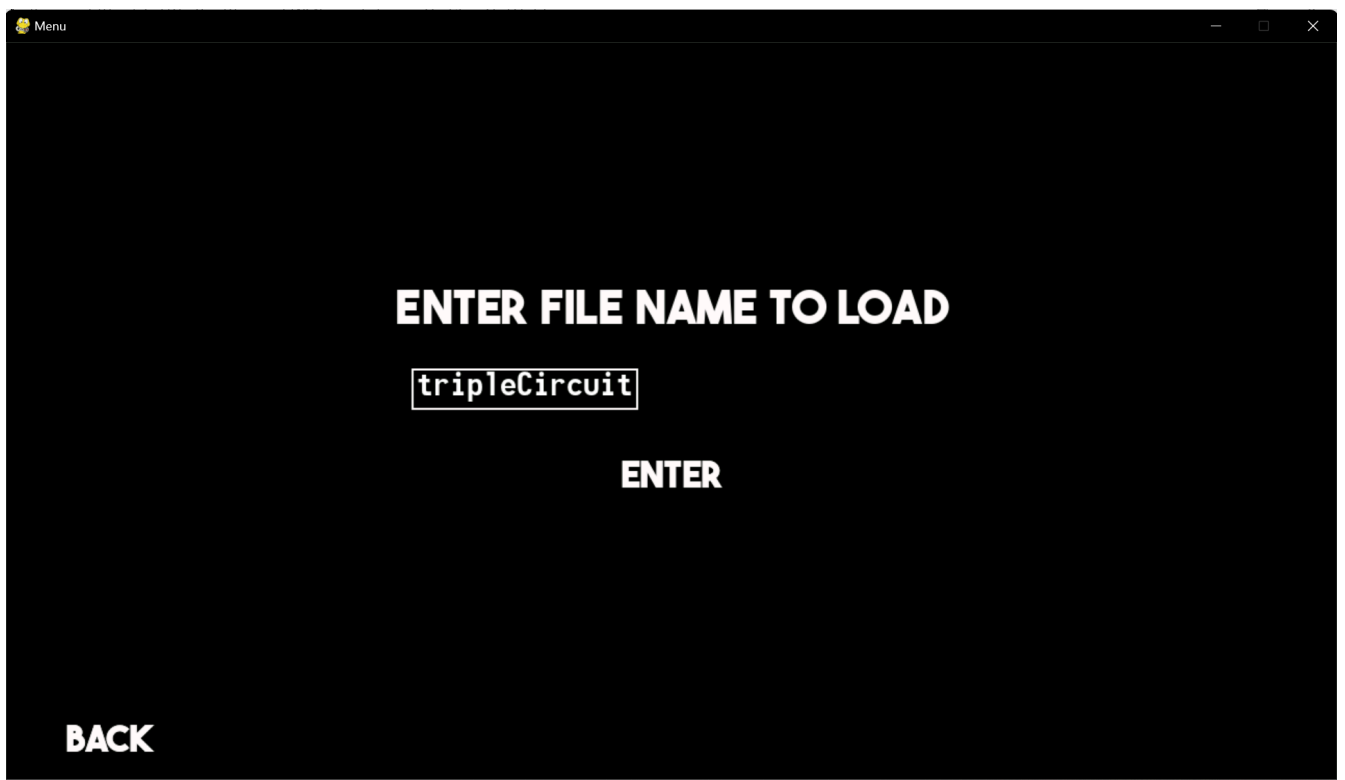
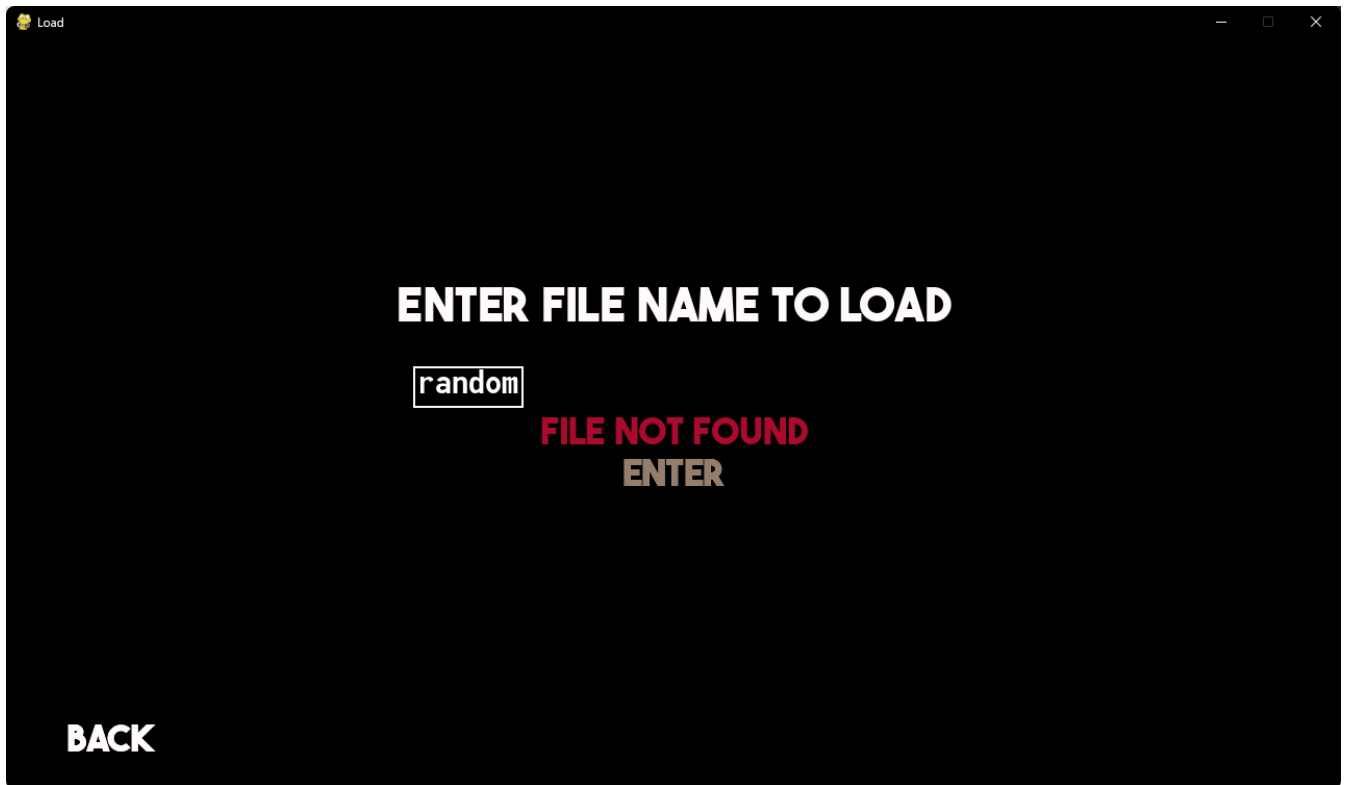


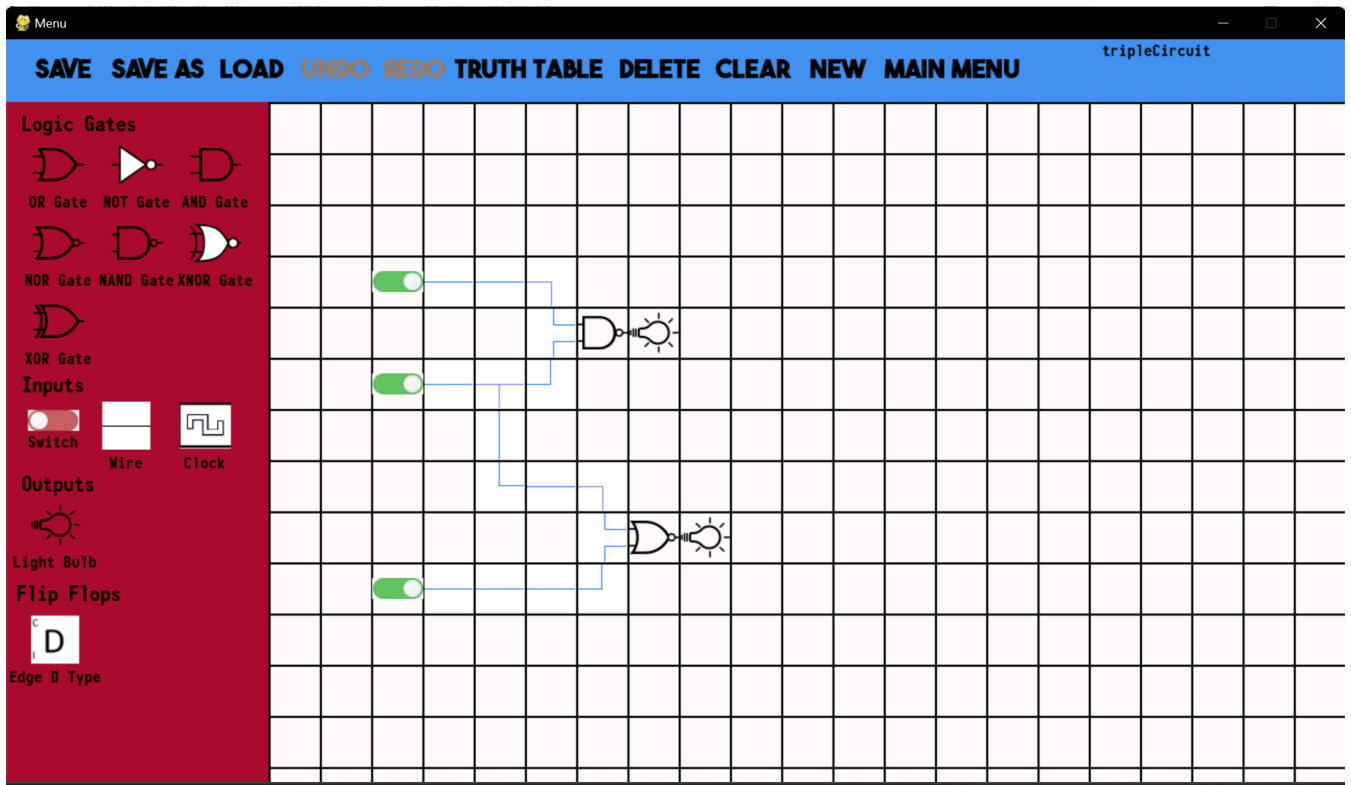


2.5. Load - **Pass**

Load functionality works as expected, users can load previously saved files.

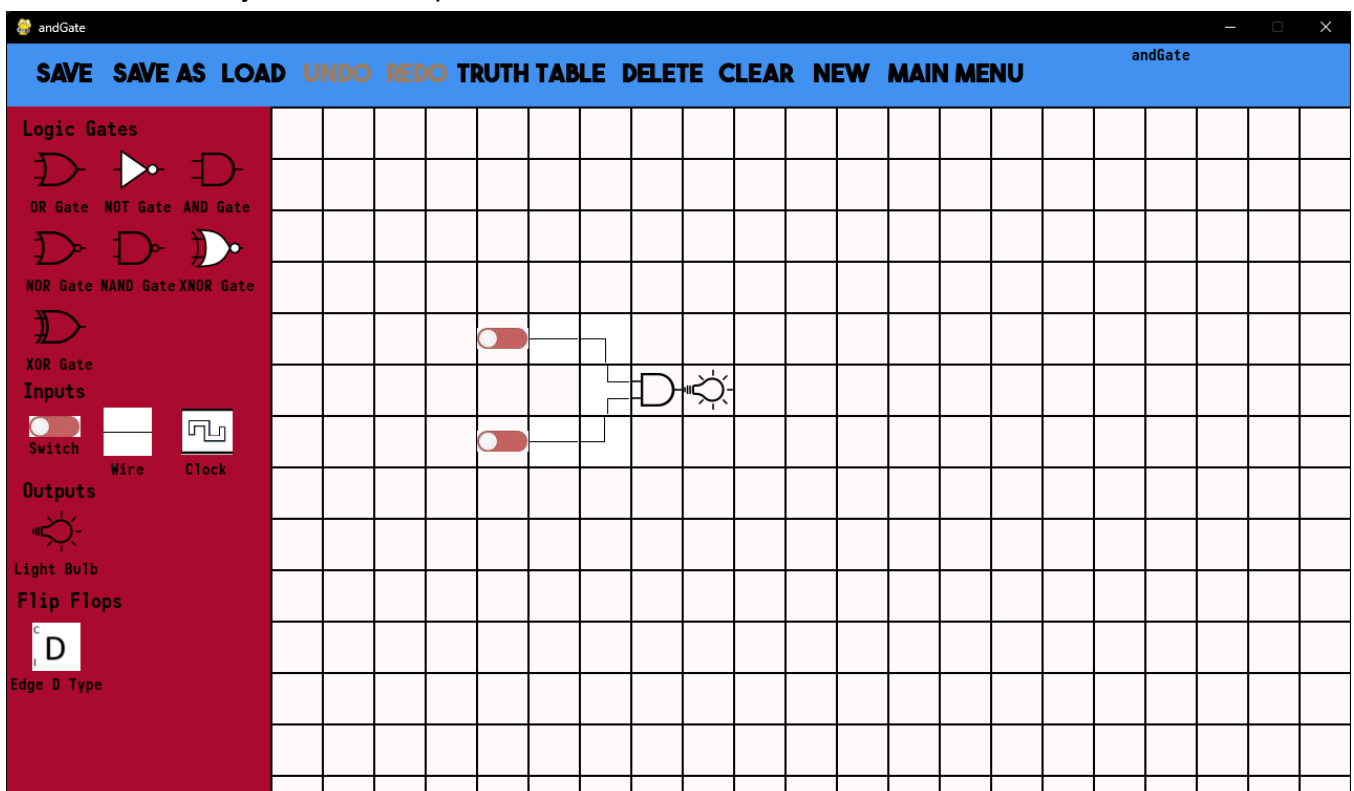






2.6. Clear - Pass

Clear functionality works as expected.



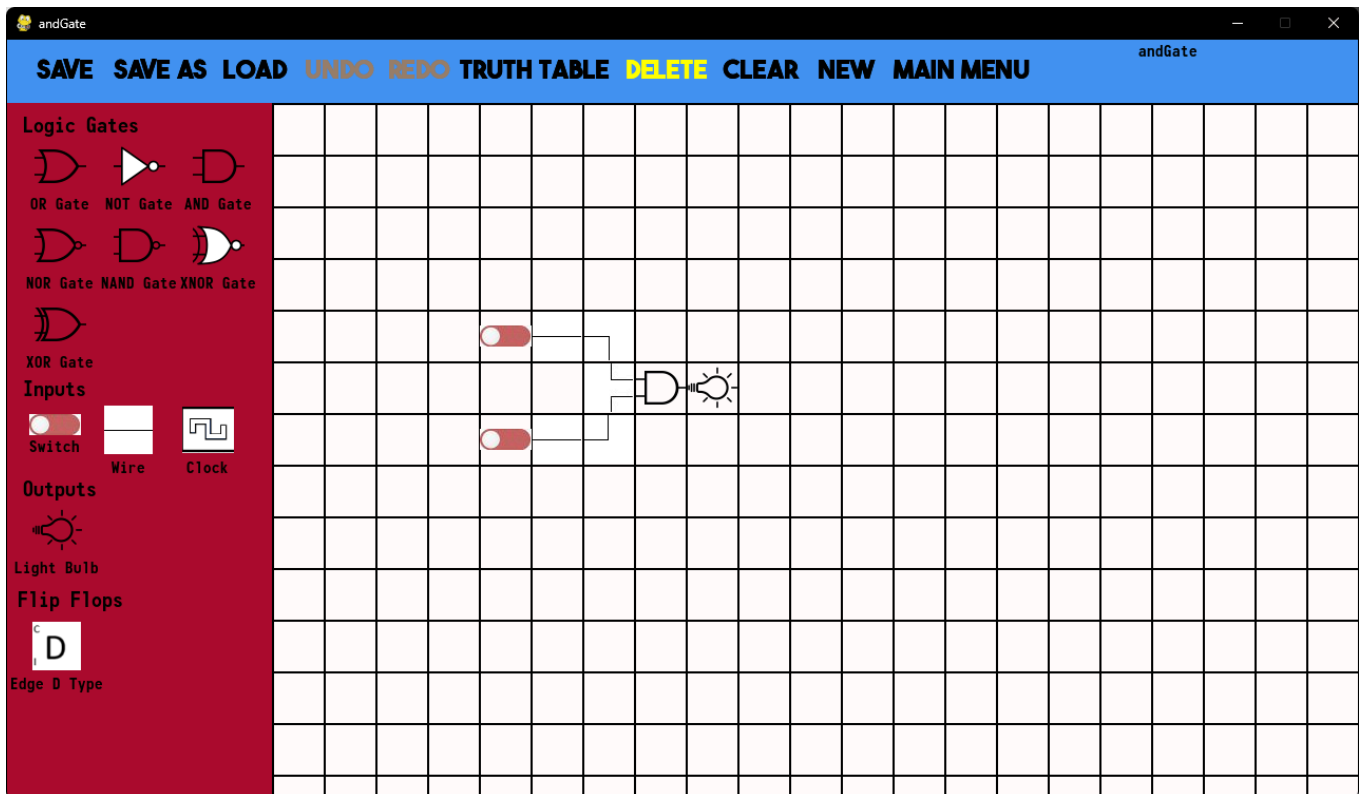


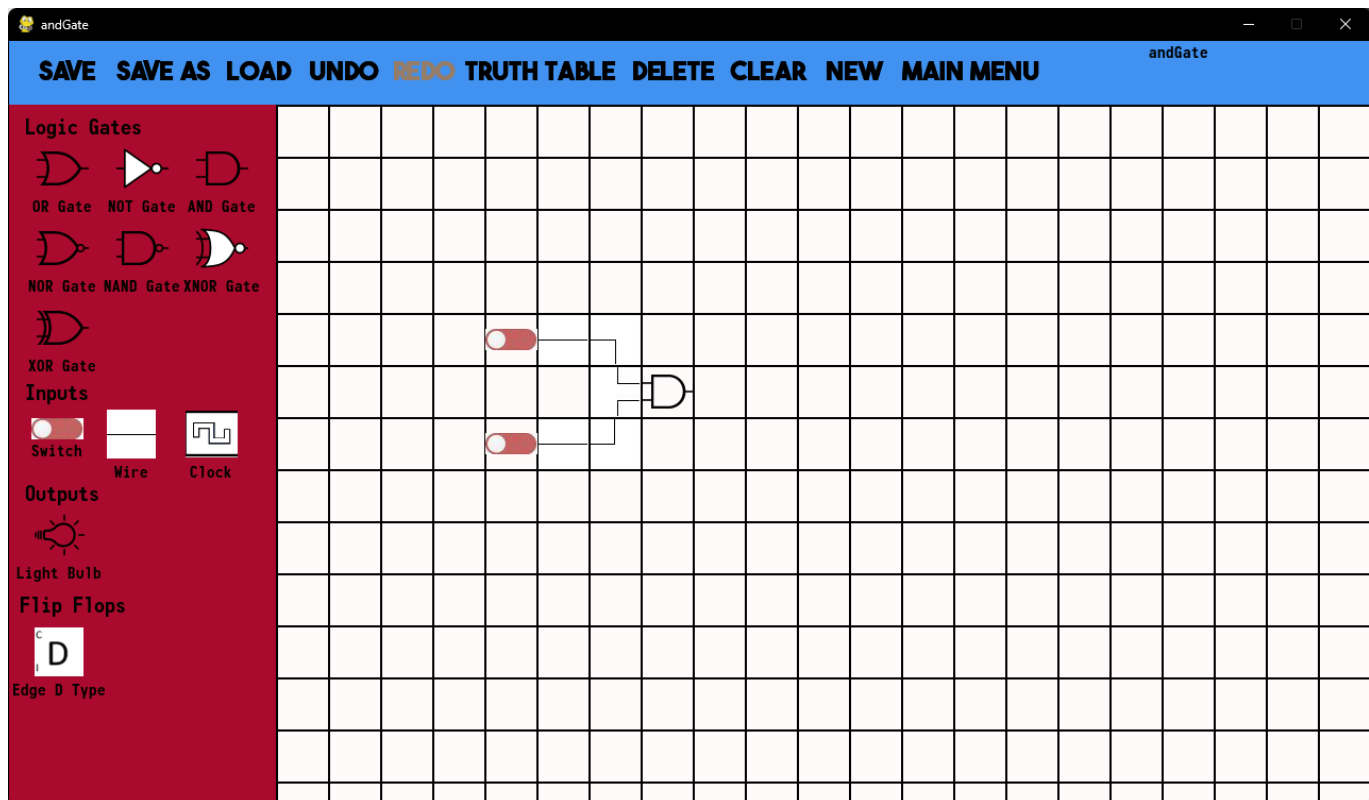
2.7. Translate between logic circuits and boolean algebra - **Fail**

Functionality not included in the final project.

2.8. Delete logic element - **Pass**

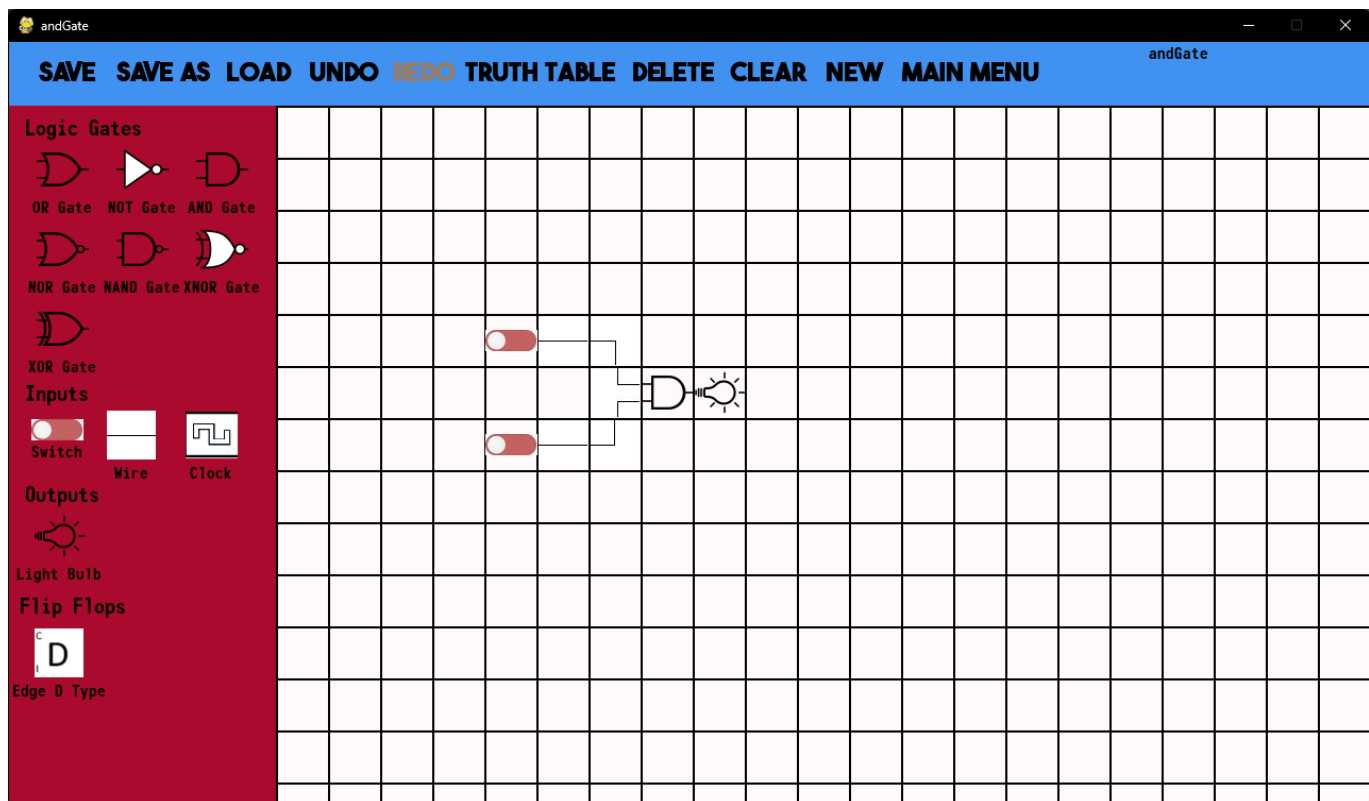
Delete functionality works as expected.





2.9. Truth table - Pass

Truth tables function as expected in all scenarios



Truth Table

Inputs		Outputs
0	0	0
0	1	0
1	0	0
1	1	1

BACK

tripleCircuit

SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

Logic Gates

OR Gate NOT Gate AND Gate

NOR Gate NAND Gate XNOR Gate

XOR Gate

Inputs

Switch Wire Clock

Outputs

Light Bulb

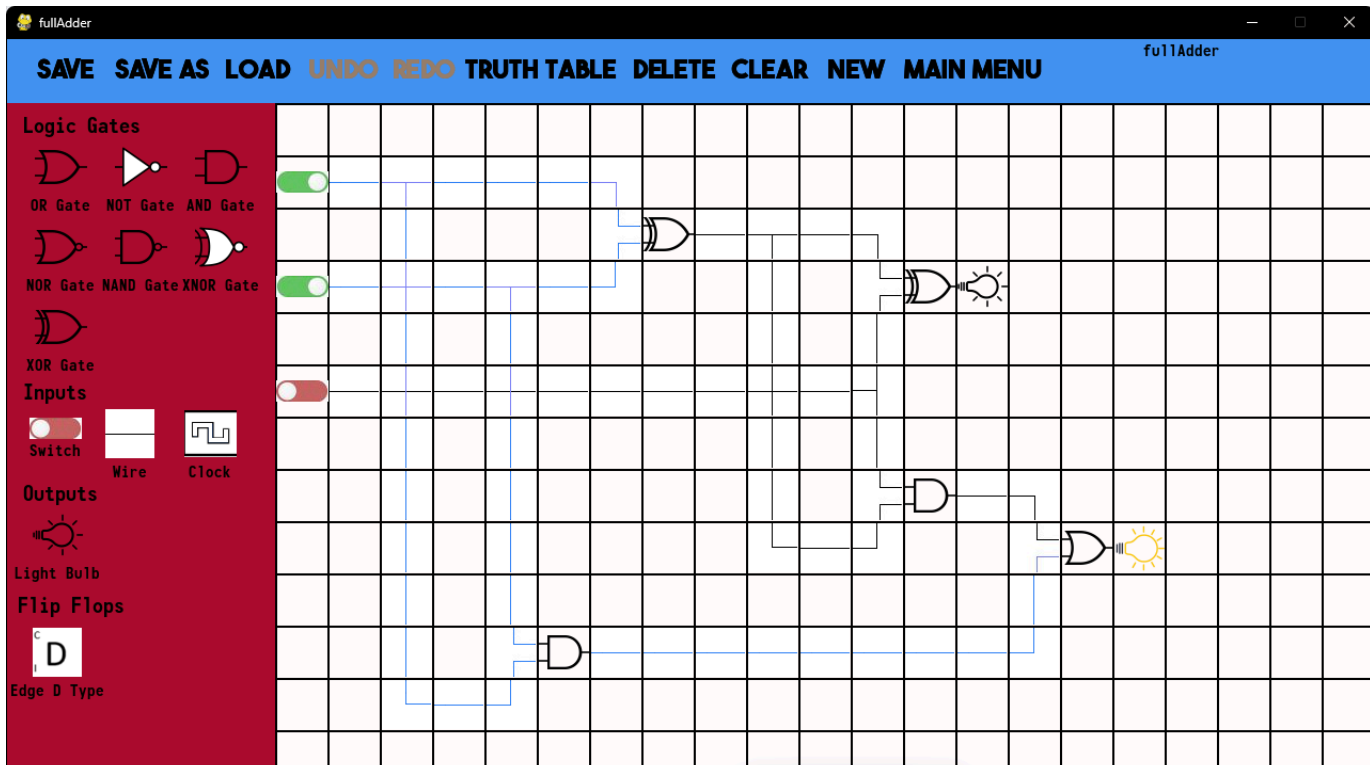
Flip Flops

Edge D Type

Truth Table

Inputs			Outputs	
0	0	0	1	1
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	0	0

BACK



Truth Table

Inputs		Outputs		
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

BACK

quadrupleCircuit

SAVE SAVE AS LOAD UNDO REDO TRUTH TABLE DELETE CLEAR NEW MAIN MENU

quadrupleCircuit

Logic Gates

OR Gate NOT Gate AND Gate

NOR Gate NAND Gate XNOR Gate

XOR Gate

Inputs

Switch Wire Clock

Outputs

Light Bulb

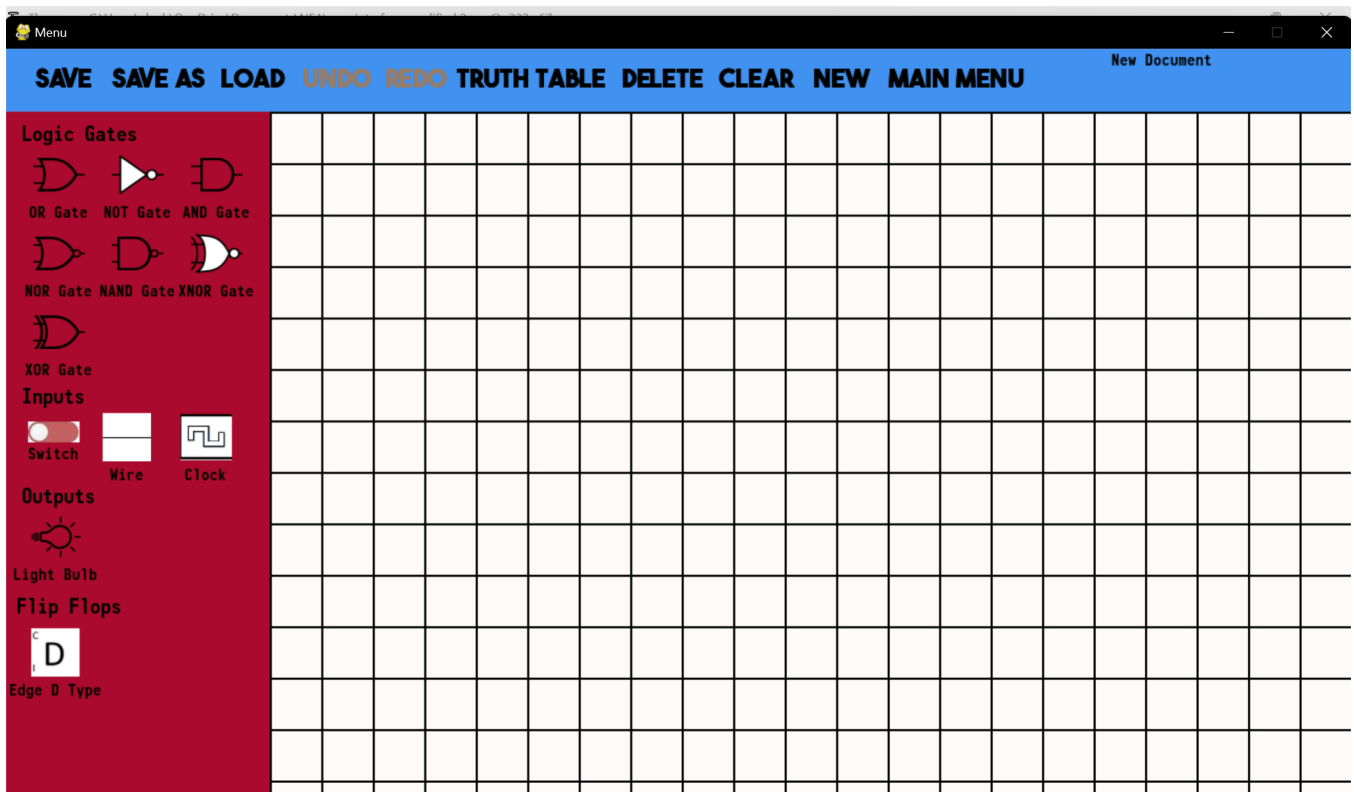
Flip Flops

Edge D Type

Truth Table				
Inputs				Outputs
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

2.10. New - Pass

New document functionality works as expected.



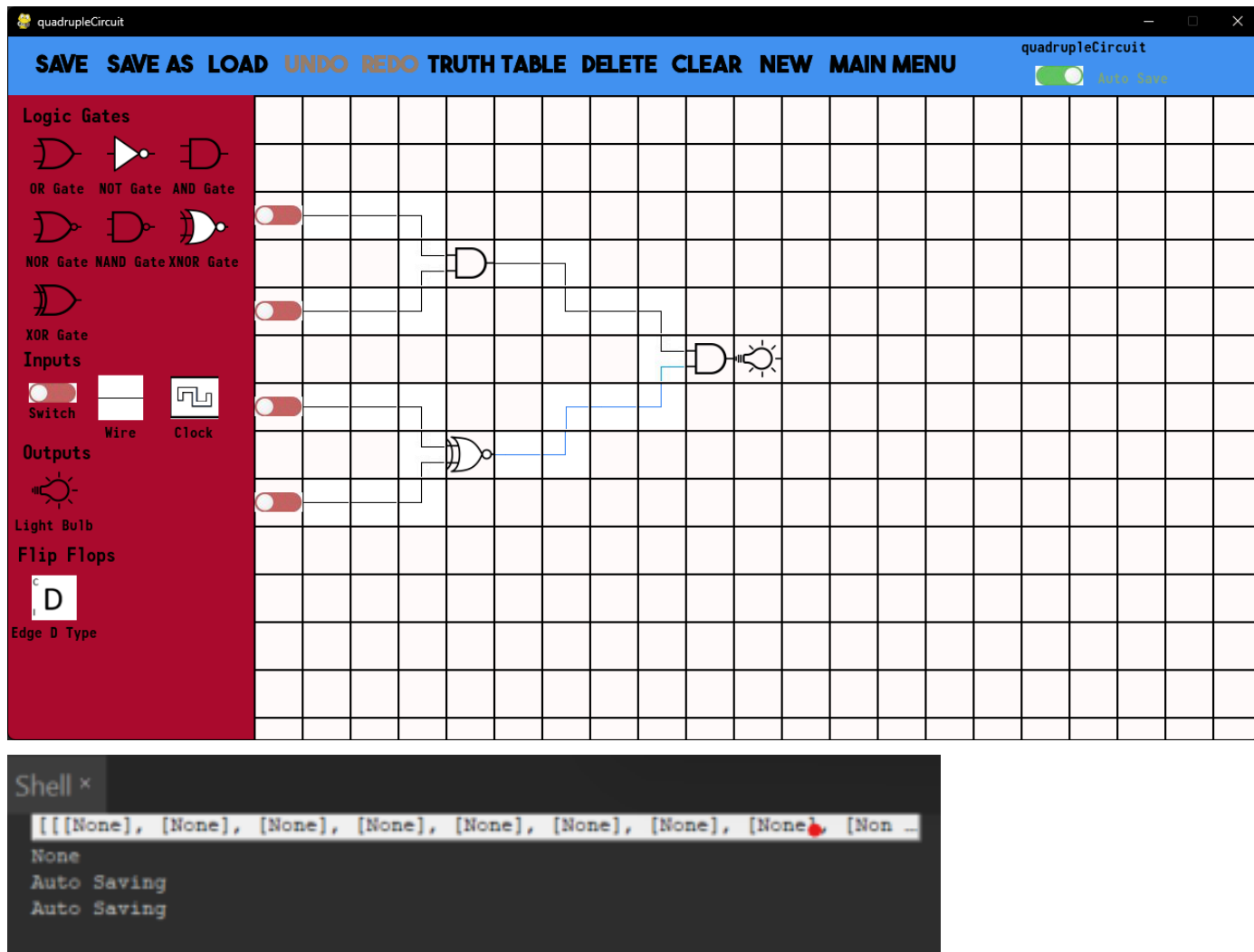
2.11. Main menu - **Pass**

Main menu functionality works as expected.



2.12. Auto save - **Pass**

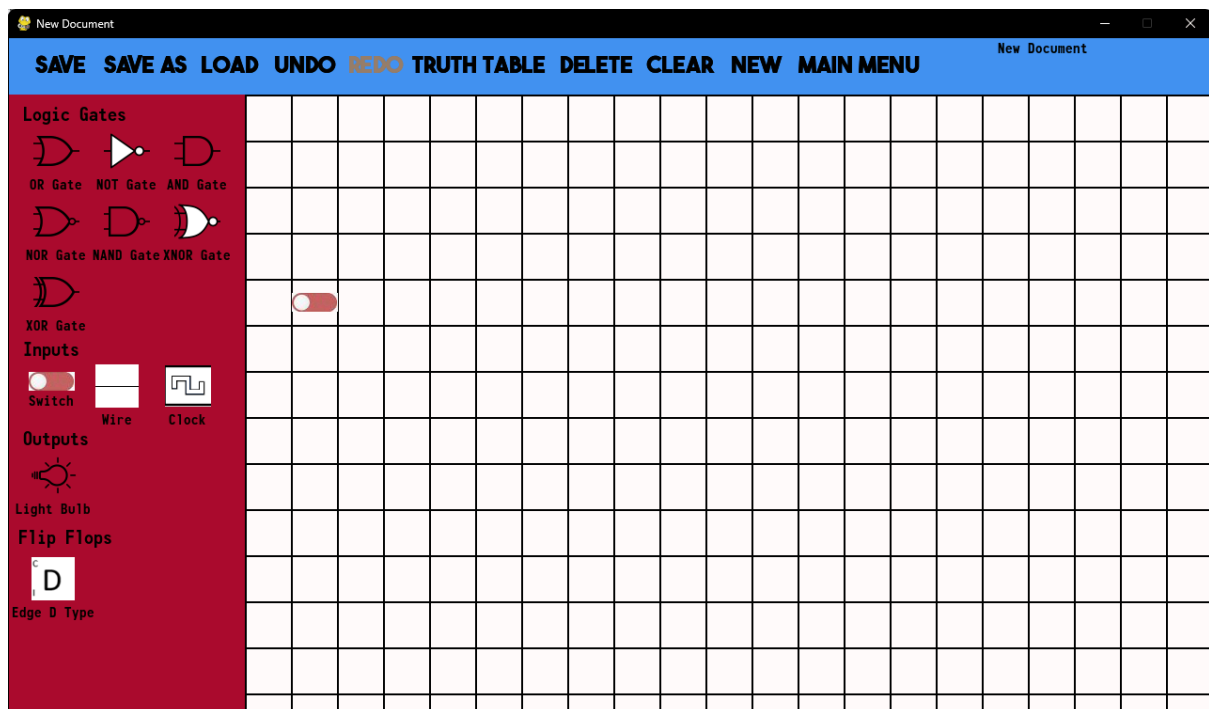
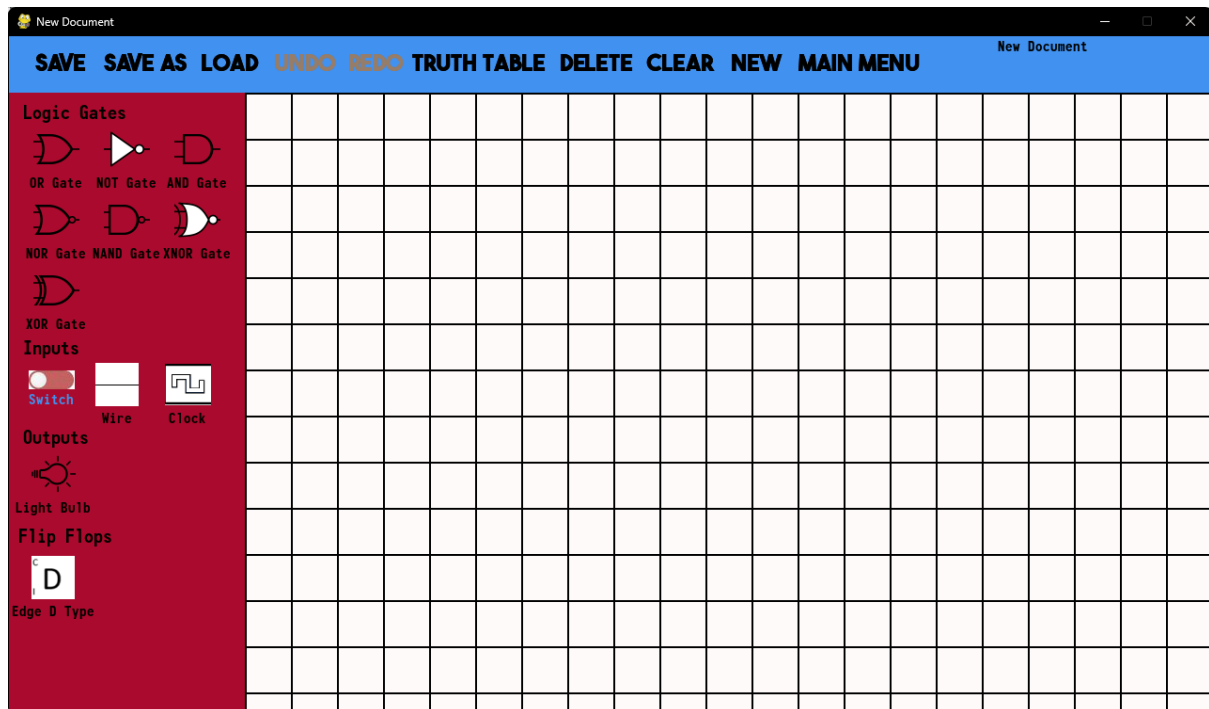
Auto saving functionality works as expected

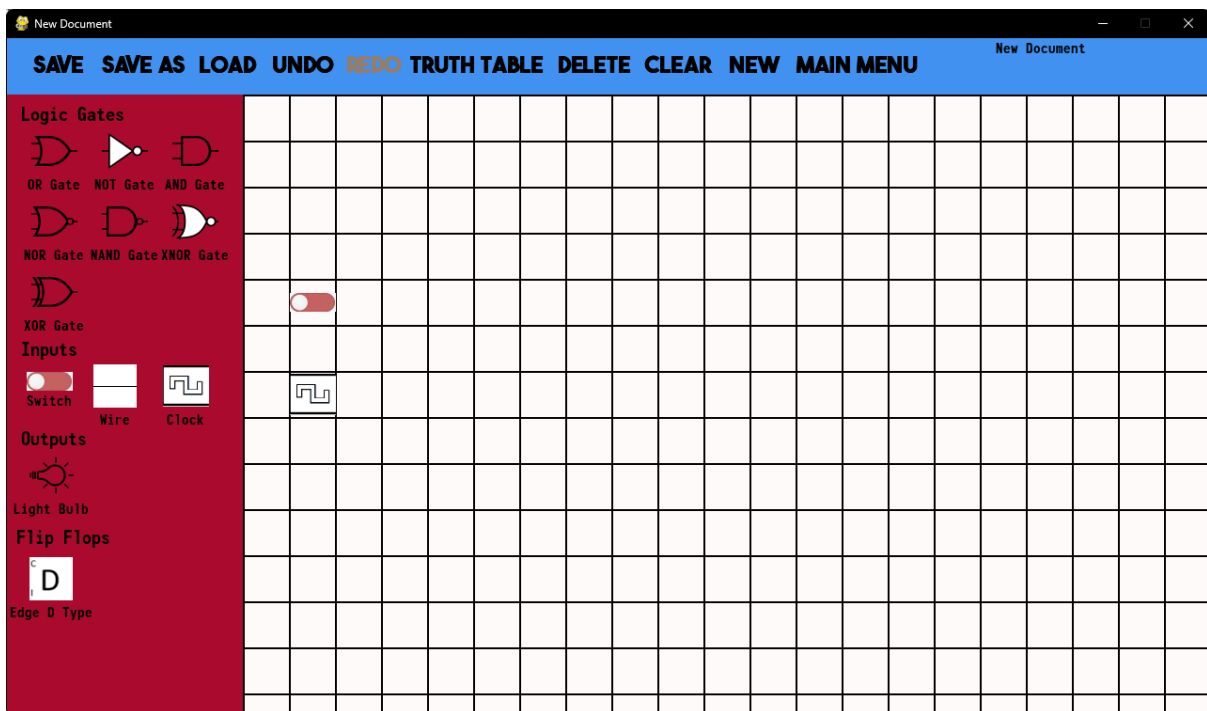
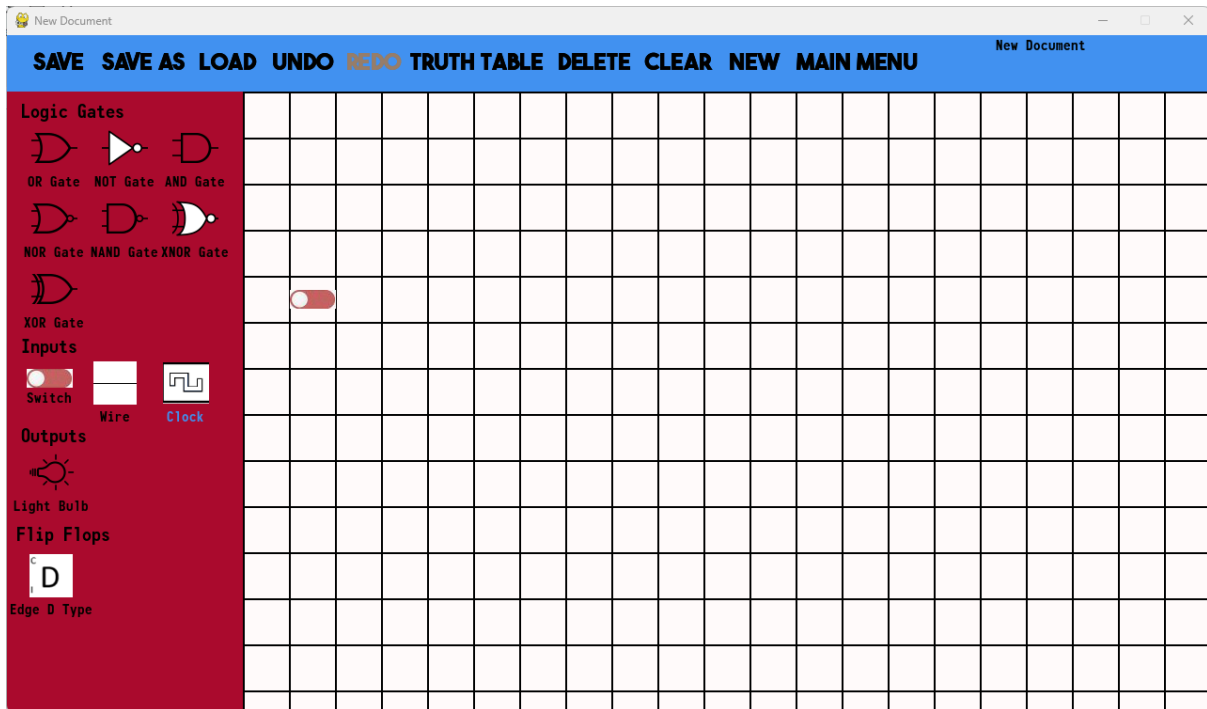


Objective 3: Logic interface

3.1. Inputs - Pass

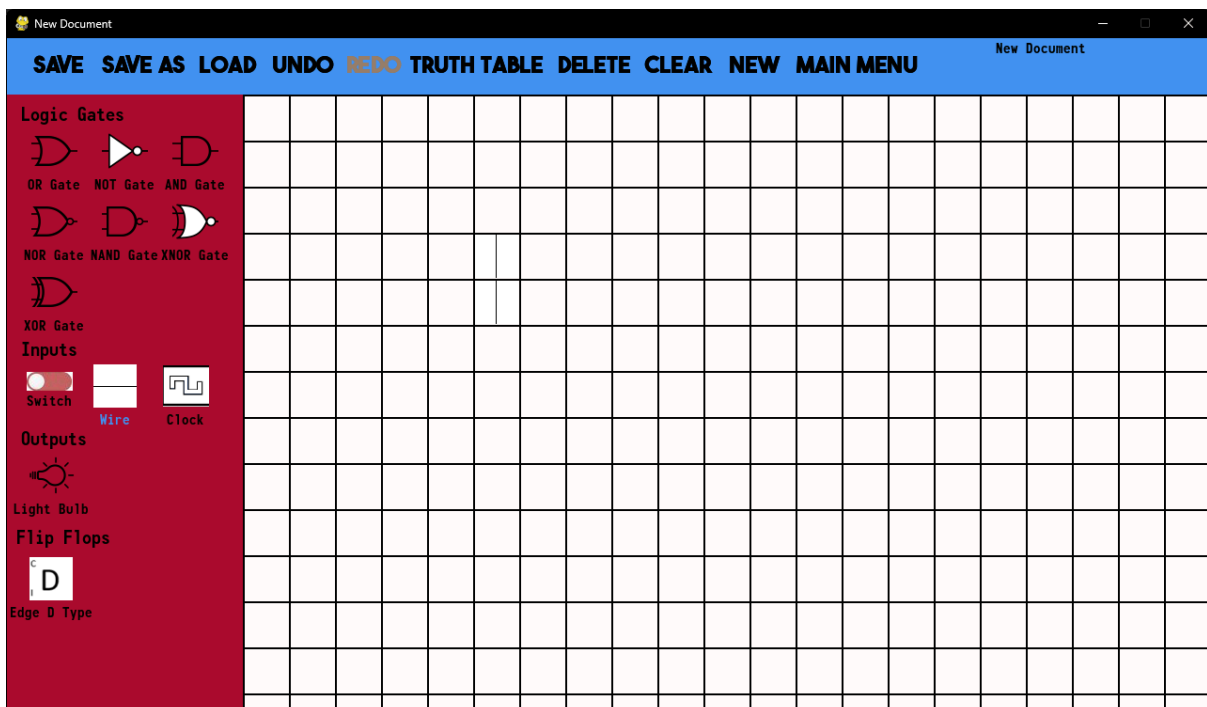
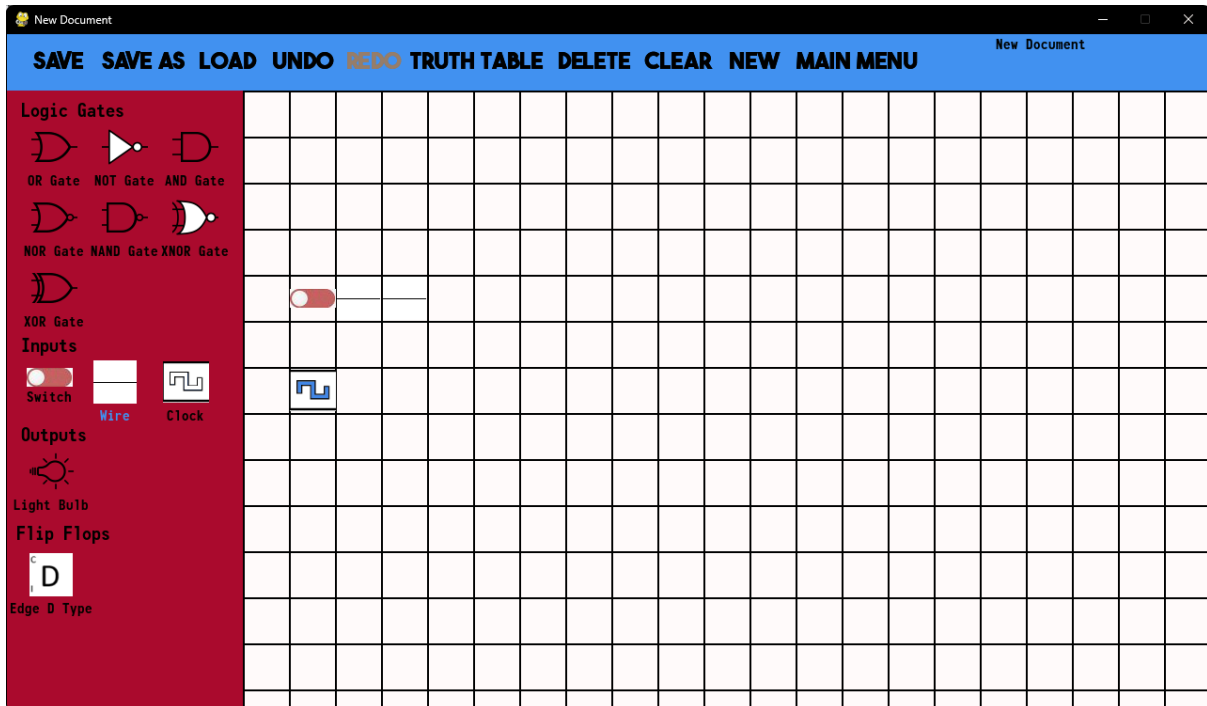
Clocks and switches behave as expected

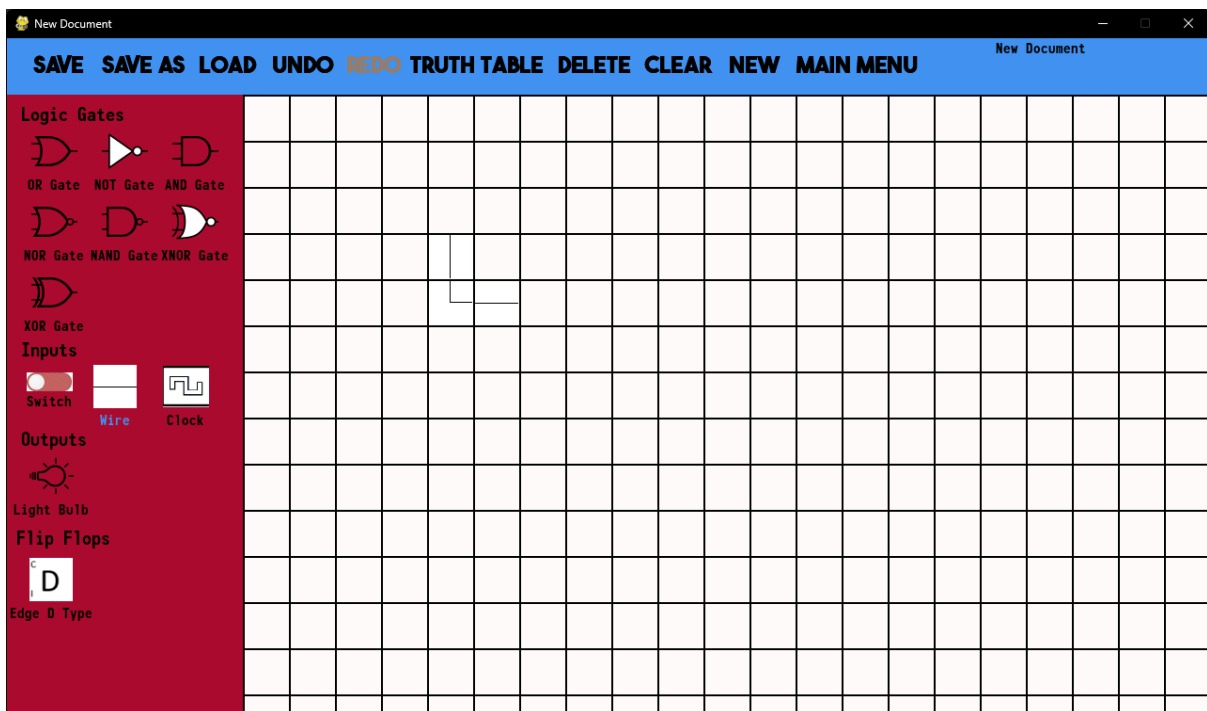
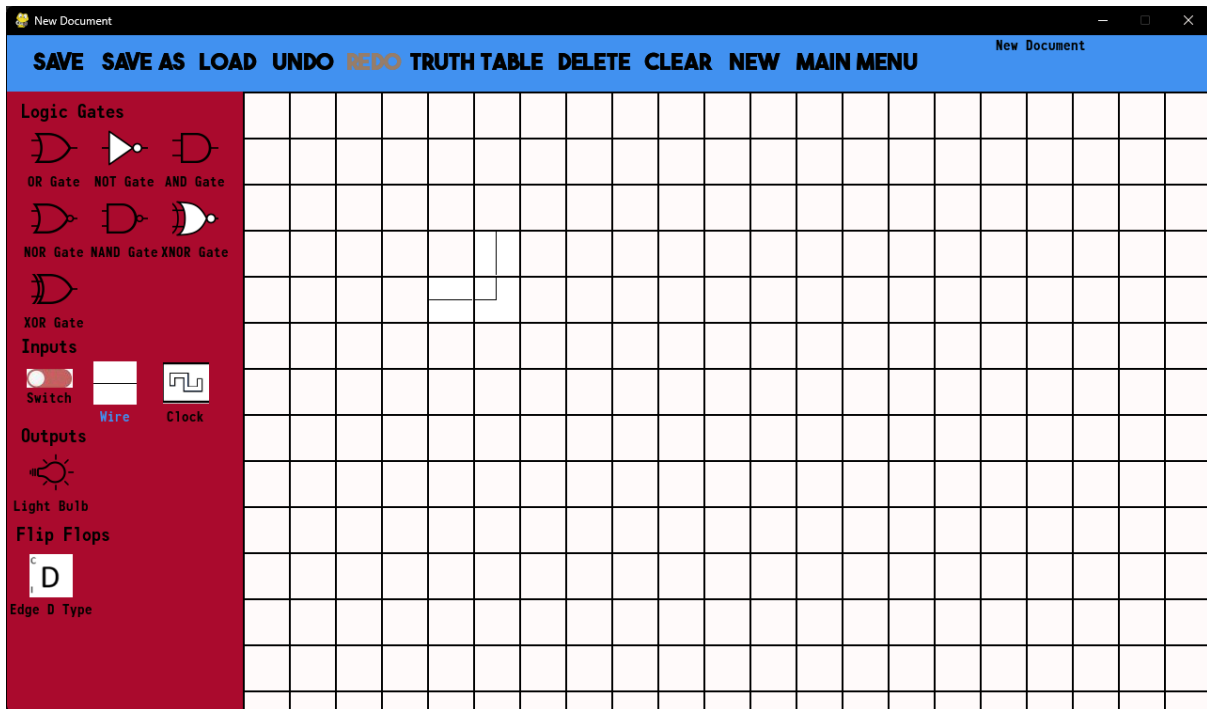


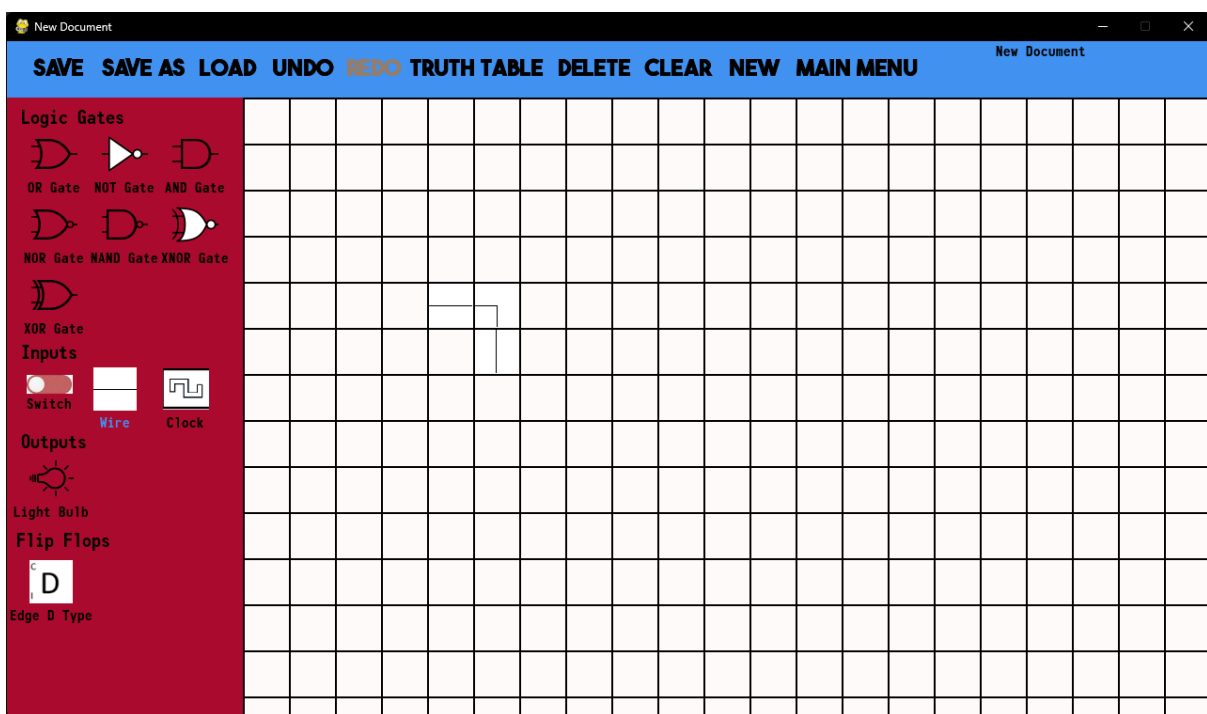


3.2. Wires - Pass

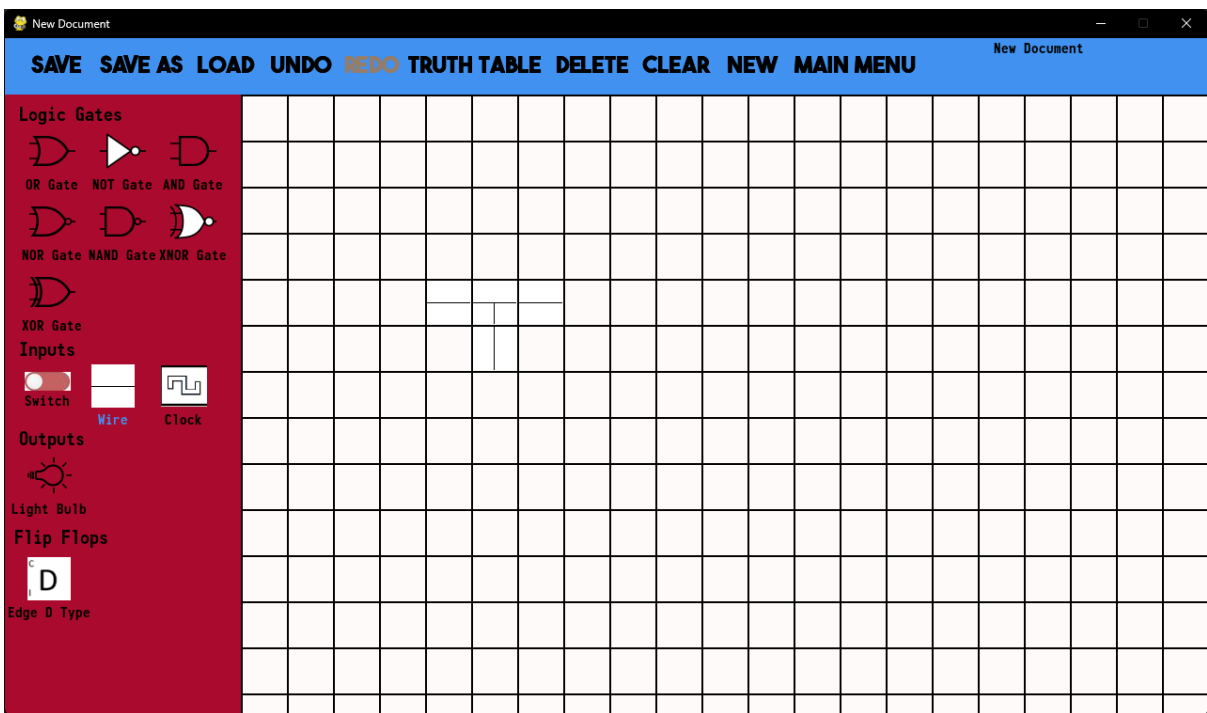
All wire combinations behave as expected

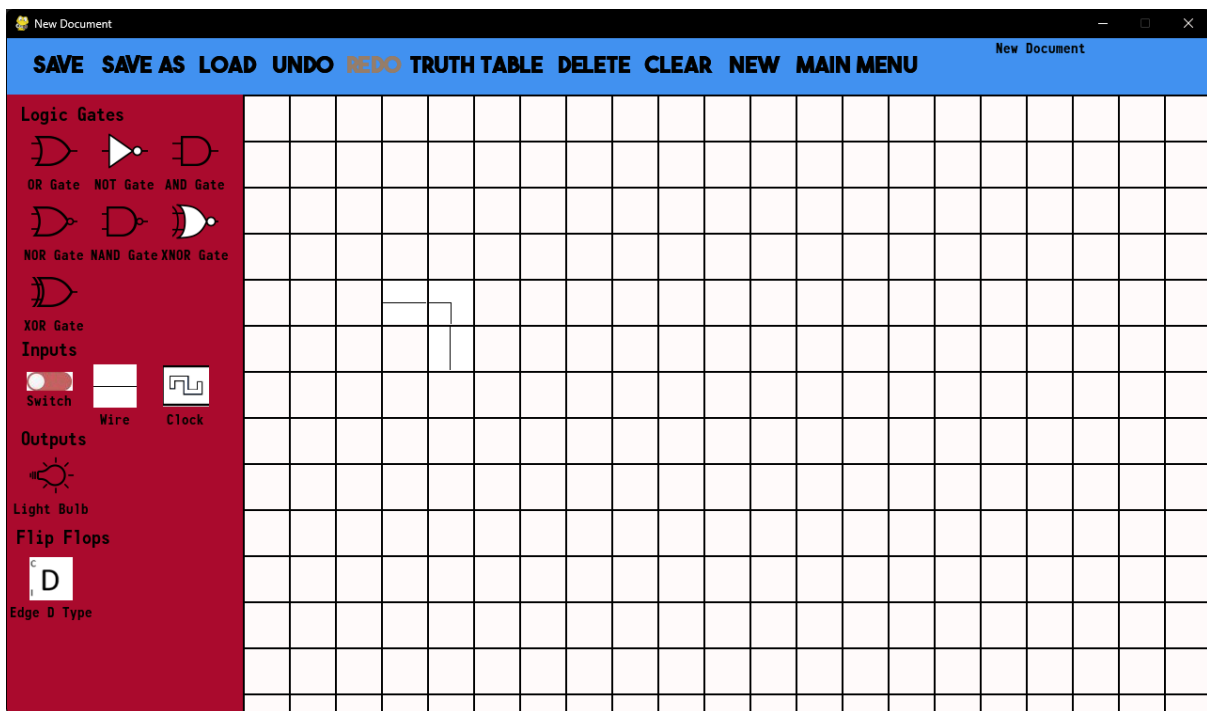




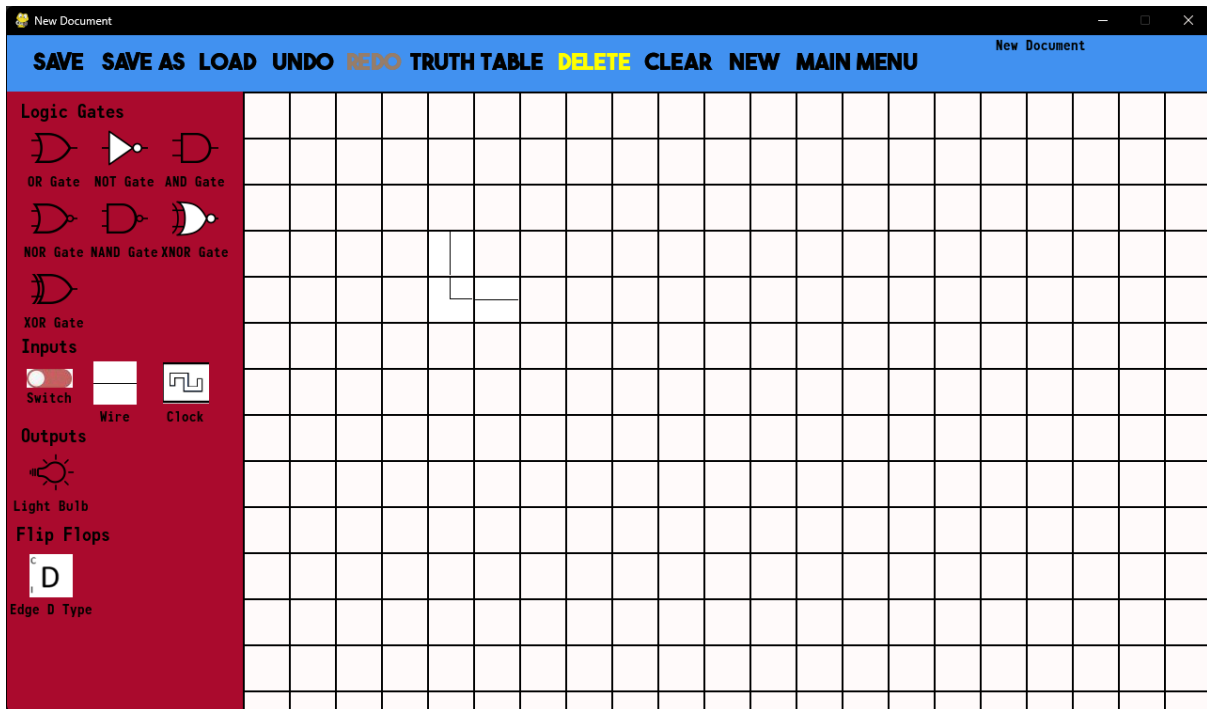


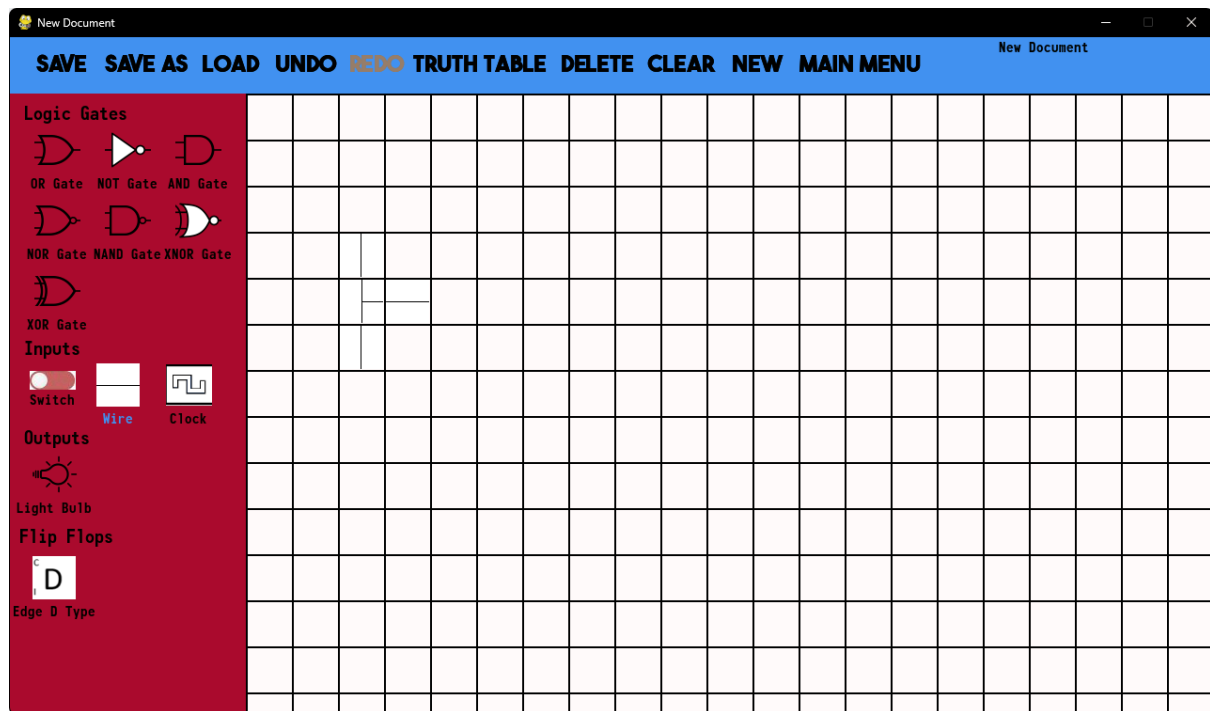
Logic gate simulator - Harry Taylor - 7173 - 65301

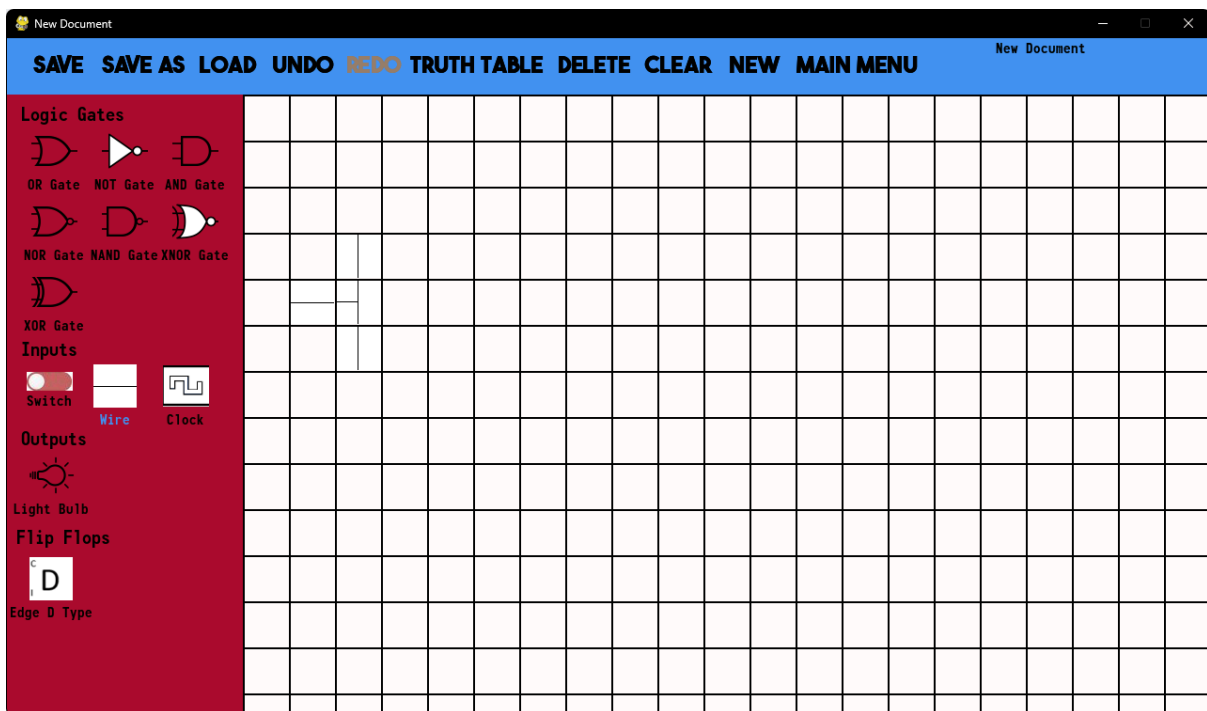
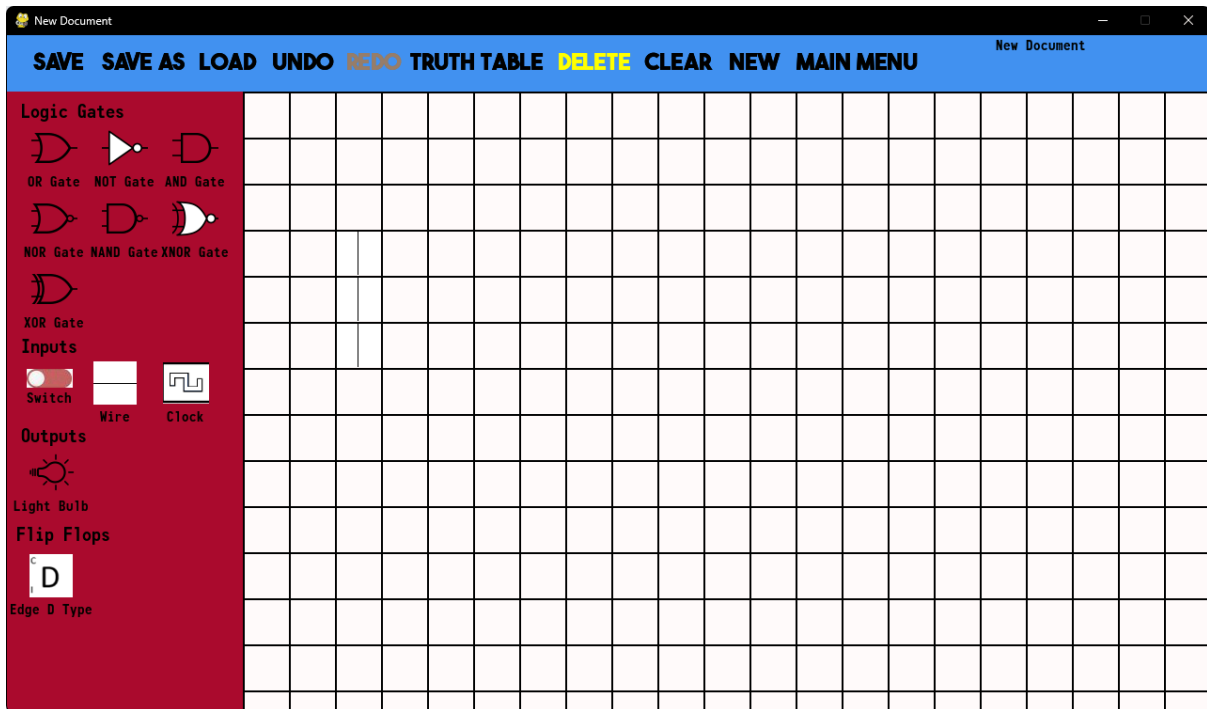


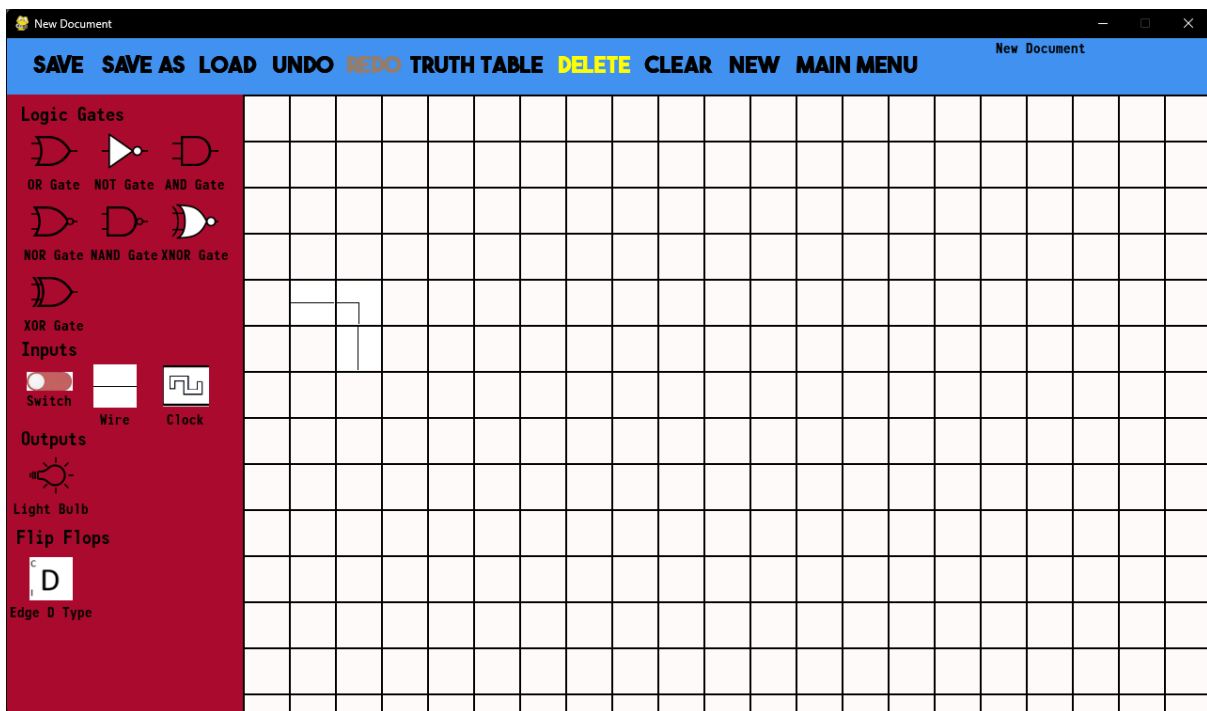
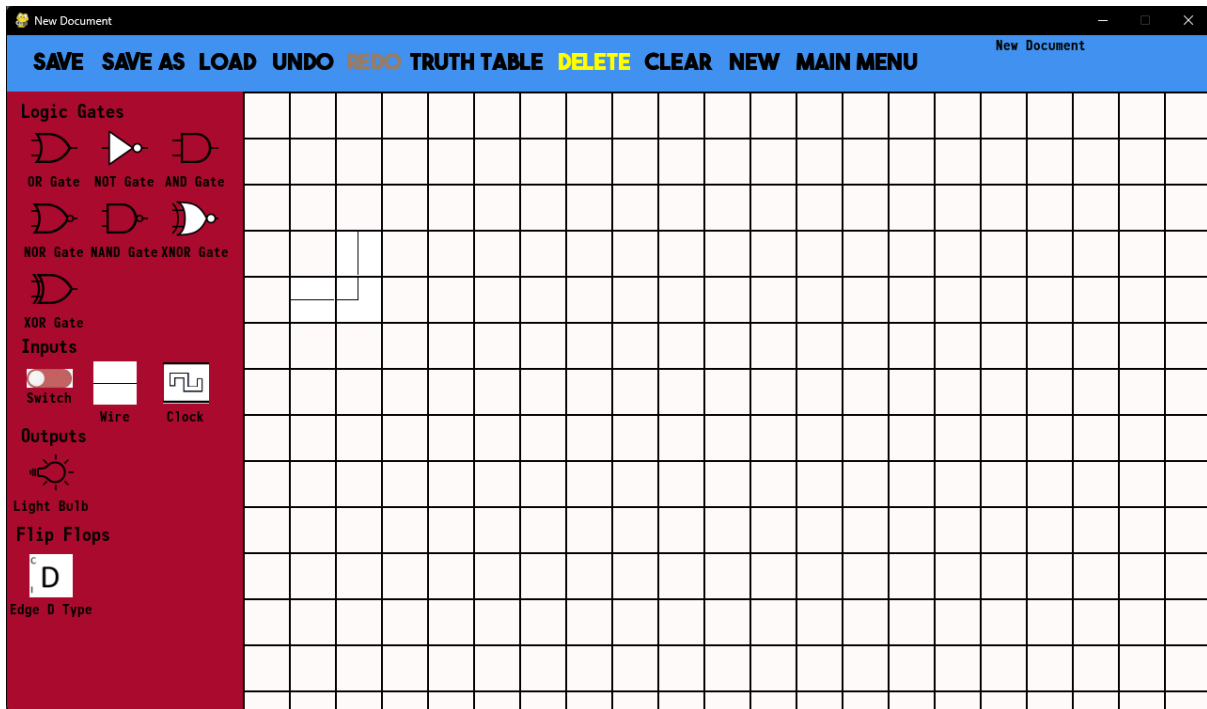


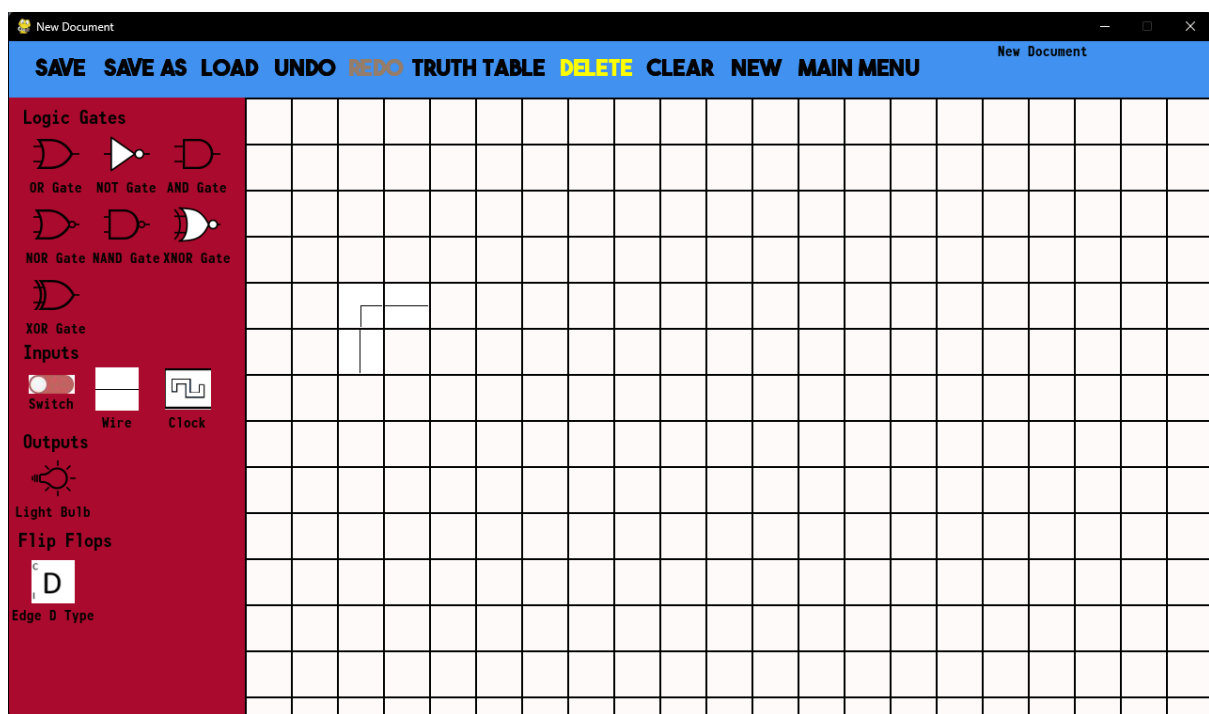


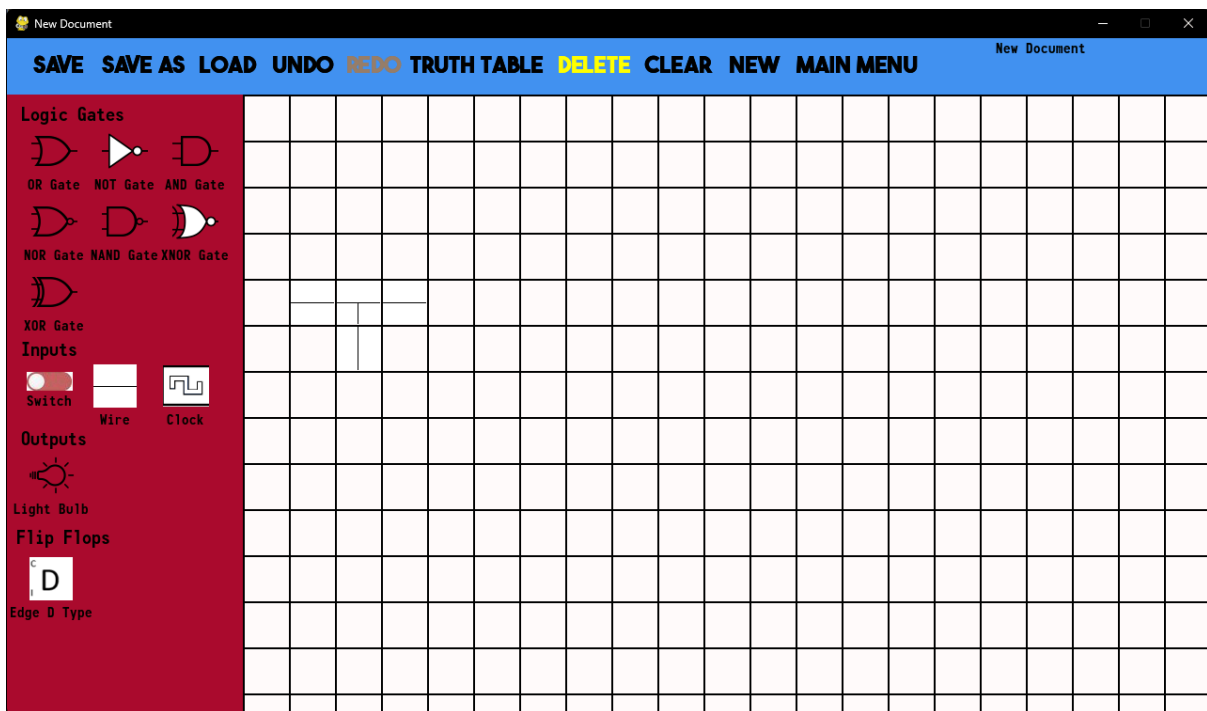


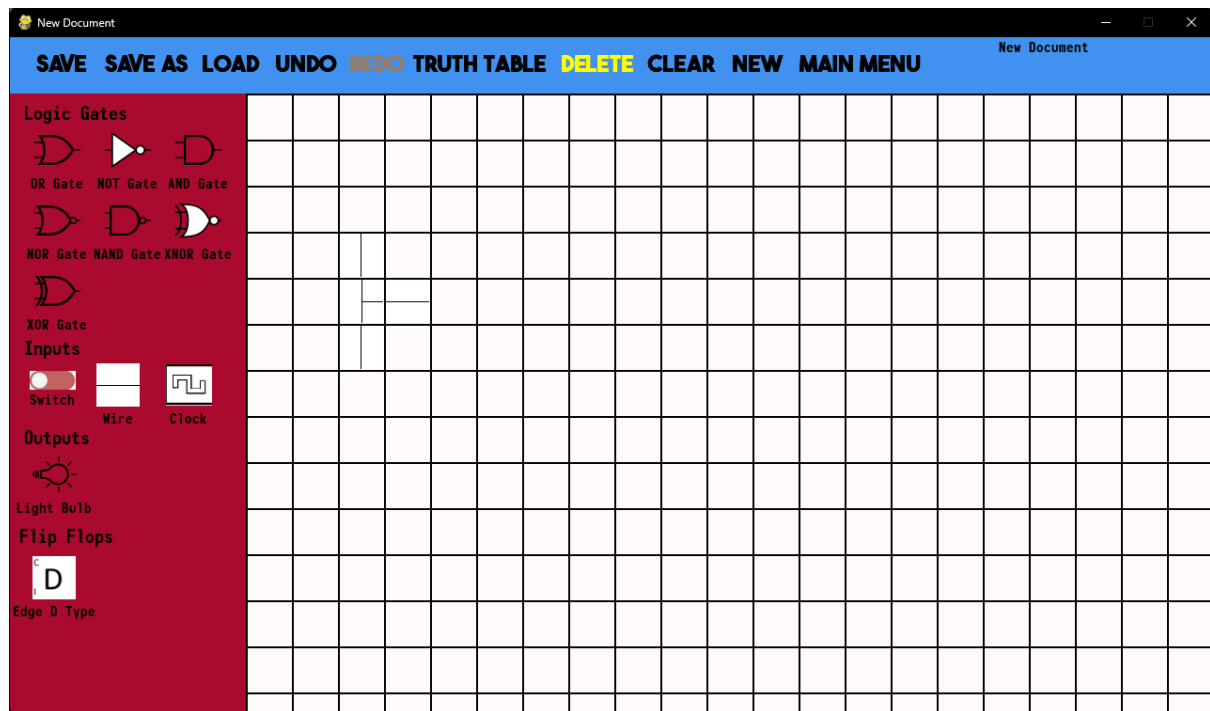


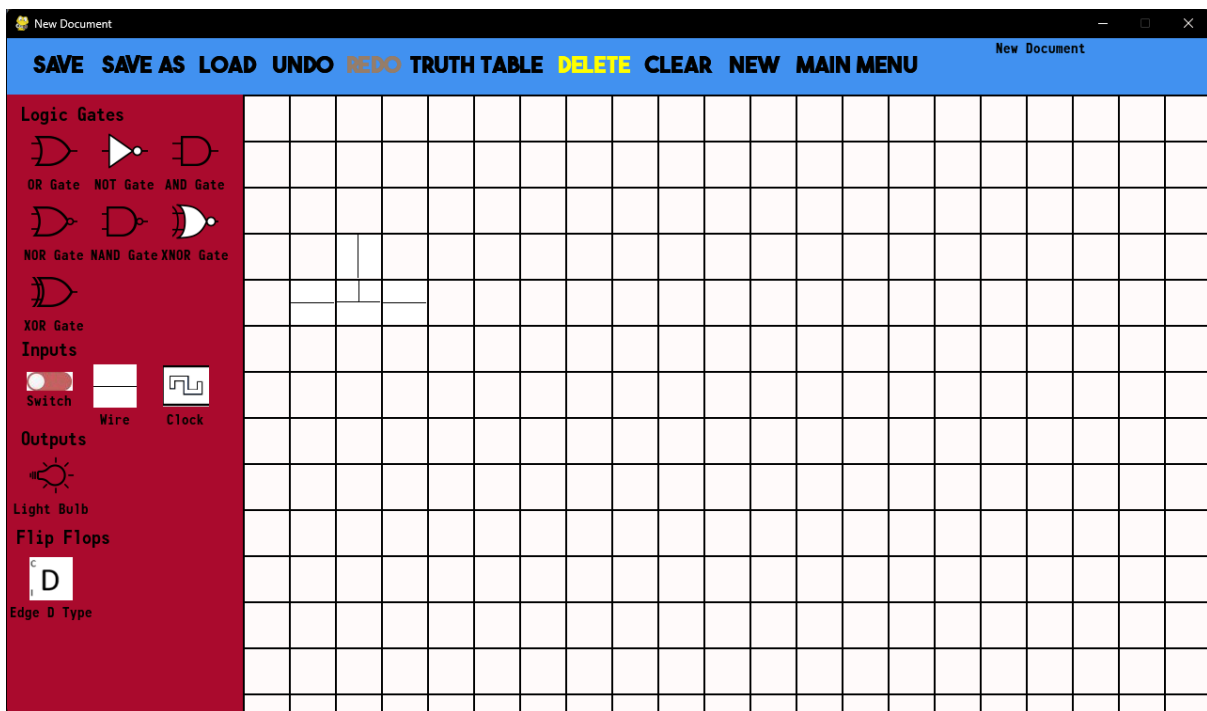
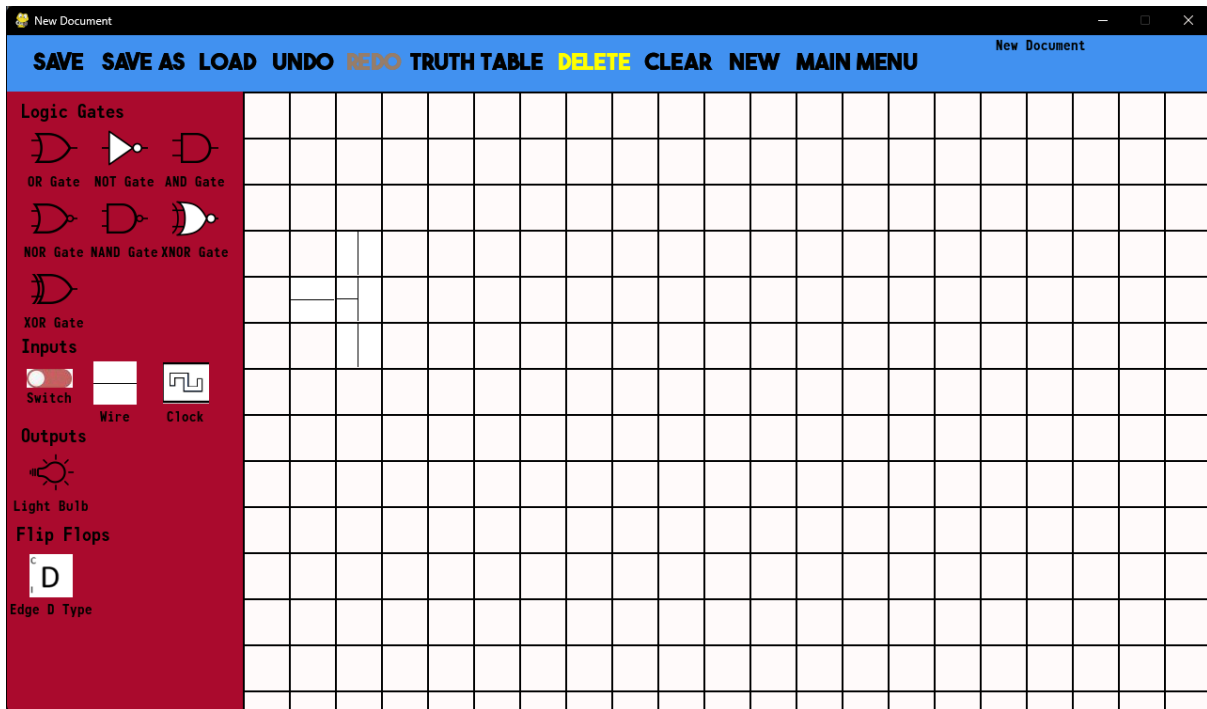






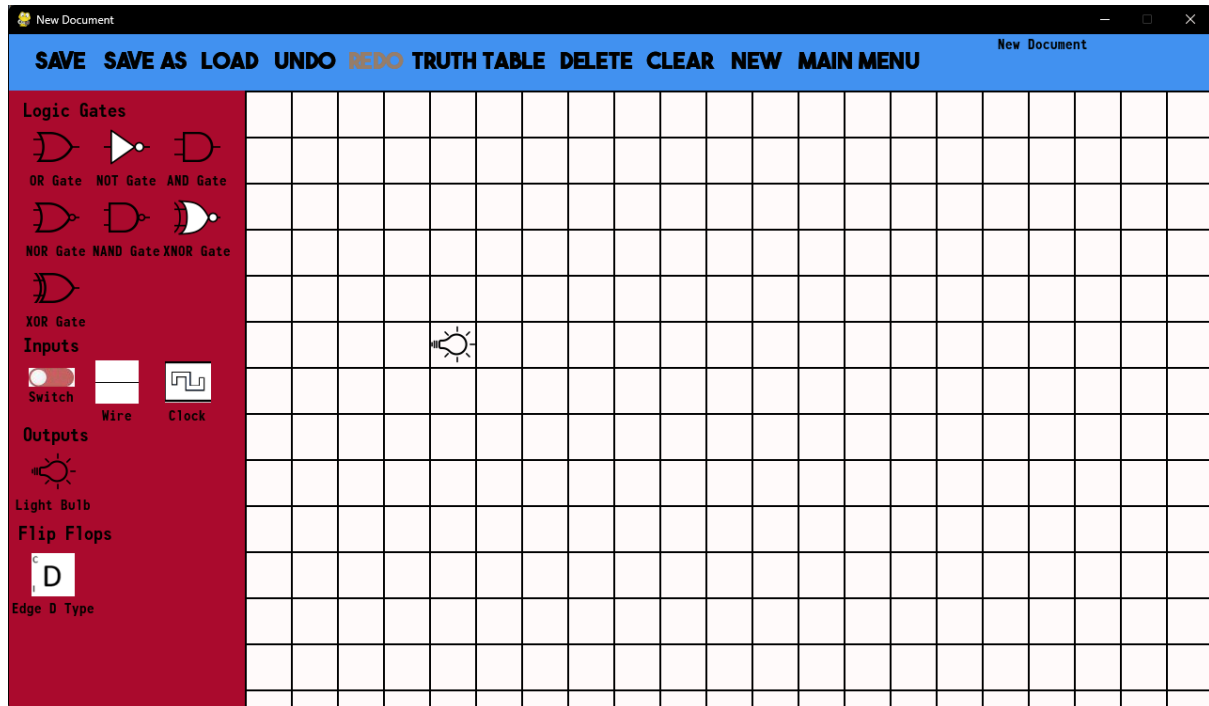






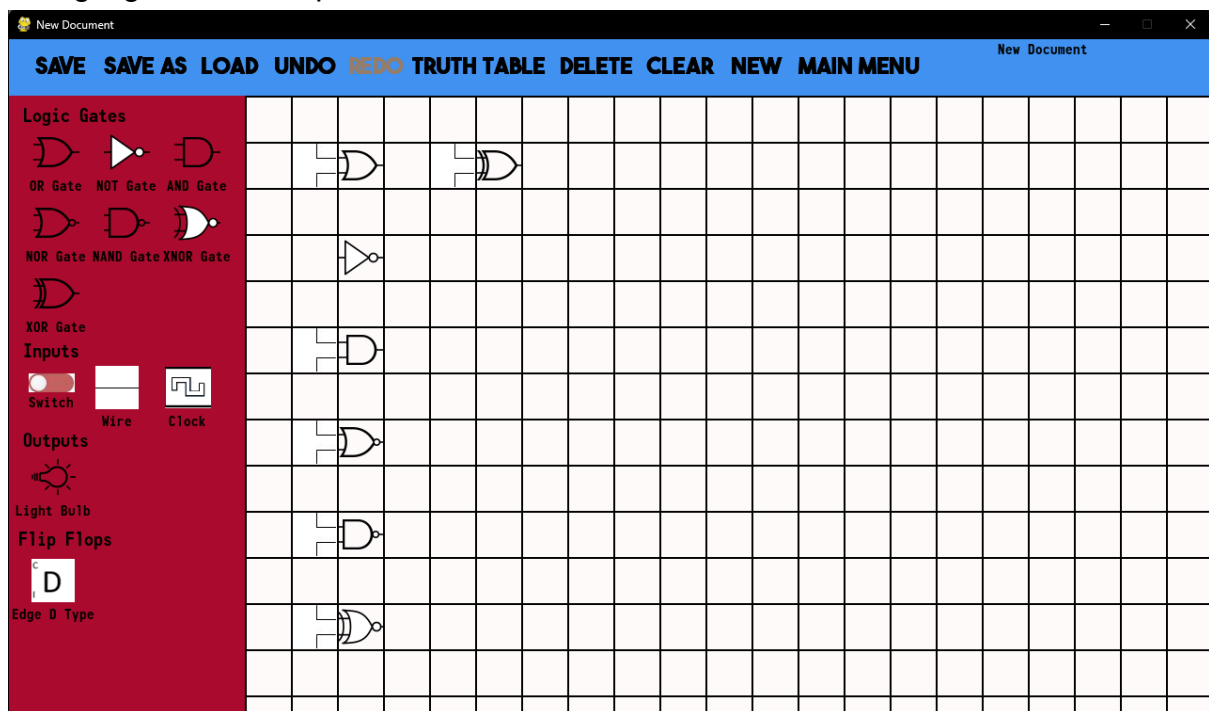
3.3. Outputs - Pass

Users are able to place light bulbs into their documents



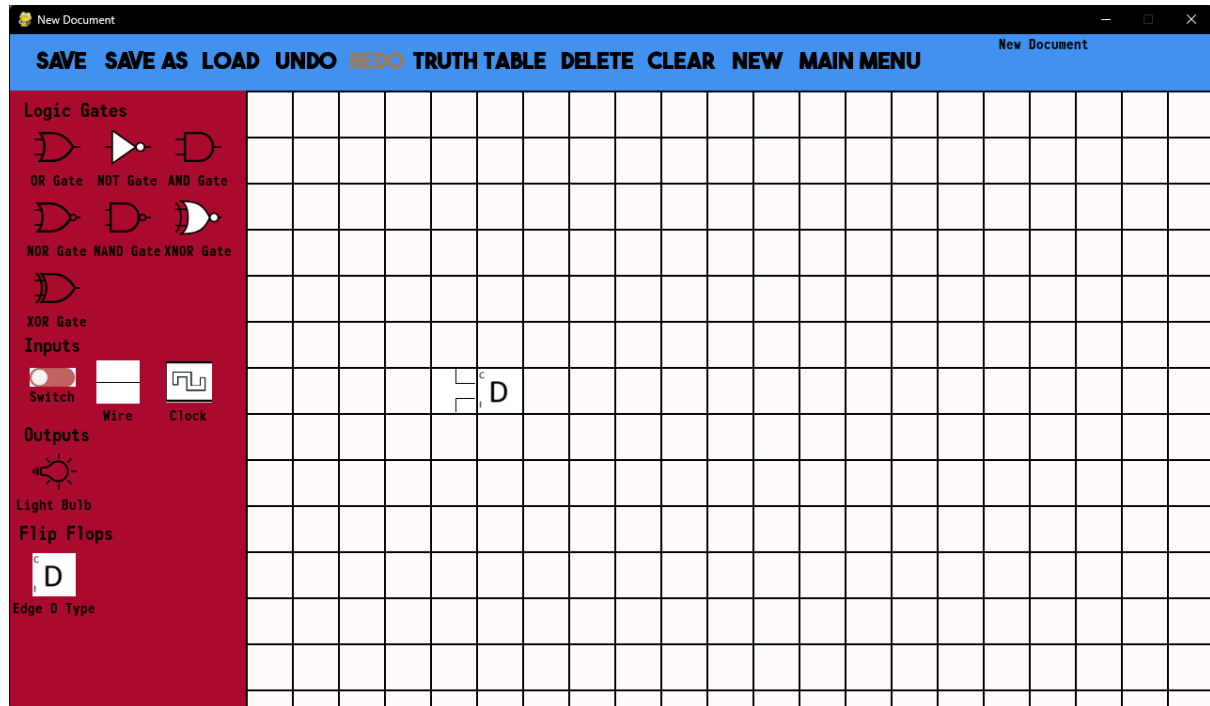
3.4. Logic gates - Pass

All logic gates can be placed



3.5. Flip-Flop gates - **Pass**

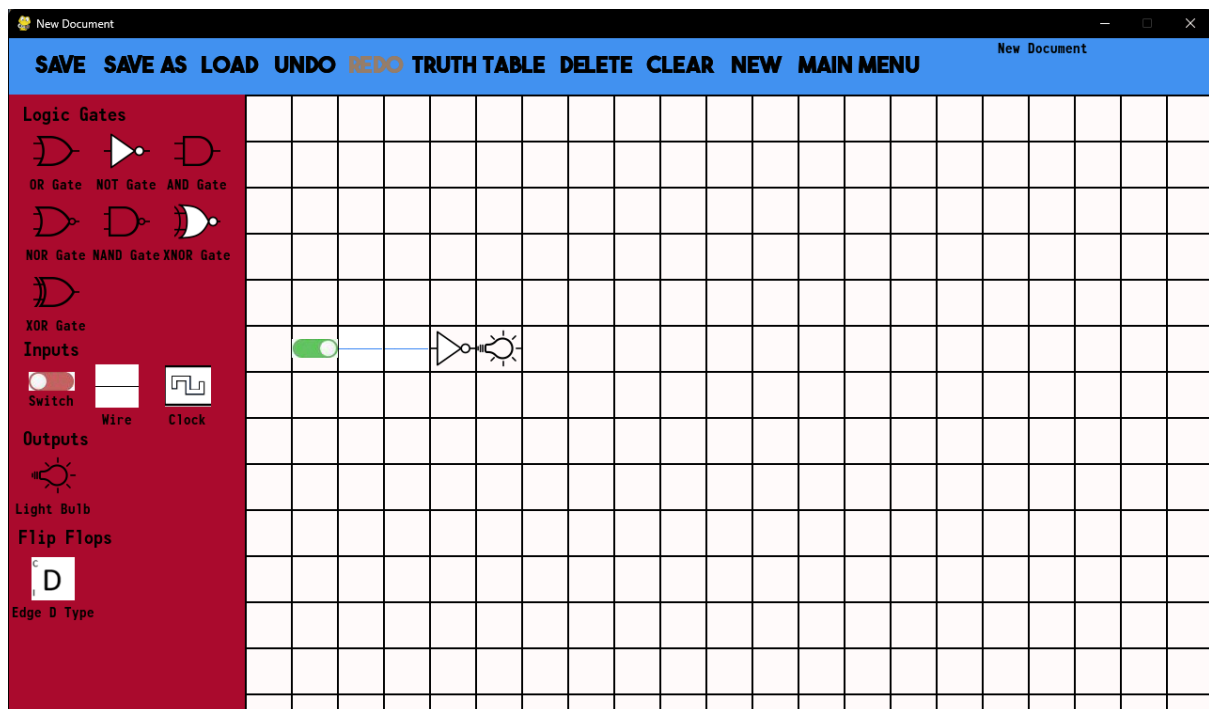
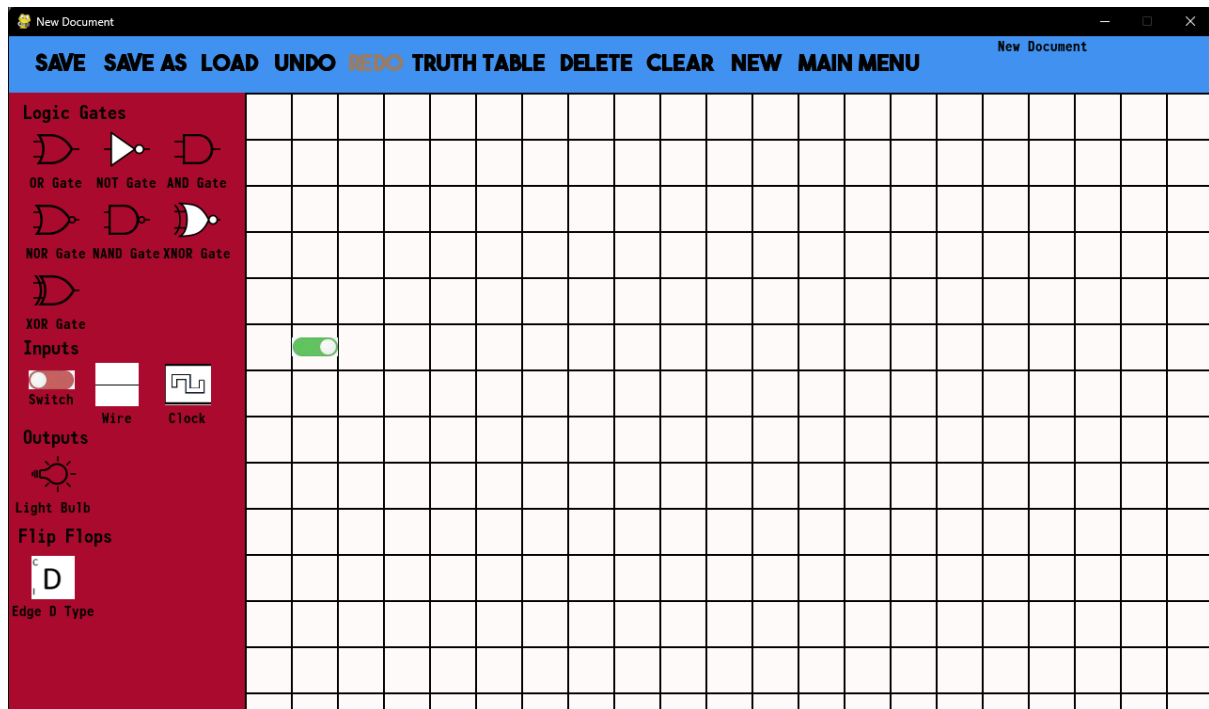
Flip flops place as expected

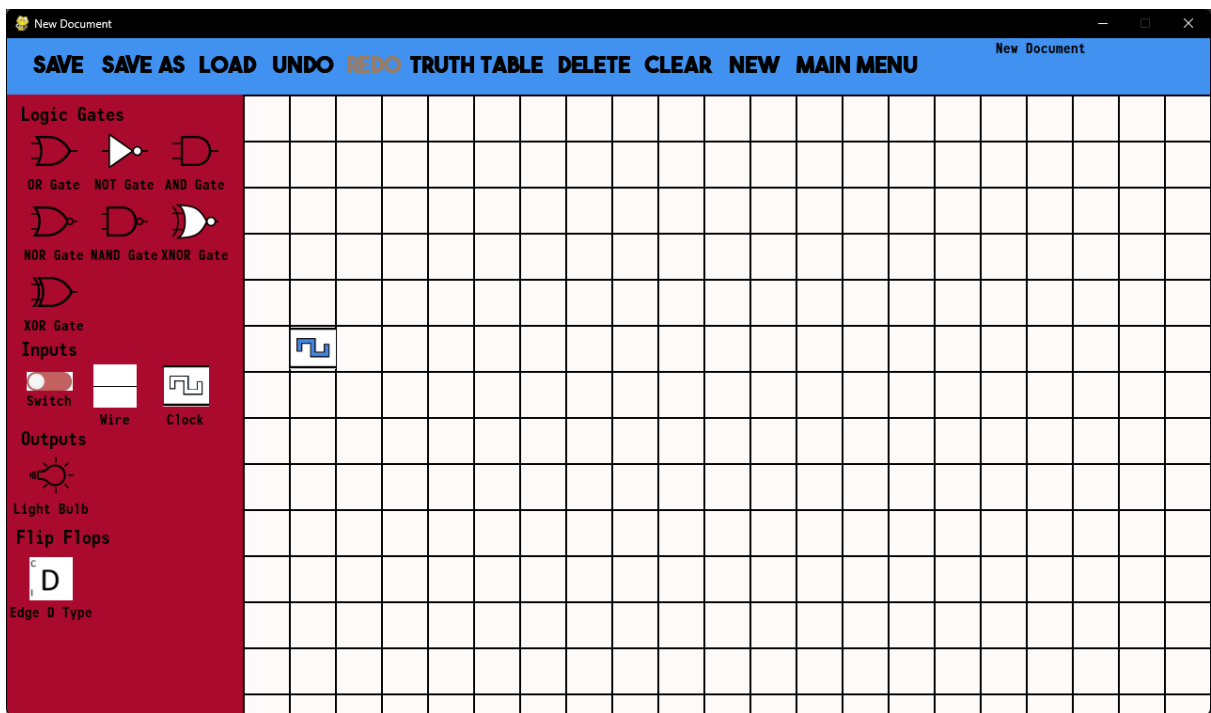
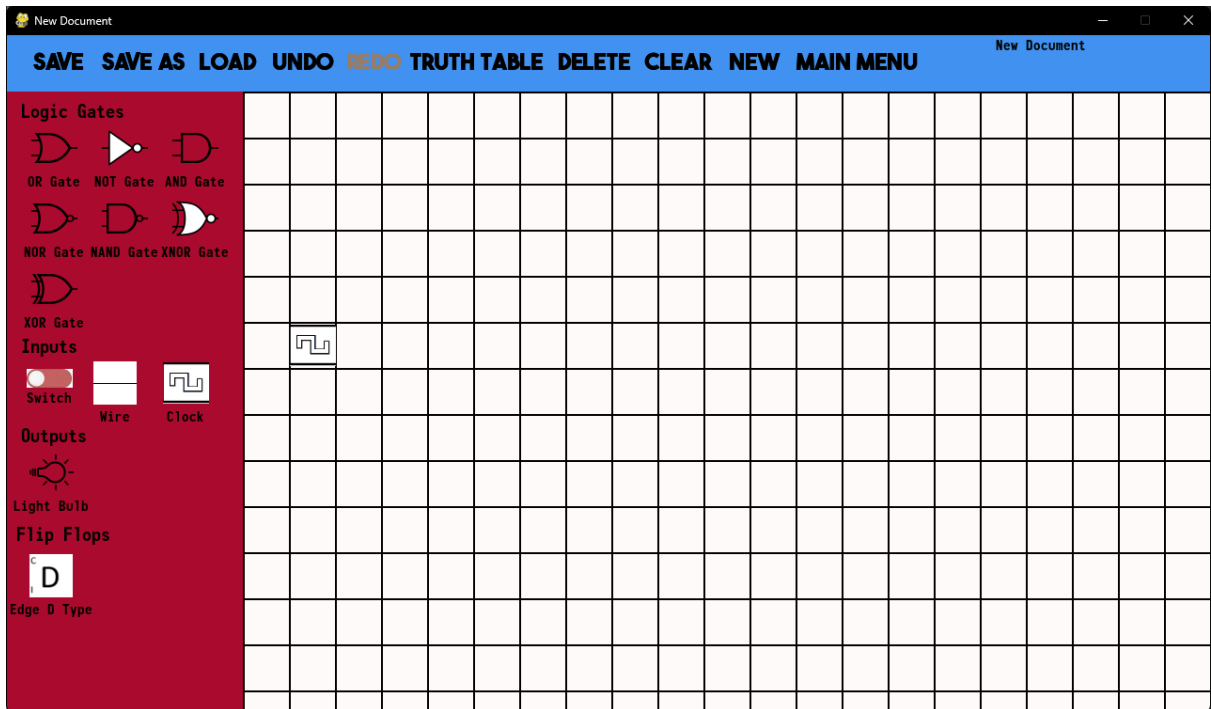


Objective 4: Logic gate interactions and shortcuts

4.1. Inputs - Pass

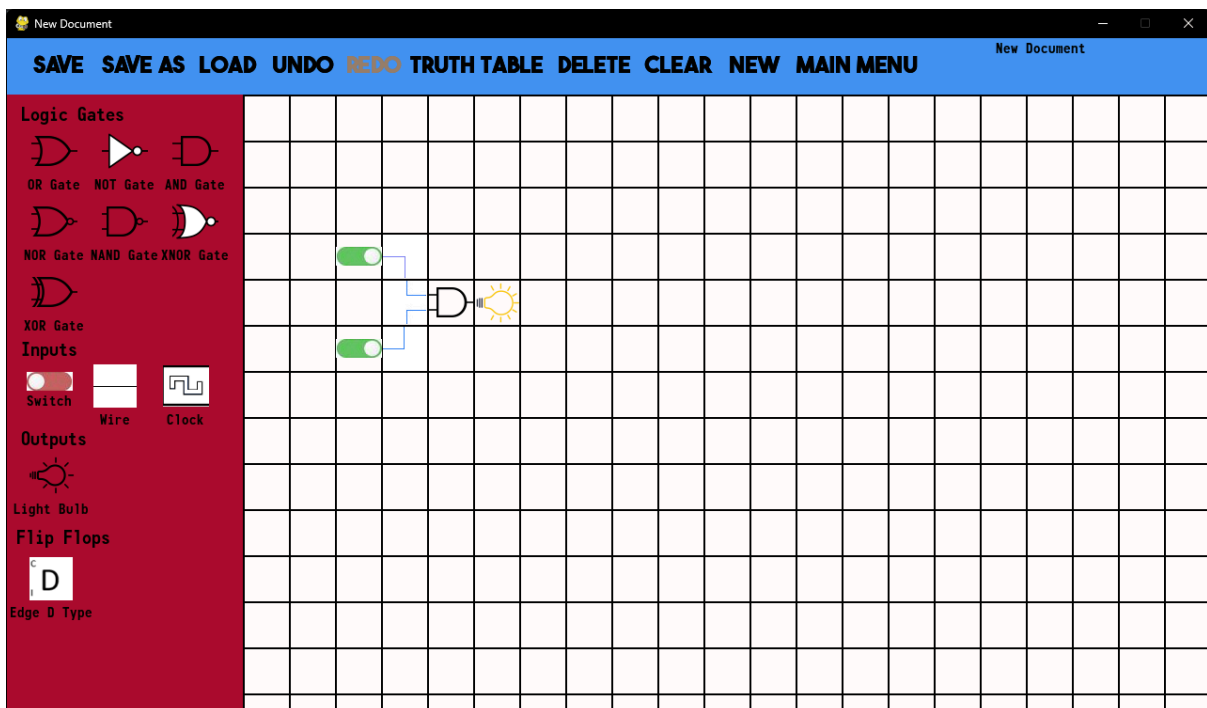
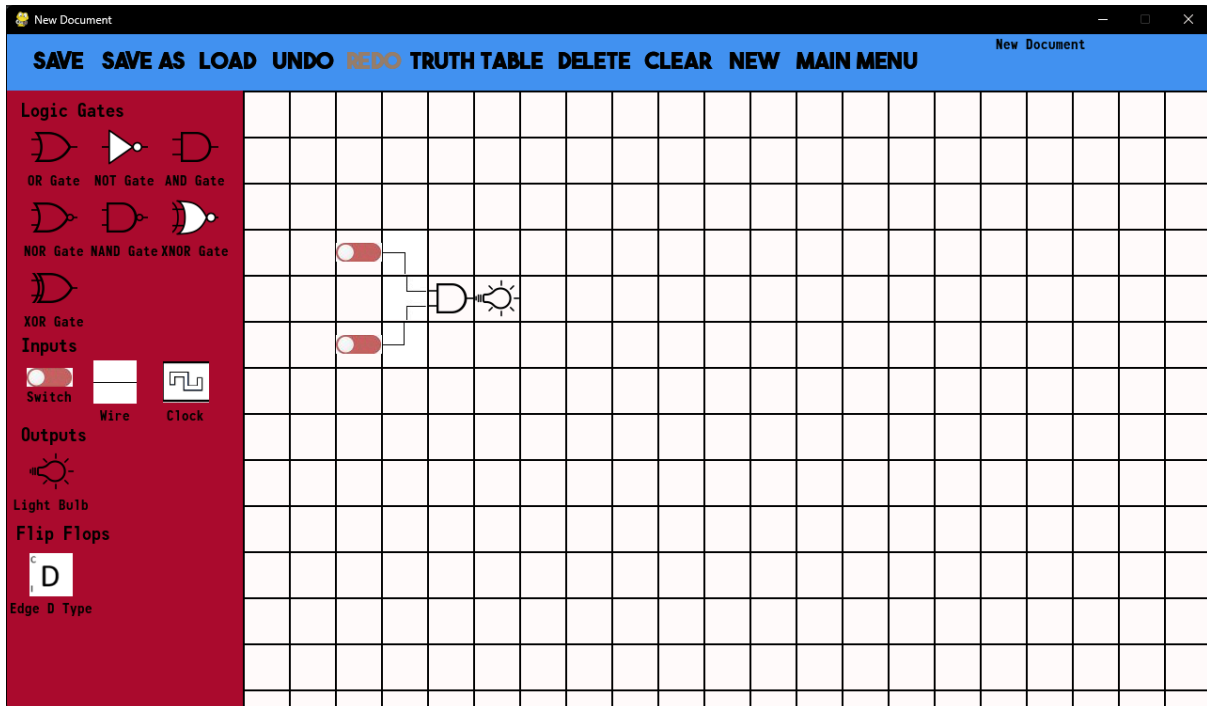
All inputs function as expected

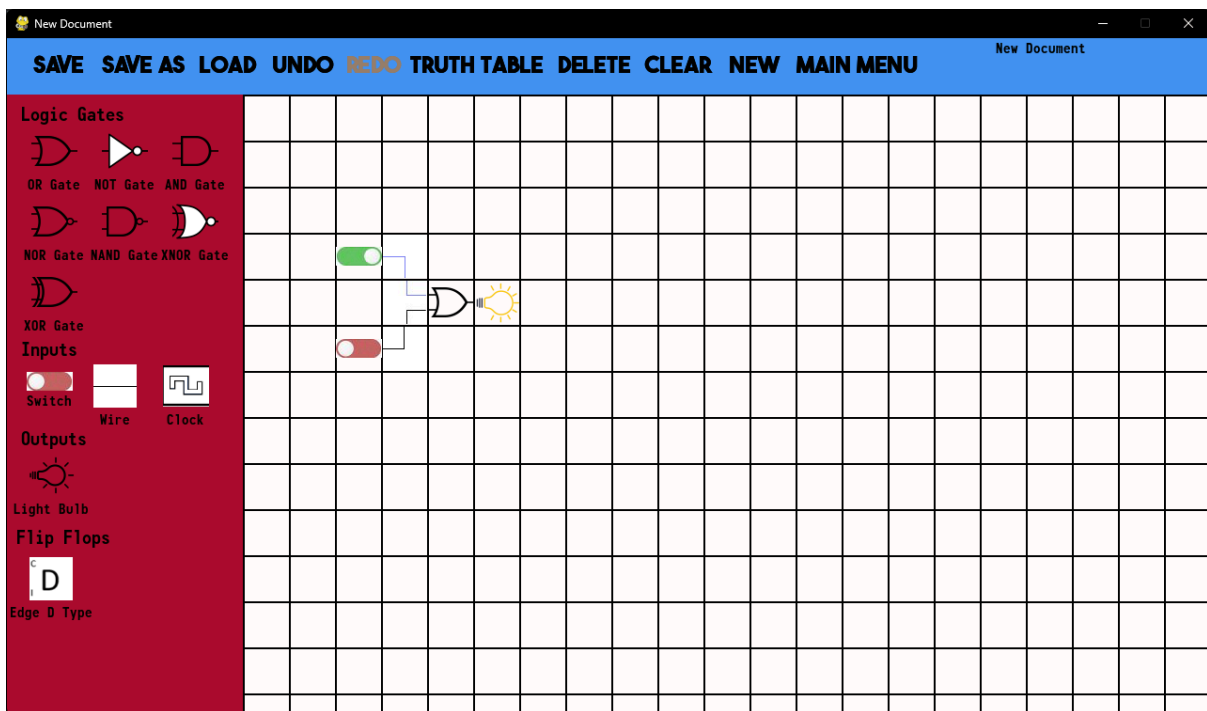
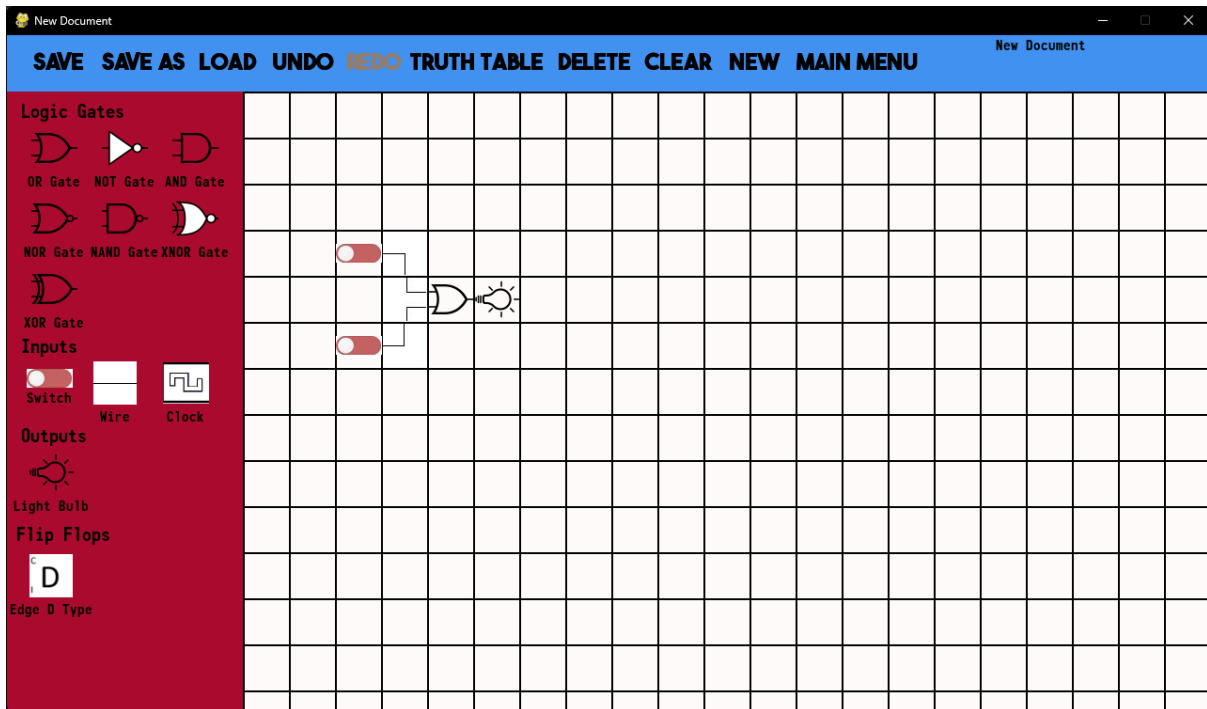


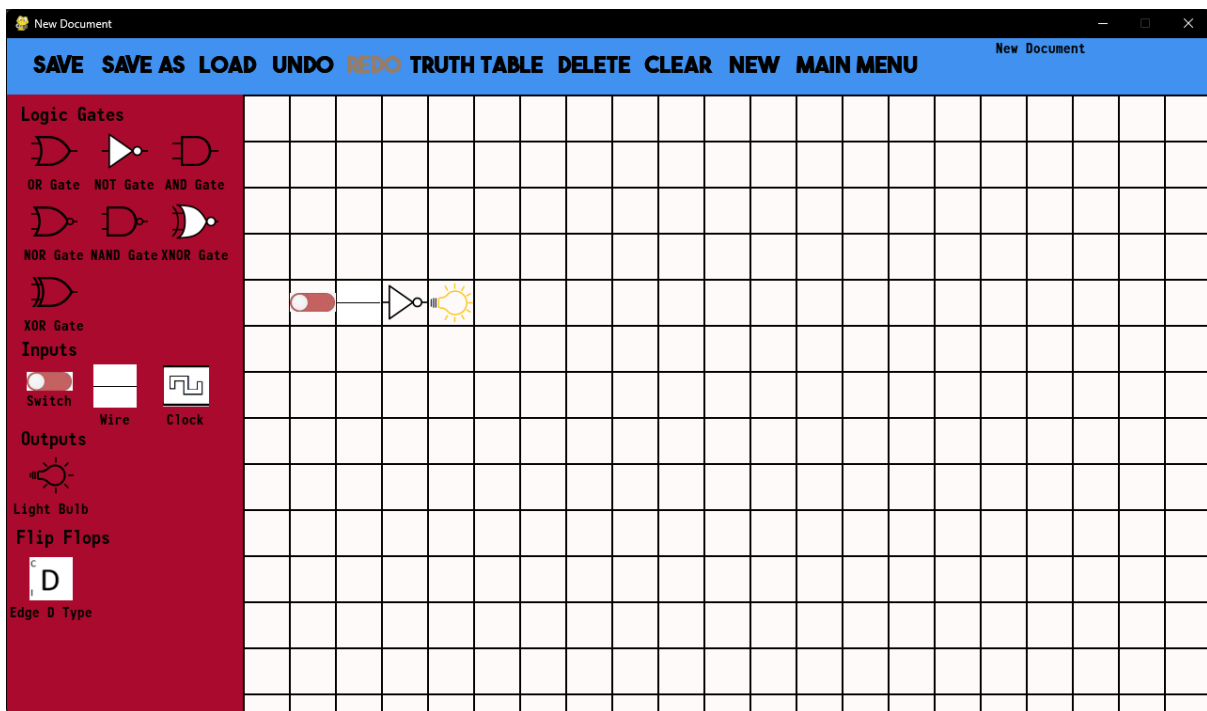
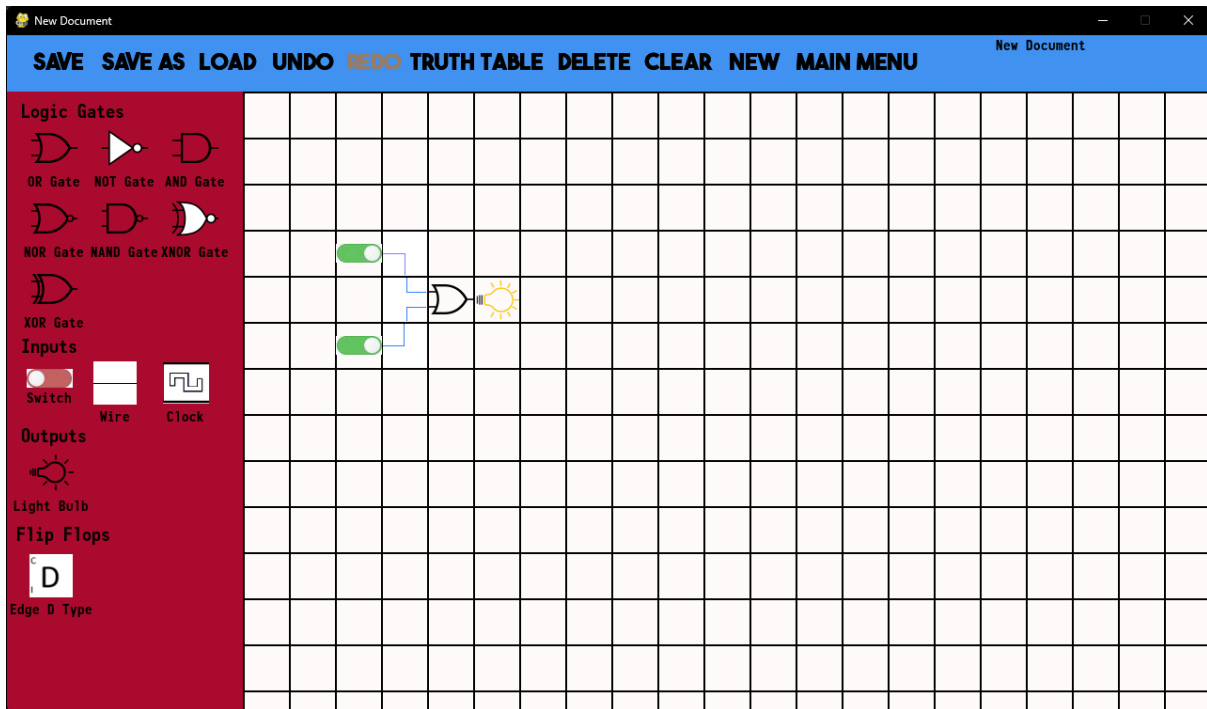


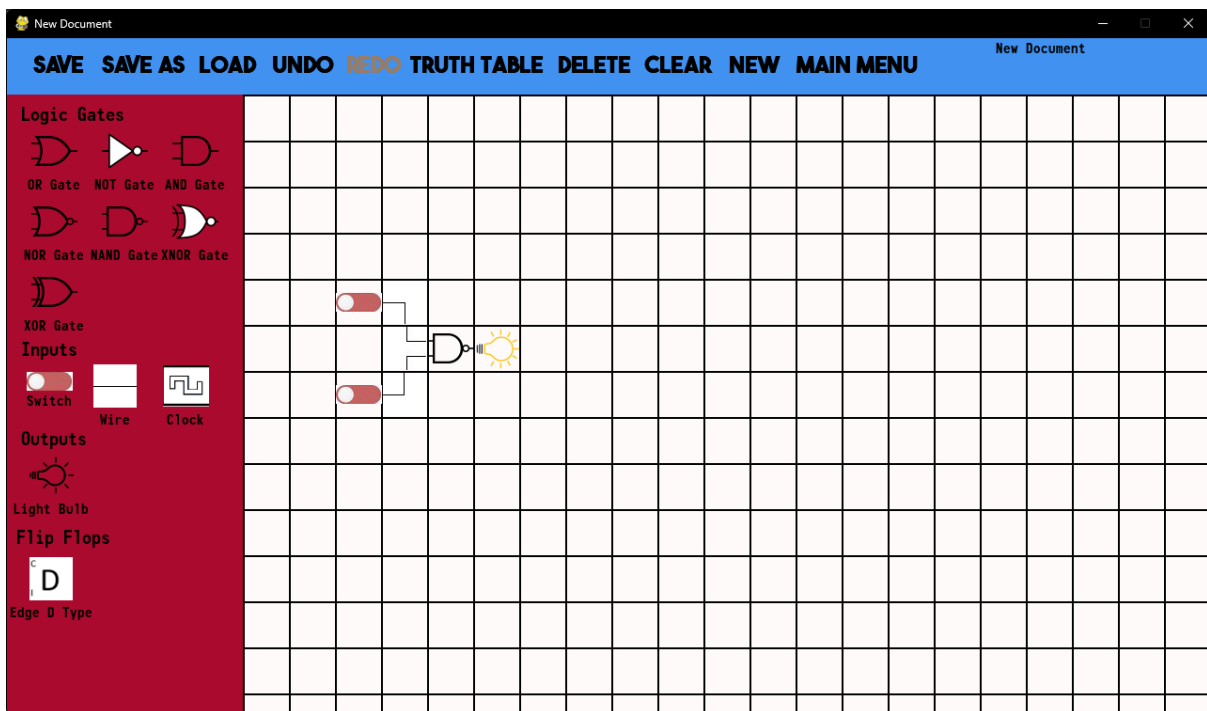
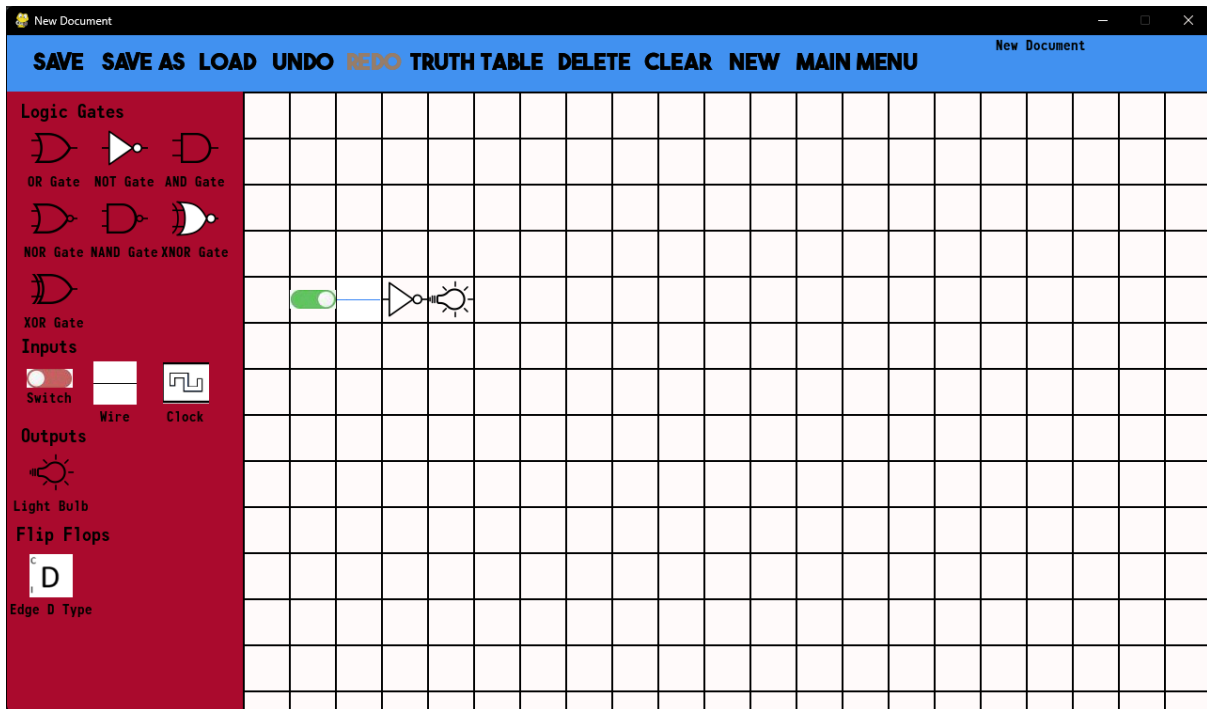
4.2. Logic gate interactions - Pass

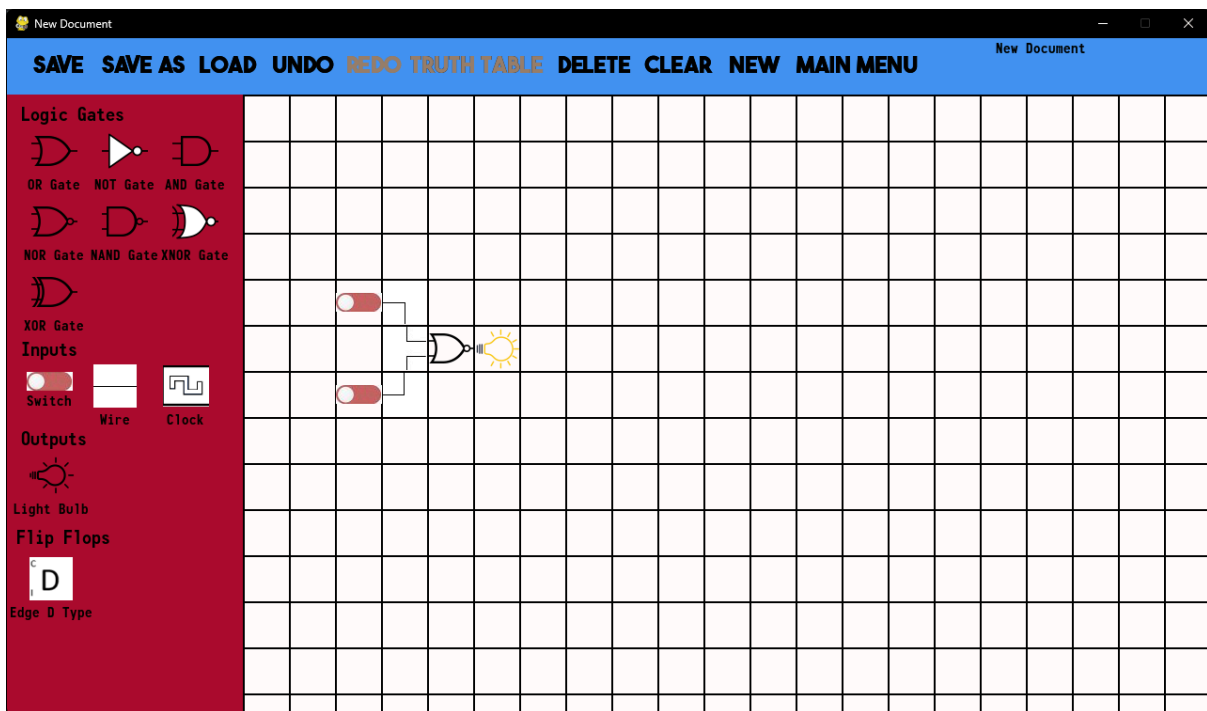
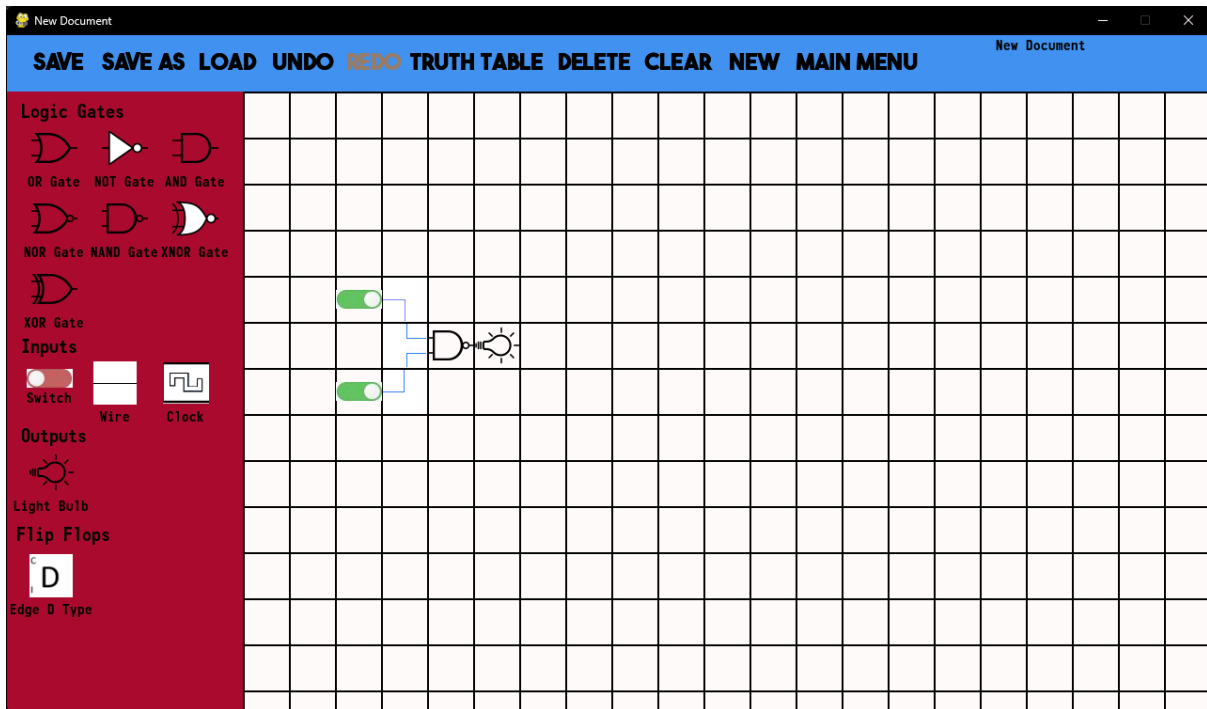
All logic gates function as expected

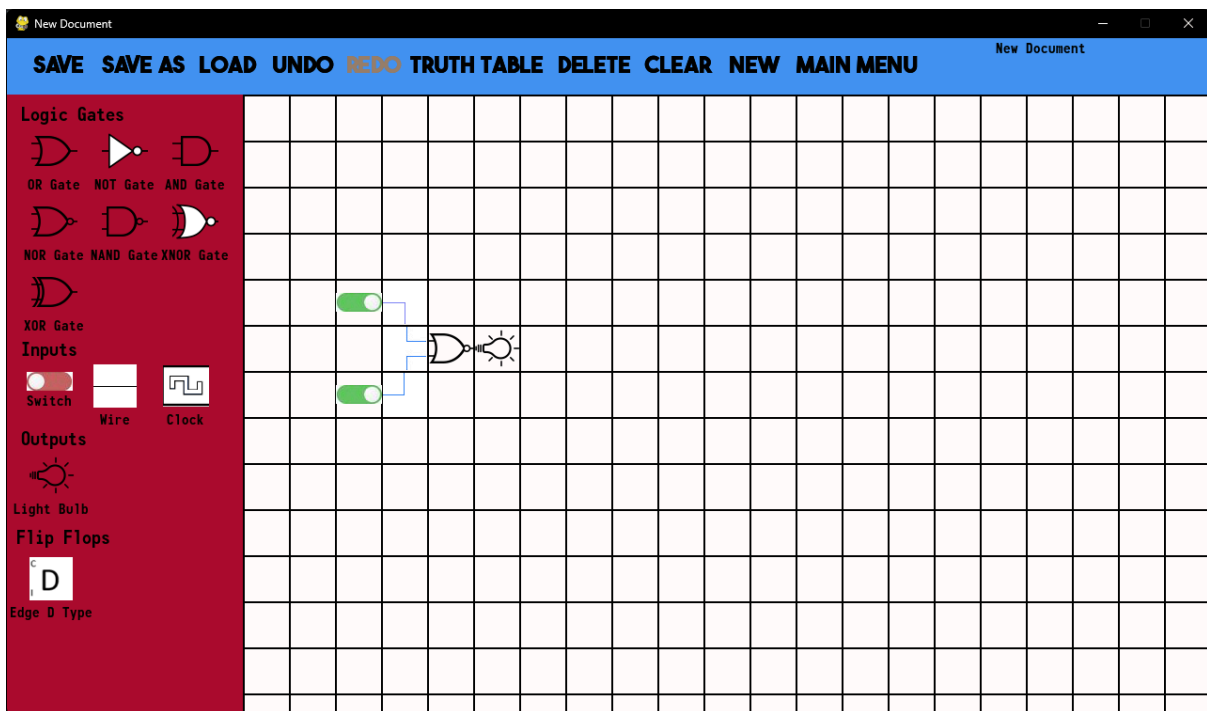
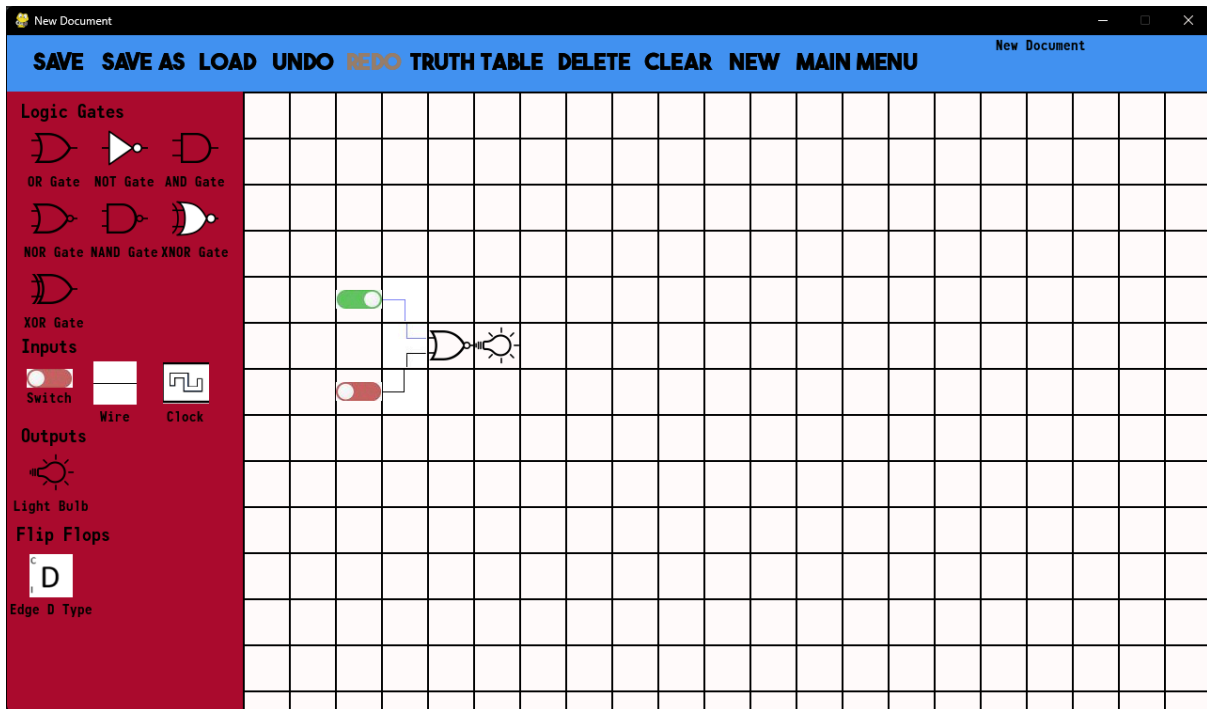


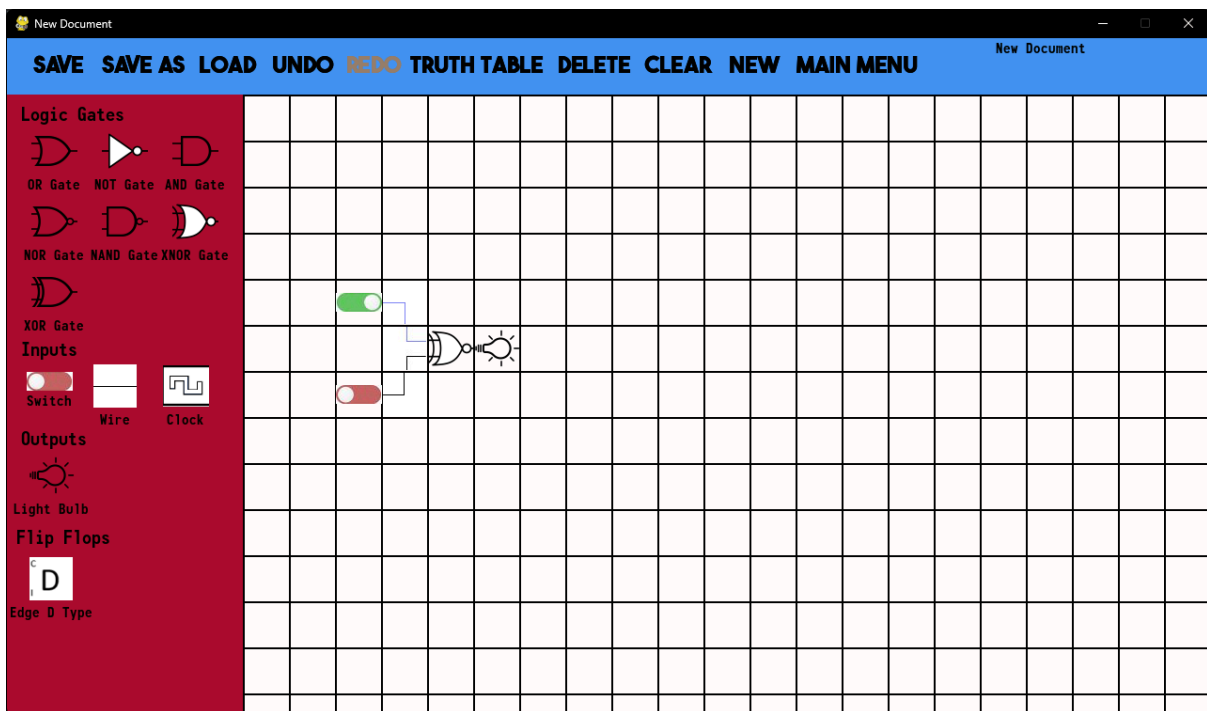
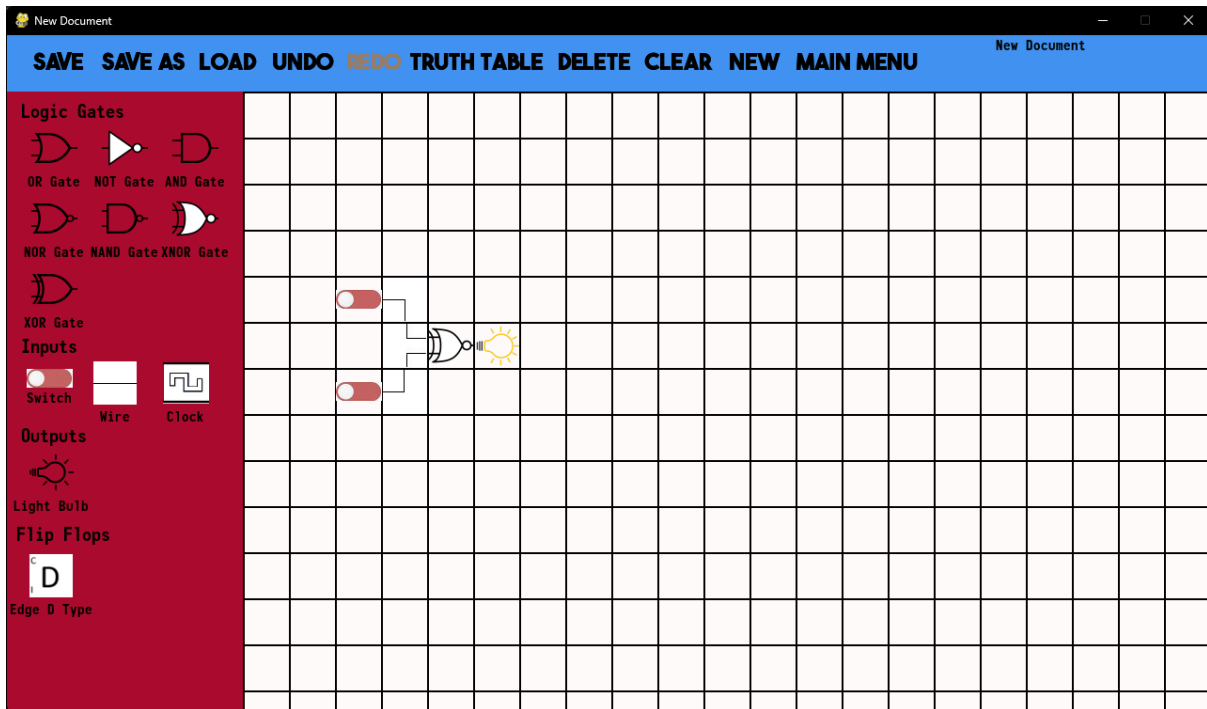


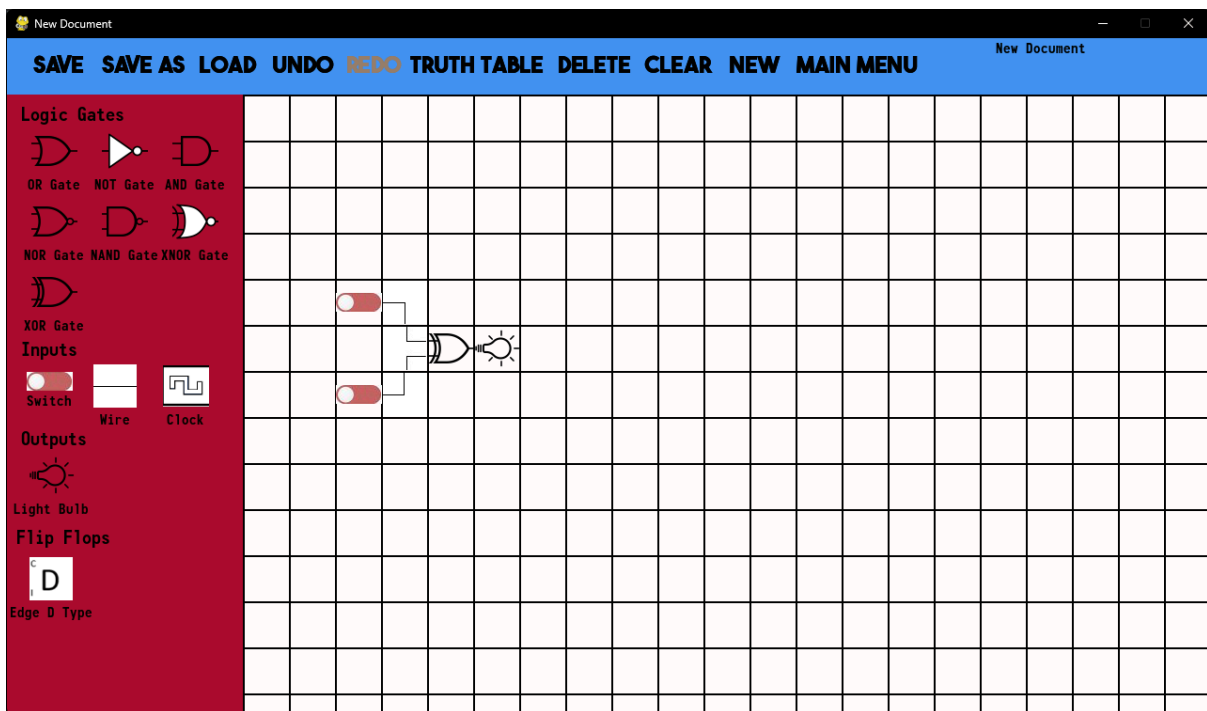
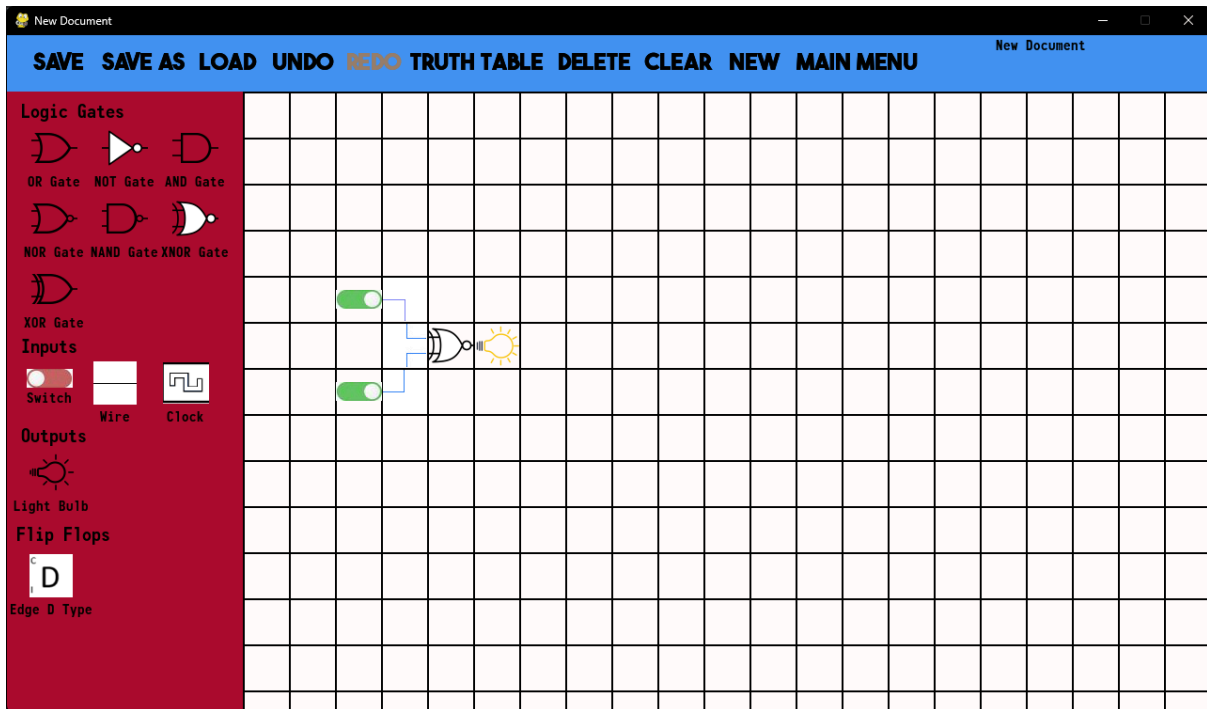


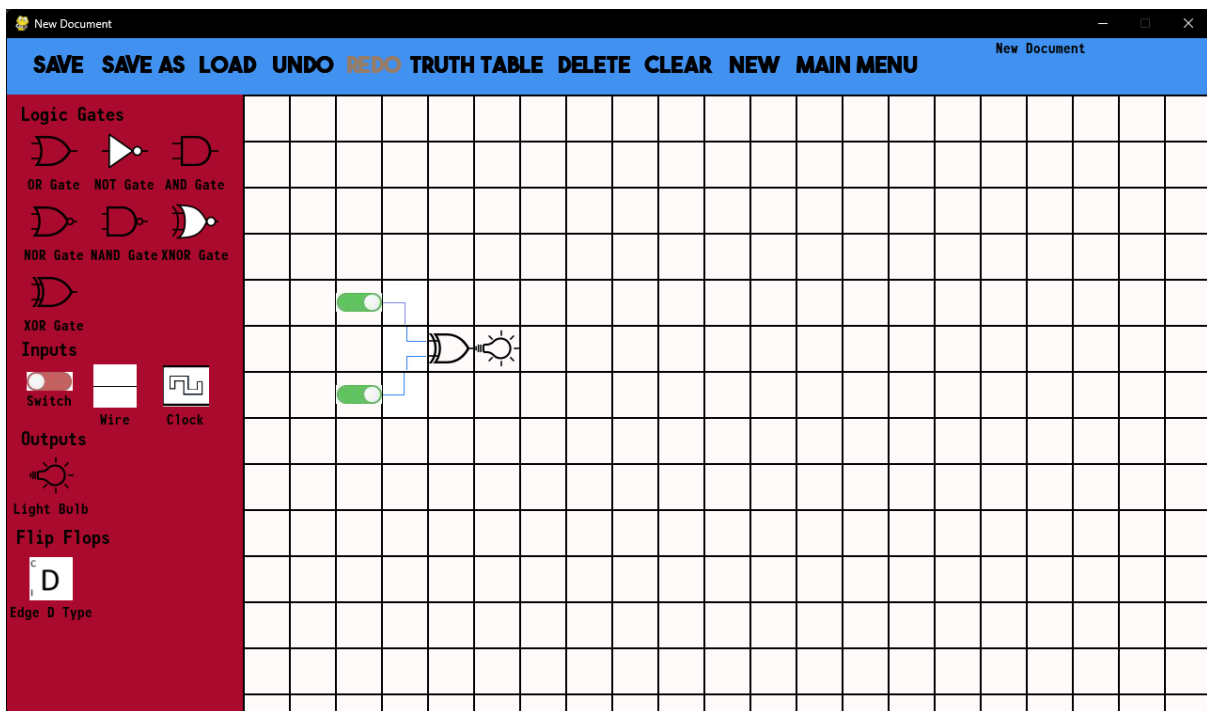
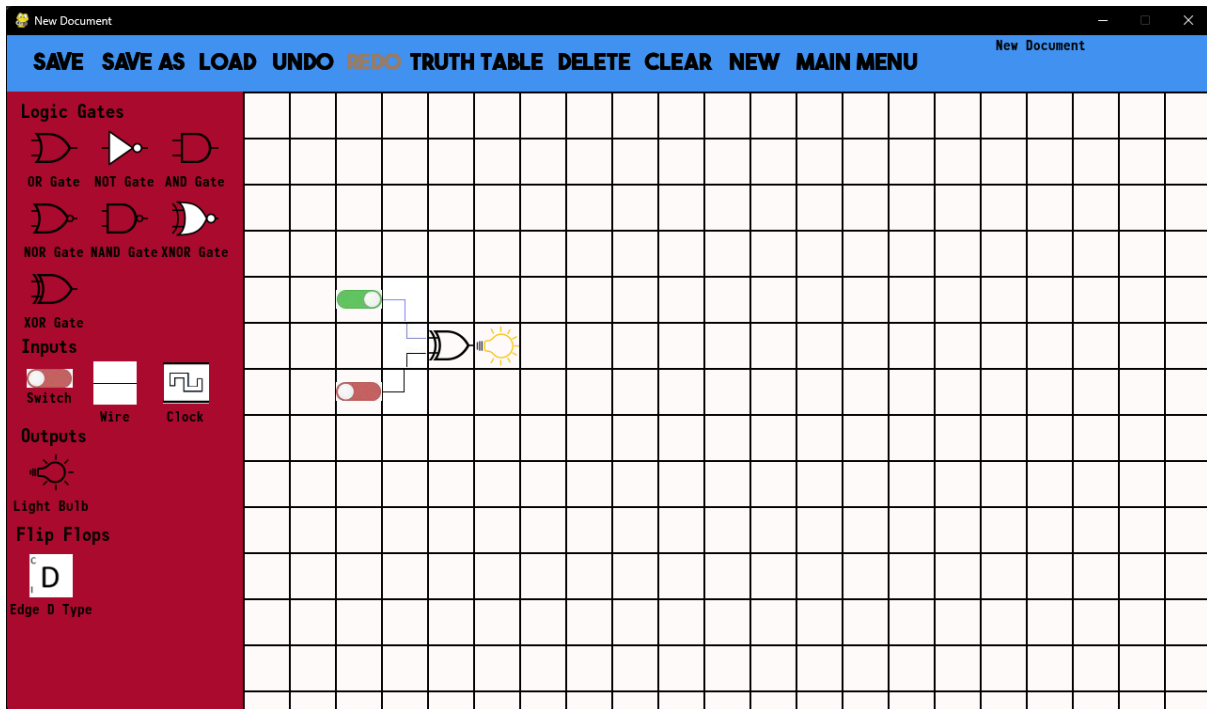






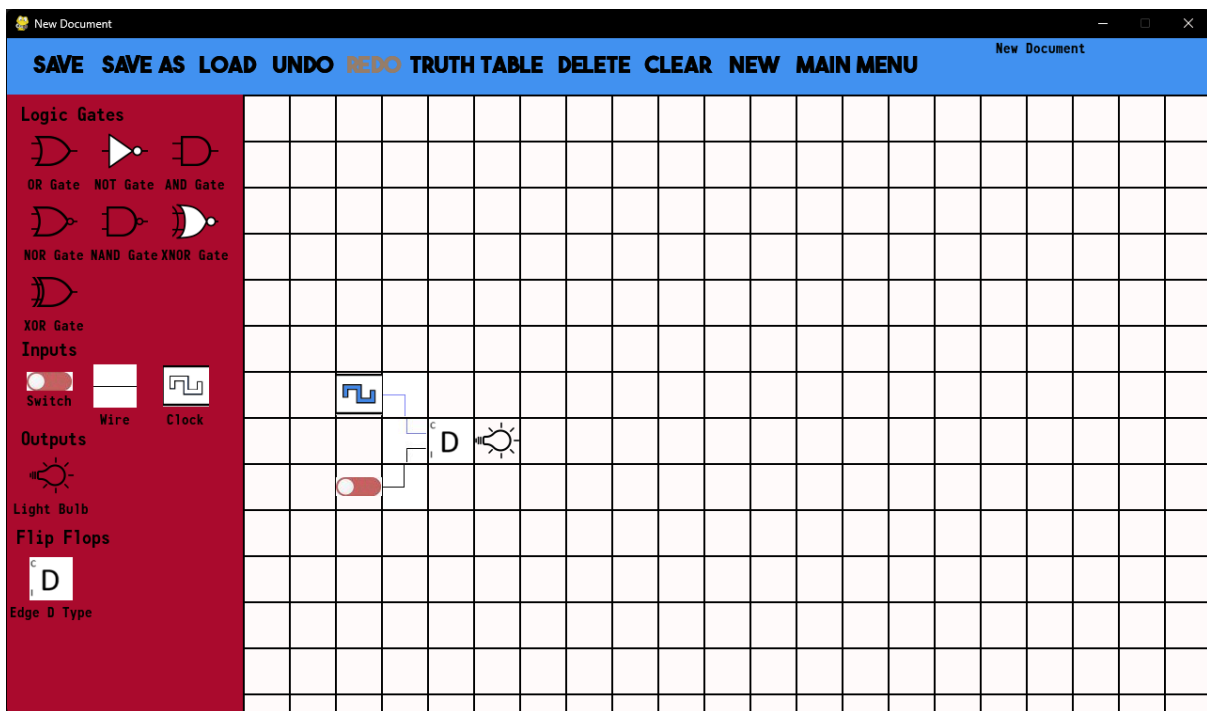
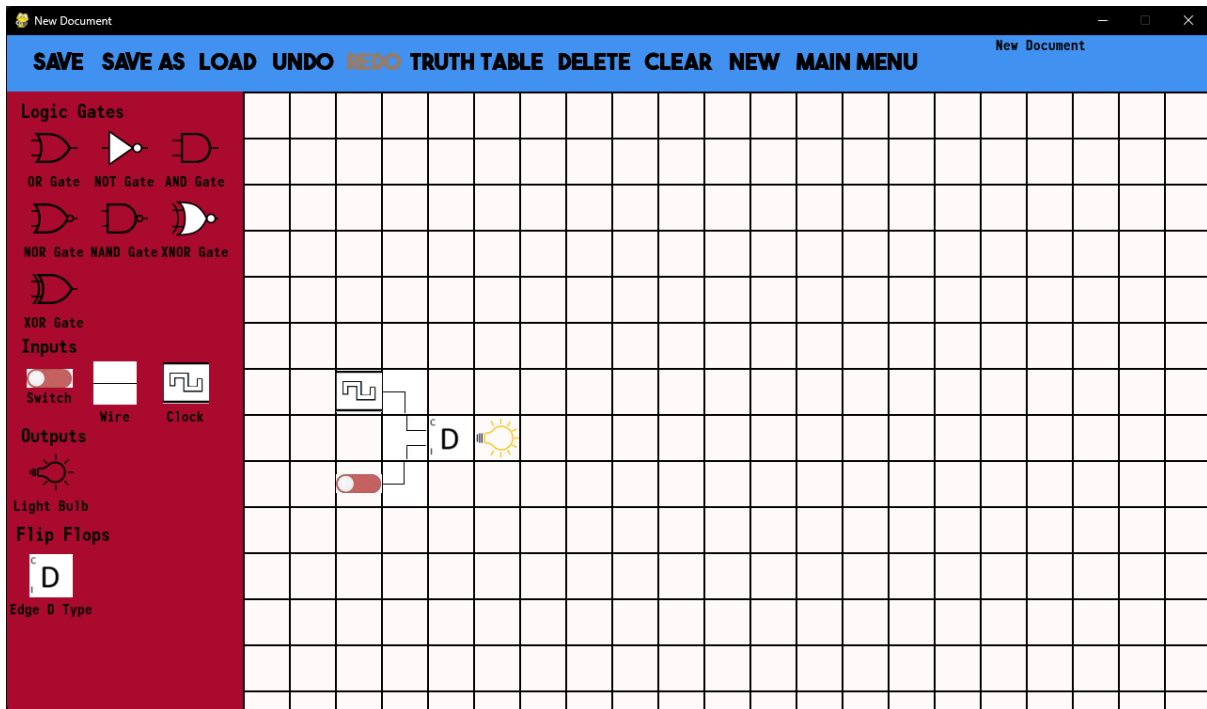






4.3. Flip-flops

Edge triggered D type flip flops function as expected



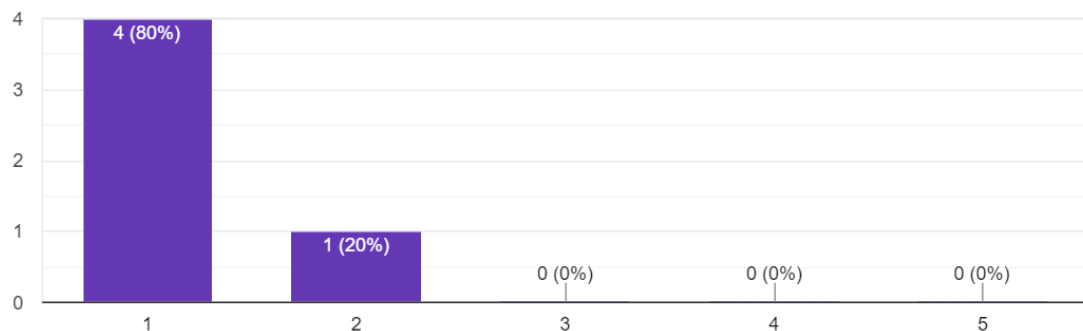
User testing

Once a finalised version of my technical solution was complete, I selected five of my six potential users to fill out a questionnaire on what they thought of my simulator, where two of my more experienced users LD and AJ, were sent my solution via email, allowing them to have an entirely unsupervised look at my solution, and the other three, AT, AS and JN were supervised by myself, with one user AT having no knowledge of logic gates prior to using my simulator program.

How easy did you find the user interface to navigate?

 Copy

5 responses



Where one is very easy and five is very difficult

As shown most users found the user interface very easy to navigate, with very few improvement comments being made:

How do you believe the user interface could have been improved?

4 responses

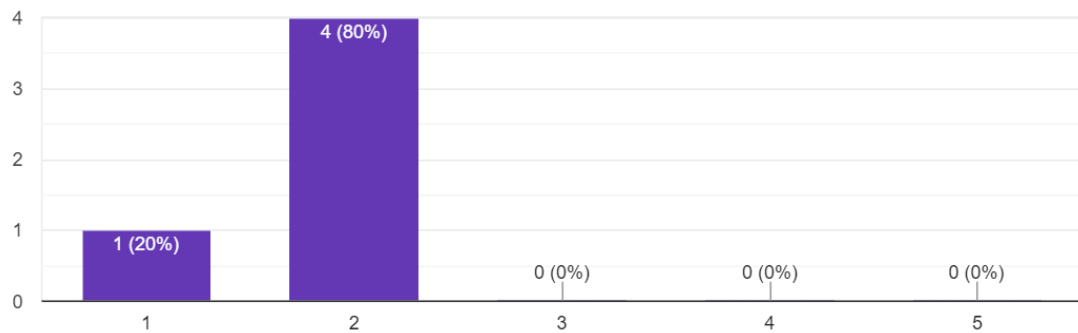
- LD** If the themes for the menu and the main game matched. Also, it would be nice if the images for the logic gates, wires and switches had transparent backgrounds and if the gates being filled in was consistent.
- AJ** The user interface was well designed, only thing I might say is to have the icons be more consistent
- JN** I'm not sure
- AS** Prompts / pop outs (i did not do the tutorial though or else it would have been onbviouse)

Most comments that were made were based on inconsistent icons, as some gate images had transparent images and some did not, one user AS stated that pop up menus could have been used to describe the functions of each logic gate, and how they work. Doing this would reduce the reliance the simulator has on its tutorial to teach users, although the same user noted that they had not used the tutorial system when trialling the simulator.

How easy did you find it to place and link logic gates, wires and switches?

 Copy

5 responses



Where one is very easy and five is very difficult

Most users found the interface for placing and linking logic elements to be moderately easy to use, with most users responding with comments on how the system could be better improved:

How do you believe placement could have been improved?

4 responses

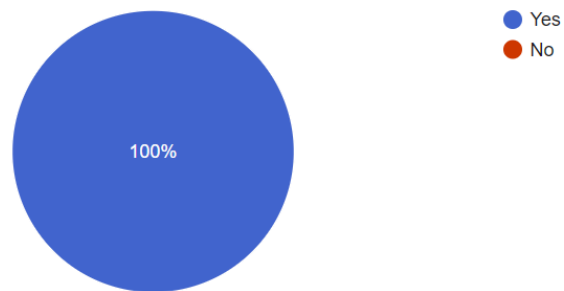
- LD Being able to have a switch directly next to a gate (specifically AND, OR, etc). Also making the addition of the switched on a selection basis like with the wires.
- AJ Allow switches and clocks to connect to wires from any side, or add a clear right side port to the icons to make the placement more obvious
- JN Maybe the lines for the wires could be a little thicker and when you select a wire, gate or switch if they changed colour this might help a little to remember what was selected.
- AS improved wirs

Both users LD and AJ who utilised my simulator without my supervision noted that switches cannot be used to turn on wires vertically, such as in an OR or AND gate, where inputs are accepted from both above and below the gate. In response I have since made changes to this system, in which switches can be connected to vertical wire pieces, including those in a logic gate itself. Some users also noted that the wire system did not work for all situations as expected, and that the wires themselves could be made more visible. The system by which wires change their orientation based on surrounding wires and logic elements was a complex system to develop and provided more time I may have been able to perfect this system fully.

Did the logic gates in the simulator function as expected?

 Copy

5 responses



If the logic gates did not function as expected, how do you believe they could function better?

0 responses

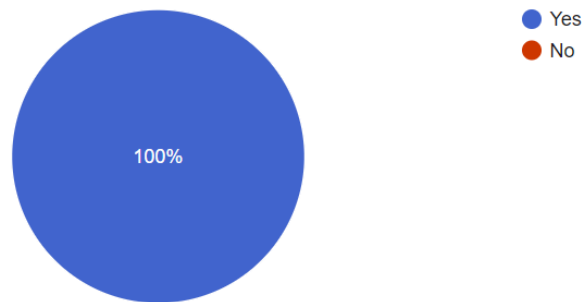
No responses yet for this question.

All users found that the logic gates themselves functioned as expected, with no improvement comments being raised.

Did you find the save and loading functionality worked as expected?

 Copy

5 responses



All users found that the saving and loading system functioned as expected, and would save and load files as requested. Only one improvement comment was made:

If no, what problems did you experience?

1 response

LD

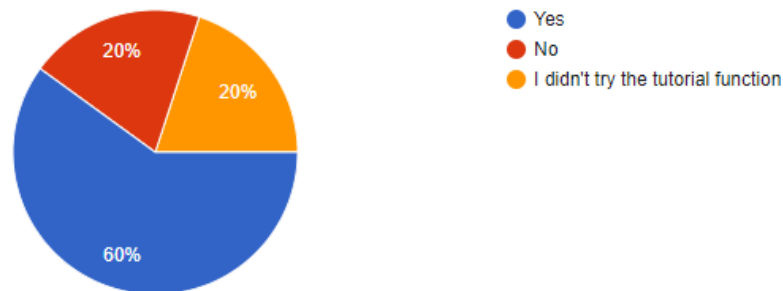
Not a problem per say, but maybe warning people that a file by that name has already been creating to stop overwriting?

LD stated that an improvement could be made to warn users of a previously saved file name if the user were to intentionally or unintentionally overwrite the file. This could have been implemented to prevent users from unintentionally changing the contents of their previously saved files.

Did you find the tutorial function an effective learning tool?

 Copy

5 responses



The majority of users found the tutorial to function as expected, and provide an effective learning experience with one user LD not finding the tutorial section to be an effective tool and one user AS having not tried to use the tutorial section at all. Users who had used this functionality made improvement comments:

What could be done to improve the tutorial function?

4 responses

LD

Have tutorials for gates other than just the NOT, because I didn't know how to work the flip-flop. Also, it would be nice to explain all of the functions (save, load, undo, redo, delete, etc.). As well as this, maybe explaining what each gate did, or at least having an area with an in-depth explanation paragraph.

AJ

Make it more obvious that switches only connect from the left

AT

slightly clearer invitation to further edit the gate before exiting tutorial

JN

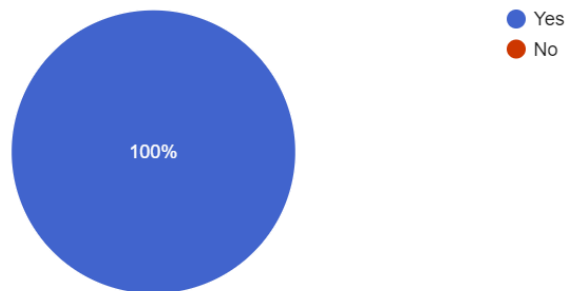
Maybe a few more descriptions about other logic gates and examples of how they can be linked with other gates.

As seen, users LD and JN commented on a lack of explanation for all logic gates given that my simulator only explains how to use AND and NOT gates. This could potentially be something to improve on, by adding an explanation for each logic gate at the end of the simulator when the logic gate is changed. LD noted that this could be done for flip-flops and the options menu, which may be more difficult to include. AJ mentioned that the switches only connecting from the left could be mentioned within the tutorial, I have remedied this by allowing users to turn on wires in any direction connected to a switch in a revised version of my program. AT had commented that the invitation to place different logic gates could become more clear, which could be done by adding an extra step in the tutorial.

Did the functions for removing items from logic circuits function as expected? e.g. delete, clear, new, undo, redo

 Copy

5 responses



If no, what problems did you experience?

0 responses

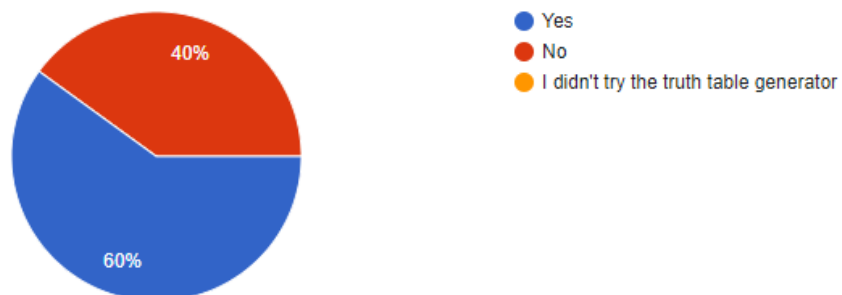
No responses yet for this question.

All users found that the functionality of adding and removing items from the logic circuit via the options such as delete, redo and undo all functioned as expected, with no improvement comments being made.

Did the truth table generator work as expected?

 Copy

5 responses



The majority of users stated that truth table generation functioned as expected, however users that tested externally stated that they were unable to get the functionality to work:

if no, what problems did you experience?

2 responses

LD It would just generate the basic truth tables for the gates placed, whether things were connected. Also the game crashed upon trying to go back.

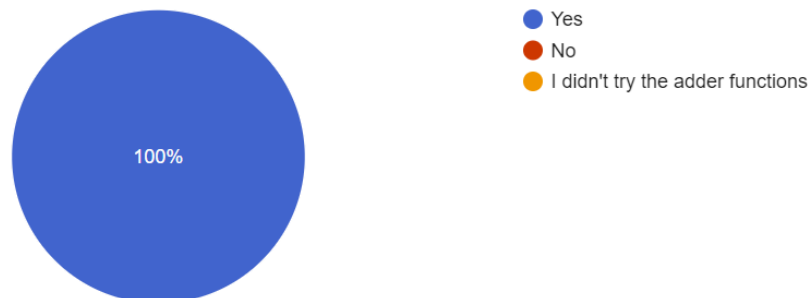
AJ I couldn't get this to work, no table appeared

Both users LD and AJ stated they were unable to get the truth table translation tool to work. Both users conducted their testing on their own systems away from my supervision. After this feedback I tested this functionality myself on a system that had never run my program before, only to receive no issue. Being unable to re-communicate with these users, I found this feedback to be inconclusive.

Did the adders generated from the adders menu perform as expected?

 Copy

5 responses



All users found that the adders menu and adder circuits themselves functioned as expected, with only one improvement comment being made:

if no, what problems did you experience?

1 response

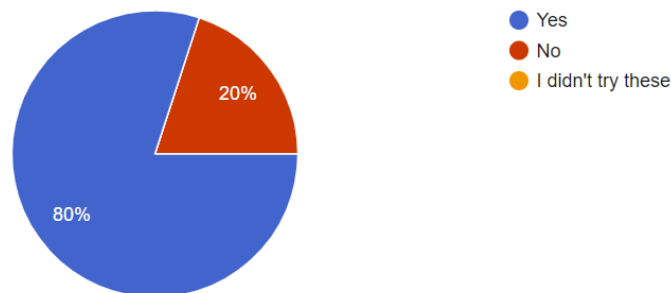
LD (genuinely confused to how adders work, but you're smart and they didn't crash so presume they're right??)

Whilst not explicitly an improvement, the lack of knowledge on how adders worked by user LD indicates that a brief description or tutorial of some sort could be used to explain the functionality of each adder circuit, that way the teaching element of the simulator could be further improved.

Did the flip flop circuit and clock function as expected?

 Copy

5 responses



Most users were able to utilise the flip flop circuit functionality of my software, with only one user LD not being able to utilise flip flops or clocks; they have made an improvement comment on how this could be rectified:

if no, what problems did you experience?

1 response

LD Didn't know how to use them and they wouldn't work when connected to switches and bulbs.

User LD stated that they did not understand how to use flip flop circuits, and thus they were unable to understand the functionality included in the simulator. This shows a level of improvement that could be made to teaching users how to use flip flop circuits. This could have been done by implementing the feature into the tutorial section given more time, this could have been achieved. The flip flop functionality of the system was developed after the tutorial, and thus due to limited time availability at such a late stage in development, I was unable to implement this improvement on time.

Technical solution

Subroutine and class definition and skills utilised

In order to better structure each section included in my final project and to best advocate the types of skills I have used during the course of my development, I have created a table displaying the types of programming skills I have used including the types of data structures used within each subroutine and class. The table also includes a brief description of each subroutines function as well as all the parameters passed into each subroutine:

Class	Parameters	Definition	Skills utilised
Button	- text_input - Font - Base_color - hovering_color	A class used to define the parameters of a button object for use in selecting menu options within the simulators menus	- Object oriented programming - Nested subroutine calls

Subroutine	Parameters	Definition	Skills utilised
mainMenu	N/A	Used as an interface for the user to access all the functions of the simulator. This is the first subroutine called, to supply the user with an interface upon start-up.	- Pygame - Object oriented programming (used for button class in all cases) - Variable declaration - Nested subroutine calls
adders	N/A	Used as an interface for the user to access adder circuits, selecting one will load it in the main logic interface.	- Pygame - Reading from a python pickle file - Object oriented programming - Nested subroutine calls

saveAsInterface	- logicArray - fileName	Used as an interface for the user to save files created in the simulator under a unique name.	- Pygame - Keyboard input processing - Writing to a python pickle file - Object oriented programming - Nested subroutine calls
loadInterface	- logicArray - fileName - menuBack	Used as an interface for the user to load files that pre-exist in the programs file directory	- Pygame - Keyboard input processing - Reading from a python pickle file - Object oriented programming - Nested subroutine calls - Exception handling
logicInterface	- logicArray - fileName - menuBack	Used as an accumulative interface for accessing the simulator and all of its option functions.	- Pygame - Multi dimensional arrays (up to 4D) - Python time package - Nested iteration - Object oriented programming - Nested subroutines - Nested subroutine calls - Extensive Input processing - Stacks
translate	- truthTable - switchCount - lightCount - switchCombinations - logicArray - fileName	Used to visually display a truth table generated by the simulator	- Pygame - Multi dimensional arrays - Nested iteration - Arithmetic solving

Nested in logicInterface			
turnOnVerticalUp	- i - j	Used to turn on all items in the row vertically above from the starting position	- Nested iteration - Multi dimensional arrays - Recursive calls - Nested calls
turnOnVerticalDown		Used to turn on all items in the row vertically below from the starting position	
turnOffVerticalUp		Used to turn off all items in the row vertically above from the starting position	
turnOffVerticalDown		Used to turn off all items in the row vertically below from the starting position	
turnOnHorizontalRight		Used to turn on all items in the column to the right of the starting position	
turnOnHorizontalLeft		Used to turn on all items in the column to the left of the starting position	
turnOffHorizontalLeft		Used to turn off all items in the column to the left of starting position	
turnOffHorizontalRight		Used to turn off all items in the column to the right of the starting position	
mainLoop	N/A	Used to decide the orientation of wires	- Python time package

		and decide the states of logic gates based on inputs. The turn on/off subroutines are also first called from here depending on the states of the switches in the circuit	- Nested subroutine calls - Multi dimensional arrays - Nested iteration
reInsert	- reArrayLast - state	Used to reinsert or remove items from the logic array based on the array at the top of the stack and its contents	- Stacks - Multi dimensional arrays - Nested iteration
checkOutput	- combination List - switchPosArray - lightPosArray	Based on the combinations within the combination list, the states of the inputs in the combination list are applied to the switches in the logic circuit as shown by the switchPosArray. The outputs at the positions of the lightPosArray are then recorded and returned.	- Multi dimensional arrays - Algorithm imports - Arithmetic solving

Full code

My final solution utilised Python, the pygame package to supply a gui and the pickle python package for saving and loading files. Files that are created by the user are stored as binary data files, with the logicArray variable being dumped under the user's specific file name. I also utilised a button class, and binary translation algorithm of my own design that were imported into the main program.

Imported algorithms

Button class

```
class Button():
    def __init__(self, pos, text_input, font, base_color, hovering_color):
        self.image = None
        self.x_pos = pos[0]
        self.y_pos = pos[1]
        self.font = font
        self.base_color, self.hovering_color = base_color, hovering_color
        self.text_input = text_input
        self.text = self.font.render(self.text_input, True, self.base_color)
        if self.image is None:
            self.image = self.text
        self.rect = self.image.get_rect(center=(self.x_pos, self.y_pos))
        self.text_rect = self.text.get_rect(center=(self.x_pos, self.y_pos))

    def update(self, screen):
        if self.image is not None:
            screen.blit(self.image, self.rect)
            screen.blit(self.text, self.text_rect)

    def checkForInput(self, position):
        if position[0] in range(self.rect.left, self.rect.right) and position[1] in
            range(self.rect.top, self.rect.bottom):
            return True
        return False

    def changeColor(self, position):
        if position[0] in range(self.rect.left, self.rect.right) and position[1] in
            range(self.rect.top, self.rect.bottom):
            self.text = self.font.render(self.text_input, True,
                self.hovering_color)
        else:
```

```
self.text = self.font.render(self.text_input, True, self.base_color)
```

Binary translator

```
def binaryConvert(number):
    binaryText = ""
    reverseBinaryList = []
    if number == 0:
        return 0
    while number != 0:
        numberRemainder = number%2
        reverseBinaryList.append(numberRemainder)
        number = number//2
    for i in range(len(reverseBinaryList)-1,-1, -1):
        binaryText = binaryText + str(reverseBinaryList[i])
    return binaryText
```

Main code

```
#imports
import pygame, sys
from button_modified import Button
import pickle
import time
import os
from binaryConvert import binaryConvert

#clock pulse variables set
finalTime = None
clockPulse = None
activeGate = None
fileName = None

#stack lists set
undoStack = []
redoStack = []

#pygame initialised
pygame.init()

#screen instantiated
SCREEN = pygame.display.set_mode((1280, 720))

#counts set
```

```
count = 0
lastCountChange = 0

#background image loaded for main menu
BG = pygame.image.load("Background.png")

#colours set
WHITE = "#FFFBFC"
BLACK = "#010400"
BLUE = "#4392F1"
RED = "#AE0E2E"
#930E29
GREEN = "#7CA982"
GREY = "#967D69"

#font creation
def get_font1(size):
    return pygame.font.Font("font1.ttf", size)
def get_font2(size):
    return pygame.font.Font("font2.ttf", size)
def get_font3(size):
    return pygame.font.Font("font3.otf", size)

#logicArray and fileName parameters are used to return the user to their document
once they have saved or go back
def saveAsInterface(logicArray, fileName):
    pygame.display.set_caption("Save As")
    #font for user text
    baseFont = get_font3(32)
    userText = ""
    #text input is shown to user in a rectangle
    inputRect = pygame.Rect(390,315,200,40)
    colour = pygame.Color(WHITE)
    while True:
        saveAs_MOUSE_POS = pygame.mouse.get_pos()
        SCREEN.fill(BLACK)
        saveFile_TEXT = get_font2(45).render("Enter New File Name", True, WHITE)
        saveFile_RECT = saveFile_TEXT.get_rect(center=(640, 260))
        SCREEN.blit(saveFile_TEXT, saveFile_RECT)
        saveFileBack = Button(pos=(100, 675), text_input="BACK", font=get_font2(35),
        base_color=WHITE, hovering_color=GREY)
        #enter button only works if the user has inputted a name
        if userText != "":
```

```
    saveFileEnter = Button(pos=(640, 420), text_input="ENTER",
font=get_font2(35), base_color=WHITE, hovering_color=GREY)
    else:
        saveFileEnter = Button(pos=(640, 420), text_input="ENTER",
font=get_font2(35), base_color=GREY, hovering_color=GREY)
        #user text is placed onto pygame window
        textSurface = baseFont.render(userText, True, (255,255,255))
        SCREEN.blit(textSurface, (inputRect.x + 5, inputRect.y - 5))
        pygame.draw.rect(SCREEN, colour, inputRect, 2)
        #rectangle changes with the length of the text
        inputRect.w = textSurface.get_width()+10

for button in [saveFileBack, saveFileEnter]:
    button.changeColor(saveAs_MOUSE_POS)
    button.update(SCREEN)

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.MOUSEBUTTONDOWN:
        #pressing back returns the user to the logic interface in the state it was
before
        if saveFileBack.checkForInput(saveAs_MOUSE_POS):
            logicInterface(logicArray, fileName, False)
        if saveFileEnter.checkForInput(saveAs_MOUSE_POS):
            #enter button only works if the user has inputted a name
            if userText != "":
                logicFile = open(userText, 'wb')
                pickle.dump(logicArray, logicFile)
                logicFile.close()
                print("Save Successful")
                #save successful prompt displayed
                saveText = get_font2(35).render("Save Successful", True, GREEN)
                saveTextRect = saveText.get_rect()
                saveTextRect.center = (640,380)
                SCREEN.blit(saveText, saveTextRect)
                pygame.display.update()
                #prompt is displayed for one second
                time.sleep(1)
                #returns user to logic document
                logicInterface(logicArray, userText, False)
            if event.type == pygame.KEYDOWN:
                #pressing backspace removes a letter from the text
```

```
if event.key == pygame.K_BACKSPACE:
    userText = userText[:-1]
    #if the key being pressed is not alphanumeric nothing happens
    elif event.key == pygame.K_TAB or event.key == pygame.K_ESCAPE or
event.key == pygame.K_DELETE:
    pass
    #pressing enter attempts a save
    elif event.key == pygame.K_RETURN:
        if userText != "":
            logicFile = open(userText, 'wb')
            pickle.dump(logicArray, logicFile)
            logicFile.close()
            print("Save Successful")
            saveText = get_font2(35).render("Save Successful", True, GREEN)
            saveTextRect = saveText.get_rect()
            saveTextRect.center = (640,380)
            SCREEN.blit(saveText, saveTextRect)
            pygame.display.update()
            time.sleep(1)
            logicInterface(logicArray, userText, False)
        #pressing any other key adds it to the text input
    else:
        #max character limit 30
        if len(userText) <= 30:
            userText += event.unicode
pygame.display.update()
```

#parameters used to determine the menu to return to, as well the contents of the logic interface, if the user is accessing the menu from there

```
def loadInterface(logicArray, fileName, menuBack):
    pygame.display.set_caption("Load")
    #font for user text
    baseFont = get_font3(32)
    userText = ""
    #text input is shown in a rectangle
    inputRect = pygame.Rect(390,315,200,40)
    colour = pygame.Color(WHITE)
    while True:
        load_MOUSE_POS = pygame.mouse.get_pos()
        SCREEN.fill(BLACK)
        loadFile_TEXT = get_font2(45).render("Enter file name to load", True, WHITE)
        loadFile_RECT = loadFile_TEXT.get_rect(center=(640, 260))
        SCREEN.blit(loadFile_TEXT, loadFile_RECT)
```

```
loadFileBack = Button(pos=(100, 675), text_input="BACK", font=get_font2(35),
base_color=WHITE, hovering_color=GREY)
#enter button only works if the user has inputted a name
if userText != "":
    loadFileEnter = Button(pos=(640, 420), text_input="ENTER",
font=get_font2(35), base_color=WHITE, hovering_color=GREY)
else:
    loadFileEnter = Button(pos=(640, 420), text_input="ENTER",
font=get_font2(35), base_color=GREY, hovering_color=GREY)
#user text is placed into pygame window
textSurface = baseFont.render(userText,True,(255,255,255))
SCREEN.blit(textSurface, (inputRect.x + 5, inputRect.y - 5))
pygame.draw.rect(SCREEN,colour,inputRect,2)
inputRect.w = textSurface.get_width()+10

for button in [loadFileBack, loadFileEnter]:
    button.changeColor(load_MOUSE_POS)
    button.update(SCREEN)

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.MOUSEBUTTONDOWN:
        if loadFileBack.checkForInput(load_MOUSE_POS):
            #if the user accessed the load interface from the main menu they are
returned to the main menu
            if menuBack == True:
                mainMenu()
            #otherwise the user is returned to their logic circuit in the state it was
previously
        else:
            logicInterface(logicArray,fileName, False)
        if loadFileEnter.checkForInput(load_MOUSE_POS):
            print(logicArray)
            #exception handling used to check if file exists
            try:
                logicFile = open(userText, 'rb')
            except:
                loadFailText = get_font2(35).render("File Not Found", True, RED)
                loadFailTextRect = loadFailText.get_rect()
                loadFailTextRect.center = (640,380)
                SCREEN.blit(loadFailText, loadFailTextRect)
                pygame.display.update()
```



```
        time.sleep(1)
    else:
        logicArray = pickle.load(logicFile)
        logicFile.close()
        logicInterface(logicArray,userText, False)
    if event.type == pygame.KEYDOWN:
        #pressing backspace removes a letter from the text
        if event.key == pygame.K_BACKSPACE:
            userText = userText[:-1]
        #if the key being pressed is not alphanumeric nothing happens
        elif event.key == pygame.K_TAB or event.key == pygame.K_ESCAPE or
event.key == pygame.K_DELETE:
            pass
        #pressing enter attempts to load a file
        elif event.key == pygame.K_RETURN:
            try:
                logicFile = open(userText, 'rb')
            except:
                loadFailText = get_font2(35).render("File Not Found", True, RED)
                loadFailTextRect = loadFailText.get_rect()
                loadFailTextRect.center = (640,380)
                SCREEN.blit(loadFailText, loadFailTextRect)
                pygame.display.update()
                time.sleep(1)
            else:
                logicArray = pickle.load(logicFile)
                logicFile.close()
                logicInterface(logicArray,userText, False)
        #pressing any other key adds it to text input
    else:
        if len(userText) <= 30:
            userText += event.unicode
```

```
pygame.display.update()
```

```
def translate(truthTable, switchCount, lightCount, switchCombinations, logicArray,
fileName):
```

```
    pygame.display.set_caption("Truth Table")
    #screen height increased to 850 from 720
    SCREEN = pygame.display.set_mode((1280, 850))
    #combined count determined
    combinedCount = switchCount + lightCount
    print(combinedCount)
    print((combinedCount*49))
```

```
print(switchCombinations)
while True:
    #grid pixel values change depending on number of switches and lights
    if switchCount == 1:
        SCREEN.fill(BLACK)
        #column lines drawn
        for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+40),2)
        #row lines drawn
        for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+542,i+40),2)
    elif switchCount == 2:
        SCREEN.fill(BLACK)
        if lightCount == 1:
            for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+89),2)
            for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+493,i+40),2)
            if lightCount == 2:
                for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+40),2)
                for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+542,i+40),2)
            elif switchCount == 3:
                SCREEN.fill(BLACK)
                if lightCount == 2:
                    for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+40+147),2
)
                    for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+542-147,i+4
0),2)
            elif lightCount == 1:
                for i in range(0,(combinedCount*49)+1,49):
```

```
pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+40+147+49),2)
    for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+542-196,i+40),2)
    #max number of inputs accepted is 4
    elif switchCount == 4:
        SCREEN.fill(BLACK)
        if lightCount == 1:
            for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+578),2)
    for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+4,i+40),2)
    elif lightCount == 2:
        for i in range(0,(combinedCount*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(i+542,40),(i+542,combinedCount*49+529),2)
    for i in range(0,(switchCombinations*49)+1,49):

pygame.draw.line(SCREEN,WHITE,(542,i+40),(switchCombinations*49+53,i+40),2)
    else:
        SCREEN.fill(BLACK)

#used to define position in logic array
countHorizontal = 0
countVertical = 0
#used to determine where to place inputs and outputs text
rectangleWidth = 0
#input states are generated in grid
for i in range(40,(switchCombinations*49)+40,49):
    rectangleWidth += 49
    for j in range(542,(switchCount*49)+542,49):
        #state being placed depends on horizontal and vertical position in truth
table array
        truthTable_TEXT =
get_font1(25).render(str(truthTable[0][countVertical][countHorizontal]), True, WHITE)
        #i and j values equal the pixel count at the top left corner of the grid
square, 25 is added to each value to find the centre
        truthTable_RECT = truthTable_TEXT.get_rect(center=(j+25, i+25))
        SCREEN.blit(truthTable_TEXT, truthTable_RECT)
```

```
#if the number of the position of the input value <= number of inputs
if countHorizontal < switchCount-1:
    countHorizontal += 1
#otherwise move onto next row
else:
    countHorizontal = 0
    countVertical += 1

#input header text is generated at a fixed position
inputsText = get_font1(15).render("Inputs", True, WHITE)
inputTextRect = pygame.Rect(542,20,rectangleWidth, 20)
SCREEN.blit(inputsText, inputTextRect)

countHorizontal = 0
countVertical = 0
rectangleWidth = 0
#output states are generated in grid
for i in range(40,(switchCombinations*49)+40,49):
    for j in
range((switchCount*49)+542,(lightCount*49)+(switchCount*49)+542,49):
        truthTable_TEXT =
get_font1(25).render(str(truthTable[1][countVertical][countHorizontal]), True, GREY)
        truthTable_RECT = truthTable_TEXT.get_rect(center=(j+25, i+25))
        SCREEN.blit(truthTable_TEXT, truthTable_RECT)
        if countHorizontal < lightCount-1:
            countHorizontal += 1
        else:
            countHorizontal = 0
            countVertical += 1

#output header text is generated based on the amount of inputs compared to
inputs and outputs
outputText = get_font1(15).render("Outputs", True, WHITE)
outputTextRect = pygame.Rect((switchCount*49)+542,20,rectangleWidth, 20)
SCREEN.blit(outputText, outputTextRect)

translate_MOUSE_POS = pygame.mouse.get_pos()

translateBack = Button(pos=(100, 675), text_input="BACK", font=get_font2(35),
base_color=WHITE, hovering_color=GREY)

translateBack.changeColor(translate_MOUSE_POS)
translateBack.update(SCREEN)
```

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.MOUSEBUTTONDOWN:
        mouseX = event.pos[0]
        mouseY = event.pos[1]
        print("X pos " + str(mouseX))
        print("Y pos " + str(mouseY))
        #back button returns user to original document
        if translateBack.checkForInput(translate_MOUSE_POS):
            logicInterface(logicArray, fileName, False)

pygame.display.update()
```

```
def logicInterface(logicArray, fileName, tutorialMode):
```

```
    #window naming
```

```
    if tutorialMode == True:
```

```
        pygame.display.set_caption("Tutorial")
```

```
    elif fileName == None:
```

```
        pygame.display.set_caption("New Document")
```

```
    else:
```

```
        pygame.display.set_caption(fileName)
```

```
    #global variables
```

```
    global count
```

```
    global lastCountChange
```

```
    global activeGate
```

```
    global undoStack
```

```
    global redoStack
```

```
    global finalTime
```

```
    global clockPulse
```

```
    #display set
```

```
    SCREEN = pygame.display.set_mode((1280, 720))
```

```
    #clock pulse synchronisation
```

```
    lastChange = time.time()
```

```
    clockPulse = False
```

```
    finalTime = time.time() + 1
```

```
    #tutorial mode selection
```

```
if tutorialMode == True:
    tutorialPhase = 1
if tutorialMode == False:
    tutorialPhase = None

#auto save initial values
autoSave = False
autoSaveStartTime = time.time()

while True:
    count += 1
    logicInterface_MOUSE_POS = pygame.mouse.get_pos()

    #save check
    if fileName != None:
        try:
            logicFileTest = open(fileName, "rb")
        except:
            fileName = None
        else:
            logicFileTest.close()

    SCREEN.fill(WHITE)
    #logic menu draw
    #load gate image
    ORGate = pygame.image.load("ORGate.png")
    #creation of background rectangle needed for the OR gate to be displayed
    ORrect = ORGate.get_rect()
    ORrect.center = (50,120)
    #creation of the text description of the gate, colours change depending on
events in the logic interface
    if activeGate == "ORGate":
        ORText = get_font3(15).render("OR Gate", True, BLUE)
    elif tutorialPhase == None or tutorialPhase == 10:
        ORText = get_font3(15).render("OR Gate", True, BLACK)
    elif tutorialPhase != None:
        ORText = get_font3(15).render("OR Gate", True, GREY)
    #creation of the background rectangle needed for text description
    ORrectText = ORText.get_rect()
    ORrectText.center = (50,155)

    #all other logic elements follow the same format
```

```
NOTGate = pygame.image.load("NOTGate.png")
NOTrect = NOTGate.get_rect()
NOTrect.center = (125,120)
if activeGate == "NOTGate":
    NOTText = get_font3(15).render("NOT Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 3:
    NOTText = get_font3(15).render("NOT Gate", True, BLACK)
elif tutorialPhase != None:
    NOTText = get_font3(15).render("NOT Gate", True, GREY)
NOTrectText = NOTText.get_rect()
NOTrectText.center = (125,155)

ANDGate = pygame.image.load("ANDGate.png")
ANDrect = ANDGate.get_rect()
ANDrect.center = (200,120)
if activeGate == "ANDGate":
    ANDText = get_font3(15).render("AND Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 8 or tutorialPhase == 10:
    ANDText = get_font3(15).render("AND Gate", True, BLACK)
elif tutorialPhase != None:
    ANDText = get_font3(15).render("AND Gate", True, GREY)
ANDrectText = ANDText.get_rect()
ANDrectText.center = (200,155)

NORGate = pygame.image.load("NORGate.png")
NORrect = NORGate.get_rect()
NORrect.center = (50,195)
if activeGate == "NORGate":
    NORText = get_font3(15).render("NOR Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 10:
    NORText = get_font3(15).render("NOR Gate", True, BLACK)
elif tutorialPhase != None:
    NORText = get_font3(15).render("NOR Gate", True, GREY)
NORrectText = NORText.get_rect()
NORrectText.center = (50,230)

NANDGate = pygame.image.load("NANDGate.png")
NANDrect = NANDGate.get_rect()
NANDrect.center = (125,195)
if activeGate == "NANDGate":
    NANDText = get_font3(15).render("NAND Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 10:
    NANDText = get_font3(15).render("NAND Gate", True, BLACK)
elif tutorialPhase != None:
```

```
NANDText = get_font3(15).render("NAND Gate", True, GREY)
NANDrectText = NANDText.get_rect()
NANDrectText.center = (125,230)

XNORGate = pygame.image.load("XNORGate.png")
XNORrect = ANDGate.get_rect()
XNORrect.center = (200,195)
if activeGate == "XNORGate":
    XNORText = get_font3(15).render("XNOR Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 10:
    XNORText = get_font3(15).render("XNOR Gate", True, BLACK)
elif tutorialPhase != None:
    XNORText = get_font3(15).render("XNOR Gate", True, GREY)
XNORrectText = XNORText.get_rect()
XNORrectText.center = (200,230)

XORGate = pygame.image.load("XORGate.png")
XORrect = XORGate.get_rect()
XORrect.center = (50,270)
if activeGate == "XORGate":
    XORText = get_font3(15).render("XOR Gate", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 10:
    XORText = get_font3(15).render("XOR Gate", True, BLACK)
elif tutorialPhase != None:
    XORText = get_font3(15).render("XOR Gate", True, GREY)
XORrectText = XORText.get_rect()
XORrectText.center = (50,305)

offSwitch = pygame.image.load("off_switch.jpg")
switchRect = offSwitch.get_rect()
switchRect.center = (45,365)
if activeGate == "Switch_OFF":
    switchText = get_font3(15).render("Switch", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 1 or tutorialPhase == 6:
    switchText = get_font3(15).render("Switch", True, BLACK)
elif tutorialPhase != None:
    switchText = get_font3(15).render("Switch", True, GREY)
switchRectText = switchText.get_rect()
switchRectText.center = (45,385)

wireHorizontal = pygame.image.load("wire_horizontal.png")
wireRect = wireHorizontal.get_rect()
wireRect.center = (115,370)
if activeGate == "wireHorizontal":
```



```
wireText = get_font3(15).render("Wire", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 2 or tutorialPhase == 7:
    wireText = get_font3(15).render("Wire", True, BLACK)
elif tutorialPhase != None:
    wireText = get_font3(15).render("Wire", True, GREY)
wireRectText = wireText.get_rect()
wireRectText.center = (115,405)

clock = pygame.image.load("Clock_OFF.png")
clockRect = clock.get_rect()
clockRect.center = (190,370)
if activeGate == "Clock":
    clockText = get_font3(15).render("Clock", True, BLUE)
elif tutorialPhase != None:
    clockText = get_font3(15).render("Clock", True, GREY)
else:
    clockText = get_font3(15).render("Clock", True, BLACK)
clockTextRect = clockText.get_rect()
clockTextRect.center = (190,405)

offLight = pygame.image.load("off_bulb.png")
lightRect = offLight.get_rect()
lightRect.center = (47,465)
if activeGate == "Light_OFF":
    lightText = get_font3(15).render("Light Bulb", True, BLUE)
elif tutorialPhase == None or tutorialPhase == 4 or tutorialPhase == 9:
    lightText = get_font3(15).render("Light Bulb", True, BLACK)
elif tutorialPhase != None:
    lightText = get_font3(15).render("Light Bulb", True, GREY)
lightRectText = lightText.get_rect()
lightRectText.center = (47,500)

DFlipFlop = pygame.image.load("DflipFlop.png")
DFlipFlopRect = DFlipFlop.get_rect()
DFlipFlopRect.center = (47,575)
if activeGate == "DFlipFlop":
    DFlipFlopText = get_font3(15).render("Edge D Type", True, BLUE)
elif tutorialPhase == None:
    DFlipFlopText = get_font3(15).render("Edge D Type", True, BLACK)
else:
    DFlipFlopText = get_font3(15).render("Edge D Type", True, GREY)
DFlipFlopTextRect = DFlipFlopText.get_rect()
DFlipFlopTextRect.center = (47,610)
```

```
#auto save switch and text display
#if auto save is off, switch is off
if autoSave == False:
    autoSaveSwitch = pygame.image.load("off_switch.jpg")
#if auto save is on, switch is on
else:
    autoSaveSwitch = pygame.image.load("on_switch.jpg")
autoSaveSwitchRect = autoSaveSwitch.get_rect()
autoSaveSwitchRect = (1050,30)
#auto save unavailable
if fileName == None or tutorialMode == True:
    autoSaveText = get_font3(15).render("Auto Save", True, GREY)
#auto save off
elif autoSave == False:
    autoSaveText = get_font3(15).render("Auto Save", True, BLACK)
#auto save on
elif autoSave == True:
    autoSaveText = get_font3(15).render("Auto Save", True, GREEN)
autoSaveTextRect = autoSaveText.get_rect()
autoSaveTextRect.center = (1150,42)

#load wires, buttons, lights images

toBeSquare = pygame.image.load("toBeSquare.png")

wireIntersection = pygame.image.load("wire_intersection.jpg")
wireIntersection_ON_BOTTOM =
pygame.image.load("wire_intersection_ON_BOTTOM.jpg")
wireIntersection_ON_TOP =
pygame.image.load("wire_intersection_ON_TOP.jpg")
wireIntersection_ON = pygame.image.load("wire_intersection_ON.jpg")

wireHorizontal = pygame.image.load("wire_horizontal.png")
wireHorizontal_ON = pygame.image.load("wire_horizontal_ON.png")

wireBottomRight = pygame.image.load("wire_bottomRight.png")
wireBottomRight_ON = pygame.image.load("wire_bottomRight_ON.png")

wireBottomLeft = pygame.image.load("wire_bottomLeft.png")
wireBottomLeft_ON = pygame.image.load("wire_bottomLeft_ON.png")

wireTopRight = pygame.image.load("wire_topRight.png")
wireTopRight_ON = pygame.image.load("wire_topRight_ON.png")
```

```
wireTopLeft = pygame.image.load("wire_topLeft.png")
wireTopLeft_ON = pygame.image.load("wire_topLeft_ON.png")

wireVertical = pygame.image.load("wire_vertical.png")
wireVertical_ON = pygame.image.load("wire_vertical_ON.png")

wireSplitterDown = pygame.image.load("wire_splitter_down.png")
wireSplitterDown_ON = pygame.image.load("wire_splitter_down_ON.png")

wireSplitterUp = pygame.image.load("wire_splitter_up.png")
wireSplitterUp_ON = pygame.image.load("wire_splitter_up_ON.png")

wireSplitterRight = pygame.image.load("wire_splitter_right.png")
wireSplitterRight_ON = pygame.image.load("wire_splitter_right_ON.png")

wireSplitterLeft = pygame.image.load("wire_splitter_left.png")
wireSplitterLeft_ON = pygame.image.load("wire_splitter_left_ON.png")

wireCross = pygame.image.load("wire_cross.png")
wireCross_ON_HORIZONTAL =
pygame.image.load("wire_cross_ON_HORIZONTAL.png")
wireCross_ON_VERTICAL =
pygame.image.load("wire_cross_ON_VERTICAL.png")
wireCross_ON = pygame.image.load("wire_cross_ON.png")

onSwitch = pygame.image.load("on_switch.jpg")
onLight = pygame.image.load("on_bulb.png")

clock_OFF = pygame.image.load("Clock_OFF.png")
clock_ON = pygame.image.load("Clock_ON.png")

DFlipFlop = pygame.image.load("DFlipFlop.png")

#draw grid
for i in range(251,1280,49):
    pygame.draw.line(SCREEN,BLACK,(i,60),(i,720),2)
for i in range(60,720,49):
    pygame.draw.line(SCREEN,BLACK,(60,i),(1280,i),2)
pygame.draw.rect(SCREEN,BLUE, [0,0,1280,60])
pygame.draw.rect(SCREEN,RED, [0,60,251,720])

#drawing gates in array
for i in range(0,len(logicArray)):
    for j in range(0,len(logicArray[i])):
```

```
if logicArray[i][j] == None:
    pass
elif logicArray[i][j][0] == "ORGate":
    #position in array used to determine which grid square to place in
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    #gate is drawn
    DrawORrect = ORGate.get_rect()
    DrawORrect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(ORGate, DrawORrect)
elif logicArray[i][j][0] == "NOTGate":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawNOTrect = NOTGate.get_rect()
    DrawNOTrect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(NOTGate, DrawNOTrect)
elif logicArray[i][j][0] == "ANDGate":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawANDrect = ANDGate.get_rect()
    DrawANDrect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(ANDGate, DrawANDrect)
elif logicArray[i][j][0] == "NORGate":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawNORrect = NORGate.get_rect()
    DrawNORrect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(NORGate, DrawNORrect)
elif logicArray[i][j][0] == "NANDGate":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawNANDrect = NANDGate.get_rect()
    DrawNANDrect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(NANDGate, DrawNANDrect)
elif logicArray[i][j][0] == "XNORGate":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawXNORrect = XNORGate.get_rect()
    DrawXNORrect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(XNORGate, DrawXNORrect)
elif logicArray[i][j][0] == "XORGate":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawXORrect = XORGate.get_rect()
```

```
    DrawXORrect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(XORGate, DrawXORrect)
elif logicArray[i][j][0] == "Switch_OFF":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawOffSwitchRect = offSwitch.get_rect()
    DrawOffSwitchRect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(offSwitch,DrawOffSwitchRect)
elif logicArray[i][j][0] == "Switch_ON":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawOnSwitchRect = onSwitch.get_rect()
    DrawOnSwitchRect.center = (XdrawPos, YdrawPos)
    SCREEN.blit(onSwitch,DrawOnSwitchRect)
elif logicArray[i][j][0] == "Light_OFF":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawOffLightRect = offLight.get_rect()
    DrawOffLightRect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(offLight,DrawOffLightRect)
elif logicArray[i][j][0] == "Light_ON":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawOnLightRect = onLight.get_rect()
    DrawOnLightRect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(onLight,DrawOnLightRect)
elif logicArray[i][j][0] == "wireIntersection":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawWireIntersectionRect = wireIntersection.get_rect()
    DrawWireIntersectionRect.center = (XdrawPos,YdrawPos)
    #depending on state of wire, different wire colours are used
    if logicArray[i][j][1] == True and logicArray[i][j][2] == True:
        SCREEN.blit(wireIntersection_ON,DrawWireIntersectionRect)
    elif logicArray[i][j][1] == True:
        SCREEN.blit(wireIntersection_ON_TOP,DrawWireIntersectionRect)
    elif logicArray[i][j][2] == True:
        SCREEN.blit(wireIntersection_ON_BOTTOM,DrawWireIntersectionRect)
    else:
        SCREEN.blit(wireIntersection,DrawWireIntersectionRect)
elif logicArray[i][j][0] == "wireHorizontal":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
```

```
DrawWireHorizontalRect = wireHorizontal.get_rect()
DrawWireHorizontalRect.center = (XdrawPos,YdrawPos)
if logicArray[i][j][1] == True:
    SCREEN.blit(wireHorizontal_ON,DrawWireHorizontalRect)
else:
    SCREEN.blit(wireHorizontal, DrawWireHorizontalRect)
elif logicArray[i][j][0] == "wireVertical":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawWireVerticalRect = wireVertical.get_rect()
    DrawWireVerticalRect.center = (XdrawPos,YdrawPos)
    if logicArray[i][j][1] == True:
        SCREEN.blit(wireVertical_ON,DrawWireVerticalRect)
    else:
        SCREEN.blit(wireVertical, DrawWireVerticalRect)
elif logicArray[i][j][0] == "wireBottomRight":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawWireBottomRightRect = wireBottomRight.get_rect()
    DrawWireBottomRightRect.center = (XdrawPos,YdrawPos)
    if logicArray[i][j][1] == True:
        SCREEN.blit(wireBottomRight_ON,DrawWireBottomRightRect)
    else:
        SCREEN.blit(wireBottomRight, DrawWireBottomRightRect)
elif logicArray[i][j][0] == "wireBottomLeft":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawWireBottomLeftRect = wireBottomLeft.get_rect()
    DrawWireBottomLeftRect.center = (XdrawPos,YdrawPos)
    if logicArray[i][j][1] == True:
        SCREEN.blit(wireBottomLeft_ON,DrawWireBottomLeftRect)
    else:
        SCREEN.blit(wireBottomLeft, DrawWireBottomLeftRect)
elif logicArray[i][j][0] == "wireTopRight":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawWireTopRightRect = wireTopRight.get_rect()
    DrawWireTopRightRect.center = (XdrawPos,YdrawPos)
    if logicArray[i][j][1] == True:
        SCREEN.blit(wireTopRight_ON,DrawWireTopRightRect)
    else:
        SCREEN.blit(wireTopRight, DrawWireTopRightRect)
elif logicArray[i][j][0] == "wireTopLeft":
    XdrawPos = (j+1)*49 + 227
```

```
YdrawPos = (i+1)*49 + 36
DrawWireTopLeftRect = wireBottomRight.get_rect()
DrawWireTopLeftRect.center = (XdrawPos,YdrawPos)
if logicArray[i][j][1] == True:
    SCREEN.blit(wireTopLeft_ON,DrawWireTopLeftRect)
else:
    SCREEN.blit(wireTopLeft, DrawWireTopLeftRect)
elif logicArray[i][j][0] == "wireSplitterDown":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawWireSplitterDownRect = wireSplitterDown.get_rect()
    DrawWireSplitterDownRect.center = (XdrawPos,YdrawPos)
    if logicArray[i][j][1] == True:
        SCREEN.blit(wireSplitterDown_ON,DrawWireSplitterDownRect)
    else:
        SCREEN.blit(wireSplitterDown, DrawWireSplitterDownRect)
elif logicArray[i][j][0] == "wireSplitterUp":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawWireSplitterUpRect = wireSplitterUp.get_rect()
    DrawWireSplitterUpRect.center = (XdrawPos,YdrawPos)
    if logicArray[i][j][1] == True:
        SCREEN.blit(wireSplitterUp_ON,DrawWireSplitterUpRect)
    else:
        SCREEN.blit(wireSplitterUp, DrawWireSplitterUpRect)
elif logicArray[i][j][0] == "wireSplitterRight":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawWireSplitterRightRect = wireSplitterRight.get_rect()
    DrawWireSplitterRightRect.center = (XdrawPos,YdrawPos)
    if logicArray[i][j][1] == True:
        SCREEN.blit(wireSplitterRight_ON,DrawWireSplitterRightRect)
    else:
        SCREEN.blit(wireSplitterRight, DrawWireSplitterRightRect)
elif logicArray[i][j][0] == "wireSplitterLeft":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawWireSplitterLeftRect = wireSplitterLeft.get_rect()
    DrawWireSplitterLeftRect.center = (XdrawPos,YdrawPos)
    if logicArray[i][j][1] == True:
        SCREEN.blit(wireSplitterLeft_ON,DrawWireSplitterLeftRect)
    else:
        SCREEN.blit(wireSplitterLeft, DrawWireSplitterLeftRect)
elif logicArray[i][j][0] == "wireCross":
```

```
XdrawPos = (j+1)*49 + 227
YdrawPos = (i+1)*49 + 36
DrawWireCrossRect = wireSplitterUp.get_rect()
DrawWireCrossRect.center = (XdrawPos,YdrawPos)
if logicArray[i][j][1] == True and logicArray[i][j][2] == True:
    SCREEN.blit(wireCross_ON,DrawWireCrossRect)
elif logicArray[i][j][1] == True:
    SCREEN.blit(wireCross_ON_VERTICAL,DrawWireCrossRect)
elif logicArray[i][j][2] == True:
    SCREEN.blit(wireCross_ON_HORIZONTAL,DrawWireCrossRect)
else:
    SCREEN.blit(wireCross, DrawWireCrossRect)
elif logicArray[i][j][0] == "Clock":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawClockRect = clock_OFF.get_rect()
    DrawClockRect.center = (XdrawPos,YdrawPos)
    if logicArray[i][j][1] == True:
        SCREEN.blit(clock_ON,DrawClockRect)
    else:
        SCREEN.blit(clock_OFF,DrawClockRect)
elif logicArray[i][j][0] == "DFlipFlop":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    DrawDFlipFlopRect = DFlipFlop.get_rect()
    DrawDFlipFlopRect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(DFlipFlop,DrawDFlipFlopRect)
#used for tutorial mode
elif logicArray[i][j][0] == "wireToBe" or logicArray[i][j][0] == "NOTGateToBe"
or logicArray[i][j][0] == "lightToBe" or logicArray[i][j][0] == "switchToBe" or
logicArray[i][j][0] == "gateToBe":
    XdrawPos = (j+1)*49 + 227
    YdrawPos = (i+1)*49 + 36
    drawToBeSquareRect = toBeSquare.get_rect()
    drawToBeSquareRect.center = (XdrawPos,YdrawPos)
    SCREEN.blit(toBeSquare,drawToBeSquareRect)

#button creation
if tutorialPhase == None:
    logicInterfaceSave = Button(pos=(55,30), text_input="Save",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
```



```
    logicInterfaceSaveAs = Button(pos=(145,30), text_input="Save As",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
    logicInterfaceLoad = Button(pos=(235,30), text_input="Load",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
    #undo button changes colour based on stack being empty
    if undoStack == []:
        logicInterfaceUndo = Button(pos=(315,30), text_input="Undo",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
    else:
        logicInterfaceUndo = Button(pos=(315,30), text_input="Undo",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
    #undo button changes colour based on stack being empty
    if redoStack == []:
        logicInterfaceRedo = Button(pos=(390,30), text_input="Redo",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
    else:
        logicInterfaceRedo = Button(pos=(390,30), text_input="Redo",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
    logicInterfaceTruthTable = Button(pos=(500,30), text_input="Truth Table",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
    #delete button changes colour based on delete function being active
    if activeGate != "Delete":
        logicInterfaceDelete = Button(pos=(625,30), text_input="Delete",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
    logicInterfaceClear = Button(pos=(715,30), text_input="Clear",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
    logicInterfaceNew = Button(pos=(795,30), text_input="New",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
    logicInterfaceMainMenu = Button(pos=(905,30), text_input="Main Menu",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
    #all options are disabled in tutorial
    elif tutorialPhase == 1:
        logicInterfaceSave = Button(pos=(55,30), text_input="Save",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
        logicInterfaceSaveAs = Button(pos=(145,30), text_input="Save As",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
        logicInterfaceLoad = Button(pos=(235,30), text_input="Load",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
        logicInterfaceUndo = Button(pos=(315,30), text_input="Undo",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
        logicInterfaceRedo = Button(pos=(390,30), text_input="Redo",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
        logicInterfaceTruthTable = Button(pos=(500,30), text_input="Truth Table",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
```

```
logicInterfaceDelete = Button(pos=(625,30), text_input="Delete",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
logicInterfaceClear = Button(pos=(715,30), text_input="Clear",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
logicInterfaceNew = Button(pos=(795,30), text_input="New",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
logicInterfaceMainMenu = Button(pos=(905,30), text_input="Main Menu",
font=get_font2(25), base_color=GREY, hovering_color=GREY)
logicInterfaceTutorialBack = Button(pos=(110,680), text_input="Back",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
elif tutorialPhase == 4 or tutorialPhase == 10:
    logicInterfaceTutorialNext = Button(pos=(200,680), text_input="Next",
font=get_font2(25), base_color=BLACK, hovering_color=GREY)
```

```
#logic gate menu text creation
```

```
logicGateText = get_font3(20).render("Logic Gates", True, BLACK)
logicGateRectText = logicGateText.get_rect()
logicGateRectText.center = (70,80)
SCREEN.blit(logicGateText, logicGateRectText)
```

```
inputText = get_font3(20).render("Inputs", True, BLACK)
inputTextRect = inputText.get_rect()
inputTextRect.center = (45,330)
SCREEN.blit(inputText, inputTextRect)
```

```
outputText = get_font3(20).render("Outputs", True, BLACK)
outputTextRect = outputText.get_rect()
outputTextRect.center = (50, 425)
SCREEN.blit(outputText, outputTextRect)
```

```
flipFlopText = get_font3(20).render("Flip Flops", True, BLACK)
flipFlopTextRect = outputText.get_rect()
flipFlopTextRect.center = (45, 530)
SCREEN.blit(flipFlopText, flipFlopTextRect)
```

```
#document name text
```

```
if tutorialPhase != None:
```

```
    fileNameText = get_font3(15).render("Tutorial", True, BLACK)
```

```
elif fileName == None:
```

```
    fileNameText = get_font3(15).render("New Document", True, BLACK)
```

```
else:
```

```
    fileNameText = get_font3(15).render(fileName, True, BLACK)
```

```
fileNameTextRect = fileNameText.get_rect()
```

```
fileNameTextRect.center = (1100,10)
```

```
SCREEN.blit(fileNameText, fileNameTextRect)

#block transfer of images and text
SCREEN.blit(ORGate, ORrect)
SCREEN.blit(ORText, ORrectText)

SCREEN.blit(NOTGate, NOTrect)
SCREEN.blit(NOTText, NOTrectText)

SCREEN.blit(ANDGate, ANDrect)
SCREEN.blit(ANDText, ANDrectText)

SCREEN.blit(NORGate, NORrect)
SCREEN.blit(NORText, NORrectText)

SCREEN.blit(NANDGate, NANDrect)
SCREEN.blit(NANDText, NANDrectText)

SCREEN.blit(XNORGate, XNORrect)
SCREEN.blit(XNORText, XNORrectText)

SCREEN.blit(XORGate, XORrect)
SCREEN.blit(XORText, XORrectText)

SCREEN.blit(offSwitch, switchRect)
SCREEN.blit(switchText, switchRectText)

SCREEN.blit(offLight, lightRect)
SCREEN.blit(lightText, lightRectText)

SCREEN.blit(wireHorizontal, wireRect)
SCREEN.blit(wireText, wireRectText)

SCREEN.blit(clock, clockRect)
SCREEN.blit(clockText, clockTextRect)

SCREEN.blit(DFlipFlop, DFlipFlopRect)
SCREEN.blit(DFlipFlopText, DFlipFlopTextRect)

SCREEN.blit(autoSaveSwitch, autoSaveSwitchRect)
SCREEN.blit(autoSaveText, autoSaveTextRect)

#main logic
#on/off subroutines
```

```
def turnOnVerticalUp(i,j):
    startPoint = i-1
    for k in range(startPoint,-1,-1):
        if logicArray[k][j] == [None]:
            break
        elif logicArray[k][j][0] == "wireVertical":
            logicArray[k][j][1] = True
            logicArray[k][j][2] = "Up"
        elif logicArray[k][j][0] == "wireTopLeft":
            logicArray[k][j][1] = True
            logicArray[k][j][2] = "Right"
            turnOnHorizontalRight(k,j)
        elif logicArray[k][j][0] == "wireTopRight":
            logicArray[k][j][1] = True
            logicArray[k][j][2] = "Left"
            turnOnHorizontalLeft(k,j)
        elif logicArray[k][j][0] == "wireIntersection":
            logicArray[k][j][2] = True
            break
        elif logicArray[k][j][0] == "wireCross":
            logicArray[k][j][1] = True
        elif logicArray[k][j][0] == "wireSplitterLeft":
            logicArray[k][j][1] = True
            logicArray[k][j][2] = "Up"
            turnOnVerticalUp(k,j)
            turnOnHorizontalLeft(k,j)
        elif logicArray[k][j][0] == "wireSplitterRight":
            logicArray[k][j][1] = True
            logicArray[k][j][2] = "Up"
            turnOnVerticalUp(k,j)
            turnOnHorizontalRight(k,j)
        elif logicArray[k][j][0] == "wireSplitterDown":
            logicArray[k][j][1] = True
            logicArray[k][j][2] = "Up"
            turnOnHorizontalRight(i,k)
            turnOnVerticalLeft(i,k)
```

```
def turnOnVerticalDown(i,j):
    startPoint = i+1
    for k in range(startPoint,13):
        if logicArray[k][j] == [None]:
            break
        elif logicArray[k][j][0] == "wireVertical":
            logicArray[k][j][1] = True
```

```
    logicArray[k][j][2] = "Down"
elif logicArray[k][j][0] == "wireBottomLeft":
    logicArray[k][j][1] = True
    logicArray[k][j][2] = "Right"
    turnOnHorizontalRight(k,j)
elif logicArray[k][j][0] == "wireBottomRight":
    logicArray[k][j][1] = True
    logicArray[k][j][2] = "Left"
    turnOnHorizontalLeft(k,j)
elif logicArray[k][j][0] == "wireIntersection":
    logicArray[k][j][1] = True
    break
elif logicArray[k][j][0] == "wireCross":
    logicArray[k][j][1] = True
elif logicArray[k][j][0] == "wireSplitterLeft":
    logicArray[k][j][1] = True
    logicArray[k][j][2] = "Down"
    turnOnVerticalDown(k,j)
    turnOnHorizontalLeft(k,j)
elif logicArray[k][j][0] == "wireSplitterRight":
    logicArray[k][j][1] = True
    logicArray[k][j][2] = "Down"
    turnOnVerticalDown(k,j)
    turnOnHorizontalRight(k,j)
elif logicArray[k][j][1] == "wireSplitterUp":
    logicArray[k][j][1] = True
    logicArray[k][j][2] = "Down"
    turnOnHorizontalRight(k,j)
    turnOnHorizontalLeft(k,j)

def turnOffVerticalUp(i,j):
    startPoint = i-1
    for k in range(startPoint,-1,-1):
        if logicArray[k][j] == [None]:
            break
        elif logicArray[k][j][0] == "wireVertical":
            logicArray[k][j][1] = False
            logicArray[k][j][2] = None
        elif logicArray[k][j][0] == "wireTopLeft":
            logicArray[k][j][1] = False
            logicArray[k][j][2] = None
            turnOffHorizontalRight(k,j)
        elif logicArray[k][j][0] == "wireTopRight":
            logicArray[k][j][1] = False
```

```
    logicArray[k][j][2] = None
    turnOffHorizontalLeft(k,j)
elif logicArray[k][j][0] == "wireIntersection":
    logicArray[k][j][2] = False
    break
elif logicArray[k][j][0] == "wireCross":
    logicArray[k][j][1] = False
elif logicArray[k][j][0] == "wireSplitterLeft":
    logicArray[k][j][1] = False
    logicArray[k][j][2] = None
    turnOffVerticalUp(k,j)
    turnOffHorizontalLeft(k,j)
elif logicArray[k][j][0] == "wireSplitterRight":
    logicArray[k][j][1] = False
    logicArray[k][j][2] = None
    turnOffVerticalUp(k,j)
    turnOffHorizontalRight(k,j)
elif logicArray[k][j][0] == "wireSplitterDown":
    logicArray[k][j][1] = False
    logicArray[k][j][2] = None
    turnOffHorizontalRight(i,k)
    turnOffVerticalLeft(i,k)

def turnOffVerticalDown(i,j):
    startPoint = i+1
    for k in range(startPoint,13):
        if logicArray[k][j] == [None]:
            break
        elif logicArray[k][j][0] == "wireVertical":
            logicArray[k][j][1] = False
            logicArray[k][j][2] = None
        elif logicArray[k][j][0] == "wireBottomLeft":
            logicArray[k][j][1] = False
            logicArray[k][j][2] = None
            turnOffHorizontalRight(k,j)
        elif logicArray[k][j][0] == "wireBottomRight":
            logicArray[k][j][1] = False
            logicArray[k][j][2] = None
            turnOffHorizontalLeft(k,j)
        elif logicArray[k][j][0] == "wireIntersection":
            logicArray[k][j][1] = False
            break
        elif logicArray[k][j][0] == "wireCross":
            logicArray[k][j][1] = False
```

```
elif logicArray[k][j][0] == "wireSplitterLeft":
    logicArray[k][j][1] = False
    logicArray[k][j][2] = None
    turnOffVerticalUp(k,j)
    turnOffHorizontalLeft(k,j)
elif logicArray[k][j][0] == "wireSplitterRight":
    logicArray[k][j][1] = False
    logicArray[k][j][2] = None
    turnOffVerticalUp(k,j)
    turnOnHorizontalRight(k,j)
elif logicArray[k][j][0] == "wireSplitterUp":
    logicArray[k][j][1] = False
    logicArray[k][j][2] = None
    turnOffHorizontalRight(k,j)
    turnOffHorizontalLeft(k,j)
```

#i refers to vertical position, j refers to horizontal position

```
def turnOnHorizontalRight(i,j):
    startPoint = j+1
    #entire row is searched
    for k in range(startPoint,21):
        #when blank square is found, subroutine ends
        if logicArray[i][k] == [None]:
            break
        #state of wire is changed, direction of power flow is defined as "right"
        elif logicArray[i][k][0] == "wireHorizontal":
            logicArray[i][k][1] = True
            logicArray[i][k][2] = "Right"
        elif logicArray[i][k][0] == "wireBottomRight":
            logicArray[i][k][1] = True
            logicArray[i][k][2] = "Up"
            #wire direction changes, so vertical up subroutine is called
            turnOnVerticalUp(i,k)
        elif logicArray[i][k][0] == "wireTopRight":
            logicArray[i][k][1] = True
            logicArray[i][k][2] = "Down"
            #wire direction changes so vertical down subroutine is called
            turnOnVerticalDown(i,k)
        elif logicArray[i][k][0] == "wireSplitterDown":
            logicArray[i][k][1] = True
            logicArray[i][k][2] = "Right"
            #power is splitm so both vertical down and horizontal right subroutines
            are called
            turnOnVerticalDown(i,k)
```

```
        turnOnHorizontalRight(i,k)
    elif logicArray[i][k][0] == "wireSplitterUp":
        logicArray[i][k][1] = True
        logicArray[i][k][2] = "Right"
        #power is split, so both vertical up and horizontal right subroutines are
called
        turnOnVerticalUp(i,k)
        turnOnHorizontalRight(i,k)
    elif logicArray[i][k][0] == "wireSplitterLeft":
        logicArray[i][k][1] = True
        logicArray[i][k][2] = "Right"
        #power is split, so both vertical up and vertical down subroutines are
called
        turnOnVerticalDown(i,k)
        turnOnVerticalUp(i,k)
    elif logicArray[i][k][0] == "wireCross":
        #wire cross functions in the same way as a horizontal wire, but with a
different index
        logicArray[i][k][2] = True

def turnOnHorizontalLeft(i,j):
    startPoint = j-1
    for k in range(startPoint,-1,-1):
        if logicArray[i][k] == [None]:
            break
        elif logicArray[i][k][0] == "wireHorizontal":
            logicArray[i][k][1] = True
            logicArray[i][k][2] = "Left"
        elif logicArray[i][k][0] == "wireBottomLeft":
            logicArray[i][k][1] = True
            turnOnVerticalUp(i,k)
            logicArray[i][k][2] = "Up"
        elif logicArray[i][k][0] == "wireTopLeft":
            logicArray[i][k][1] = True
            turnOnVerticalDown(i,k)
            logicArray[i][k][2] = "Down"
        elif logicArray[i][k][0] == "wireSplitterDown":
            logicArray[i][k][1] = True
            logicArray[i][k][2] = "Left"
            turnOnVerticalDown(i,k)
            turnOnHorizontalLeft(i,k)
        elif logicArray[i][k][0] == "wireSplitterUp":
            logicArray[i][k][1] = True
            logicArray[i][k][2] = "Left"
```



```
    turnOnVerticalUp(i,k)
    turnOnHorizontalRight(i,k)
elif logicArray[i][k][0] == "wireSplitterRight":
    logicArray[i][k][1] = True
    logicArray[i][k][2] = "Left"
    turnOnVerticalDown(i,k)
    turnOnVerticalUp(i,k)
elif logicArray[i][k][0] == "wireCross":
    logicArray[i][k][2] = True
```

#i refers to vertical position, j refers to horizontal position

```
def turnOffHorizontalRight(i,j):
    startPoint = j+1
    #entire row is searched
    for k in range(startPoint,21):
        #when blank square is found, subroutine ends
        if logicArray[i][k] == [None]:
            break
        elif logicArray[i][k][0] == "wireHorizontal":
            #state of wire is changed to off, direction of power flow is reset
            logicArray[i][k][2] = None
            logicArray[i][k][1] = False
        elif logicArray[i][k][0] == "wireBottomRight":
            logicArray[i][k][2] = None
            logicArray[i][k][1] = False
            #wire direction changes, so vertical subroutine is called
            turnOffVerticalUp(i, k)
        elif logicArray[i][k][0] == "wireTopRight":
            logicArray[i][k][2] = None
            logicArray[i][k][1] = False
            #wire direction changes, so vertical down subroutine is called
            turnOffVerticalDown(i,k)
        elif logicArray[i][k][0] == "wireSplitterDown":
            logicArray[i][k][2] = None
            logicArray[i][k][1] = False
            #power is split, both horizontal right and vertical down subroutines are
            called
            turnOffVerticalDown(i,k)
            turnOffHorizontalRight(i,k)
        elif logicArray[i][k][0] == "wireSplitterUp":
            logicArray[i][k][2] = None
            logicArray[i][k][1] = False
            #power is split, both horizontal right and vertical up subroutines are
            called
```

```
    turnOffVerticalUp(i,k)
    turnOffHorizontalRight(i,k)
elif logicArray[i][k][0] == "wireSplitterLeft":
    logicArray[i][k][1] = False
    logicArray[i][k][2] = None
    #power is split, both vertical up and down subroutines are called
    turnOffVerticalDown(i,k)
    turnOffVerticalUp(i,k)
elif logicArray[i][k][0] == "wireCross":
    #wire cross functions in the same way as a horizontal wire but with a
different index
    logicArray[i][k][2] = False

def turnOffHorizontalLeft(i,j):
    startPoint = j-1
    for k in range(startPoint,-1,-1):
        if logicArray[i][k] == [None]:
            break
        elif logicArray[i][k][0] == "wireHorizontal":
            logicArray[i][k][1] = False
            logicArray[i][k][2] = None
        elif logicArray[i][k][0] == "wireBottomLeft":
            logicArray[i][k][1] = False
            logicArray[i][k][2] = None
            turnOffVerticalUp(i,k)
        elif logicArray[i][k][0] == "wireTopLeft":
            logicArray[i][k][1] = False
            logicArray[i][k][2] = None
            turnOffVerticalDown(i,k)
        elif logicArray[i][k][0] == "wireSplitterDown":
            logicArray[i][k][1] = False
            logicArray[i][k][2] = None
            turnOffVerticalDown(i,k)
            turnOffHorizontalLeft(i,k)
        elif logicArray[i][k][0] == "wireSplitterUp":
            logicArray[i][k][1] = False
            logicArray[i][k][2] = None
            turnOffVerticalUp(i,k)
            turnOffHorizontalRight(i,k)
        elif logicArray[i][k][0] == "wireSplitterRight":
            logicArray[i][k][1] = False
            logicArray[i][k][2] = None
            turnOnVerticalDown(i,k)
            turnOnVerticalUp(i,k)
```

```
elif logicArray[i][k][0] == "wireCross":  
    logicArray[i][k][2] = False
```

```
#auto save logic  
if autoSave == True:  
    # if time passed since last save is 30 seconds  
    if time.time() - autoSaveStartTime >= 30:  
        print("Auto Saving")  
        #time reset  
        autoSaveStartTime = time.time()  
        logicFile = open(fileName, "wb")  
        #file saved  
        pickle.dump(logicArray, logicFile)  
        logicFile.close()
```

```
#main loop  
def mainLoop():  
    #global variables  
    global lastCountChange  
    global count  
    global finalTime  
    global clockPulse  
    global lastChange  
    #clock pulses  
    if time.time() > finalTime:  
        if clockPulse == False:  
            clockPulse = True  
        else:  
            clockPulse = False  
        lastChange = finalTime  
        finalTime = time.time() + 1  
    for i in range(0,len(logicArray)):  
        for j in range(0,len(logicArray[i])):  
            #algorithm passes over empty squares  
            if logicArray[i][j][0] == None:  
                pass  
            #turning a switch on causes corresponding elements in all directions to  
turn on  
            elif logicArray[i][j][0] == "Switch_ON":  
                turnOnHorizontalRight(i,j)  
                turnOnHorizontalLeft(i,j)  
                turnOnVerticalUp(i,j)  
                turnOnVerticalDown(i,j)
```

```
turn off
#turning a switch off causes corresponding elements in all directions to
turn off
elif logicArray[i][j][0] == "Switch_OFF":
    turnOffHorizontalRight(i,j)
    turnOffHorizontalLeft(i,j)
    turnOffVerticalUp(i,j)
    turnOffVerticalDown(i,j)
elif logicArray[i][j][0] == "Clock":
    #clock changes state based on pulse
    if clockPulse == True:
        logicArray[i][j][1] = True
        turnOnHorizontalRight(i,j)
        turnOnHorizontalLeft(i,j)
        turnOnVerticalDown(i,j)
        turnOnVerticalUp(i,j)
    else:
        logicArray[i][j][1] = False
        turnOffHorizontalRight(i,j)
        turnOffHorizontalLeft(i,j)
        turnOffVerticalDown(i,j)
        turnOffVerticalUp(i,j)
elif logicArray[i][j][0] == "NOTGate":
    #NOT operation
    if logicArray[i][j-1][0] == None:
        pass
    elif logicArray[i][j-1][1] == True:
        logicArray[i][j] = ["NOTGate", False]
        turnOffHorizontalRight(i,j)
    else:
        logicArray[i][j] = ["NOTGate", True]
        turnOnHorizontalRight(i,j)
elif logicArray[i][j][0] == "ORGate":
    #OR operation
    if logicArray[i][j-1][0] == "wireIntersection" and logicArray[i][j-1][1] ==
True or logicArray[i][j-1][2] == True:
        logicArray[i][j][1] = True
        turnOnHorizontalRight(i,j)
    else:
        logicArray[i][j][1] = False
        turnOffHorizontalRight(i,j)
elif logicArray[i][j][0] == "ANDGate":
    #AND operation
    if logicArray[i][j-1][0] == "wireIntersection" and logicArray[i][j-1][1] ==
True and logicArray[i][j-1][2] == True:
```

```
    logicArray[i][j][1] = True
    turnOnHorizontalRight(i,j)
else:
    logicArray[i][j][1] = False
    turnOffHorizontalRight(i,j)
elif logicArray[i][j][0] == "NORGate":
    #NOR operation
    if logicArray[i][j-1][0] == "wireIntersection" and logicArray[i][j-1][1] ==
True or logicArray[i][j-1][2] == True:
        logicArray[i][j][1] = False
        turnOffHorizontalRight(i,j)
    else:
        logicArray[i][j][1] = True
        turnOnHorizontalRight(i,j)
elif logicArray[i][j][0] == "NANDGate":
    #NAND operation
    if logicArray[i][j-1][0] == "wireIntersection" and logicArray[i][j-1][1] ==
True and logicArray[i][j-1][2] == True:
        logicArray[i][j][1] = False
        turnOffHorizontalRight(i,j)
    else:
        logicArray[i][j][1] = True
        turnOnHorizontalRight(i,j)
elif logicArray[i][j][0] == "XNORGate":
    #XNOR operation
    if logicArray[i][j-1][0] == "wireIntersection" and logicArray[i][j-1][1] ==
True and logicArray[i][j-1][2] == True:
        logicArray[i][j][1] = True
    elif logicArray[i][j-1][0] == "wireIntersection" and logicArray[i][j-1][1] ==
True or logicArray[i][j-1][2] == True:
        logicArray[i][j][1] = False
    else:
        logicArray[i][j][1] = True
    if logicArray[i][j][1] == True:
        turnOnHorizontalRight(i,j)
    else:
        turnOffHorizontalRight(i,j)
elif logicArray[i][j][0] == "XORGate":
    #XOR operation
    if logicArray[i][j-1][0] == "wireIntersection" and logicArray[i][j-1][1] ==
True and logicArray[i][j-1][2] == True:
        logicArray[i][j][1] = False
    elif logicArray[i][j-1][0] == "wireIntersection" and logicArray[i][j-1][1] ==
True or logicArray[i][j-1][2] == True:
```

```
        logicArray[i][j][1] = True
    else:
        logicArray[i][j][1] = False
    if logicArray[i][j][1] == True:
        turnOnHorizontalRight(i,j)
    else:
        turnOffHorizontalRight(i,j)
    elif logicArray[i][j][0] == "DFlipFlop":
        #state of D flip flop changes based on last switch state change and
time since last clock pulse
        if time.time() < lastChange + 0.5 and count == lastCountChange + 1:
            if logicArray[i][j-1][1] == True:
                if logicArray[i][j-1][2] == True:
                    logicArray[i][j][1] = False
                    turnOffHorizontalRight(i,j)
                elif logicArray[i][j-1][2] == False:
                    logicArray[i][j][1] = True
                    turnOnHorizontalRight(i,j)
            elif logicArray[i][j][1] == True:
                turnOnHorizontalRight(i,j)
            elif logicArray[i][j][1] == False:
                turnOffHorizontalRight(i,j)
        #light turns off if wire to the left is off
    elif logicArray[i][j][0] == "Light_OFF":
        if logicArray[i][j-1][0] == None:
            logicArray[i][j] = ["Light_OFF", False]
        elif logicArray[i][j-1][1] == True:
            logicArray[i][j] = ["Light_ON", True]
        #wire turns on if wire to the left is on
    elif logicArray[i][j][0] == "Light_ON":
        if logicArray[i][j-1][0] == None:
            pass
        elif logicArray[i][j-1][1] == False:
            logicArray[i][j] = ["Light_OFF", False]
    elif logicArray[i][j][0] == "wireHorizontal":
        #change to wire splitter up
        if (logicArray[i-1][j][0] == "wireHorizontal" or logicArray[i-1][j][0] ==
"wireVertical") and (logicArray[i][j-1][0] == "wireHorizontal" or logicArray[i][j-1][0] ==
"wireTopLeft") and (logicArray[i][j+1][0] == "wireHorizontal" or logicArray[i][j+1][0] ==
"wireBottomRight"):
            logicArray[i][j][0] = "wireSplitterUp"
        #change to bottom right
```

```

        elif (logicArray[i-1][j][0] == "wireHorizontal" or logicArray[i-1][j][0] ==
"wireIntersection") and (logicArray[i][j-1][0] == "wireHorizontal" or logicArray[i][j-1][0]
== "Switch_ON" or logicArray[i][j-1][0] == "Switch_OFF"):
            logicArray[i][j][0] = "wireBottomRight"
            if logicArray[i-1][j][0] == "wireHorizontal":
                logicArray[i][j][0] = "wireVertical"
            #change to top right
            elif (logicArray[i+1][j][0] == "wireHorizontal" or logicArray[i+1][j][0] ==
"wireIntersection") and logicArray[i][j-1][0] == "wireHorizontal":
                logicArray[i][j][0] = "wireTopRight"
                if logicArray[i+1][j][0] == "wireHorizontal":
                    logicArray[i+1][j][0] = "wireVertical"
            #change to vertical
            elif logicArray[i+1][j][0] == "wireVertical" or logicArray[i+1][j][0] ==
"wireIntersection" or logicArray[i+1][j][0] == "wireBottomRight" or logicArray[i+1][j][0]
== "wireSplitterUp" or logicArray[i+1][j][0] == "Switch_OFF" or logicArray[i+1][j][0] ==
"Switch_ON" or logicArray[i+1][j][0] == "Clock":
                logicArray[i][j][0] = "wireVertical"
            elif logicArray[i-1][j][0] == "wireVertical" or logicArray[i-1][j][0] ==
"wireIntersection" or logicArray[i-1][j][0] == "wireSplitterDown" or logicArray[i-1][j][0]
== "wireSplitterLeft" or logicArray[i-1][j][0] == "wireSplitterRight" or logicArray[i-1][j][0]
== "wireTopRight" or logicArray[i-1][j][0] == "Switch_OFF" or logicArray[i-1][j][0] ==
"Switch_ON" or logicArray[i-1][j][0] == "Clock":
                logicArray[i][j][0] = "wireVertical"
            elif logicArray[i+1][j][0] == "wireHorizontal":
                logicArray[i+1][j][0] = "wireVertical"
                logicArray[i][j][0] = "wireVertical"
            elif logicArray[i-1][j][0] == "wireHorizontal":
                logicArray[i-1][j][0] = "wireVertical"
                logicArray[i][j][0] = "wireVertical"
            elif logicArray[i][j][0] == "wireVertical":
                #change to wire splitter left
                if (logicArray[i-1][j][0] == "wireVertical" or logicArray[i-1][j][0] ==
"wireTopLeft") and (logicArray[i+1][j][0] == "wireVertical" or logicArray[i+1][j][0] ==
"wireBottomLeft") and logicArray[i][j-1][0] == "wireHorizontal":
                    logicArray[i][j][0] = "wireSplitterLeft"
                #change to wire splitter right
                elif (logicArray[i-1][j][0] == "wireVertical" or logicArray[i-1][j][0] ==
"wireTopRight") and (logicArray[i+1][j][0] == "wireVertical" or logicArray[i+1][j][0] ==
"wireBottomRight") and logicArray[i][j+1][0] == "wireHorizontal":
                    logicArray[i][j][0] = "wireSplitterRight"
                #change to top left

```

```
elif logicArray[i][j-1][0] != "wireHorizontal" and (logicArray[i][j+1][0] ==
"wireHorizontal" or logicArray[i][j+1][0] == "wireBottomRight") and
(logicArray[i+1][j][0] == "wireVertical" or logicArray[i+1][j][0] == "wireBottomRight"):
    logicArray[i][j][0] = "wireTopLeft"
    #change to top right
    elif (logicArray[i+1][j][0] == "wireVertical" or logicArray[i+1][j][0] ==
"wireBottomLeft" or logicArray[i+1][j][0] == "wireIntersection") and (logicArray[i][j-1][0]
== "wireHorizontal" or logicArray[i][j-1][0] == "wireBottomLeft" or logicArray[i][j-1][0]
== "Switch_ON" or logicArray[i][j-1][0] == "Switch_OFF" or logicArray[i][j-1][0] ==
"Clock"):
        logicArray[i][j][0] = "wireTopRight"
        #change to bottom left
        elif (logicArray[i-1][j][0] == "wireVertical" or logicArray[i-1][j][0] ==
"wireTopRight") and (logicArray[i][j+1][0] == "wireHorizontal" or logicArray[i][j+1][0] ==
"wireTopRight"):
            logicArray[i][j][0] = "wireBottomLeft"
            #change to bottom right
            elif (logicArray[i-1][j][0] == "wireVertical" or logicArray[i-1][j][0] ==
"wireIntersection" or logicArray[i-1][j][0] == "wireTopLeft" or logicArray[i-1][j][0] ==
"wireSplitterDown") and (logicArray[i][j-1][0] == "wireHorizontal" or logicArray[i][j-1][0]
== "wireTopLeft" or logicArray[i][j-1][0] == "Switch_ON" or logicArray[i][j-1][0] ==
"Switch_OFF" or logicArray[i][j-1][0] == "Clock"):
                logicArray[i][j][0] = "wireBottomRight"
                #top left change
                elif logicArray[i][j][0] == "wireTopLeft":
                    if logicArray[i+1][j][0] == None and (logicArray[i][j+1][0] ==
"wireHorizontal" or logicArray[i][j+1][0] == "wireBottomRight"):
                        logicArray[i][j][0] = "wireHorizontal"
                        elif (logicArray[i+1][j][0] == "wireVertical" or logicArray[i+1][j][0] ==
"wireBottomRight") and logicArray[i][j+1][0] == None:
                            logicArray[i][j][0] = "wireVertical"
                            elif logicArray[i][j-1][0] == "wireHorizontal" and logicArray[i][j+1][0] ==
"wireHorizontal":
                                logicArray[i][j][0] = "wireSplitterDown"
                                elif logicArray[i-1][j][0] == "wireHorizontal":
                                    logicArray[i][j][0] = "wireSplitterRight"
                                    logicArray[i-1][j][0] = "wireVertical"
                                #top right change
                                elif logicArray[i][j][0] == "wireTopRight":
                                    if (logicArray[i+1][j][0] == "wireVertical" or logicArray[i+1][j][0] ==
"wireVertical") and (logicArray[i][j-1][0] == "wireHorizontal" or logicArray[i][j-1][0] ==
"Switch_OFF" or logicArray[i][j-1][0] == "Switch_ON") and (logicArray[i][j+1][0] ==
"wireHorizontal" or logicArray[i][j+1][0] == "wireBottomRight"):
                                        logicArray[i][j][0] = "wireSplitterDown"
```



```
elif logicArray[i+1][j][0] == None:
    logicArray[i][j][0] = "wireHorizontal"
elif logicArray[i][j-1][0] == None:
    logicArray[i][j][0] = "wireVertical"
elif logicArray[i-1][j][0] == "wireHorizontal":
    logicArray[i][j][0] = "wireSplitterLeft"
    logicArray[i-1][j][0] = "wireVertical"
#bottom left change
elif logicArray[i][j][0] == "wireBottomLeft":
    if (logicArray[i-1][j][0] == "wireHorizontal" or logicArray[i-1][j][0] ==
"wireVertical") and (logicArray[i][j-1][0] == "wireHorizontal" or logicArray[i][j-1][0] ==
"Switch_OFF" or logicArray[i][j-1][0] == "Switch_ON") and (logicArray[i][j+1][0] ==
"wireHorizontal" or logicArray[i][j+1][0] == "wireTopRight"):
        logicArray[i][j][0] = "wireSplitterUp"
    if logicArray[i-1][j][0] == None:
        logicArray[i][j][0] = "wireHorizontal"
    elif logicArray[i][j+1][0] == None:
        logicArray[i][j][0] = "wireVertical"
    elif logicArray[i+1][j][0] == "wireHorizontal":
        logicArray[i][j][0] = "wireSplitterRight"
        logicArray[i+1][j][0] = "wireVertical"
#bottom right change
elif logicArray[i][j][0] == "wireBottomRight":
    if (logicArray[i-1][j][0] == "wireHorizontal" or logicArray[i-1][j][0] ==
"wireVertical") and (logicArray[i][j-1][0] == "wireHorizontal" or logicArray[i][j-1][0] ==
"Switch_OFF" or logicArray[i][j-1][0] == "Switch_ON") and logicArray[i][j+1][0] ==
"wireHorizontal":
        logicArray[i][j][0] = "wireSplitterUp"
    elif logicArray[i-1][j][0] == None:
        logicArray[i][j][0] = "wireHorizontal"
    elif logicArray[i][j-1][0] == None:
        logicArray[i][j][0] = "wireVertical"
    elif logicArray[i+1][j][0] == "wireHorizontal":
        logicArray[i][j][0] = "wireSplitterLeft"
        logicArray[i+1][j][0] = "wireVertical"
elif logicArray[i][j][0] == "wireIntersection":
    if logicArray[i][j+1][0] != "ANDGate" and logicArray[i][j+1][0] !=
"ORGate" and logicArray[i][j+1][0] != "NORGate" and logicArray[i][j+1][0] !=
"NANDGate" and logicArray[i][j+1][0] != "XNORGate" and logicArray[i][j+1][0] !=
"XORGate" and logicArray[i][j+1][0] != "DFlipFlop":
        logicArray[i][j][0] = [None]
#wire splitter down change
elif logicArray[i][j][0] == "wireSplitterDown":
```

```
    if (logicArray[i][j-1][0] == "wireHorizontal" and logicArray[i][j+1][0] ==
"wireHorizontal") and (logicArray[i+1][j][0] == "wireVertical" and logicArray[i-1][j][0] ==
"wireHorizontal"):
        logicArray[i][j] = ["wireCross",False,False]
        logicArray[i-1][j][0] = "wireVertical"
    elif logicArray[i+1][j][0] == None:
        logicArray[i][j][0] = "wireHorizontal"
    elif logicArray[i][j-1][0] == None:
        logicArray[i][j][0] = "wireTopLeft"
    elif logicArray[i][j+1][0] == None:
        logicArray[i][j][0] = "wireTopRight"
#wire splitter up change
elif logicArray[i][j][0] == "wireSplitterUp":
    if (logicArray[i][j-1][0] == "wireHorizontal" and logicArray[i][j+1][0] ==
"wireHorizontal") and (logicArray[i+1][j][0] == "wireHorizontal" and logicArray[i-1][j][0]
== "wireVertical"):
        logicArray[i][j] = ["wireCross",False,False]
        logicArray[i+1][j][0] = "wireVertical"
    elif logicArray[i-1][j][0] == None:
        logicArray[i][j][0] = "wireHorizontal"
    elif logicArray[i][j-1][0] == None:
        logicArray[i][j][0] = "wireBottomLeft"
    elif logicArray[i][j+1][0] == None:
        logicArray[i][j][0] = "wireBottomRight"
#wire splitter right change
elif logicArray[i][j][0] == "wireSplitterRight":
    if logicArray[i-1][j][0] == None:
        logicArray[i][j][0] = "wireTopLeft"
    elif logicArray[i+1][j][0] == None:
        logicArray[i][j][0] = "wireBottomLeft"
    elif logicArray[i][j+1][0] == None:
        logicArray[i][j][0] = "wireVertical"
    elif logicArray[i][j-1][0] == "wireHorizontal":
        logicArray[i][j] = ["wireCross",False,False]
#wire splitter left change
elif logicArray[i][j][0] == "wireSplitterLeft":
    if logicArray[i-1][j][0] == None:
        logicArray[i][j][0] = "wireTopRight"
    elif logicArray[i+1][j][0] == None:
        logicArray[i][j][0] = "wireBottomRight"
    elif logicArray[i][j-1][0] == None:
        logicArray[i][j][0] = "wireHorizontal"
    elif logicArray[i][j+1][0] == "wireHorizontal":
        logicArray[i][j] = ["wireCross",False,False]
```

```
#wire cross change
elif logicArray[i][j][0] == "wireCross":
    if logicArray[i-1][j][0] == None:
        logicArray[i][j] = ["wireSplitterDown", False, None]
    elif logicArray[i+1][j][0] == None:
        logicArray[i][j] = ["wireSplitterUp", False, None]
    elif logicArray[i][j+1][0] == None:
        logicArray[i][j] = ["wireSplitterLeft", False, None]
    elif logicArray[i][j-1][0] == None:
        logicArray[i][j] = ["wireSplitterRight", False, None]

mainLoop()

#reArrayLast consists of: ["add/remove", "logicElement", Yposition, Xposition]
def reInsert(reArrayLast, state):
    #logic element removal
    if reArrayLast[0] == "remove":
        #if logic element is a wire or NOT gate
        if reArrayLast[1] == "NOTGate" or reArrayLast[1] == "Switch_OFF" or
reArrayLast[1] == "Light_OFF" or reArrayLast[1] == "wireHorizontal" or
reArrayLast[1] == "wireVertical" or reArrayLast[1] == "wireTopLeft" or reArrayLast[1]
== "wireTopRight" or reArrayLast[1] == "wireBottomRight" or reArrayLast[1] ==
"wireBottomLeft":
            #turning off connected wires
            if len(logicArray[reArrayLast[2]][reArrayLast[3]]) == 3:
                print(logicArray[reArrayLast[2]][reArrayLast[3]][2])
                if logicArray[reArrayLast[2]][reArrayLast[3]][2] == "Up":
                    turnOffVerticalUp(reArrayLast[2],reArrayLast[3])
                elif logicArray[reArrayLast[2]][reArrayLast[3]][2] == "Down":
                    turnOffVerticalDown(reArrayLast[2],reArrayLast[3])
                elif logicArray[reArrayLast[2]][reArrayLast[3]][2] == "Right":
                    turnOffHorizontalRight(reArrayLast[2],reArrayLast[3])
                elif logicArray[reArrayLast[2]][reArrayLast[3]][2] == "Left":
                    turnOffHorizontalLeft(reArrayLast[2],reArrayLast[3])
            elif reArrayLast[1] == "Clock":
                pass
            else:
                turnOffHorizontalRight(reArrayLast[2],reArrayLast[3])
                logicArray[reArrayLast[2]][reArrayLast[3]] = [None]
        #if logic element is a logic gate
    else:
        turnOffHorizontalRight(reArrayLast[2],reArrayLast[3])
        logicArray[reArrayLast[2]][reArrayLast[3]] = [None]
```

```
    logicArray[reArrayLast[2]][reArrayLast[3]-1] = [None]
    if state == "undo":
        #reArrayLast is popped from undo stack and pushed to redo stack with
function reversed from add to remove
        reArrayLast[0] = "add"
        redoStack.append(reArrayLast)
        del undoStack[len(undoStack)-1]
    elif state == "redo":
        #reArrayLast is popped from redo stack and pushed to undo stack with
function reversed from add to remove
        reArrayLast[0] = "add"
        undoStack.append(reArrayLast)
        del redoStack[len(redoStack)-1]
#logic element insertion
elif reArrayLast[0] == "add":
    #NOT gate is reinserted
    if reArrayLast[1] == "NOTGate":
        logicArray[reArrayLast[2]][reArrayLast[3]] = ["NOTGate", False, False]
    #switch or light is reinserted
    elif reArrayLast[1] == "Switch_OFF" or reArrayLast[1] == "Light_OFF":
        logicArray[reArrayLast[2]][reArrayLast[3]] = [reArrayLast[1], False]
    #wire is reinserted
    elif reArrayLast[1] == "wireHorizontal" or reArrayLast[1] == "wireVertical" or
reArrayLast[1] == "wireTopLeft" or reArrayLast[1] == "wireTopRight" or reArrayLast[1]
== "wireBottomRight" or reArrayLast[1] == "wireBottomLeft":
        logicArray[reArrayLast[2]][reArrayLast[3]] = ["wireHorizontal", False,
None]
    #logic gate is reinserted
    else:
        logicArray[reArrayLast[2]][reArrayLast[3]] = [reArrayLast[1], False]
        if reArrayLast[1] == "Clock":
            pass
        else:
            logicArray[reArrayLast[2]][reArrayLast[3]-1] = ["wireIntersection",
False, False]
    if state == "redo":
        #reArrayLast is popped from redo stack and pushed to undo stack with
function reversed from add to remove
        reArrayLast[0] = "remove"
        undoStack.append(reArrayLast)
        del redoStack[len(redoStack)-1]
    elif state == "undo":
        #reArrayLast is popped from undo stack and pushed to redo stack with
function reversed from add to remove
```

```
reArrayLast[0] = "remove"
redoStack.append(reArrayLast)
del undoStack[len(undoStack)-1]

#tutorial text
if tutorialPhase == 1:
    #text and constants
    paragraphText = [
        "Welcome to the Tutorial",
        "First we are going to build a NOT Gate circuit",
        "Select the switch and place it in the glowing square"]
    yPos = 280
    lineSpacing = 20

    #width and height definitions
    textWidth = 400
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20

    #drawing background rectangle
    rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
    rectangleSurface.fill(BLUE)
    rectangleRect = rectangleSurface.get_rect()
    rectangleRect.topleft = (830, 280)
    SCREEN.blit(rectangleSurface, rectangleRect)
    #drawing lines of text
    for line in paragraphText:
        textSurface = get_font3(15).render(line, True, BLACK)
        SCREEN.blit(textSurface, (835, yPos))
        yPos += lineSpacing
elif tutorialPhase == 2:
    paragraphText = [
        "Well done!",
        "Now to place the wires that will be used to connect the circuit",
        "Select the wire and place it in the glowing squares"]
    yPos = 280
    lineSpacing = 20

    textWidth = 480
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20
```

```
rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
rectangleSurface.fill(BLUE)
rectangleRect = rectangleSurface.get_rect()
rectangleRect.topleft = (760, 280)
SCREEN.blit(rectangleSurface, rectangleRect)
for line in paragraphText:
    textSurface = get_font3(15).render(line, True, BLACK)
    SCREEN.blit(textSurface, (765, yPos))
    yPos += lineSpacing
elif tutorialPhase == 3:
    paragraphText = [
        "Now we need to add the logic gate to the circuit",
        "The gate we will be using is a NOT gate, which inverts the input",
        "Select the NOT gate and place it in the glowing square"]
    yPos = 280
    lineSpacing = 20

    textWidth = 490
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20

    rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
    rectangleSurface.fill(BLUE)
    rectangleRect = rectangleSurface.get_rect()
    rectangleRect.topleft = (755, 280)
    SCREEN.blit(rectangleSurface, rectangleRect)
    for line in paragraphText:
        textSurface = get_font3(15).render(line, True, BLACK)
        SCREEN.blit(textSurface, (760, yPos))
        yPos += lineSpacing
elif tutorialPhase == 4:
    paragraphText = [
        "We now need to put in the light bulb, completing the circuit",
        "Select the light bulb and place it in the glowing square"]
    yPos = 280
    lineSpacing = 20

    textWidth = 465
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20

    rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
```

```
rectangleSurface.fill(BLUE)
rectangleRect = rectangleSurface.get_rect()
rectangleRect.topleft = (775, 280)
SCREEN.blit(rectangleSurface, rectangleRect)
for line in paragraphText:
    textSurface = get_font3(15).render(line, True, BLACK)
    SCREEN.blit(textSurface, (780, yPos))
    yPos += lineSpacing
elif tutorialPhase == 5:
    paragraphText = [
        "Now the circuit has been completed",
        "Feel free to test the circuit by turning it on and off",
        "Press the next button to continue"]
    yPos = 280
    lineSpacing = 20

    textWidth = 410
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20

    rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
    rectangleSurface.fill(BLUE)
    rectangleRect = rectangleSurface.get_rect()
    rectangleRect.topleft = (800, 280)
    SCREEN.blit(rectangleSurface, rectangleRect)
    for line in paragraphText:
        textSurface = get_font3(15).render(line, True, BLACK)
        SCREEN.blit(textSurface, (805, yPos))
        yPos += lineSpacing
elif tutorialPhase == 6:
    paragraphText = [
        "Now we are going to make an AND circuit",
        "We are going to need two switches for this circuit",
        "Select the switch and place it in the glowing squares"]
    yPos = 280
    lineSpacing = 20

    textWidth = 410
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20

    rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
```

```
rectangleSurface.fill(BLUE)
rectangleRect = rectangleSurface.get_rect()
rectangleRect.topleft = (830, 280)
SCREEN.blit(rectangleSurface, rectangleRect)
for line in paragraphText:
    textSurface = get_font3(15).render(line, True, BLACK)
    SCREEN.blit(textSurface, (835, yPos))
    yPos += lineSpacing
elif tutorialPhase == 7:
    paragraphText = [
        "Now we need to place the wires needed for the circuit",
        "This will allow both of the switches to connect to the AND gate",
        "Select the wire and place it in the glowing squares"]
    yPos = 280
    lineSpacing = 20

    textWidth = 480
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20

    rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
    rectangleSurface.fill(BLUE)
    rectangleRect = rectangleSurface.get_rect()
    rectangleRect.topleft = (750, 280)
    SCREEN.blit(rectangleSurface, rectangleRect)
    for line in paragraphText:
        textSurface = get_font3(15).render(line, True, BLACK)
        SCREEN.blit(textSurface, (755, yPos))
        yPos += lineSpacing
elif tutorialPhase == 8:
    paragraphText = [
        "Now we need to place the logic gate needed for the circuit",
        "The gate we will be using an AND gate, which turns on when both inputs
are on",
        "Select the AND gate and place it in the glowing square"]
    yPos = 280
    lineSpacing = 20

    textWidth = 595
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20
```



```
rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
rectangleSurface.fill(BLUE)
rectangleRect = rectangleSurface.get_rect()
rectangleRect.topleft = (650, 280)
SCREEN.blit(rectangleSurface, rectangleRect)
for line in paragraphText:
    textSurface = get_font3(15).render(line, True, BLACK)
    SCREEN.blit(textSurface, (655, yPos))
    yPos += lineSpacing
elif tutorialPhase == 9:
    paragraphText = [
        "We now need to put in the light bulb, completing the circuit",
        "Select the light bulb and place it in the glowing square"]
    yPos = 280
    lineSpacing = 20

    textWidth = 465
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
    rectangleHeight = textHeight + 20

    rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
    rectangleSurface.fill(BLUE)
    rectangleRect = rectangleSurface.get_rect()
    rectangleRect.topleft = (775, 280)
    SCREEN.blit(rectangleSurface, rectangleRect)
    for line in paragraphText:
        textSurface = get_font3(15).render(line, True, BLACK)
        SCREEN.blit(textSurface, (780, yPos))
        yPos += lineSpacing
elif tutorialPhase == 10:
    paragraphText = [
        "The AND gate circuit is now complete",
        "Feel free to test it again, you can also change the gate to one of your",
        "choosing",
        "Select the gate you want and place it in the same place as the AND",
        "Gate",
        "Press next to finish the tutorial"]
    yPos = 280
    lineSpacing = 20

    textWidth = 620
    textHeight = len(paragraphText) * lineSpacing
    rectangleWidth = textWidth + 30
```

```
rectangleHeight = textHeight + 20
```

```
rectangleSurface = pygame.Surface((rectangleWidth, rectangleHeight))
rectangleSurface.fill(BLUE)
rectangleRect = rectangleSurface.get_rect()
rectangleRect.topleft = (620, 280)
SCREEN.blit(rectangleSurface, rectangleRect)
for line in paragraphText:
    textSurface = get_font3(15).render(line, True, BLACK)
    SCREEN.blit(textSurface, (625, yPos))
    yPos += lineSpacing
```

```
#drawing buttons
for button in [logicInterfaceSave, logicInterfaceSaveAs, logicInterfaceLoad,
logicInterfaceUndo, logicInterfaceRedo, logicInterfaceTruthTable,
logicInterfaceDelete, logicInterfaceClear, logicInterfaceNew,
logicInterfaceMainMenu]:
    button.changeColor(logicInterface_MOUSE_POS)
    button.update(SCREEN)
if tutorialPhase != None and tutorialPhase != 10:
    logicInterfaceTutorialBack.changeColor(logicInterface_MOUSE_POS)
    logicInterfaceTutorialBack.update(SCREEN)
#input checker
if tutorialPhase == None:
    #keyboard shortcuts
    keys = pygame.key.get_pressed()
    #undo
    if keys[pygame.K_LCTRL] and keys[pygame.K_z]:
        if undoStack == []:
            pass
        else:
            print(undoStack)
            reInsert(undoStack[len(undoStack)-1], "undo")
            print(undoStack)
            print(redoStack)
    #redo
    elif keys[pygame.K_LCTRL] and keys[pygame.K_y]:
        if redoStack == []:
            pass
        else:
            print(redoStack)
            reInsert(redoStack[len(redoStack)-1], "redo")
            print(redoStack)
```

```
        print(undoStack)
    #save
    elif keys[pygame.K_LCTRL] and keys[pygame.K_s]:
        if fileName == None:
            saveAsInterface(logicArray, fileName)
        else:
            logicFile = open(fileName, "wb")
            pickle.dump(logicArray, logicFile)
            logicFile.close()
            logicSaveText = get_font2(25).render("Save Successful", True, GREEN)
            logicSaveTextRect = logicSaveText.get_rect()
            logicSaveTextRect.center = (1100,30)
            SCREEN.blit(logicSaveText, logicSaveTextRect)
            pygame.display.update()
            time.sleep(1)
            print("Save Successful")
    #load
    elif keys[pygame.K_LCTRL] and keys[pygame.K_l]:
        loadInterface(logicArray, fileName,False)
        print(logicArray)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            #button checker
            #save as pressed
            if logicInterfaceSaveAs.checkForInput(logicInterface_MOUSE_POS):
                saveAsInterface(logicArray, fileName)
            #delete pressed
            if logicInterfaceDelete.checkForInput(logicInterface_MOUSE_POS):
                #delete functionality acts as an active gate
                if activeGate != "Delete":
                    activeGate = "Delete"
                    #button changes colour if pressed
                    logicInterfaceDelete = Button(pos=(625,30), text_input="Delete",
font=get_font2(25), base_color="Yellow", hovering_color=GREY)
                    logicInterfaceDelete.changeColor(logicInterface_MOUSE_POS)
                    logicInterfaceDelete.update(SCREEN)
                    pygame.display.update()
```

```
        print(activeGate)
    else:
        logicInterfaceDelete = Button(pos=(525,30),
            text_input="Delete", font=get_font2(25), base_color="Black",
            hovering_color=GREY)
        activeGate = None
    #undo button pressed
    if logicInterfaceUndo.checkForInput(logicInterface_MOUSE_POS):
        if undoStack == []:
            pass
        else:
            print(undoStack)
            reInsert(undoStack[len(undoStack)-1], "undo")
            print(undoStack)
            print(redoStack)
    #redo button pressed
    if logicInterfaceRedo.checkForInput(logicInterface_MOUSE_POS):
        if redoStack == []:
            pass
        else:
            print(redoStack)
            reInsert(redoStack[len(redoStack)-1], "redo")
            print(redoStack)
            print(undoStack)
    #truth table button pressed
    if logicInterfaceTruthTable.checkForInput(logicInterface_MOUSE_POS):
        activeGate = None
        #finds the outputted information from a combination of inputs in the
truth table
        def checkOutput(combinationList,switchPosArray,lightPosArray):
            output = []
            print("combination list: " + str(combinationList))
            #iterates through the combination of inputs
            for i in range(0,len(combinationList)):
                #switch is turned on, if combination specifies
                if combinationList[i] == 1:
                    turnOnHorizontalRight(switchPosArray[i][0],switchPosArray[i][1])
                    logicArray[switchPosArray[i][0]][switchPosArray[i][1]] =
["Switch_ON",True]
                    #main loop run to produce changes in logic array
                    mainLoop()
                #switch is turned off, if combination specifies
                elif combinationList[i] == 0:
```

```
turnOffHorizontalRight(switchPosArray[i][0],switchPosArray[i][1])
    logicArray[switchPosArray[i][0]][switchPosArray[i][1]] =
["Switch_OFF",False]
    #main loop run to produce changes in logic array
    mainLoop()
    print("the length: " + str(len(lightPosArray)))
    #checking if light bulbs turn on in correspondence, iterates through
list of light positions
    outputNumber = 0
    while outputNumber < len(lightPosArray):
        if
logicArray[lightPosArray[outputNumber][0]][lightPosArray[outputNumber][1]][0] ==
"Light_ON":
            #output of light is appended to overall output
            output.append(1)
        elif
logicArray[lightPosArray[outputNumber][0]][lightPosArray[outputNumber][1]][0] ==
"Light_OFF":
            #output of light is appended to overall output
            output.append(0)
            outputNumber += 1
            print(output)
            #overall output appended to truth table row
            truthTable[1].append(output)

#initial values and data structures set
switchCount = 0
lightCount = 0
switchPosArray = []
lightPosArray = []
truthTable = [[],[]]
print(switchCount)
print(lightCount)
#logic array is searched for switches and lights
for i in range(0,len(logicArray)):
    for j in range(0,len(logicArray[i])):
        #switch found
        if logicArray[i][j][0] == "Switch_OFF" or logicArray[i][j][0] ==
"Switch_ON":
            switchCount += 1
            switchPosArray.append([i,j])
        #light found
```

```
elif logicArray[i][j][0] == "Light_OFF" or logicArray[i][j][0] ==
"Light_ON":
    lightCount += 1
    lightPosArray.append([i,j])

#amount of possible switch combinations is calculated as 2^n
switchCombinations = 2**switchCount
#print values in shell
print("switchCount: " + str(switchCount))
print("switchPosArray: " + str(switchPosArray))
print("lightCount: " + str(lightCount))
print("lightPosArray: " + str(lightPosArray))
for i in range(0,switchCombinations):
    rowList = []
    #each switch combination is converted to binary
    convertedNumber = binaryConvert(i)
    if len(str(convertedNumber)) < switchCount:
        difference = switchCount - len(str(convertedNumber))
        #if binary number is less than the number of bits, zeros are
        appended to the combination
        for i in range(0,difference):
            rowList.append(0)
    #binary number is converted to list format
    convertedNumberList = list(str(convertedNumber))
    #items in list are converted to integers
    for j in range(0, len(convertedNumberList)):
        convertedNumberList[j] = int(convertedNumberList[j])
    #binary number is added to contents of rowList
    rowList = rowList + convertedNumberList
    #binary combination is appended to truth table
    truthTable[0].append(rowList)

#translation only runs if switches and lights are detected
if switchCount > 0 and switchCount < 5 and lightCount > 0:
    for i in range(0,len(truthTable[0])):
        checkOutput(truthTable[0][i],switchPosArray,lightPosArray)

    print(truthTable)

    translate(truthTable,switchCount,lightCount, switchCombinations,
logicArray, fileName)

#save button pressed
if logicInterfaceSave.checkForInput(logicInterface_MOUSE_POS):
```

```
menu    #file is a new document, pressing save directs the user to the save as
        if fileName == None:
            activeGate = None
            saveAsInterface(logicArray, fileName)
        #file is a previously saved document, pressing save overwrites file
contents
        else:
            logicFile = open(fileName, "wb")
            pickle.dump(logicArray, logicFile)
            logicFile.close()
            logicSaveText = get_font2(25).render("Save Successful", True,
GREEN)
            logicSaveTextRect = logicSaveText.get_rect()
            logicSaveTextRect.center = (1100,30)
            SCREEN.blit(logicSaveText, logicSaveTextRect)
            pygame.display.update()
            time.sleep(1)
            print("Save Successful")
        #load button pressed
        if logicInterfaceLoad.checkForInput(logicInterface_MOUSE_POS):
            activeGate = None
            loadInterface(logicArray, fileName,False)
            print(logicArray)
        #clear button pressed
        if logicInterfaceClear.checkForInput(logicInterface_MOUSE_POS):
            #stacks reset
            undoStack = []
            redoStack = []
            #logic array reset
            logicArray = []
            for i in range(0,14):
                logicArray.append([None], [None], [None],[None], [None],
[None],[None], [None], [None],[None], [None], [None],[None], [None],
[None], [None],[None], [None], [None]])
            #new button pressed
            if logicInterfaceNew.checkForInput(logicInterface_MOUSE_POS):
                #stacks reset
                undoStack = []
                redoStack = []
                #array reset
                logicArray = []
                #fileName reset
                fileName = None
```

```
for i in range(0,14):
    logicArray.append([None], [None], [None],[None], [None],
[None],[None], [None], [None],[None], [None], [None],[None], [None], [None],[None],
[None], [None],[None], [None], [None]])
    #main menu button pressed
    if logicInterfaceMainMenu.checkForInput(logicInterface_MOUSE_POS):
        mainMenu()
    if event.button == 1:
        #logic menu input checker
        if mouseX >= 25 and mouseX <= 75 and mouseY >= 100 and
mouseY <= 140:
            if activeGate == "ORGate":
                #if the gate is already active, active gate is reset
                activeGate = None
            else:
                #clicking the gate causes it to become active
                activeGate = "ORGate"
        elif mouseX >= 99 and mouseX <= 125 and mouseY >= 100 and
mouseY <= 140:
            if activeGate == "NOTGate":
                activeGate = None
            else:
                activeGate = "NOTGate"
        elif mouseX >= 175 and mouseX <= 225 and mouseY >= 100 and
mouseY <= 140:
            if activeGate == "ANDGate":
                activeGate = None
            else:
                activeGate = "ANDGate"
        elif mouseX >= 25 and mouseX <= 75 and mouseY >= 175 and
mouseY <= 215:
            if activeGate == "NORGate":
                activeGate = None
            else:
                activeGate = "NORGate"
        elif mouseX >= 99 and mouseX <= 125 and mouseY >= 175 and
mouseY <= 215:
            if activeGate == "NANDGate":
                activeGate = None
            else:
                activeGate = "NANDGate"
        elif mouseX >= 175 and mouseX <= 225 and mouseY >= 175 and
mouseY <= 215:
            if activeGate == "XNORGate":
```



```
        activeGate = None
    else:
        activeGate = "XNORGate"
    elif mouseX >= 25 and mouseX <= 75 and mouseY >= 250 and
mouseY <= 290:
        if activeGate == "XORGate":
            activeGate = None
        else:
            activeGate = "XORGate"
    elif mouseX >= 20 and mouseX <= 69 and mouseY >= 355 and
mouseY <= 375:
        if activeGate == "Switch_OFF":
            activeGate = None
        else:
            activeGate = "Switch_OFF"
    elif mouseX >= 27 and mouseX <= 67 and mouseY >= 440 and
mouseY <= 491:
        if activeGate == "Light_OFF":
            activeGate = None
        else:
            activeGate = "Light_OFF"
    elif mouseX >= 90 and mouseX <= 140 and mouseY >= 345 and
mouseY <= 395:
        if activeGate == "wireHorizontal":
            activeGate = None
        else:
            activeGate = "wireHorizontal"
    elif mouseX >= 165 and mouseX <= 215 and mouseY >= 348 and
mouseY <= 398:
        if activeGate == "Clock":
            activeGate = None
        else:
            activeGate = "Clock"
    elif mouseX >= 25 and mouseX <= 75 and mouseY >= 550 and
mouseY <= 605:
        if activeGate == "DFlipFlop":
            activeGate = None
        else:
            activeGate = "DFlipFlop"
    elif mouseX >= 1050 and mouseX <= 1100 and mouseY >= 30 and
mouseY <= 50:
        #toggle on/off for auto save
        if autoSave == False and fileName != None:
            autoSave = True
```

```
else:  
    autoSave = False
```

```
#placement checker  
elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and  
mouseY <= 697:  
    #grid position is calculated  
    Xsquare = (mouseX-251)//49  
    Ysquare = (mouseY-60)//49  
    print(Xsquare)  
    print(Ysquare)  
    if activeGate != None:  
        redoStack = []  
    if activeGate == "ORGate":  
        #OR gate is added to grid  
        logicArray[Ysquare][Xsquare] = ["ORGate", False]  
        #wire intersection is added to grid  
        logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,  
False]  
  
        #active gate is reset  
        activeGate = None  
        #action is added to undo stack  
        undoStack.append(["remove", "ORGate", Ysquare, Xsquare])  
        print(activeGate)  
    elif activeGate == "NOTGate":  
        #NOT gate is added to grid  
        logicArray[Ysquare][Xsquare] = ["NOTGate", False]  
        activeGate = None  
        undoStack.append(["remove", "NOTGate", Ysquare, Xsquare])  
        print(activeGate)  
    elif activeGate == "ANDGate":  
        #AND gate is added to grid  
        logicArray[Ysquare][Xsquare] = ["ANDGate", False]  
        logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,  
False]  
  
        activeGate = None  
        undoStack.append(["remove", "ANDGate", Ysquare, Xsquare])  
        print(activeGate)  
    elif activeGate == "NORGate":  
        #NOR gate is added to grid  
        logicArray[Ysquare][Xsquare] = ["NORGate", False]  
        logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,  
False]  
  
        activeGate = None
```

```
undoStack.append(["remove", "NORGate", Ysquare, Xsquare])
print(activeGate)
elif activeGate == "NANDGate":
    #NAND gate is added to grid
    logicArray[Ysquare][Xsquare] = ["NANDGate", False]
    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    undoStack.append(["remove", "NANDGate", Ysquare, Xsquare])
    print(activeGate)
elif activeGate == "XNORGate":
    #XNOR gate is added to grid
    logicArray[Ysquare][Xsquare] = ["XNORGate", False]
    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    undoStack.append(["remove", "XNORGate", Ysquare, Xsquare])
    print(activeGate)
elif activeGate == "XORGate":
    #XOR gate is added to grid
    logicArray[Ysquare][Xsquare] = ["XORGate", False]
    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    undoStack.append(["remove", "XORGate", Ysquare, Xsquare])
    print(activeGate)
elif activeGate == "Light_OFF":
    #light is added to grid
    logicArray[Ysquare][Xsquare] = ["Light_OFF", False]
    activeGate = None
    undoStack.append(["remove", "Light_OFF", Ysquare, Xsquare])
    print(activeGate)
elif activeGate == "Switch_OFF":
    #switch is added to grid
    logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
    activeGate = None
    undoStack.append(["remove", "Switch_OFF", Ysquare,
Xsquare])

    print(activeGate)
elif activeGate == "wireHorizontal":
    #horizontal wire is added to grid
    logicArray[Ysquare][Xsquare] = ["wireHorizontal", False, None]
    undoStack.append(["remove", "wireHorizontal", Ysquare,
Xsquare])
```

```
elif activeGate == "Clock":
    #clock is added to grid
    logicArray[Ysquare][Xsquare] = ["Clock", False]
    activeGate = None
    undoStack.append(["remove", "Clock", Ysquare, Xsquare])
elif activeGate == "DFlipFlop":
    #D type flip flop is added to grid
    logicArray[Ysquare][Xsquare] = ["DFlipFlop", False]
    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    undoStack.append(["remove", "DFlipFlop", Ysquare, Xsquare])
elif activeGate == "Delete":
    undoStack.append(["add", logicArray[Ysquare][Xsquare][0],
Ysquare, Xsquare])
    print(undoStack)
    if len(logicArray[Ysquare][Xsquare]) == 3:
        if logicArray[Ysquare][Xsquare][2] == "Up":
            turnOffVerticalUp(Ysquare, Xsquare)
        elif logicArray[Ysquare][Xsquare][2] == "Down":
            turnOffVerticalDown(Ysquare, Xsquare)
        elif logicArray[Ysquare][Xsquare][2] == "Right":
            if logicArray[Ysquare][Xsquare][0] == "wireSplitterDown":
                turnOffVerticalDown(Ysquare, Xsquare)
                turnOffHorizontalRight(Ysquare, Xsquare)
            elif logicArray[Ysquare][Xsquare][2] == "Left":
                if logicArray[Ysquare][Xsquare][0] == "wireSplitterDown":
                    turnOffVerticalDown(Ysquare, Xsquare)
                    turnOffHorizontalLeft(Ysquare, Xsquare)
            if logicArray[Ysquare][Xsquare][0] == "wireIntersection":
                logicArray[Ysquare][Xsquare+1] = [None]
            elif logicArray[Ysquare][Xsquare-1][0] == "wireIntersection":
                logicArray[Ysquare][Xsquare-1] = [None]
            logicArray[Ysquare][Xsquare] = [None]
        elif logicArray[Ysquare][Xsquare] == None:
            break
    elif logicArray[Ysquare][Xsquare][0] == "Switch_OFF":
        logicArray[Ysquare][Xsquare] = ["Switch_ON", True]
        lastCountChange = count
    elif logicArray[Ysquare][Xsquare][0] == "Switch_ON":
        logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
        lastCountChange = count
    print(logicArray)
    print(activeGate)
```

```
if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_UP:
        print(logicArray)
    if event.key == pygame.K_DOWN:
        print(redoArray)
        print(undoArray)
elif tutorialPhase != None:
    #functionality of stage 1 of the tutorial
    if tutorialPhase == 1:
        #location for the user to place a switch is set
        logicArray[6][1] = ["switchToBe"]
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.MOUSEBUTTONDOWN:
                #pixel position of click determined
                mouseX = event.pos[0]
                mouseY = event.pos[1]
                print("X pos " + str(mouseX))
                print("Y pos " + str(mouseY))
                if event.button == 1:
                    #selection only allows for a switch to be selected
                    if mouseX >= 20 and mouseX <= 69 and mouseY >= 355 and
mouseY <= 375:
                        if activeGate == "Switch_OFF":
                            activeGate = None
                        else:
                            activeGate = "Switch_OFF"
                            print(activeGate)
                    elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                        #x and y squares in grid are determined
                        Xsquare = (mouseX-251)//49
                        Ysquare = (mouseY-60)//49
                        print(Xsquare)
                        print(Ysquare)
                        if activeGate == "Switch_OFF":
                            #user can only place in a specific square
                            if logicArray[Ysquare][Xsquare][0] == "switchToBe":
                                logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
                                activeGate = None
                            #proceed to next phase
                            tutorialPhase += 1
```

```
        if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    #user returns to main menu if back button is pressed
    mainMenu()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_UP:
            print(logicArray)

elif tutorialPhase == 2:
    for i in range(2,7):
        if i != 4:
            #location for each wire piece is set
            if logicArray[6][i][0] == None:
                logicArray[6][i] = ["wireToBe"]
            #variable checking if all wires have been placed
            complete = True
            for i in range(2,7):
                if i != 4:
                    if logicArray[6][i][0] == "wireToBe":
                        complete = False
            if complete == True:
                tutorialPhase += 1
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    sys.exit()
                if event.type == pygame.MOUSEBUTTONDOWN:
                    mouseX = event.pos[0]
                    mouseY = event.pos[1]
                    print("X pos " + str(mouseX))
                    print("Y pos " + str(mouseY))
                    if event.button == 1:
                        if mouseX >= 90 and mouseX <= 140 and mouseY >= 345 and
mouseY <= 395:
                            if activeGate == "wireHorizontal":
                                activeGate = None
                            else:
                                activeGate = "wireHorizontal"
                        elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                            Xsquare = (mouseX-251)//49
                            Ysquare = (mouseY-60)//49
                            print(Xsquare)
                            print(Ysquare)
```

```
        if activeGate == "wireHorizontal":
            if logicArray[Ysquare][Xsquare][0] == "wireToBe":
                logicArray[Ysquare][Xsquare] = ["wireHorizontal", False,
None]
            if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
                mainMenu()

elif tutorialPhase == 3:
    #activeGate is reset
    if activeGate == "wireHorizontal":
        activeGate = None
    #location for NOT gate is set
    if logicArray[6][4][0] == None:
        logicArray[6][4][0] = "NOTGateToBe"
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            if event.button == 1:
                #selection only allows for NOT gates to selected
                if mouseX >= 99 and mouseX <= 125 and mouseY >= 100 and
mouseY <= 140:
                    if activeGate == "NOTGate":
                        activeGate = None
                    else:
                        activeGate = "NOTGate"
                elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                    Xsquare = (mouseX-251)//49
                    Ysquare = (mouseY-60)//49
                    print(Xsquare)
                    print(Ysquare)
                    if activeGate == "NOTGate":
                        if logicArray[Ysquare][Xsquare][0] == "NOTGateToBe":
                            logicArray[Ysquare][Xsquare] = ["NOTGate", False]
                            activeGate = None
                            tutorialPhase += 1
```

```
        if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
        mainMenu()

elif tutorialPhase == 4:
    #location for user to place a light is set
    if logicArray[6][7][0] == None:
        logicArray[6][7][0] = "lightToBe"
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            if event.button == 1:
                #selection only allows lights to be selected
                if mouseX >= 27 and mouseX <= 67 and mouseY >= 440 and
mouseY <= 491:
                    if activeGate == "Light_OFF":
                        activeGate = None
                    else:
                        activeGate = "Light_OFF"
                elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                    Xsquare = (mouseX-251)//49
                    Ysquare = (mouseY-60)//49
                    print(Xsquare)
                    print(Ysquare)
                    if activeGate == "Light_OFF":
                        if logicArray[Ysquare][Xsquare][0] == "lightToBe":
                            logicArray[Ysquare][Xsquare] = ["Light_OFF", False]
                            activeGate = None
                            tutorialPhase += 1
            if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
        mainMenu()

elif tutorialPhase == 5:
    #next button becomes visible to user
    logicInterfaceTutorialNext.changeColor(logicInterface_MOUSE_POS)
```



```
logicInterfaceTutorialNext.update(SCREEN)
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.MOUSEBUTTONDOWN:
        mouseX = event.pos[0]
        mouseY = event.pos[1]
        print("X pos " + str(mouseX))
        print("Y pos " + str(mouseY))
        if event.button == 1:
            if mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                Xsquare = (mouseX-251)//49
                Ysquare = (mouseY-60)//49
                print(Xsquare)
                print(Ysquare)
                #user can turn the switch on and off
                if logicArray[Ysquare][Xsquare][0] == "Switch_OFF":
                    logicArray[Ysquare][Xsquare] = ["Switch_ON", True]
                elif logicArray[Ysquare][Xsquare][0] == "Switch_ON":
                    logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
            if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    mainMenu()
    if
logicInterfaceTutorialNext.checkForInput(logicInterface_MOUSE_POS):
    #pressing next procedes to AND circuit building
    tutorialPhase += 1

elif tutorialPhase == 6:
    #logic array reset
    if logicArray[6][1][0] == "Switch_OFF" or logicArray[6][1][0] ==
"Switch_ON":
        logicArray = []
        for i in range(0,14):
            logicArray.append([[None], [None], [None],[None], [None],
[None],[None], [None], [None],[None], [None], [None],[None],[None],
[None], [None],[None], [None], [None]])
            #locations for switches to be placed are set
            logicArray[5][1][0] = "switchToBe"
            logicArray[7][1][0] = "switchToBe"
            if logicArray[5][1][0] == "Switch_OFF" and logicArray[7][1][0] ==
"Switch_OFF":
```

```
tutorialPhase += 1
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.MOUSEBUTTONDOWN:
        mouseX = event.pos[0]
        mouseY = event.pos[1]
        print("X pos " + str(mouseX))
        print("Y pos " + str(mouseY))
        if event.button == 1:
            #oselection only allows for a switch to be placed
            if mouseX >= 20 and mouseX <= 69 and mouseY >= 355 and
mouseY <= 375:
                if activeGate == "Switch_OFF":
                    activeGate = None
                else:
                    activeGate = "Switch_OFF"
                    print(activeGate)
            elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                Xsquare = (mouseX-251)//49
                Ysquare = (mouseY-60)//49
                print(Xsquare)
                print(Ysquare)
                if activeGate == "Switch_OFF":
                    if logicArray[Ysquare][Xsquare][0] == "switchToBe":
                        logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
                        activeGate = None
                if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    mainMenu()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_UP:
            print(logicArray)

elif tutorialPhase == 7:
    if logicArray[5][2][0] == None:
        #locations for wires to be placed are set
        for i in range(2,4):
            logicArray[5][i][0] = "wireToBe"
            logicArray[7][i][0] = "wireToBe"
```

```
    if logicArray[5][2][0] == "wireHorizontal" and logicArray[7][2][0] ==
"wireHorizontal" and logicArray[5][3][0] == "wireHorizontal" and logicArray[7][3][0] ==
"wireHorizontal":
        activeGate = None
        tutorialPhase += 1
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            if event.button == 1:
                #selection only allows for a wire to be placed
                if mouseX >= 90 and mouseX <= 140 and mouseY >= 345 and
mouseY <= 395:
                    if activeGate == "wireHorizontal":
                        activeGate = None
                    else:
                        activeGate = "wireHorizontal"
                elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
                    Xsquare = (mouseX-251)//49
                    Ysquare = (mouseY-60)//49
                    print(Xsquare)
                    print(Ysquare)
                    if activeGate == "wireHorizontal":
                        if logicArray[Ysquare][Xsquare][0] == "wireToBe":
                            logicArray[Ysquare][Xsquare] = ["wireHorizontal", False,
None]
                        if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
                            mainMenu()
                    if event.type == pygame.KEYDOWN:
                        if event.key == pygame.K_UP:
                            print(logicArray)
                elif tutorialPhase == 8:
                    #location for AND gate is set
                    if logicArray[6][4][0] == None:
                        logicArray[6][4][0] = "gateToBe"
                    for event in pygame.event.get():
                        if event.type == pygame.QUIT:
```

```
pygame.quit()
sys.exit()
if event.type == pygame.MOUSEBUTTONDOWN:
    mouseX = event.pos[0]
    mouseY = event.pos[1]
    print("X pos " + str(mouseX))
    print("Y pos " + str(mouseY))
    if event.button == 1:
        if mouseX >= 175 and mouseX <= 225 and mouseY >= 100 and
mouseY <= 140:
            if activeGate == "ANDGate":
                activeGate = None
            else:
                activeGate = "ANDGate"
        elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
            Xsquare = (mouseX-251)//49
            Ysquare = (mouseY-60)//49
            print(Xsquare)
            print(Ysquare)
            if activeGate == "ANDGate":
                if logicArray[Ysquare][Xsquare][0] == "gateToBe":
                    logicArray[Ysquare][Xsquare] = ["ANDGate", False]
                    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

                    activeGate = None
                    tutorialPhase += 1
                    print(activeGate)
            if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    mainMenu()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_UP:
            print(logicArray)
elif tutorialPhase == 9:
    #location for light is set
    if logicArray[6][5][0] == None:
        logicArray[6][5][0] = "lightToBe"
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
```

```
mouseY = event.pos[1]
print("X pos " + str(mouseX))
print("Y pos " + str(mouseY))
if event.button == 1:
    if mouseX >= 27 and mouseX <= 67 and mouseY >= 440 and
mouseY <= 491:
        if activeGate == "Light_OFF":
            activeGate = None
        else:
            activeGate = "Light_OFF"
    elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
        Xsquare = (mouseX-251)//49
        Ysquare = (mouseY-60)//49
        print(Xsquare)
        print(Ysquare)
        if activeGate == "Light_OFF":
            if logicArray[Ysquare][Xsquare][0] == "lightToBe":
                logicArray[Ysquare][Xsquare] = ["Light_OFF", False]
                activeGate = None
                tutorialPhase += 1
        if
logicInterfaceTutorialBack.checkForInput(logicInterface_MOUSE_POS):
    mainMenu()
elif tutorialPhase == 10:
    #next button appears visible to user
    logicInterfaceTutorialNext.changeColor(logicInterface_MOUSE_POS)
    logicInterfaceTutorialNext.update(SCREEN)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            mouseX = event.pos[0]
            mouseY = event.pos[1]
            print("X pos " + str(mouseX))
            print("Y pos " + str(mouseY))
            if event.button == 1:
                #any logic gate can be placed in place of the AND gate
                if mouseX >= 25 and mouseX <= 75 and mouseY >= 100 and
mouseY <= 140:
                    if activeGate == "ORGate":
                        activeGate = None
                    else:
```

```
        activeGate = "ORGate"
    elif mouseX >= 175 and mouseX <= 225 and mouseY >= 100 and
mouseY <= 140:
        if activeGate == "ANDGate":
            activeGate = None
        else:
            activeGate = "ANDGate"
    elif mouseX >= 25 and mouseX <= 75 and mouseY >= 175 and
mouseY <= 215:
        if activeGate == "NORGate":
            activeGate = None
        else:
            activeGate = "NORGate"
    elif mouseX >= 99 and mouseX <= 125 and mouseY >= 175 and
mouseY <= 215:
        if activeGate == "NANDGate":
            activeGate = None
        else:
            activeGate = "NANDGate"
    elif mouseX >= 175 and mouseX <= 225 and mouseY >= 175 and
mouseY <= 215:
        if activeGate == "XNORGate":
            activeGate = None
        else:
            activeGate = "XNORGate"
    elif mouseX >= 25 and mouseX <= 75 and mouseY >= 250 and
mouseY <= 290:
        if activeGate == "XORGate":
            activeGate = None
        else:
            activeGate = "XORGate"
    elif mouseX >= 251 and mouseX <= 1280 and mouseY >= 60 and
mouseY <= 697:
        Xsquare = (mouseX-251)//49
        Ysquare = (mouseY-60)//49
        print(Xsquare)
        print(Ysquare)
        if logicArray[Ysquare][Xsquare][0] == "ORGate" or
logicArray[Ysquare][Xsquare][0] == "ANDGate" or logicArray[Ysquare][Xsquare][0]
== "NORGate" or logicArray[Ysquare][Xsquare][0] == "NANDGate" or
logicArray[Ysquare][Xsquare][0] == "XNORGate" or logicArray[Ysquare][Xsquare][0]
== "XORGate":
            if activeGate == "ORGate":
                logicArray[Ysquare][Xsquare] = ["ORGate", False]
```

```
logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    print(activeGate)
elif activeGate == "ANDGate":
    logicArray[Ysquare][Xsquare] = ["ANDGate", False]
    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    undoStack.append(["remove", "ANDGate", Ysquare,
Xsquare])

    print(activeGate)
elif activeGate == "NORGate":
    logicArray[Ysquare][Xsquare] = ["NORGate", False]
    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    print(activeGate)
elif activeGate == "NANDGate":
    logicArray[Ysquare][Xsquare] = ["NANDGate", False]
    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    print(activeGate)
elif activeGate == "XNORGate":
    logicArray[Ysquare][Xsquare] = ["XNORGate", False]
    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    print(activeGate)
elif activeGate == "XORGate":
    logicArray[Ysquare][Xsquare] = ["XORGate", False]
    logicArray[Ysquare][Xsquare-1] = ["wireIntersection", False,
False]

    activeGate = None
    print(activeGate)
if logicArray[Ysquare][Xsquare][0] == "Switch_OFF":
    logicArray[Ysquare][Xsquare] = ["Switch_ON", True]
elif logicArray[Ysquare][Xsquare][0] == "Switch_ON":
    logicArray[Ysquare][Xsquare] = ["Switch_OFF", False]
if
logicInterfaceTutorialNext.checkForInput(logicInterface_MOUSE_POS):
    #tutorial ends
    mainMenu()
```

```
pygame.display.update()

def adders():
    pygame.display.set_caption("Adders")
    while True:
        adders_MOUSE_POS = pygame.mouse.get_pos()

        SCREEN.fill(BLACK)

        #adder text created
        adders_TEXT = get_font1(45).render("Please select an adder to load", True,
        WHITE)
        adders_RECT = adders_TEXT.get_rect(center=(640, 260))
        SCREEN.blit(adders_TEXT, adders_RECT)

        #buttons created
        addersBack = Button(pos=(640,440),text_input="Back", font=get_font2(30),
        base_color=WHITE, hovering_color=GREEN)
        addersHalfAdder = Button(pos=(550, 360), text_input="Half Adder",
        font=get_font2(30), base_color=WHITE, hovering_color=GREEN)
        addersFullAdder = Button(pos=(730,360), text_input="Full Adder", font =
        get_font2(30), base_color=WHITE, hovering_color=GREEN)

        #buttons draw in in interface
        for button in [addersBack, addersHalfAdder, addersFullAdder]:
            button.changeColor(adders_MOUSE_POS)
            button.update(SCREEN)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.MOUSEBUTTONDOWN:
                #back button pressed
                if addersBack.checkForInput(adders_MOUSE_POS):
                    mainMenu()
                #half adder button pressed
                if addersHalfAdder.checkForInput(adders_MOUSE_POS):
                    #file is loaded from adders folder
                    currentDirectory = os.path.dirname(os.path.abspath(__file__))
                    folderPath = os.path.join(currentDirectory, "adders")
                    filePath = os.path.join(folderPath, "halfAdder")
```



```
    logicFile = open(filePath,"rb")
    logicArray = pickle.load(logicFile)
    #logic interface opens
    logicInterface(logicArray,"halfAdder", False)
#full addder button pressed
if addersFullAdder.checkForInput(adders_MOUSE_POS):
    #file is loaded from adders folder
    currentDirectory = os.path.dirname(os.path.abspath(__file__))
    folderPath = os.path.join(currentDirectory, "adders")
    filePath = os.path.join(folderPath, "fullAdder")
    logicFile = open(filePath,"rb")
    logicArray = pickle.load(logicFile)
    #logic interface opens
    logicInterface(logicArray,"fullAdder", False)

pygame.display.update()

def mainMenu():
    pygame.display.set_caption("Main Menu")
    #logic array reset
    logicArray = []
    for i in range(0,14):
        logicArray.append([[None], [None], [None],[None], [None], [None],[None],
[None], [None],[None], [None], [None],[None], [None], [None],[None], [None],
[None],[None], [None], [None]])
    while True:
        SCREEN.blit(BG, (0, 0))

        MENU_MOUSE_POS = pygame.mouse.get_pos()

        menuText = get_font1(100).render("Logic Gate Simulator", True, "#b68f40")
        menuRect = menuText.get_rect(center=(640, 100))

        #menu buttons created
        newFileButton = Button(pos=(640, 200),
                                text_input="New File", font=get_font1(75), base_color="#d7fcd4",
                                hovering_color=WHITE)
        loadFileButton = Button(pos=(640, 300),
                                text_input="Load File", font=get_font1(75), base_color="#d7fcd4",
                                hovering_color=WHITE)
        tutorialButton = Button(pos=(640, 400),
                                text_input="Tutorial", font=get_font1(75), base_color="#d7fcd4",
                                hovering_color=WHITE)
        adderButton = Button(pos=(640, 500),
```

```
        text_input="Adders", font=get_font1(75), base_color="#d7fcd4",
hovering_color=WHITE)
    quitButton = Button(pos=(640, 600),
        text_input="Quit", font=get_font1(75), base_color="#d7fcd4",
hovering_color=WHITE)
```

```
SCREEN.blit(menuText, menuRect)
```

```
for button in [newFileButton, loadFileButton, tutorialButton, adderButton,
quitButton]:
```

```
    button.changeColor(MENU_MOUSE_POS)
    button.update(SCREEN)
```

```
for event in pygame.event.get():
```

```
    if event.type == pygame.QUIT:
```

```
        pygame.quit()
```

```
        sys.exit()
```

```
    if event.type == pygame.MOUSEBUTTONDOWN:
```

```
        #new file button pressed
```

```
        if newFileButton.checkForInput(MENU_MOUSE_POS):
```

```
            logicInterface(logicArray, None, False)
```

```
        #load file button pressed
```

```
        if loadFileButton.checkForInput(MENU_MOUSE_POS):
```

```
            loadInterface(logicArray, None, True)
```

```
        #adder button pressed
```

```
        if adderButton.checkForInput(MENU_MOUSE_POS):
```

```
            adders()
```

```
        #tutorial button pressed
```

```
        if tutorialButton.checkForInput(MENU_MOUSE_POS):
```

```
            logicInterface(logicArray, None, True)
```

```
        #quit button pressed
```

```
        if quitButton.checkForInput(MENU_MOUSE_POS):
```

```
            pygame.quit()
```

```
            sys.exit()
```

```
pygame.display.update()
```

```
#main menu opens
```

```
mainMenu()
```

Evaluation

MOSCOW chart

Within my research and analysis section, I created a MOSCOW chart to outline objectives that must, should and could be included in my project. I have since colour coded this chart for the objectives I did and didn't achieve. All of the must objectives have been completed as expected, with only a few of the should and could objectives left uncompleted.

Must	Should	Could	Won't
Contain all logic gate types from GCSE and A-Level	Allow users to swap out logic gates that have already been placed in the circuit	Contain a feature that allows auto saving of logic circuits	Allow sharing of files between multiple users
Drag and drop logic gates, input buttons and output lights onto a display	Contain a description for each logic gate providing a logic table for that specific gate	Allow users to select an area of their work they would like to replicate for use in another part of the circuit	
Contain at least a tutorial screen instructing users how to use the software	Contain an undo and redo function for ease of use	Contain a login system for users to access their individual files	
Connect wires to different inputs, gates and outputs	Allow a user to select the exam board and syllabus they are learning so that the logic gates that are part of their syllabus can be highlighted or those that are not can be hidden	Allow a user to save their circuit and reuse it at a later date	
Contain switches for linking to logic gates for input and	Allow the user to view a truth table of their work	Allow a user to change the zoom and menu size	

lights for outputs			
	Contain prebuilt adder functions	Translate boolean expressions to logic circuits and vice versa	
	Contain flip-flop gates		

Requirements

1 User opens simulation: **Met**

Upon starting the simulator the user is presented with all the options I intended on including. This includes opening a new document, loading an existing one, opening the tutorial menu, and opening the adders menu. The functionality of each menu works fully and as expected.

2 Options menu: **Partially Met**

Upon accessing the main logic interface, the user is presented with a variety of options relating to their document, ranging from functions relating to interacting with the circuit such as by translating it to a truth table, to file management, by loading and saving files. Most of this functionality works completely, however the intended translation tool between boolean algebra and logic circuits was not included due to its overall complexity; this could perhaps be added given better skill on my part as well as more time.

3 Logic Interface: **Met**

The function of the main logic interface including placing logic gates and changing wire orientation all function as expected. Users can place logic gates, flip flops, inputs and outputs as well as change the orientation of wire squares, depending on the wires surrounding them. Wires can also change orientation back to their original state when the wires around have been deleted.

4 Logic gate interactions: **Met**

The functionality of the inputs, outputs, flip flops and logic gates themselves all function as expected. The state of wires and gates can be changed using recursive subroutines that change the state of the wires in a given direction. Light bulbs also respond in accordance to the state of whatever device that is connected to the left of them

Improvements and further features

Boolean algebra translation

As mentioned in my design and testing sections, I was unable to complete my initially planned boolean algebra translation function. This would allow users to both translate boolean algebra to a logic circuit, and a logic circuit to boolean algebra; the former likely being easier to implement. I left this feature out as it was not essential for the overall function of the simulator, and would have been a complex feature to implement taking a large amount of my limited time that I felt would be better invested in other aspects of my project.

When translating to a logic circuit the feature would have taken a string entered by a user. This string would obviously contain special characters for boolean statements that would have likely been shown using an on screen keyboard. This would then be used to create a logic circuit.

For translating from a logic circuit to boolean, the entire array would have to be searched for first inputs, then logic gates connected to these inputs. This would then be used to create the boolean algebra expression that would be displayed in a separate menu.

Tutorial

As mentioned in a few responses from my user testing, the tutorial function could have done a better job in informing users of both how logic gates and flip flops work as well as the functionality of the options menu in the main logic interface. These could have been included by better management of my tutorial section, likely requiring more stages of the tutorial to be added.

Users also suggested that individual tutorials could be selected by the user to allow users to skip over parts of the tutorial they don't need and select specific tutorials for what circuit they would like to learn about. This feature could have also been extended to adder circuits, as a few test users weren't familiar with how adder circuits were meant to function.

Truth table functionality

A small improvement that could have been made would be in the way in which my project's truth tables are displayed, as users are unable to view their circuit alongside a truth table translation. With this improvement users would be able to open a separate window alongside their circuit window demonstrating a truth table for their created circuit. Unfortunately this functionality could not be implemented as pygame does not allow for two separate windows to be opened simultaneously, this might have been remedied by using a different GUI package with this feature available.

Another improvement would be in the way large truth tables are displayed as depending on the display, they can be out of bounds of the display resolution, this could have been potentially resolved by having the ability to scroll through the table.

Bibliography

Number	Source
1	 Football Management Simulator - NEA Project Report Football Management Simulator - NEA Project Report Used for reference for project report [First accessed 30 Mar 2023]
2	 Example_Non-Exam Assessment 2019 - Chess Chess game Used for reference for project report [First accessed 30 Mar 2023]
3	https://isaacomputerscience.org/ Isaac computer science Used for reference for project concept and for some analysis images [First accessed 30 Mar 2023]
4	https://logic.ly/demo/ Logic.ly Used as an existing system in research [First accessed 1 Apr 2023]
5	https://scratch.mit.edu/projects/785418/ Logic Gate Circuit Builder Used as an existing system in research

	[First accessed 1 Apr 2023]
4	https://www.youtube.com/watch?v=GMBqjxcKogA https://github.com/baraltech/Menu-System-PyGame HOW TO MAKE A MENU SCREEN IN PYGAME! Used to create main menu and logic document user interface [First accessed 6 Jul 2023]
5	https://www.dafont.com/ Dafont Used to obtain fonts used in the simulator [First accessed 11 Jul 2023]
6	https://www.geeksforgeeks.org/pygame-tutorial/ GeeksforGeeks Pygame tutorial Used to learn how pygame methods function [First accessed 12 Aug 2023]
7	https://www.youtube.com/watch?v=1Rc_MmMW5PI how to make a grid with pygame Used to create a grid for the logic user interface [First accessed 12 Aug 2023]
8	https://chat.openai.com/ ChatGPT Used throughout for fact checking [First accessed 12 Aug 2023]
9	https://www.youtube.com/watch?v=OGDJdeiuB5M&t=211s Pygame (Python Game Development) Tutorial - 6 - Draw Rect and Fill Used to learn how to draw rectangles in pygame [First accessed 12 Aug 2023]
10	https://www.youtube.com/watch?v=Lw8ndzcWFFw&list=PLuzRlnGbYC1VwDnabB00Pw8x2rfaCBDJ&index=3 How to add images in Pygame Pygame Basics Part 2 Used to learn how to add images into pygame [First accessed 12 Aug 2023]
11	https://drive.google.com/file/d/1g4Z9Rdsnsnw4p5mIUUXAKIM2oJvVNSvG/view?usp=drive_link Pygame noughts and crosses Used to create a grid for the logic user interface [First accessed 16 Aug 2023]
12	https://en.wikipedia.org/wiki/AND_gate#/media/File:AND_ANSI.svg https://en.wikipedia.org/wiki/OR_gate#/media/File:OR_ANSI_Labelled.svg

	https://en.wikipedia.org/wiki/Inverter_(logic_gate)#/media/File:Not-gate-en.svg https://en.wikipedia.org/wiki/NOR_gate#/media/File:NOR_ANSI_Labelled.svg https://en.wikipedia.org/wiki/NAND_gate#/media/File:NAND_ANSI_Labelled.svg https://en.wikipedia.org/wiki/XOR_gate#/media/File:XOR_ANSI.svg https://en.wikipedia.org/wiki/XNOR_gate#/media/File:Xnor-gate-en.svg Wikipedia logic gate images Used as sprites for logic gates in the simulator [First accessed 18 Aug 2023]
13	https://www.youtube.com/watch?v=Ro82dac_J1Y How to Drag and Drop Items in Pygame Used to create moveable logic gates - unused [First accessed 11 Sept 2023]
14	https://www.geeksforgeeks.org/understanding-python-pickling-example/ Understanding Python Pickling with Example Used to create saving system for documents [First accessed 16 Nov 2023]
15	https://www.youtube.com/watch?v=Rvcyf4HsWiw Python / Pygame tutorial: Getting text input Used for text input in user interfaces [First accessed 19 Nov 2023]
16	https://gamedev.stackexchange.com/questions/182459/how-to-register-multiple-key-presses-at-the-same-time-in-pygame How to register multiple key presses at the same time in pygame Used for undo/redo keybinds [First accessed 29 Jan 2024]
17	 COMP4 COMP4- Practical Project (Rugby Player Management System) Used for reference for project report [First accessed 16 Feb 2024]

Appendices

Appendix A:

Here are screenshots of my user feedback form I sent to my test users before testing my project.

...

Full Name *

Short-answer text

Age *

Short-answer text

What level are you currently studying at? *

☐ Secondary school education

☐ GCSE

☐ A-Level

☐ Degree level or higher

☐ Not studying

Do you have a background in the computer science industry? *

☐ Yes

☐ No

If yes, what is your background?

Short-answer text

If you would like, please try logic.ly, a pre-existing logic gate simulator that I am using as inspiration for my project

<https://logic.ly/demo/>

...

Have you previously used any logic gate simulators e.g. logic.ly? *

☐ Yes

☐ No

If yes, which logic gate simulators have you used?

Short-answer text

What features do you like about the simulator(s) you have used?

Long-answer text

Is there any features lacking from these simulators that you would like to see implemented?

Long-answer text

Have you had any previous experience working with/using logic gates? *

☐ Yes

☐ No

⋮

If yes, how would you rate your experience?

1

2

3

4

5

☐

☐

☐

☐

☐

If no, are you open to learning about logic gates?

☐ Yes

☐ No

Which of these features do you find necessary in a logic gate simulator? *

- ☐ Saving and loading files
- ☐ Translation tools for Boolean algebra
- ☐ Translation tools for Truth tables
- ☐ Pre-built adder circuits
- ☐ Keyboard shortcuts for menu functions
- ☐ I don't know
- ☐ Other...

Would you like to be a test user for the logic simulator software? *

- ☐ Yes
- ☐ No

Would you like to be sent a complete build of the software once it is available? *

- ☐ Yes
- ☐ No

Appendix B:

Here are screenshots of my user feedback form I sent to my test users after having tested my project.

Full Name *

Your answer

How easy did you find the user interface to navigate *

1 2 3 4 5

Very easy ☐ ☐ ☐ ☐ ☐ Very difficult

How do you believe the user interface could have been improved?

Your answer

How easy did you find it to place and link logic gates, wires and switches? *

1 2 3 4 5

Very Easy ☐ ☐ ☐ ☐ ☐ Very Difficult

How do you believe placement could have been improved?

Your answer

Did the logic gates in the simulator function as expected? *

☐ Yes

☐ No

If the logic gates did not function as expected, how do you believe they could function better?

Your answer

Did you find the save and loading functionality worked as expected? *

☐ Yes

☐ No

If no, what problems did you experience?

Your answer

Did you find the tutorial function an effective learning tool? *

- ☐ Yes
- ☐ No
- ☐ I didn't try the tutorial function

What could be done to improve the tutorial function?

Your answer

Did the functions for removing items from logic circuits function as expected? *
e.g. delete, clear, new, undo, redo

- ☐ Yes
- ☐ No

If no, what problems did you experience?

Your answer

Did the truth table generator work as expected? *

- ☐ Yes
- ☐ No
- ☐ I didn't try the truth table generator

if no, what problems did you experience?

Your answer

Did the adders generated from the adders menu perform as expected? *

- ☐ Yes
- ☐ No
- ☐ I didn't try the adder functions

if no, what problems did you experience?

Your answer

Did the flip flop circuit and clock function as expected? *

- ☐ Yes
- ☐ No
- ☐ I didn't try these

if no, what problems did you experience?

Your answer

What other features could have been added to improve the experience of the simulator?

Your answer